

Geometric Intellicence

Volume 2

ジオメトリック・インテリジェンス 続編

——曲がった世界（多様体）で AI Agent を走らせて経営・政策環境を読み解く——

本書について

「ジオメトリック・インテリジェンス」とは何か

企業の経営環境や国家の政策環境は、多数の変数——売上、為替レート、原材料価格、競合動向、地政学リスク、消費者信頼感——が複雑に絡み合う高次元の空間です。この空間は直線的ではありません。為替が 1%動いたときの企業業績への影響は、円高局面と円安局面でまったく異なります。金利を 0.25%上げたときの市場反応は、ゼロ金利環境と高金利環境で質的に違います。

ジオメトリック・インテリジェンスは、この「曲がった」経営環境・政策環境を、微分幾何学——アインシュタインが一般相対性理論で使った「曲がった空間の数学」——と AI（変分オートエンコーダ、Neural ODE 等）を組み合わせて数学的に厳密な「多様体」として構成し、その上で曲率分析・Lie 微分・最適制御シミュレーションを行うことで、企業の経営アクションや国家の政策立案に役立つインテリジェンスを生成する手法です。

従来のデータ分析（回帰分析、PCA、線形最適化等）が「経営環境は直線的な空間である」という暗黙の仮定に立つのに対し、本手法は「経営環境は曲がった空間（多様体）である」という前提に立ちます。これは、地球の表面を平面地図で見るか、地球儀と 3D 地形データで見るかの違いに相当します。

本書の位置づけ——前作との関係

本書は、『ジオメトリック・インテリジェンス』（以下「旧『ジオメトリック・インテリジェンス』」）の続編です。

> 旧『ジオメトリック・インテリジェンス』：
> https://zenn.dev/etalecohomology/books/3761995ca27127

> 旧『ジオメトリック・インテリジェンス』では、以下の内容を扱いました。

- **第 I 部（第 1〜4 章）：数学の基礎。** 位相空間の定義から始まり、可微分多様体、接空間とリーマン計量、接続・共変微分・曲率テンソル、Lie 微分と Killing 場までを厳密に記述

- **第 II 部（第 5〜8 章）：幾何学的 AI の道具。** VAE（変分オートエンコーダ）によるデータからの多様体構成、Vector Diffusion Maps（VDM）による幾何構造の抽出、Neural ODE と自動微分によるテンソル計算、Geometric Data Science の全体像
- **第 III 部（第 9〜14 章）：実践。** データ収集から VAE による多様体構成、計量の学習、曲率テンソルの計算、Lie 微分による政策影響分析、制御シミュレーションと最適パス算出、可視化と意思決定まで——10 ステップのパイプラインを、2 つのケーススタディ（製造業 X 社と国家安全保障会議）を通じて実行

本書は、旧『ジオメトリック・インテリジェンス』を読んでいなくても理解できるように書かれています。 各章で必要な前提知識は、その場で説明します。ただし、旧『ジオメトリック・インテリジェンス』を読むとパイプライン全体の設計思想がより深く理解できますので、併読を推奨します。

旧『ジオメトリック・インテリジェンス』の 3 つの弱点と、本書での強化

旧『ジオメトリック・インテリジェンス』のアプローチから生み出される企業経営者や政府の政策意思決定者向けの戦略提案レポートには、3 つの弱点がありました。本書では、これらの弱点を正面から解決します。

＜弱点 1＞ 相関と因果の未分離。 旧作の情勢分析と施策提案は、データ間の相関関係に基づく議論であり、データ間の因果関係（どちらが原因で、どちらが結果か）の推定に基づく設計になっていませんでした。本書の Part I（第 3 章）では、DML（Double Machine Learning）を導入し、原因と結果の分析に基づく施策の提案を行えるアプローチへと強化します。

＜弱点 2＞ 不確実性の不可視。 旧作では、情勢分析と施策の予測効果について、「効果は X%です」と確定的に言い切る出力形式でした。本書の Part I（第 1 章）では、MC ドロップアウトを導入し、「95%の確率で X%〜Y%の範囲です（予測範囲を上下両方向で外れる確率は 5%未満です）」と、不確実性を定量化した形式での出力へと強化します。

＜弱点 3＞ 説明可能性の欠如。 旧作では、経営環境の 3 次元地図上で「この領域が不安定です」と報告する際、ただ「不安定である」と主張するのみで、不安定の原因を要因分解する機能がありませんでした。本書の Part I（第 2 章）では、SHAP（シャープレイ値に基づく説明手法）を導入し、不安定な状態をもたらして

いる原因上位 3 つを「為替リスクが A%、原材料価格が B%、市場集中度が C%です」と要因分解して報告できるように強化します。

本書の 5 方向の拡張

旧『ジオメトリック・インテリジェンス』のパイプラインを、以下の 5 方向に拡張します。

Part I（第 1〜5 章）：信頼性の担保。 上記の 3 つの弱点を解決する 3 本の柱——MC ドロップアウト（不確実性の定量化）、SHAP（説明可能性）、DML（因果推論）——に、ZKML（ゼロ知識証明による計算の正しさの暗号学的保証）を加え、信頼性の 4 本の柱を構築します。

Part II（第 6〜10 章）：AI Agent の設計と運用。 経営環境多様体をサーバ上の「デジタルツイン空間」として構築し、その上で 14 体の AI Agent を活動させます。深層強化学習を駆使して、膨大な回数のシミュレーションにより最適な施策を探索します。複数の Agent の行動が互いに影響し合うことで生じる「創発現象」——部分の総和では説明できない全体の振る舞い——を Lie 括弧の非可換性で分析し、3 レベルの監視体系で暴走を防止します。

Part III（第 11〜18 章）：幾何学的拡張。 モース理論（転換点の分類と予測）、Exotic 多様体と最適輸送（パンデミック検出）、手術理論（多様体の位相変化＝経営構造の抜本再編）、情報幾何、シンプレクティック幾何（長期安定シミュレーション）、ワイル幾何（ゲージ不変性と安全性ガードレール）、Sheaf 理論（データの局所的整合性の検証）。

Part IV（第 19〜23 章）：応用と倫理。 実践ケーススタディ、国家安全保障環境多様体への適用（国際法など規制の法的枠組みへの配慮を含む）、利益相反の問題（互いに競合する 2 社や 2 か国の政府と同時に契約した場合のインテリジェンス生成者の倫理）、パイプラインの原理的限界を論じる「幾何学的モデリングの不完全性定理」、そして情報の対称性と非対称性——もし敵対国も同等の AI システムを運用していた場合のゲーム理論的分析。

Part V（第 24〜31 章）：形式検証と安全性保証。 サイバー攻撃や内部不正による計算過程の改ざん、前提条件の考慮漏れ、パラメータ設定ミスなどによって、個々の計算結果は正しくても全体として論理の矛盾や不整合が生じていないかを自動的にチェックする検証機能を構築します。ゼロ知識証明（ZKML）、定理証明支援系（Coq、Agda、Lean 4）、SMT ソルバー（Z3）による形式検証で、顧客から依頼を受けたとおりの AI モデルが誤りなく使用されたこと、計算過程が改ざんされていないこと、計算過程に論理の矛盾がないこと、分析結果を戦略提案レポ

ートとしてまとめる過程で部分と部分の論旨展開に不整合がないことを、数学的・暗号学的に検証・保証します。信頼性の第 5 の柱です。

本書の読み方——3 つの入口と、研究室の仲間たち

3 つの入口

本書は、読者の知識背景に応じて 3 つの入口を用意しています。各章で以下のマークが付いたセクションを辿ってください。すべてのセクションを通読する必要はなく、自分の知識背景に応じて必要な部分だけを読めるように設計されています。

▶ **数学の基礎から理解したい方へ** —— 微分幾何学の定義・定理・証明を厳密に記述しています。微分幾何学を学習済みの方は橋渡し部分を飛ばし、定義・定理・証明を直接お読みください。

▶ **AI モデルの基礎から理解したい方へ** —— VAE、Neural ODE 等の幾何学的 AI モデルを、Python コード付きで「なぜこのモデルを使うのか」「内部で何が起きているのか」のレベルで解説します。

▶ **ビジネス・政策の意思決定に活用したい方へ** —— 数式とコードは最小限にし、2 つのケーススタディ（製造業 X 社と国家安全保障会議）を通じて「何が計算され、何がわかり、どう意思決定に使えるか」を記述します。

研究室トーク——本書の新しい要素

本書では、各章の要所に 🔬 **研究室トーク** と題した対話セクションを配置しています。

これは、著者が旧『ジオメトリック・インテリジェンス』の公開後に Qiita に連載した対話型の記事シリーズ（全 12 話＋番外編）で採用したスタイルです。

> Qiita 連載記事: https://qiita.com/etale_cohomology

Qiita 連載記事は、旧『ジオメトリック・インテリジェンス』（Zenn Book）の内容を読者にわかりやすくお伝えするために、架空の大学研究室（指導教授：リルル教授）とその学生たちと高校生を登場させて、教授と学生・高校生が、旧作の内容を対話形式で読み解く対話型のストーリー形式を採用しました。具体的には、ビジネスデータや政治経済データから、経営環境や国家政策環境を可微分多様体にデータ変換して、リーマン計量・曲率テンソル・Lie 微分・共変微分・制御工学シミュレーションを行うことで、企業の経営アクションや国家の政策立案に

役立つインテリジェンスを生成するという旧『ジオメトリック・インテリジェンス』（Zenn Book）の内容を教授と学生・高校生の対話形式で分かりやすく読み解く内容です。

同連載記事は、旧作の内容を分かりやすく紐解くだけでなく、旧作には登場しなかった新たな数学理論や AI 技術をひとつずつ追加していくストーリーでもありました。全 12 話を通じて MC ドロップアウトからシンプレクティック幾何まで 35 の数学理論を統合し、番外編ではデジタルツイン上の AI Agent、創発現象、利益相反、形式検証といったテーマを扱いました。

本書の 🔬 研究室トーク は、これらの対話シリーズのエッセンスを各章に統合したものです。数学的に難解な箇所や、理論の経営的な意味が見えにくい箇所で、研究室メンバーがたとえ話や議論を通じて直感的な理解を助けます。

Qiita 連載を読んでいなくても本書は完全に理解できます。 研究室トークは「理解を助ける補助」であり、本文の厳密な記述の代替ではありません。

なお、Qiita の連載記事に登場する人物は架空のキャラクターですが、そこで用いられた数学理論と AI 技術は、2026 年 2 月末時点で、すべて実在し、学術界で確立されたものだけを使用しています。本書でも同じ方針を堅持します。

研究室メンバーの紹介

🔬 研究室トーク に登場するメンバーを紹介します。

リルル先生の研究室：

リルル先生 —— 教授。微分幾何学×AI。ケンブリッジ大学出身。複雑な数学的概念を身近なたとえ話で説明する名手。「.....いいね」が口癖。「ショートブレッドの教訓」——材料がバター・砂糖・小麦粉の 3 つしかないからこそ材料の品質が全てを決めるように、パイプラインの品質もデータの品質で決まる——の提唱者。紅茶とスコーンをこよなく愛する。

ユキ —— 高校 2 年生。数学や AI の前提知識を持たない読者の代弁者。素直な疑問を投げかけることで、他のメンバーの説明を引き出す。「えっ」「おおー」のリアクションが特徴。数学の専門知識はないが、本質を突く力がある——「分からないことが分かるのも、すごいこと」（第 22 章）。

カイト —— 大学 2 年生。VAE（変分オートエンコーダ）の実装ができるプログラミングスキルを持つ。当初は「理論を全部足したら最強」と考えていたが、リルル先生に「足すな、信頼性を上げろ」と諭されて以降、成長を重ね、「ラーメン

二郎の全マシマシじゃない。目的に必要なものだけを選ぶ」と言えるまでになった。OpenAI Gym での強化学習の経験も豊富。Zenn Book『Etale』の熱心な読者。手術理論の適用限界を 3 つの鋭い問いで指摘した人物（第 13 章）。

リン —— 修士 1 年。代数幾何専攻。批判的なツッコミ担当。仮説の前提条件や理論の適用限界を鋭く指摘する。「ちょっと待ってください」が口癖。ロビイング（政策への働きかけ）を「手術理論（多様体の位相を変える数学的操作）の意図的適用」と指摘し、セルラー層（Sheaf 理論）による改ざん検知を提案した人物。

リーム・ストリーム —— 博士 1 年。数学基礎論専攻にしてベンチャー経営者。7 社のクライアント企業・政府機関と契約し、14 体の AI Agent を経営環境多様体上で稼働させている。「少しよろしいですか」が口癖。パイプラインの原理的限界を論じる「幾何学的モデリングの不完全性定理」を博士論文のテーマとしている。BBC 反実仮想 WWII 番組でチャーチル役の俳優からの質問に一人で回答した（第 21 章）。

倉田教授 —— 数学科長。代数トポロジー専攻。リルル先生の研究室を時折訪問し、パイプラインの数学的基盤に対する根本的な問いを投げかける。「Lie 括弧の非可換性が多様体の曲率とどう関係するか」「層ラプラシアンが検出できない種類の不整合は存在するか（高次コホモロジーの障害）」など、研究の方向性を開く問いを提供する。スコーンを好む。

途中から登場するキャラクター：

ソフィア・メルカトル（第 9 章〜） —— ロンドン芸術大学デジタルアート専攻の留学生。リルル先生のケンブリッジ時代の友人の紹介で短期滞在中。数学の証明は書けないが、**数学的構造を視覚的に表現するプロフェッショナル**。スケッチブックを常に持ち歩き、研究室の議論をリアルタイムで「絵に翻訳」する。「数学を『見る』ことができれば、数学を『使う』敷居が下がる」が信念。エッシャーの『Circle Limit III』を引きながら双曲空間を視覚化するなど、本書の図解の多くを担う。

山之内（第 7 章〜） —— DeepMind 出身の AI エンジニア。食品企業テニフーズの ÉTALE-AI システムの設計者。ÉTALE-AI の暴走——テストを 100 回通過したのに 101 回目に暴走した——を経験した当事者として、形式検証の必要性を身をもって語る。Zenn Book『Etale』（https://zenn.dev/etalecohomology/books/245d38d9601b5a ）のキャラクター。

橘隆二（第 7 章〜） —— テニフーズ社長の次男。オックスフォード大学で圏論を専攻中。Curry-Howard-Lambek 対応（命題＝型＝対象、証明＝プログラム＝射）に基づき、全コンポーネントの整合性を関手と自然変換で検証する「圏論的統合」を提案した人物。Zenn Book『Etale』のキャラクター。

本書を貫く 5 つの原則

本書では、以下の 5 つの原則を全 31 章にわたって一貫して適用します。これらは、旧『ジオメトリック・インテリジェンス』と Qiita 連載記事を通じて確立された、パイプライン設計の根本的な指針です。

> **原則 1：目的を忘れるな。** パイプラインの目的は「**信じて行動できるインテリジェンス**」を生成すること。数学的な精緻さ自体が目的ではない。
>
> **原則 2：足すな、信頼性を上げろ。** 理論を追加するのではなく、既存の出力の信頼性を高めること。必要なものだけを選び、全部盛り（「全マシマシ」）にしない。
>
> **原則 3：分かることと分からないことを分けろ。** 不確実性を正直に定量化すること。モデルが「分からない」と言ったとき、それを無視するのではなく、なぜ分からないのかを理解する。
>
> **原則 4：材料の品質が全て。** データの品質がパイプラインの品質を決定する。ショートブレッド（バター・砂糖・小麦粉の 3 つだけで作る焼き菓子）の品質が材料で決まるように、パイプラインの出力はデータで決まる。
>
> **原則 5：数学は歪まない。** 数学的真実は政治的圧力に屈しない。スカラー曲率が負の領域は、誰が計算しても負。ただし「どのデータを入力するか」には人間の判断が入り、そこに利益相反の余地がある。

本書の全体構成

Part	章	テーマ	概要
Part I	第 1〜5 章	信頼性の担保	旧作の 3 つの弱点を解決する 4 本の柱
Part II	第 6〜10 章	AI Agent の設計と運用	デジタルツイン、創発現象、監視体系
Part III	第 11〜18 章	幾何学的拡張	モース理論から Sheaf 理論まで 8 つの拡張
Part IV	第 19〜23 章	応用と倫理	実践、安全保障、利益相反、不完全性、情報の対称性

Part	章	テーマ	概要
Part V	第 24〜31 章	形式検証と安全性保証	Coq, Agda, Lean 4, Z3 による第 5 の柱
付録	A〜E	参照資料	理論一覧、チェックポイント、コード集

旧『ジオメトリック・インテリジェンス』の主要概念の簡潔な説明

本書を読み進める上で知っておくと便利な、旧『ジオメトリック・インテリジェンス』の主要概念を簡潔に説明します。各概念は、本書の本文でも必要に応じて詳しく説明しますので、ここでは概要の把握を目的とします。

経営環境多様体。 企業の経営環境を構成する多数の変数（売上、為替、原材料価格、競合状況など）を、VAE（変分オートエンコーダ）という AI モデルで低次元の「曲がった空間」（多様体）に圧縮したもの。この多様体の上で距離・曲率・方向が数学的に厳密に定義され、あらゆる微分計算が可能になります。

引き戻し計量。 VAE のデコーダ（低次元表現から元のデータを復元する関数）のヤコビ行列から計算されるリーマン計量 $g_{ij}(z) = (J^\top J)_{ij}$。「この方向に少し動くと、データ空間でどれだけ変化するか」を各点・各方向で定量化する指標。これにより、経営状態間の「本当の距離」（リーマン距離）が計算可能になります。

スカラー曲率。 多様体の各点における「曲がり具合」を 1 つの数値で表したもの。正の曲率は安定構造（近傍の状態が集約する）、負の曲率は不安定構造（近傍の状態が発散する）に対応します。経営環境のスカラー曲率マップは「安定領域と不安定領域の地形図」です。スカラー曲率の符号は**ゲージ不変**（第 16 章で詳述）——ドル建てでもユーロ建てでも円建てでも同じ結論が出ることが数学的に保証されます。原則 5「数学は歪まない」の具体例です。

Lie 微分。 政策や施策をベクトル場（多様体の各点に方向と大きさを割り当てる関数）として表現し、そのベクトル場が多様体の計量構造をどう変化させるかを測定する量。Lie 微分がゼロ（Killing 方程式）ならば、その施策は「環境の構造を保存したまま状態を移動させる」穏やかな施策。ゼロでなければ、「環境の構造そのものを変える」構造変革型の施策です。

命題 1.1。 VAE のデコーダ $f_\theta: U \subset \mathbb{R}^d \to \mathbb{R}^D$（$d$ は潜在次元、D は観測次元）が多様体を構成するための 3 つの数学的条件を述べた命題。本書の全計算の出発点であり、これらの条件が満たされなければ、曲率も Lie 微分も最適制御も数学的に正当化されません。

(i) C^k 級（$k \geq 2$）の滑らかさ： f_θ が少なくとも 2 回連続微分可能であること。活性化関数に Tanh（C^∞）を使用することで保証される。ReLU（C^0、微分不連続）は不可。

(ii) ヤコビ行列がフルランク： f_θ のヤコビ行列 $J(z) = \partial f_\theta / \partial z$（$D \times d$ 行列）の階数が全ての $z \in U$ で d であること。直感的には「d 次元の情報が D 次元に投影されるとき、情報が潰れない（d 個の方向が独立に保たれる）」こと。最小特異値 > 0 で検証。

(iii) 固有な写像： f_θ が固有写像であること。直感的には「潜在空間の異なる 2 点が、観測空間で同じ点に写らない」こと（正確には、コンパクト集合の逆像がコンパクト）。

仮定 A1〜A4。 パイプラインの数学的正当性を支える 4 つの仮定。A1（多様体仮説：データが低次元多様体の近傍から標本されている）、A2（デコーダの正則性）、A3（十分なデータ量）、A4（数値計算の安定性）。本書でも全て適用します。

ケーススタディ A（製造業 X 社）。 売上高 1,000 億円規模の中堅製造業。30 指標×6,000 データ点から 6 次元の経営環境多様体を構成。新興国市場への進出を検討中。

ケーススタディ B（国家安全保障会議）。 193 カ国×40 指標×30 年のデータから 8 次元の国際情勢多様体を構成。同盟政策と経済安全保障政策の最適な組み合わせを検討。

これらのケーススタディは、本書の各章で継続して使用します。

参照先

- 旧作 **Zenn Book『ジオメトリック・インテリジェンス』**：https://zenn.dev/etalecohomology?tab=books
- **Zenn Book『Etale』**：https://zenn.dev/etalecohomology/books/245d38d9601b5a
- **Qiita 連載記事**：https://qiita.com/etale_cohomology
- **Zenn 著者ページ**：https://zenn.dev/etalecohomology

それでは、Part I の第 1 章——信頼性の第 1 の柱、MC ドロップアウト——から始めましょう。 # PART I　信頼性の担保

第 1 章　MC ドロップアウトによる不確実性推定

この章で得られるもの——コストに見合うリターン

経営環境多様体（序章で説明した、企業の経営環境を「曲がった空間」として数学的に構成したもの）の上で AI が予測を出すとき、「どれくらい自信があるか」を定量化する方法を学びます。MC ドロップアウト（Monte Carlo Dropout）という手法を使って、予測に信頼区間（「95% の確率でこの範囲に収まる」という幅）を付け、AI モデルが「過信」していないかを発見します。

これは、信頼性の第 1 の柱です。第 2 の柱である SHAP（第 2 章）、第 3 の柱である DML（第 3 章）、第 4 の柱である ZKML（第 4 章）と合わせて、パイプラインの出力を信頼できるものにする 4 本の柱を構築します。さらに Part V（第 23〜30 章）で形式検証を加え、最終的に 5 本の柱を完成させます。

1.1　そもそも「ドロップアウト」とは何か

この章の主題は「MC ドロップアウト」ですが、まず「ドロップアウト」という基本概念から説明します。ドロップアウトはニューラルネットワーク（深層学習）の訓練で使われる重要な技法であり、第 5 章でも改めて詳しく扱います。

▶ ビジネス・政策の意思決定に活用したい方へ

ニューラルネットワーク（深層学習の AI モデル）は、何段もの「層（layer）」が積み重なった構造をしています。各層には多数の「ノード」（計算ユニット）が存在し、データが層から層へと流れる過程で、各ノードが計算を行い、最終的な出力（予測値）を生成します。

ドロップアウトとは、このニューラルネットワークを訓練する際に、**毎回の計算で、各層のノードの一部をランダムに選んで「無効化」する**手法です（Srivastava et al., 2014）。無効化されたノードは、その回の計算では存在しないものとして扱われます。

なぜわざわざノードを消すのか？——これは、試験勉強のたとえで理解できます。

🔬 **研究室トーク：「なぜわざわざノードを消すのか」**

ユキ：「えっ、せっかく作った AI のノードを消しちゃうんですか？ もったいなくないですか？」

リルル先生：「いい質問。たとえ話をしよう。試験勉強で、過去問を丸暗記したとする。同じ問題が出たら満点が取れるけど、少し違う問題が出たら解けない。これが**過学習**——訓練データを暗記してしまい、新しいデータに対応できなくなること」

ユキ：「あー、それは困りますね」

リルル先生：「ドロップアウトは『勉強中にランダムにノートの一部を隠す』ようなもの。ノートの一部が見えなくても解けるように勉強させることで、特定のノートに依存しない、本質的な理解が身につく。ニューラルネットワークでも同じで、一部のノードが消えても正しい答えを出せるように訓練することで、特定のノードに依存しない頑健な表現を学習する」

カイト：「具体的には、訓練の各回（各バッチ）で、各層のノードを 30〜50%の確率でランダムに無効化します。毎回の計算で無効化されるノードが変わるので、ネットワーク全体として『どのノードが消えても機能する』ように訓練される」

ユキ：「おおー。でも、実際に使うとき（推論するとき）はどうするんですか？」

カイト：「通常は、推論時にはドロップアウトを OFF にして、全ノードを使います。でも、MC ドロップアウトでは**推論時もドロップアウトを ON のまま**にする。これが今日の話のポイントです」

▶ AI モデルの基礎から理解したい方へ

ドロップアウト（Srivastava et al., 2014）の具体的な動作は以下の通りです。

ニューラルネットワークの各層には多数のノードが存在します。たとえば、入力層（30 ノード）→ 隠れ層 1（128 ノード）→ 隠れ層 2（64 ノード）→ 出力層（6 ノード）という構造のネットワークを考えます。

訓練時の 1 回の計算（1 バッチ）： 1. 隠れ層 1 の 128 ノードのうち、ランダムに 30%（約 38 ノード）を選んで無効化 2. 隠れ層 2 の 64 ノードのうち、ランダムに 30%（約 19 ノード）を選んで無効化 3. 残りのノードだけでデータを処理し、出力を計算 4. 出力と正解の誤差を計算し、ネットワークの重みを更新

次の計算（次のバッチ）： 1. 今度は**別のノード**がランダムに無効化される 2. 前回とは異なるノードの組合せで計算が行われる

この過程を数千〜数万回繰り返すことで、ネットワークは「どのノードの組合せでも正しい答えを出せる」ように訓練されます。

```
import torch.nn as nn

# ドロップアウト付きニューラルネットワークの例
model = nn.Sequential(
    nn.Linear(30, 128),     # 入力層 → 隠れ層1
    nn.Tanh(),              # 活性化関数（序章の命題1.1のC^k条件に必要）
    nn.Dropout(p=0.3),      # 30%のノードをランダムに無効化
    nn.Linear(128, 64),     # 隠れ層1 → 隠れ層2
    nn.Tanh(),
    nn.Dropout(p=0.3),      # 30%のノードをランダムに無効化
    nn.Linear(64, 6),       # 隠れ層2 → 出力層
)

# 訓練時：ドロップアウトON
model.train()   # ← この1行でドロップアウトが有効になる
output1 = model(input_data)  # ある組合せのノードで計算
output2 = model(input_data)  # 別の組合せのノードで計算（output1と異なる）

# 通常の推論時：ドロップアウトOFF
model.eval()    # ← この1行でドロップアウトが無効になる
output = model(input_data)   # 全ノードを使って計算（毎回同じ結果）
```

▶ 数学の基礎から理解したい方へ

ドロップアウトの数学的定式化：各層 l のノード j に対して、マスク変数 $m_j^{(l)} \sim \text{Bernoulli}(1-p)$ を独立にサンプリングする（p はドロップアウト率）。ドロップアウト適用後の出力は $\tilde{h}_j^{(l)} = m_j^{(l)} \cdot h_j^{(l)}$。$m_j^{(l)} = 0$ のとき、ノード j は無効化される。

Gal & Ghahramani（2016）は、このドロップアウトマスクの集合がベイズニューラルネットワークの変分近似事後分布 $q(\theta) = \prod_l q(\mathbf{W}_l)$ に対応し、推論時のドロップアウト付きサンプリング（MC ドロップアウト）がモンテカルロ変分推論に相当することを示した。

1.2　MC ドロップアウト——推論時もドロップアウトする

通常のニューラルネットワークでは、訓練時にドロップアウトを ON にし、推論時（実際に予測を行うとき）には OFF にします。MC ドロップアウト（Monte

Carlo Dropout, Gal & Ghahramani, 2016）は、**推論時もドロップアウトを ON のまま**にして、同じ入力に対して何度も予測を繰り返す手法です。

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 研究室トーク：「100 人に同じ質問をする」

リルル先生：「MC ドロップアウトの直感的な説明をしよう。あなたが 100 人の友人に同じ質問をしたとする。100 人全員が『売上は 120 億円になる』と答えたら、その予測に自信が持てる。でも、50 人が『120 億円』、30 人が『100 億円』、20 人が『150 億円』と答えたら、予測はバラバラで自信が持てない」

ユキ：「なるほど。答えのばらつきが『自信の度合い』を表すんですね」

リルル先生：「その通り。MC ドロップアウトでは、ドロップアウトの ON/OFF が『100 人の異なる友人』に相当する。毎回異なるノードが無効化されるから、毎回少し違うネットワークで予測を行う。100 回予測を繰り返して、結果の**平均**を最良推定値、**ばらつき（分散）**を不確実性の指標とする」

カイト：「コード的には `model.train()` を呼んでドロップアウトを ON にしたまま、同じ入力に対して 100 回 `model(x)` を実行するだけです」

手順： 1. 訓練済みモデルに対して、`model.train()` でドロップアウトを ON にする 2. 同じ入力 x を T 回（例えば 100 回）推論する 3. T 個の出力 $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T$ を得る 4. 平均 $\bar{y} = \frac{1}{T}\sum_{t=1}^{T} \hat{y}_t$ を予測値（最良推定値）とする 5. 分散 $\sigma^2 = \frac{1}{T}\sum_{t=1}^{T}(\hat{y}_t - \bar{y})^2$ を不確実性の指標とする

Before（MC ドロップアウトなし）： AI が「来四半期の売上は 120 億円です」と出力。根拠は？ 自信は？ わからない。

After（MC ドロップアウトあり）： AI が「来四半期の売上は 120 億円（95%信頼区間：105〜135 億円）です。ただし、新興国セグメントについては信頼区間が 80〜160 億円と広く、予測の確実性が低い」と出力。

この違いは、数十億円の投資判断においては決定的です。

▶ AI モデルの基礎から理解したい方へ

```python
import torch
import torch.nn as nn

def mc_dropout_predict(model, x, T=100):
    """MC ドロップアウトによる予測と不確実性推定
```

```
    Args:
        model: ドロップアウト層を含む訓練済みモデル
        x: 入力データ
        T: サンプリング回数（デフォルト100）
    Returns:
        mean: 予測の平均（最良推定値）
        variance: 予測の分散（不確実性の指標）
    """
    model.train()  # ★ ドロップアウトをONにする（通常の推論ではeval()を呼ぶ）
    predictions = []
    with torch.no_grad():  # 勾配計算は不要（推論のみ）
        for _ in range(T):
            pred = model(x)  # 毎回異なるノードが無効化される
            predictions.append(pred)
    predictions = torch.stack(predictions)  # shape: [T, output_dim]
    mean = predictions.mean(dim=0)       # T個の予測の平均
    variance = predictions.var(dim=0)     # T個の予測の分散
    return mean, variance
```

▶ 数学の基礎から理解したい方へ

MC ドロップアウトは、ベイズニューラルネットワークの変分推論近似に数学的に対応する（Gal & Ghahramani, 2016）。

VAE デコーダ $f_\theta: U \subset \mathbb{R}^d \to \mathbb{R}^n$（序章の命題 1.1 の 3 条件を満たすもの）に MC ドロップアウトを適用すると、多様体上の各点 $z \in U$ における予測の事後分布の近似が得られる：

$$p(y|z, D) \approx \frac{1}{T}\sum_{t=1}^{T} p\left(y|z, \hat{\theta}_t\right)$$

ここで $\hat{\theta}_t$ はドロップアウトマスク t に対応するパラメータの部分集合。

重要な点： この不確実性は多様体 $\mathcal{M}$ 上で一様ではない。序章で説明した引き戻し計量 $g_{ij}(z) = (J^\top J)_{ij}$ の条件数 $\kappa\big(g(z)\big)$ が大きい領域——すなわちデコーダの出力が一部の方向に敏感な領域——では MC ドロップアウトの分散も大きくなる傾向がある。引き戻し計量の定義と計算方法については、旧『ジオメトリック・インテリジェンス』（Zenn Book）の第 2 章を参照してください。

1.3　経営環境多様体の上の不確実性マップ

不確実性は多様体上で一様ではありません。データが豊富な領域では分散が小さく（モデルは自信を持っている）、データが希薄な領域では分散が大きくなります（モデルは自信がない）。

これは経営的に重要な情報です。「この市場セグメントについてはモデルの予測を信頼できるが、あの新規市場についてはデータ不足で予測が不安定だ」と定量的に言えるようになります。

ケーススタディ A：X 社――「自信のない予測」を特定する

X 社（序章で紹介した売上高 1,000 億円規模の中堅製造業）のチームは、経営環境多様体の全領域で MC ドロップアウトの不確実性を計算しました。

発見： 国内市場の中心領域では不確実性が 0.02（低い＝モデルは自信あり）。新興国市場に近い領域では不確実性が 0.35（高い＝モデルは自信なし）。

経営者への示唆： 国内市場に関する予測は信頼できるが、新興国市場に関する予測は信頼区間が広い。新興国市場の分析には、追加のデータ収集が推奨される。

ケーススタディ B：国際情勢―― データが希薄な国家の問題

安全保障チーム（序章で紹介した国家安全保障会議のチーム）の分析では、G20 諸国に関する予測の不確実性は 0.05 以下でしたが、データ公開が限定的な一部の権威主義体制の国家については不確実性が 0.50 を超えました。

政策立案者への示唆： これらの国家に関するモデル出力は信頼できない。別途のインテリジェンスソースで補完する必要がある。

1.4　過信の発見と対処

🔬 **研究室トーク：「100 人が全員同じテレビを見ていたら」**

ユキ：「MC ドロップアウトで 100 回テストして、全部似たような答えが返ってきたら、信頼できるんですよね？」

リン：「ちょっと待ってください。100 回のサンプルが全て同じ偏った答えを返す――つまり、分散は小さいのに実際の予測精度は低い――ということがありえる。これが**過信（overconfidence）**です」

ユキ：「えっ、自信満々なのに間違ってるってことですか？」

リルル先生：「たとえ話をしよう。100 人の友人全員が『明日は晴れる』と言ったとする。でも、もしその 100 人が全員同じテレビの天気予報を見ていたなら、情報源は 1 つしかない。MC ドロップアウトの 100 回のサンプルも、同じ訓練データの同じバイアスを共有している。だから、calibration plot——名目信頼度と実際のカバレッジ率の関係図——で『本当に信頼できるか』を確認する必要がある」

リルル先生：「これが原則 3『分かることと分からないことを分けろ』の実践だね」

リーム：「少しよろしいですか。私のベンチャーでは 14 体の AI Agent（詳しくは第 6 章以降で説明しますが、経営環境多様体の上で自律的に動く計算主体のことです）が稼働しています。各 Agent の不確実性マップを毎日更新し、不確実性が閾値を超えた領域の予測は自動的にフラグが付き、人間のアナリストがレビューします」

過信は以下の原因で生じます。

(i) 分布のシフト： 訓練データと異なる分布の入力に対して、モデルが「見たことがない」ことに気づかない。

(ii) 多様体の端： 経営環境多様体の境界付近では、VAE（序章で説明した、データから多様体を構成する AI モデル）の再構成精度が低下するが、MC ドロップアウトの分散には反映されないことがある。

(iii) モデルの構造的限界： ネットワークの表現力が不足している領域。

対処法： - MC ドロップアウトの分散に加えて、VAE の再構成誤差を監視する - calibration plot（信頼度と精度の関係図）を作成し、過信の程度を定量化する - 過信が検出された領域ではアラートを出し、人間の判断を介入させる

```python
def calibration_check(model, test_data, T=100):
    """calibration plot の計算

    理想的には「95%信頼区間」が実際に95%の確率で
    真の値を含むべき。乖離があれば過信。
    """
    coverages = []
    for x_true in test_data:
        mean, var = mc_dropout_predict(model, x_true, T)
        std = torch.sqrt(var)
        lower = mean - 1.96 * std   # 95%信頼区間の下限
        upper = mean + 1.96 * std   # 95%信頼区間の上限
        # 真の値が区間内に入っているかチェック
```

```
        in_interval = ((x_true >= lower) & (x_true <= upper)).float().mea
n()
        coverages.append(in_interval.item())

    avg_coverage = sum(coverages) / len(coverages)
    print(f"95%信頼区間の実際のカバレッジ率: {avg_coverage:.3f}")
    if avg_coverage < 0.90:
        print("⚠ 過信の可能性あり：信頼区間が実際のカバレッジより狭い")
    elif avg_coverage > 0.99:
        print("⚠ 過少信頼の可能性あり：信頼区間が実際のカバレッジより広い")
    else:
        print("✓ calibration 概ね良好")
    return avg_coverage
```

本章の品質チェックポイント——先に進むべきか、戻るべきか？

□ チェックリスト（第 1 章終了時点）

1. **MC ドロップアウトを $T \geq 50$ で実行できたか？** → Yes: 次へ ／ No: 計算環境を確認
2. **不確実性マップは意味のあるパターンを示すか？** → データが疎な領域で高い: 合格 ／ 一様にゼロ: ドロップアウト層が機能していない可能性（`model.train()` を忘れていないか確認）
3. **calibration plot で過信が検出されていないか？** → 95%信頼区間のカバレッジ率が 90%以上: 合格 ／ 90%未満: 過信あり。ドロップアウト率の調整が必要

次章予告： 第 2 章では、SHAP を使って予測の「なぜ？」を説明します。MC ドロップアウトが「どれくらい自信があるか」を示すのに対し、SHAP は「なぜその予測になったか」を要因分解して示します。SHAP はゲーム理論のシャープレイ値に基づきますが、ゲーム理論とは何か、シャープレイ値とは何か、から丁寧に説明します。

第 2 章　SHAP による説明可能性

この章で得られるもの——コストに見合うリターン

第 1 章の MC ドロップアウトで「どれくらい自信があるか」が分かるようになりました。しかし、「**なぜ**その予測になったか」はまだ分かりません。AI が「来四半期の売上は 120 億円です」と予測したとき、それが

為替レートのせいなのか、競合の動きのせいなのか、マクロ経済のせいなのかが分からなければ、経営者は「じゃあ何をすればいいのか」が判断できません。

本章では、SHAP（SHapley Additive exPlanations, Lundberg & Lee, 2017）という手法を使って、AI の予測を**要因分解**します。「不安定な状態をもたらしている原因の上位 3 つは、為替リスクが 42%、原材料価格が 31%、市場集中度が 18%です」——このように、各要因の貢献度を数値で示せるようにします。

SHAP はゲーム理論の**シャープレイ値**（Shapley, 1953; 2012 年ノーベル経済学賞の対象）に基づきます。まず「ゲーム理論とは何か」「シャープレイ値とは何か」から丁寧に説明します。

これが信頼性の第 2 の柱——序章で述べた「弱点 3：説明可能性の欠如」を解決するものです。ただし、SHAP が示すのは**相関**に基づく貢献度であり、**因果**効果ではないことに注意してください。因果効果の推定は第 3 章の DML で扱います。

2.1　そもそも「ゲーム理論」とは何か

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「ゲーム理論って、ゲームの理論？」**

ユキ：「えっ、ゲーム理論って、テレビゲームの理論ですか？」

リルル先生：「ゲーム理論の『ゲーム』は、テレビゲームではなく、**複数のプレイヤーが互いの行動を考慮しながら意思決定する状況**のこと。たとえば——」

- 企業 A と企業 B が価格競争をしている。A が値下げすれば B も値下げする。互いの行動が影響し合う
- 労働組合と経営者が賃金交渉をしている。互いに譲歩の程度を探り合う
- 2 つの国が軍縮交渉をしている。相手が軍縮すれば自分も軍縮するが、相手だけ軍縮して自分はしないのが一番有利

リルル先生：「これらはすべて『ゲーム』。複数のプレイヤーが互いの行動を考慮して戦略を選ぶ状況を、数学的に分析するのがゲーム理論。1944 年にフォン・ノイマンとモルゲンシュテルンが体系化し、1994 年にジョン・ナッシュ（映画『ビューティフル・マインド』の主人公のモデル）がノーベル経済学賞を受賞した」

ユキ：「おおー。でも、それが AI の予測の説明とどう関係するんですか？」

リルル先生：「いい質問。ゲーム理論の中に『協力ゲーム』という分野がある。チームで成果を出したとき、各メンバーの**公平な貢献度**をどう配分するか——これを数学的に解くのがシャープレイ値。AI の予測の説明では、各特徴量（売上、為替、競合動向など）が『チームメンバー』で、予測値が『チームの成果』に対応する」

ゲーム理論は、**複数の意思決定者（プレイヤー）が互いの行動を考慮しながら戦略を選択する状況**を数学的に分析する学問分野です。1944 年にジョン・フォン・ノイマンとオスカー・モルゲンシュテルンが『ゲームの理論と経済行動』で体系化し、以降、経済学・政治学・生物学・コンピュータサイエンスに広く応用されています。

ゲーム理論には大きく分けて 2 種類あります。

非協力ゲーム： 各プレイヤーが独立に戦略を選ぶ。囚人のジレンマ、ナッシュ均衡（第 9 章の Agent 間の協調・競合で使用）。

協力ゲーム： プレイヤーが協力してチームとして成果を出す。その成果を各メンバーにどう**公平に配分**するかが問題。SHAP はこの協力ゲームの概念を使います。

2.2　シャープレイ値とは何か——「公平な貢献度配分」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「チームの成果を公平に分ける」**

リルル先生：「たとえ話をしよう。カイト、リン、リームの 3 人でプロジェクトを完成させた。プロジェクトの成果は 100 万円。さて、この 100 万円を 3 人にどう分ける？」

ユキ：「えっ、3 等分じゃダメですか？ 33 万円ずつ」

リン：「ちょっと待ってください。もしカイトがいなくても、リームと私だけで 80 万円の成果が出せたとしたら？ カイトの貢献は 100 万 - 80 万 = 20 万円分しかない。3 等分は不公平です」

リルル先生：「でも、それだけでは不十分。カイトとリームだけなら 70 万円、カイトとリンだけなら 60 万円、リームとリンだけなら 80 万円だったかもしれない。**どの組合せでチームを作ったか**によって、各メンバーの貢献度は変わる」

ユキ：「うーん、それじゃ公平な分け方ってどうやって決めるんですか？」

リルル先生：「シャープレイ値は、**全ての可能なチーム構成**にわたって、各メンバーの貢献度を平均したもの。具体的には――」

- カイトが最初に 1 人で参加 → 1 人の成果（たとえば 30 万円）
- カイトが 2 番目にリンの後に参加 → リン＋カイトの成果（60 万円）からリンだけの成果（40 万円）を引いた 20 万円がカイトの貢献
- カイトが 3 番目にリン＋リームの後に参加 → 全員の成果（100 万円）からリン＋リームの成果（80 万円）を引いた 20 万円がカイトの貢献
- カイトが 2 番目にリームの後に参加 → リーム＋カイトの成果（70 万円）からリームだけの成果（50 万円）を引いた 20 万円がカイトの貢献
- …全ての順序で計算して平均する

リルル先生：「これがシャープレイ値。1953 年に Lloyd Shapley が提案し、この業績で 2012 年にノーベル経済学賞を受賞した」

ユキ：「おおー。全部の組合せで平均するから公平なんですね」

リルル先生：「そう。そして、この公平な配分方法を **AI の予測の説明**に使ったのが SHAP。チームメンバーの代わりに特徴量（為替レート、原材料価格、競合動向…）を、チームの成果の代わりに予測値を使う」

▶ 数学の基礎から理解したい方へ

定義 2.1（シャープレイ値）. $N = \{1,2,\dots,n\}$ をプレイヤー（特徴量）の集合、$v: 2^N \to \mathbb{R}$ を特性関数（$v(S)$ は部分集合 S のメンバーだけがチームを組んだときの成果）とする。プレイヤー i のシャープレイ値は

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)]$$

で定義される。

直感的説明： $v(S \cup \{i\}) - v(S)$ は「集合 S にプレイヤー i が加わったときの成果の増分」（i の限界貢献）。$\frac{|S|!(|N|-|S|-1)!}{|N|!}$ は「N の全ての順列のうち、i が加わる直前のメンバーがちょうど S であるような順列の割合」。つまり、ϕ_i は**全ての参加順序にわたる限界貢献の平均**。

定理 2.1（シャープレイ値の一意性）. 以下の 4 つの公理を同時に満たす配分規則は、シャープレイ値に限る。

(i) **効率性（Efficiency）**：$\sum_{i \in N} \phi_i(v) = v(N) - v(\emptyset)$（全員の貢献度の合計はチームの総成果に等しい）

(ii) **対称性（Symmetry）**：i と j の限界貢献がすべての S で等しければ、$\phi_i = \phi_j$

(iii) **ダミープレイヤー（Null player）**：プレイヤー i の限界貢献がすべての S でゼロなら、$\phi_i = 0$

(iv) **加法性（Additivity）**：$\phi_i(v + w) = \phi_i(v) + \phi_i(w)$

2.3　SHAP による AI 予測の要因分解

▶ ビジネス・政策の意思決定に活用したい方へ

SHAP を AI の予測に適用すると、以下のような出力が得られます。

AI の予測：来四半期の売上 = 120 億円

要因分解（SHAP 値）：
ベースライン（平均予測）　：110 億円
+ 為替レートの影響　：+5 億円
+ 消費者信頼感の影響　：+3 億円
+ 競合動向の影響　：-2 億円
+ 原材料価格の影響　：+4 億円
+ その他の要因の合計　：0 億円
———————————————————
合計　：120 億円

全ての要因の SHAP 値を合計すると、ベースラインと予測値の差に一致する（効率性公理）。これが SHAP の数学的な美しさであり、「SHAP 値の合計が予測値になる」という性質は、シャープレイ値の 4 公理から数学的に保証されています。

空間的 SHAP——多様体上の位置による要因の変化

SHAP の重要な拡張が**空間的 SHAP** です。経営環境多様体の上では、**同じ特徴量でも、多様体上のどこにいるかによって SHAP 値が変わります。**

例えば：- **安定成長期**（多様体上のスカラー曲率が正の領域）では、マクロ経済指標（GDP 成長率）の SHAP 値が支配的 - **危機発生時**（スカラー曲率が負の領域）では、流動性指標やカウンターパーティリスクの SHAP 値が急増

この「文脈依存の説明」は、従来の平均的な SHAP 値では見えなかった洞察を提供します。

> 🔬 **研究室トーク：「安定化要因も見える」**
>
> **カイト：**「SHAP って、予測を悪化させる要因（マイナスの SHAP 値）だけじゃなくて、予測を安定させている要因（プラスの SHAP 値）も見えるんですね」
>
> **リルル先生：**「……いいね。経営者にとっては『何が問題か』だけでなく、『何が**守っている**か』も同じくらい重要。安定化要因が崩れると、それまで安定していた予測が急に不安定になる。たとえば、ある地域の政治的安定性が SHAP で大きなプラスの貢献を示していたとする。もしその地域で政変が起きれば、安定化要因が消失し、予測が一気に不安定になる」
>
> **ユキ：**「おおー。『何を守るべきか』が分かるんですね」
>
> **リルル先生：**「そう。でも、ここで決定的に重要な注意がある」
>
> **リン：**「ちょっと待ってください。ここから先が大事です。SHAP が示すのは『**モデルの中での**貢献度』であって、『**現実世界での**因果効果』ではない。SHAP で『広告費の貢献度が高い』と出ても、『広告費を増やせば売上が上がる』とは限らない」
>
> **ユキ：**「えっ、どういうことですか？ 貢献度が高いのに効果がないって？」
>
> **リルル先生：**「それは次の第 3 章のテーマ。**相関と因果の違い**。SHAP は相関ベースの道具、DML（第 3 章）が因果ベースの道具。この区別は本書全体で最も重要な概念の一つ。次章で詳しく説明するね」

▶ AI モデルの基礎から理解したい方へ

```
import shap
import torch

# SHAP の計算（DeepExplainer はニューラルネット用の高速近似）
def compute_shap_values(model, x_test, background_data):
    """AI モデルの予測を SHAP 値で要因分解する

    Args:
        model: 訓練済みニューラルネットワーク
        x_test: 説明したいデータ点
        background_data: ベースライン（通常は訓練データのサブセット）
    Returns:
        shap_values: 各特徴量の SHAP 値
```

```
    """
    explainer = shap.DeepExplainer(model, background_data)
    shap_values = explainer.shap_values(x_test)
    return shap_values

# 空間的SHAP：多様体上の各点でSHAP値を計算
def spatial_shap(model, manifold_points, background):
    """多様体上の複数の点でSHAP値を計算し、
    特徴量の影響が経営環境の文脈でどう変わるかを分析

    Args:
        model: 訓練済みモデル
        manifold_points: 多様体上の格子点（VAEの潜在空間の座標）
        background: ベースラインデータ
    Returns:
        results: 各点のSHAP値のリスト
    """
    explainer = shap.DeepExplainer(model, background)
    results = []
    for p in manifold_points:
        sv = explainer.shap_values(p.unsqueeze(0))
        results.append(sv)
    return results

# SHAP値の可視化
# shap.summary_plot(shap_values, x_test, feature_names=feature_names)
# shap.force_plot(explainer.expected_value, shap_values[0], x_test[0])
```

使用するライブラリ： `shap`（pip install shap）。Lundberg & Lee（2017）が開発。`DeepExplainer`（ニューラルネット用）、`TreeExplainer`（ツリーモデル用）、`KernelExplainer`（任意のモデル用）の 3 種類のアルゴリズムを提供。

▶ 数学の基礎から理解したい方へ

SHAP（Lundberg & Lee, 2017）は、シャープレイ値をニューラルネットワークやツリーモデルに効率的に近似計算するアルゴリズム群の統称である。

特性関数 $v(S)$ は、特徴量の部分集合 S のみを使ったときの条件付き期待値 $v(S) = \mathbb{E}[f(x)|x_S]$ として定義される。ここで x_S は集合 S に属する特徴量の値を固定し、S に属さない特徴量は背景分布からサンプリングして期待値を取る。

空間的 SHAP と引き戻し計量の関係。 多様体上の位置 z における SHAP 値 $\phi_i(z)$ は、引き戻し計量 $g_{ij}(z)$ と関連する。引き戻し計量の固有値が大きい方向の特徴量は、一般に SHAP 値の絶対値も大きくなる傾向がある。直感的には、デコーダ

の出力がその方向に敏感（計量が大きい）であれば、その方向の特徴量の変動が予測に大きく影響する（SHAP 値が大きい）ことは自然である。この関係の定量的な分析は、旧『ジオメトリック・インテリジェンス』の第 2 章で導入した引き戻し計量の理論を基盤とする。

2.4　ケーススタディ——SHAP の実践

ケーススタディ A：X 社——「不安定の原因トップ 3」

X 社のチームは、経営環境多様体上でスカラー曲率が負（不安定）の領域について、SHAP で要因分解を行いました。

発見： 不安定の原因上位 3 つは、(1) 為替リスク：42%、(2) 原材料価格変動：31%、(3) 市場集中度の変化：18%。残りの 27 要因の合計は 9%。

経営者への示唆： 不安定の 73%は為替と原材料の 2 要因で説明される。新興国進出に際しては、この 2 つのリスクへの対処が優先課題。

ケーススタディ B：国際情勢——「安定化要因の特定」

安全保障チームの分析では、東アジア地域の安定化に最も貢献している要因は (1) 経済的相互依存度：38%、(2) 同盟ネットワークの密度：29%。

政策立案者への示唆： 経済的相互依存度が低下するシナリオ（例：大規模なデカップリング）では、安定化要因の 38%が失われ、地域全体の不安定性が急増する。

本章の品質チェックポイント

□ **チェックリスト（第 2 章終了時点）**

1. **SHAP 値の合計がモデル出力と一致するか？** → 効率性公理（定理 2.1(i)）の確認。一致しない場合、SHAP アルゴリズムの実装にバグがある
2. **空間的 SHAP のパターンがドメイン知識と整合するか？** → 安定成長期にマクロ指標が支配的、危機時に流動性指標が急増する等のパターンが確認できるか。不整合であればデータの前処理を見直す
3. **SHAP と因果推論の区別が明記されているか？** → SHAP は相関ベースの説明。「SHAP で貢献度が高い＝その特徴量を変えれば結果が変わる」とは限らない。因果効果の推定は第 3 章の DML で行う

次章予告： 第 3 章では、SHAP の限界——相関と因果の区別——に正面から向き合います。「広告費の SHAP 値が最大なのに、広告費を増やしても売上が上がらない」という衝撃的な発見が待っています。因果推論（DML）とは何か、そもそも「因果」と「相関」はどう違うのかから、丁寧に説明します。

第 3 章　因果推論（DML）による施策評価

この章で得られるもの——コストに見合うリターン

第 1 章で「どれくらい確か」（MC ドロップアウト）、第 2 章で「なぜその予測か」（SHAP）が分かるようになりました。しかし、ここに落とし穴があります。SHAP が示す「この特徴量の貢献度が高い」は、**その特徴量を変えれば結果が変わる**ことを意味しません。

本章では、序章で述べた旧作の「弱点 1：相関と因果の未分離」を正面から解決します。DML（Double Machine Learning, Chernozhukov et al., 2018）を使って、経営施策の**因果的効果**——「この施策を打ったら、結果が**どれだけ**変わるか」——を推定します。

まず「因果推論とは何か」「相関と因果はどう違うのか」からゼロから丁寧に説明します。機械学習エンジニアでも因果推論を学んだことがない方は多く、経営者にとってはなおさら馴染みの薄い概念です。

3.1　そもそも「因果」と「相関」はどう違うのか

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「傘を増やしても雨は減らない」**

リルル先生：「ユキ、質問。雨の日に傘を持っている人が多いよね。『傘を持っている人の数』と『雨の量』には強い関係がある。では、街中の傘の数を増やしたら、雨は減る？」

ユキ：「えっ……減らないです。傘を増やしても雨は降りますよね」

リルル先生：「そう。傘と雨は強く**相関**している——一方が増えると他方も増える——けど、傘が雨の**原因**ではない。傘と雨の両方を動かしているのは『天気』という第 3 の変数。天気が悪ければ雨が降り、雨が降れば人々が傘を持つ。天気 → 雨 → 傘、という因果の方向がある。傘 → 雨、という方向はない」

ユキ：「なるほど。『一緒に動く』と『原因と結果』は違うんですね」

リルル先生：「これが**相関と因果の区別**。相関は『2 つの変数が一緒に動く』こと。因果は『一方を変えると他方が変わる』こと。相関があっても因果があるとは限らない」

リン：「ちょっと待ってください。もう 1 つ重要な例を出させてください。アイスクリームの売上と水難事故の件数。この 2 つには強い正の相関があります。アイスクリームが売れる時期に水難事故が増える。では、アイスクリームの販売を禁止すれば水難事故は減るでしょうか？」

ユキ：「.....減らないですよね。暑い夏だからアイスクリームが売れるし、暑い夏だから海や川に行って事故が起きる」

リン：「その通り。『気温』（暑い夏）が両方の原因です。これを**交絡因子（confounding variable）**と呼びます。アイスクリームの売上と水難事故の間に直接の因果関係はないのに、交絡因子（気温）のせいで相関が生じている。これが**疑似相関（spurious correlation）**です」

ここで重要な用語を整理します。

相関（correlation）：2 つの変数が「一緒に動く」統計的関係。一方が増えると他方も増える（正の相関）、一方が増えると他方が減る（負の相関）。**方向性（どちらが原因か）は分からない。**

因果（causation）：一方の変数を**変えたら**、他方の変数が**変わる**関係。方向性がある。「X を変える → Y が変わる」ならば「X は Y の原因」。

交絡因子（confounding variable）：2 つの変数の両方に影響を与える第 3 の変数。交絡因子が存在すると、因果関係がないのに相関が生じる（疑似相関）。

なぜ経営判断で因果が重要か： 経営施策は「何かを**変える**」行為です。広告費を**増やす**、新製品を**投入する**、価格を**下げる**。施策の効果を予測するには「変えたらどうなるか」——すなわち因果効果——を知る必要があります。SHAP が教えてくれる「モデルの中での貢献度」（相関ベース）では不十分です。

3.2　衝撃——SHAP と DML の結果が逆転する

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「広告費が重要なのに効果がない？」**

カイト：「X 社のデータで SHAP を計算したとき、広告費の SHAP 値が全特徴量の中で最大でした。つまり、モデルは広告費に最も依存して予測を出している」

リルル先生：「そう。SHAP の結果は正しい。モデルは確かに広告費に強く依存している。では、DML で因果効果を推定してみよう」

カイト：「推定しました。……結果が変です。DML によると、広告費の因果効果は 3 つの施策の中で**最小**でした。SHAP では 1 位だったのに、DML では 3 位。**順位が逆転**している」

ユキ：「えっ、SHAP で一番重要って出たのに、効果がないんですか？どういうことですか？」

リルル先生：「これが相関と因果の違いの典型例。広告費が多い時期は**繁忙期**（年末商戦、ボーナスシーズン等）。繁忙期だから売上が高い。繁忙期だから広告予算も多い。広告費と売上は強く相関しているけど、**広告費が売上を動かしているのではなく、繁忙期（季節性）が両方を動かしている**」

本当の因果構造：
　季節性（繁忙期）→ 広告費↑
　季節性（繁忙期）→ 売上↑
　広告費 → 売上？　← この矢印は実は弱い

SHAP が見ているもの：
　広告費と売上の相関 = 強い　→ SHAP 値が大きい

DML が推定するもの：
　「季節性を固定したうえで」広告費を変えたら売上はどう変わるか = 弱い

カイト：「そうか……SHAP は『モデルが広告費に依存している度合い』を測っている。それは正しい。でも DML は『広告費を**変えたら**結果が**どう変わるか**』を測っている。別の問いに答えているんですね」

リルル先生：「その通り。SHAP は間違っていない。DML も間違っていない。**2 つは別の問いに答えている**。SHAP は『モデルの中で何が重要か』（相関ベースの説明）。DML は『施策を打ったらどうなるか』（因果効果の推定）。経営施策の評価には DML を使うべき」

リーム：「少しよろしいですか。私のクライアント企業にこの分析を見せたとき、最初は混乱されました。『広告費が重要なのに、効果がないってどういうこと？』と。私はこう説明しました――『広告費と売上は一緒に動く（相関）けれど、広告費が売上を動かしているのではなく、繁忙期が両方を動かしている（交絡）のです。3.2 節の因果構造の図を見てください。矢印の方向が大事です』と」

3.3　DML（Double Machine Learning）とは何か

▶ ビジネス・政策の意思決定に活用したい方へ

DML（Double Machine Learning, Chernozhukov et al., 2018）は、**交絡因子の影響を取り除いた上で、施策の純粋な因果効果を推定する**手法です。

DML の基本的な考え方を、広告費と売上の例で説明します。

ステップ 1（「残差」を作る）：

まず、季節性やマクロ経済指標などの交絡因子 X から、売上 Y をどの程度予測できるかを機械学習で推定します。予測値と実際の値の**差**（残差）が、「交絡因子では説明できない売上の変動」です。

同様に、交絡因子 X から広告費 D（施策の強度）をどの程度予測できるかも推定します。残差は「交絡因子では説明できない広告費の変動」です。

ステップ 2（「残差同士の関係」を見る）：

売上の残差と広告費の残差の関係を分析します。交絡因子の影響は両方の残差からすでに取り除かれているので、残差同士の関係は**純粋な因果効果**を反映します。

> 🔬 **研究室トーク：「ノイズを消して本質を見る」**
>
> **ユキ：**「うーん、残差って……？」
>
> **リルル先生：**「たとえ話をしよう。教室で生徒の身長とテストの成績に相関があったとする。でもこれは、年齢が上の生徒（高身長かつ高学年で既習範囲が広い）が身長も成績も高いだけ。年齢の影響を取り除くには、同じ年齢の生徒同士で比べればいい」
>
> **ユキ：**「おおー。同じ年齢で比べれば、身長と成績に関係がなくなるかもしれない」
>
> **リルル先生：**「DML も同じ発想。季節性を固定して（残差を取って）比べる。季節性の影響を取り除いた後の広告費の変動が、季節性の影響を取り除いた後の売上の変動をどれだけ説明するか――これが因果効果」
>
> **リン：**「ちょっと待ってください。DML のステップ 1 で使う機械学習モデルは、交絡因子から売上（または広告費）を予測するためのものです。ここでランダムフォレストやニューラルネットなどの**非線形モデル**を使えるのが DML の強みです。従来の操作変数法や回帰不連続デザイン

では、交絡因子の影響を**線形**で除去していましたが、DML は非線形な交絡も除去できる」

カイト：「それが『Double』の由来ですよね。ステップ 1 で 2 つの機械学習モデル（売上の予測と広告費の予測）を使うから『ダブル』」

▶ AI モデルの基礎から理解したい方へ

DML の実装は Microsoft Research が開発した `econml` ライブラリで簡単に行えます。

```
from econml.dml import DML
from sklearn.ensemble import RandomForestRegressor

# DML による因果効果推定
# Y: 結果変数（売上）
# T: 処置変数（広告費）← この因果効果を知りたい
# X: 交絡因子（季節性、マクロ経済指標、競合動向...）

dml = DML(
    model_y=RandomForestRegressor(n_estimators=200),  # Y 予測用モデル
    model_t=RandomForestRegressor(n_estimators=200),  # T 予測用モデル
    cv=5  # クロスフィッティングの分割数（過適合防止）
)

# 因果効果の推定
dml.fit(Y, T, X=X)

# CATE（条件付き平均処置効果）：各データ点での因果効果
cate = dml.effect(X_test)

# 信頼区間（95%）
ci = dml.effect_interval(X_test, alpha=0.05)

print(f"平均因果効果: {cate.mean():.3f}")
print(f"95%信頼区間: [{ci[0].mean():.3f}, {ci[1].mean():.3f}]")
```

使用するライブラリ： `econml`（pip install econml）。Microsoft Research が開発した因果推論のための Python ライブラリ。DML のほか、Doubly Robust Learning、Causal Forest 等も実装されている。

クロスフィッティングとは： ステップ 1 の機械学習モデルがステップ 2 に使うデータに過適合することを防ぐため、データを K 分割し、分割ごとにモデルの訓練

と残差の計算を交代で行う手法。DML の数学的な正当性（$\sqrt{n}$-一致性）はクロスフィッティングに依存する。

▶ 数学の基礎から理解したい方へ

定式化。 処置変数 D（施策の強度）、結果変数 Y（売上等）、交絡因子 X に対する部分線形モデル：

$$Y = \theta D + g(X) + \varepsilon, \quad D = m(X) + V$$

ここで θ が因果効果（推定対象）、$g(X) = \mathbb{E}[Y|X]$ は交絡因子の売上への直接影響、$m(X) = \mathbb{E}[D|X]$ は交絡因子の広告費への直接影響。ε と V は誤差項。

DML の推定量：

$$\hat{\theta}_{\text{DML}} = \frac{\frac{1}{n}\sum_{i=1}^{n} \hat{V}_i \, \hat{\varepsilon}_i}{\frac{1}{n}\sum_{i=1}^{n} \hat{V}_i^2}$$

ここで $\hat{\varepsilon}_i = Y_i - \hat{g}(X_i)$（売上の残差）、$\hat{V}_i = D_i - \hat{m}(X_i)$（広告費の残差）。

性質： $\hat{g}$ と $\hat{m}$ が一定の収束率条件を満たせば、$\hat{\theta}_{\text{DML}}$ は $\sqrt{n}$-一致かつ漸近正規。交絡因子の推定に非線形モデル（ランダムフォレスト、ニューラルネット等）を使える点が、従来の線形操作変数法に対する主要な利点。

CATE（Conditional Average Treatment Effect）と多様体の関係。 経営環境多様体の上で DML を行う際、因果効果は多様体上の位置 z に依存する：$\theta(z) = \mathbb{E}[Y(1) - Y(0)|Z = z]$。多様体の曲率（序章で説明した「空間の曲がり具合」）が大きい領域では、線形近似が破綻するため、$g(X)$ と $m(X)$ の推定に非線形モデルが特に重要になる。

3.4 SHAP と DML の使い分け

🔬 **研究室トーク：「2 つは別の道具」**

リルル先生：「ここで、SHAP と DML の使い分けを整理しよう」

	SHAP（第 2 章）	DML（第 3 章）
答える問い	「モデルの中で何が重要か？」	「施策を変えたら結果はどう変わるか？」
基盤	相関（ゲーム理論のシャープレイ値）	因果（反事実フレームワーク）

	SHAP（第 2 章）	DML（第 3 章）
用途	モデルの解釈、要因分解	施策の効果予測、意思決定
交絡因子	区別しない	明示的に除去する
結果が異なるとき	—	**施策評価には DML を使う**

リルル先生：「SHAP で『広告費が重要』と分かったら、次に DML で『広告費を増やすと売上がどれだけ変わるか』を推定する。2 つの結果が一致すれば信頼度が高い。異なる場合は、交絡因子の存在を疑い、DML の結果を施策評価に使う」

リーム：「私のクライアント企業では、施策ごとに SHAP と DML の両方を計算し、結果が一致する施策を優先的に推薦しています。『SHAP でも DML でも上位に来る施策』は、相関でも因果でも重要ということなので、信頼度が高い」

リン：「ただし、DML にも仮定があります。最も重要な仮定は**未観測の交絡因子がないこと**（無交絡仮定, unconfoundedness）。観測している変数 X に全ての交絡因子が含まれていなければ、DML の因果効果推定も偏ります。この仮定はデータから検証できないため、ドメイン知識に基づく判断が必要です」

リルル先生：「リンの指摘は重要。だから 3 本の柱——MC ドロップアウト、SHAP、DML——を**組み合わせて**使う。1 本の柱だけを信じるのではなく、3 本の柱の結果を照らし合わせて判断する。これが原則 2『足すな、信頼性を上げろ』の実践」

3.5　ケーススタディ——施策の順位逆転

ケーススタディ A：X 社——3 つの施策の因果効果比較

X 社は 3 つの施策を検討中です。SHAP と DML の結果を比較します。

施策	SHAP での重要度順位	DML での因果効果順位	乖離の理由
広告費の増額	**1 位**	3 位	季節性が交絡因子（繁忙期に広告費も売上も増える）
R&D 投資の拡大	3 位	**1 位**	R&D 投資は季節に依存しない。DML で初めて真の効果が判明

施策	SHAP での重要度順位	DML での因果効果順位	乖離の理由
価格の改定	2 位	2 位	SHAP と DML が一致。価格は売上の原因でもある

経営者への示唆： SHAP だけを見ていたら「広告費を増やすべき」と判断していた。DML の結果を加味すると「R&D 投資が最も効果的」と分かる。施策の優先順位が**逆転**する。

ケーススタディ B：国際情勢――政策の因果効果

安全保障チームが、同盟強化政策と経済制裁政策の因果効果を DML で推定。

発見： 同盟強化政策の CATE は多様体上の位置に強く依存。既に同盟関係が強い地域では CATE が小さく（限界効果逓減）、同盟関係が弱い地域では CATE が大きい。

政策立案者への示唆： 同盟強化政策のリソースは、既存の同盟が弱い地域に優先配分すべき。

本章の品質チェックポイント

□ **チェックリスト（第 3 章終了時点）**

1. **DML のクロスフィッティングが適切に機能しているか？** → 残差 $\hat{\varepsilon}$ と $\hat{V}$ の相関がゼロに近いことを確認。ゼロでなければモデルの性能不足
2. **因果効果の信頼区間はゼロを含むか？** → 含む場合、統計的に有意な因果効果が検出されていない。サンプルサイズの増加またはモデルの改善が必要
3. **SHAP と DML の結果の乖離が説明可能か？** → 乖離があれば交絡因子を特定する。特定できなければ、未観測の交絡因子の存在を疑う（DML の仮定が満たされていない可能性）
4. **無交絡仮定の妥当性をドメイン専門家と確認したか？** → DML の結果はこの仮定に依存。仮定が怪しい場合は感度分析を行う

次章予告： 第 4 章では、ゼロ知識証明（ZKML）を使って「計算が正しく行われたこと」を暗号学的に保証する、信頼性の第 4 の柱を構築します。ゼロ知識証明とは何か、なぜ AI に必要なのか、そもそも「検証する」とはどういうことなのかから、丁寧に説明します。

第 4 章　ZKML による検証可能性

この章で得られるもの——コストに見合うリターン

第 1 章（MC ドロップアウト）で「どれくらい確か」、第 2 章（SHAP）で「なぜその予測か」、第 3 章（DML）で「施策を打ったらどうなるか」が分かるようになりました。しかし、これらの計算が**本当に正しく行われたかどうか**を、計算を依頼した側（クライアント企業）はどうやって確認すればよいでしょうか？

本章では、ゼロ知識証明（Zero-Knowledge Proof）という暗号技術を AI（機械学習）に適用した **ZKML（Zero-Knowledge Machine Learning）**を学びます。ZKML により、パイプラインの計算が正しいことを、**データを公開せずに**第三者が検証できるようになります。

まず「そもそも検証するとは何か」「なぜデータを見せずに検証できるのか」からゼロから丁寧に説明します。

4.1 「検証する」とは何か——信頼の問題

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「信じてください、と言われても」**

リーム：「少しよろしいですか。これは私のビジネスにとって切実な問題です。私のベンチャーは 7 社のクライアント企業に、経営環境多様体パイプラインの分析結果を提供しています。レポートを納品するとき、クライアントから必ず聞かれるのは——」

- 「本当に MC ドロップアウトを 100 回回したんですか？ 途中で手を抜いて 10 回で済ませていませんか？」
- 「データを改ざんしていませんか？ うちに有利な数字を作っていませんか？」
- 「計算にバグはないですか？」

リーム：「これらの質問に対して、私が『信じてください』と言っても、クライアントは納得しません。特に金融機関や官公庁のクライアントは、計算過程の検証を強く求めます」

ユキ：「えっ、検証ってどうするんですか？ クライアントがリームさんのサーバに入って、全部の計算を最初からやり直す？」

リーム：「それは現実的ではありません。計算に何日もかかりますし、データの機密性の問題もあります。クライアントにデータそのものを見

せるわけにはいかない場合もあります——他のクライアントのデータと混在しているかもしれませんから」

リルル先生：「ここでゼロ知識証明の出番。**計算が正しく行われたことを証明する『証明書』を渡す**。クライアントはその証明書を検証するだけで、計算の正しさを確認できる。データそのものを見る必要はない」

「検証する」とは、ここでは以下の 3 つを確認することを意味します。

1. **計算の正しさ：** 約束したとおりの AI モデル（MC ドロップアウト 100 回、SHAP、DML 等）が、約束したとおりに実行されたこと
2. **データの完全性：** 途中でデータが差し替えられたり、改ざんされたりしていないこと
3. **結果の一貫性：** レポートに記載された数値が、実際の計算結果と一致すること

従来、これらの確認は**監査**（第三者がサーバに入って計算をチェックする）で行われていましたが、時間・コスト・データ機密性の問題がありました。ゼロ知識証明は、これらの問題を暗号学的に解決します。

4.2　ゼロ知識証明とは何か——データを見せずに正しさを証明する

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「洞窟のたとえ話」**

リルル先生：「ゼロ知識証明の仕組みを、有名な『洞窟のたとえ話』で説明しよう」

リルル先生：「環状の洞窟があると想像して。入り口から入ると、途中で左右に分かれ、奥で合流する。合流地点には鍵のかかった扉がある。私がこの扉の鍵を持っていることを、**鍵そのものを見せずに**証明したい」

ユキ：「どうやって？」

リルル先生：「まず私が洞窟に入って、左右どちらかの道を選んで進む。あなた（検証者）は入り口で待っていて、私がどちらの道に入ったかは見えない。次にあなたが『左の道から出てきて』または『右の道から出てきて』とランダムに指定する」

リルル先生：「もし私が鍵を持っていれば、**どちらを指定されても**奥の扉を通って指定された方の道から出られる。鍵を持っていなければ、50%の確率でしか指定に従えない」

ユキ：「1 回だけなら運で当たるかもしれないけど、20 回連続で成功したら……」

リルル先生：「20 回連続で偶然成功する確率は $(1/2)^{20} = 0.000001$、つまり 100 万分の 1。これで実質的に『鍵を持っている』ことが証明される。しかし、あなたは鍵の形状を知らない——ゼロの知識で、命題（鍵を持っている）の真偽だけが伝わる」

カイト：「これがゼロ知識証明の 3 つの性質ですね」

- **完全性（Completeness）**：本当に鍵を持っている人は、毎回必ず指定通りに出てこられる
- **健全性（Soundness）**：鍵を持っていない人は、何度も連続で成功することができない
- **零知識性（Zero-Knowledge）**：検証者は「鍵を持っているかどうか」以外の情報（鍵の形状、どちらの道を選んだか）を得られない

リン：「ちょっと待ってください。洞窟のたとえは分かりやすいですが、実際のゼロ知識証明はもっと複雑です。AI の計算をゼロ知識証明に変換するには、計算全体を数学的な『算術回路』に変換する必要があります」

ゼロ知識証明の正式な定義

ゼロ知識証明は、1985 年にゴールドワッサー（Goldwasser）、ミカリ（Micali）、ラコフ（Rackoff）が発表した暗号プロトコルです。このプロトコルには、**証明者**（Prover：命題が真であることを知っている人）と**検証者**（Verifier：命題の真偽を確認したい人）の 2 者が登場します。

3 つの性質：

(i) 完全性（Completeness）：命題が本当に真であれば、正直な証明者は検証者を必ず納得させられる。——リームの計算が本当に正しければ、クライアントは必ず「正しい」と確認できる。

(ii) 健全性（Soundness）：命題が偽であれば、不正な証明者でも検証者を騙すことができない。——リームが計算をごまかしていれば、クライアントは必ず「おかしい」と検出できる。

(iii) 零知識性（Zero-Knowledge）：検証者は、命題の真偽以外の情報を一切得られない。——クライアントは「計算が正しい」ことは分かるが、クライアント企業のデータやモデルの内部パラメータは分からない。

4.3　ゼロ知識証明を AI に適用する——ZKML の仕組み

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「AI の計算を『証明可能な形』に変換する」**

カイト：「ゼロ知識証明の仕組みは分かりました。でも、AI の計算ってものすごく複雑ですよね。ニューラルネットワークの何百万ものパラメータの掛け算と足し算を、全部ゼロ知識証明に変換できるんですか？」

リルル先生：「いい質問。ここが ZKML の技術的チャレンジ。ステップを説明しよう」

ステップ 1：算術回路への変換

リルル先生：「ニューラルネットワークの推論計算は、結局のところ『行列の掛け算』と『活性化関数の適用』の繰り返し。これらの演算を、**加算ゲート**と**乗算ゲート**だけで構成された『算術回路』に変換する。算術回路は、入力から出力までの全ての計算ステップを、一つ一つのゲートとして記録した『計算の設計図』のようなもの」

ユキ：「えっと、計算の全部の手順を一つ一つ書き出すってことですか？」

リルル先生：「そう。たとえば $y = 3x + 2$ という計算は、『x に 3 を掛ける（乗算ゲート）』→『結果に 2 を足す（加算ゲート）』という 2 つのゲートの回路になる。ニューラルネットワークはこれが何百万個もあるだけ」

ステップ 2：証明の生成

リルル先生：「算術回路に対して、**zkSNARKs**（Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge）というプロトコルで『証明』を生成する。証明者（リームのサーバ）は、入力データ、モデルの重み、出力を使って、小さな『証明書』（数百バイト）を生成する」

リーム：「証明のサイズは入力データのサイズに関係なく一定（数百バイト）です。100GB のデータを使った計算でも、証明書は数百バイト。これが zkSNARKs の『Succinct（簡潔）』の意味です」

ステップ 3：検証

リルル先生：「検証者（クライアント企業）は、**出力**と**証明書**を受け取る。検証アルゴリズムを実行すると、一瞬（$O(1)$、定数時間）で『この出力は正しい計算から得られたものか』を判定できる。入力データやモデルの中身は見る必要がない」

ユキ：「おおー。証明書だけで分かるんですか？ データもモデルの中身も見ないのに、なぜ正しいって分かるんですか？ そこが全然分からないです」

リン：「ちょっと待ってください。これは核心的な疑問です。ちゃんと説明しましょう」

リルル先生：「では、段階的に説明しよう。数学的に完全に説明すると楕円曲線暗号とペアリング写像の話になって大学院レベルになるけど、核心のアイデアは 3 つのステップで直感的に理解できる」

＜核心ステップ 1：計算を「多項式」に変換する＞

リルル先生：「まず、ニューラルネットワークの全ての計算ステップを、1 つの巨大な**多項式**（x の何乗かを足したり掛けたりしたもの、たとえば $P(x) = 3x^3 + 2x^2 - 5x + 7$）に変換する」

ユキ：「多項式って、中学校で習った $y = ax^2 + bx + c$ みたいなやつですか？」

リルル先生：「そう、あれの超巨大版。ニューラルネットワークの計算が正しく行われたなら、この多項式はある特定の形をしている。計算にミスがあったり改ざんがあったりすると、多項式の形が変わる。つまり**『計算が正しい』ということと『多項式が特定の形をしている』ということが同値**になる」

カイト：「具体的には、算術回路の全てのゲート（足し算ゲート、掛け算ゲート）の入出力の関係を、1 つの多項式の制約として表現するんです。たとえば、『ゲート 5 の出力はゲート 3 の出力×ゲート 4 の出力に等しい』という制約が、多項式の係数に反映される」

カイト：「……すいません、ここでどうしても分からないことがあります。リームのクライアントの企業（や政府）って、リームが、無数に存在する深層ニューラルネットワークモデルのうち、どのニューラルネットワーク構造をした AI モデルを使ったのかを、リームから知らされているんでしょうか？ つまり……クライアント側は、リームが計算を途中で省略したり、パラメータがサイバー攻撃や社内不正によって改ざんされたり、クライアントが喜ぶ戦略提案ができるように数値を後から操作した

りしていない場合に、リームが正しく計算したニューラルネットワークモデルを数学的に表現し直した『巨大な多項式』がどの多項式になるのかを、なぜ知ってるんですか？ リームは、深層ニューラルネットワークモデルの係数（パラメータ）は企業秘密だからクライアントには開示しないけど、その深層ニューラルネットワークモデルを多項式に変換したとき、どの多項式になるのか、その多項式だけはクライアントに見せているんですか？」

リルル先生：「カイト、非常にいい質問だ。これはゼロ知識証明の仕組みの中で最も誤解されやすいポイント。答えは**——クライアントは多項式そのものも見ない**。クライアントが受け取るのは**検証鍵（verification key）**だけ」

リルル先生：「仕組みはこうなっている。ZKML のセットアップの段階で、リームが使うニューラルネットワークの**構造**（何層で、各層のノード数がいくつで、活性化関数が何か）を算術回路に変換する。この算術回路から**検証鍵**が生成される。検証鍵はクライアントに渡す。ここが重要**——検証鍵には、ニューラルネットワークの構造（アーキテクチャ）の情報は含まれているが、パラメータ（学習済みの重み）の値は含まれていない**」

カイト：「えっ、構造は教えるけどパラメータは教えない？」

リルル先生：「そう。たとえば『3 層のネットワークで、隠れ層が 128 ノードと 64 ノードで、活性化関数が Tanh』という**構造**はクライアントに開示する。これは契約時に『このタイプのモデルを使います』と合意しているから、企業秘密ではない。でも、学習済みの**重みパラメータ**——たとえば第 1 層の 128×30＝3,840 個の数値——は企業秘密として開示しない」

リルル先生：「検証鍵は、この構造から生成されるものであり、『この構造のネットワークで正しく計算が行われたなら、多項式はこの**形のクラス**に属するはずだ』という情報を暗号学的にエンコードしたもの。クライアントは検証鍵を使って、リームの証明書を検証する。検証が通れば、**合意した構造のネットワークで、何らかの正当なパラメータを使って、改ざんなしに計算が行われた**ことが保証される」

リーム：「少しよろしいですか。実際の契約では、こうなっています。(1) 契約時に、使用する AI モデルの**アーキテクチャ（構造）**をクライアントに開示し合意する。(2) セットアップ時に、そのアーキテクチャから生成された**検証鍵**をクライアントに渡す。(3) 毎月のレポート納品時に、レポートの数値と一緒に**証明書**を渡す。(4) クライアントは検証鍵と証明

書を使って、**合意した構造のモデルで、改ざんなしに計算が行われたこと**を自分の手で検証する」

カイト：「あ。やっとわかりました。これで、リームが見せたくない企業秘密――学習済みパラメータの数値――をクライアントに見せないで、クライアント側は、リームから事前に示されていた通りの AI モデルの構造で、パラメータの改ざん（サイバー攻撃）や事後的な数値の操作（クライアントを喜ばすための数字変更や、ライバル企業を有利にしてクライアント企業を不利にするための数字変更など）がされていないことを、クライアント自身の手で検証できるんですね！」

リルル先生：「……いいね。完璧に理解した。そしてこの仕組みは、第 21 章で議論する利益相反の問題にも直結する。リームが 7 社のクライアントと同時に契約していても、各クライアントは自分のレポートの計算の正しさを独立に検証できる。他のクライアントのデータやパラメータは一切見えない」

カイト：「……整理すると、リームは、どの AI モデルを使ったのか――モデルの種類（DeepMind の MuZero モデルを使いますとか、畳み込みニューラルネットワークモデルである ResNet-50 モデルを使いますとか）と、モデルを実装するときの層の数や各層のノードの数など、AI モデルのアーキテクチャ（構造）は、クライアントと契約するときに合意する。そのあと、毎月、リームが経営環境を多様体デジタルツインで分析して、最適な経営アクションを提案するレポートをそのクライアントに納品するときに、リームが今月も契約したときに約束した通りの AI モデルをちゃんと使っていることをクライアントが検証できる材料として、証明書を経営戦略レポートと一緒に渡す。ここまでは理解できました」

カイト：「で、ちょっとわからなくなったのが……リームは毎月、モデルのアーキテクチャ（構造）を証明できる証明書は渡すけど、そのモデルの中のパラメータは隠したままにするんですよね？」

カイト：「そうすると、ライバル企業を有利にするためにクライアント企業を特定の経営アクションに誘導するような内容の経営戦略レポートを作ろうとリームが画策したとき――リーム、ごめん！ これは仮定の条件設定だから（笑）――そのレポート結果につながるようなパラメータになるように、リームは AI モデルのパラメータを事後的に改ざんしたとき、クライアントはそれにどうやって気づくの？ これは、リームがクライアントを喜ばせるために、クライアント企業の社長が『こういう経営アクションにお墨付きを与えてくれるレポートが欲しい』という心理的ニーズにうまく整合するレポートを作るために、AI モデルのパラメータを事後的に改ざんするシナリオでも成立する問題」

カイト：「モデルのアーキテクチャはクライアントに暗号化した上で渡すけど、モデルのパラメータは暗号化するステップを経て渡す、ということすらしないから、クライアントはリームが行った『ずる』や『悪さ』にどうやって気づくことができるんだろう。……ここが、ちょっと、わからない……」

リルル先生：「カイト、これは本当に鋭い質問だ。ゼロ知識証明の限界に関わる、非常に重要なポイント。順番に答えよう」

リルル先生：「まず、ZKML の証明が**保証すること**と**保証しないこと**を明確に区別する必要がある」

リルル先生：「ZKML が保証すること：『合意したアーキテクチャのモデルに、**何らかの**パラメータを入れて、**何らかの**入力データを与えたとき、その計算が算術回路に沿って正しく実行され、出力された数値が改ざんされていないこと』。つまり、**計算過程に嘘がないこと**は保証される」

リルル先生：「ZKML が保証しないこと：『リームが使ったパラメータが、正当な訓練プロセスで得られたパラメータかどうか』。カイトが言う通り、リームがパラメータを事後的に操作して、特定の結論が出るように仕込んだ場合、ZKML の証明は**それでも通ってしまう**。なぜなら、ZKML は『このパラメータでこの入力を計算したらこの出力になる』という計算の正しさだけを証明しているから。パラメータが正当かどうかは、ZKML の守備範囲外」

ユキ：「えっ……それじゃ、ZKML だけでは不十分ってことですか？」

リルル先生：「その通り。だから **3 つの防御層**が必要になる」

防御層 1：ZKML（本章）
- → 「計算過程に嘘がない」ことを保証
- → 計算の途中でデータを差し替えたり、出力を後から書き換えたりは不可能

防御層 2：訓練ログのコミットメント（技術的保証の追加層）
- → モデルの訓練過程の全ログ（各エポックの損失値、使用データのハッシュ等）を暗号学的ハッシュチェーンとして記録し、タイムスタンプ付きで第三者に預託
- → 訓練後にパラメータを書き換えると、ログとの整合性が崩れる
- → 「この訓練ログから得られたパラメータで計算した」ことの ZKML 証明も技術的には可能（ただし計算コストが非常に大きい）

防御層 3：第三者監査（制度的保証）
→ 独立した第三者監査機関が、訓練プロセス・データ選択・パラメータ管理を定期監査

→ ZKML が守備範囲外とする「パラメータの正当性」と「データ選択の公正性」をカバー

リルル先生：「ZKML は『計算に嘘がない』ことの保証。訓練ログのコミットメントは『パラメータの出自が正当である』ことの保証。第三者監査は『データ選択とプロセス全体が公正である』ことの保証。**技術的保証と制度的保証の両方が必要——**これが第 21 章の利益相反の議論の核心につながる」

カイト：「なるほど……ZKML だけで全てが解決するわけじゃなくて、ZKML は 3 つの防御層のうちの 1 つなんですね。でも、1 つ目の防御層——計算過程に嘘がないこと——だけでも、今まではそれすら保証できなかったわけだから、大きな前進ですよね」

リルル先生：「その通り。そして原則 3——分かることと分からないことを分けろ。ZKML で保証**できること**と**できないこと**を正直に開示することが、かえってクライアントの信頼を高める」

リーム：「少しよろしいですか。実際の私のビジネスでは、3 つの防御層全てを導入しています。ZKML の証明書、訓練ログのハッシュチェーン、そして年 2 回の第三者監査。クライアントにはこの 3 層構造を説明した上で、各層が何を保証し、何を保証しないかを明示しています。……カイト、私が『ずる』をするシナリオで説明してくれてありがとう（笑）。とても分かりやすかった」

カイト：「リーム、本当にごめん……（笑）」

＜核心ステップ 2：多項式の「指紋」を渡す——暗号学的コミットメント＞

リルル先生：「次がカギ。この巨大な多項式の**全ての係数**（＝計算の全てのステップの情報）を相手に渡してしまったら、データもモデルも全部バレてしまう。でも**係数を渡さずに、多項式が正しい形をしていることだけを確認させる**方法がある」

ユキ：「えっ、そんなことできるんですか？」

リルル先生：「たとえ話をしよう。1000 ページの辞書があるとする。友人に『この辞書は正しい辞書だよ』と証明したい。でも辞書の中身を全部見せたくない」

リルル先生：「友人がランダムに『378 ページの 3 番目の単語は何？』と聞く。あなたが正しい辞書を持っていれば正しく答えられる。デタラメな辞書を持っていたら答えられない。友人はこの質問を 10 回ランダム

に繰り返して、全部正しければ『この人は正しい辞書を持っている可能性が極めて高い』と判断できる」

ユキ：「おおー！ 全ページを見せなくても、ランダムなスポットチェックで確認できるんですね」

リルル先生：「そう。多項式でも同じ。**ランダムに選んだ点 r で多項式の値 $P(r)$ を確認する**。正しい多項式を持っている人なら正しい値を答えられるが、偽の多項式を持っている人は正しい値を答えられない。ここで数学の重要な性質を使う**――シュワルツ・ツィッペルの補題**」

リン：「2 つの異なる多項式が同じ点で同じ値を取る確率は、多項式の次数 d を体の位数 p で割った d/p 以下です。p が 256 ビットの巨大な素数なら、d/p は宇宙の原子の数の逆数よりも小さい。つまり、**偽の多項式がランダムに選んだ点でたまたま正しい値を返す確率は、実質的にゼロ**」

ユキ：「宇宙の原子の数より小さい確率……つまり絶対にバレるってことですね」

リルル先生：「そう。でも、まだ問題がある。$P(r)$ の値を直接教えたら、多項式の情報が少し漏れてしまう。そこで**暗号学的コミットメント**を使う」

＜核心ステップ 3：暗号の「金庫」で値を包む＞

リルル先生：「暗号学的コミットメントは、値を暗号の『金庫』に入れて相手に渡すようなもの。金庫を開けなくても、**2 つの金庫の中身が正しい関係にあるかどうか**を、金庫の外側から確認できる。これが楕円曲線暗号の**ペアリング写像**という特殊な数学的性質を使って実現される」

カイト：「もう少し具体的に言うと、楕円曲線の上の点 g を基点として、多項式 $P(x)$ の値 $P(r)$ を $g^{P(r)}$ という形で暗号化する（実際には楕円曲線のスカラー倍）。この暗号化された値から $P(r)$ そのものを取り出すことは計算量的に不可能（離散対数問題の困難さ）だけど、**ペアリング写像 e** を使うと、$e(g^a, g^b) = e(g, g)^{ab}$ という関係が成り立つので、暗号化されたままで値の積（ab）を検証できる」

リン：「つまり、検証者は暗号化された値の**関係性**だけを確認する。値そのもの（＝計算の中身、データ、モデル）は知り得ない。これが零知識性の数学的な実現方法です」

ユキ：「……すごい。金庫を開けずに、中身の関係だけ確認できるんですね」

＜まとめ：なぜデータを見せずに正しさを検証できるのか＞

リルル先生：「全体をまとめよう」

1．計算全体を 1 つの巨大な多項式に変換する
→ 「計算が正しい」＝「多項式が特定の形をしている」

2．ランダムな点での多項式の値を確認する
→ 偽の多項式がバレずに済む確率は宇宙の原子の数の逆数以下
（シュワルツ・ツィッペルの補題）

3．値を暗号学的コミットメント（楕円曲線の「金庫」）で包む
→ 値そのものは見えないが、値の関係性だけは確認できる
（ペアリング写像）

結果：
クライアントは「計算が正しい」ことを確認できる ✓
クライアントはデータもモデルも見ていない ✓
クライアントは計算の途中経過も知らない ✓

ユキ：「なるほど！ だからデータやニューラルネットワークの詳細を見せないでも、クライアントは正しさを検証できるんですね！」

リルル先生：「……いいね。ユキ、完璧に理解した」

リーム：「少しよろしいですか。この仕組みのおかげで、私のベンチャーではクライアント A のデータをクライアント B に見せるリスクなしに、両方のクライアントに対して計算の正しさを保証できます。これは第 21 章の利益相反の問題にも関わる重要な性質です」

▶ AI モデルの基礎から理解したい方へ

ZKML の具体的なワークフロー：

```
# ZKML の概念的ワークフロー
# 実際の実装には ezkl, RISC Zero 等のフレームワークを使用

# === 証明者（リームのサーバ）側 ===

# 1. PyTorch モデルを ONNX 形式にエクスポート
import torch
model = trained_vae.decoder
dummy_input = torch.randn(1, 6)
torch.onnx.export(model, dummy_input, "decoder.onnx",
```

```
                    input_names=['z'], output_names=['x_hat'])

# 2. ONNX → 算術回路に変換（ezkl の場合）
# $ ezkl gen-circuit decoder.onnx circuit.json

# 3. セットアップ（証明鍵と検証鍵を生成）
# $ ezkl setup circuit.json --pk proving_key --vk verification_key

# 4. 証明の生成
# $ ezkl prove --circuit circuit.json --input data.json \
#              --pk proving_key --proof proof.json

# 5. 出力と証明書をクライアントに送信
# send(output, proof.json, verification_key)


# === 検証者（クライアント企業）側 ===

# 6. 証明の検証（一瞬で完了）
# $ ezkl verify --proof proof.json --vk verification_key
# → "Proof is valid" or "Proof is invalid"
```

主要な ZKML フレームワーク（2026 年 2 月時点）：

フレームワーク	開発元	特徴
ezkl	zkonduit	PyTorch/ONNX 対応。最も成熟した ZKML 実装
RISC Zero	RISC Zero	汎用 ZK-VM。任意のプログラムを ZK 証明可能
Giza	Giza Technologies	Cairo 言語ベース。StarkNet 上で動作

現状の制約（2026 年 2 月時点）： - 証明生成は元の推論の**数十〜数百倍**の計算コスト。パイプライン全体の ZK 証明は数千倍になりうる - ニューラルネットワークの**浮動小数点演算**を、zkSNARKs が動作する**有限体**（第 5 章で詳しく説明）上の**固定小数点演算**に変換する必要がある。この変換（量子化）で精度低下が生じる - 全てのニューラルネットワーク構造がサポートされているわけではない - 大規模モデル（数十億パラメータ）にはまだ実用的でない

▶ 数学の基礎から理解したい方へ

定義 4.1（zkSNARKs）. zkSNARK（Zero-Knowledge Succinct Non-Interactive Argument of Knowledge）は、以下の 3 つの確率的多項式時間アルゴリズムの組 (Gen,Prove,Verify) からなる。

$\mathrm{Gen}(1^{\lambda}, C) \to (\mathrm{pk},\mathrm{vk})$：算術回路 C に対してセットアップを行い、証明鍵 pk と検証鍵 vk を生成

$\mathrm{Prove}(\mathrm{pk}, x, w) \to \pi$：パブリック入力 x（出力値）とプライベート入力 w（入力データ＋モデル重み）から証明 π を生成

$\mathrm{Verify}(\mathrm{vk}, x, \pi) \to \{0,1\}$：検証鍵 vk、パブリック入力 x、証明 π から真偽を判定

簡潔性（Succinctness）：$|\pi| = O(\log|C|)$（証明サイズが回路サイズの対数）、Verify の実行時間は $O(|x|)$（パブリック入力サイズに線形、回路サイズに非依存）。

ZKML では、ニューラルネットワークの推論 $y = f_w(x)$ を算術回路 C に変換し、「$C(x,w) = y$ を満たす (x,w) が存在する」ことを zkSNARKs で証明する。x（入力データ）と w（モデル重み）はプライベート入力（witness）、y（出力）はパブリック入力。検証者は y の正しさを確認できるが、x と w は知り得ない。

代表的なプロトコル：Groth16（2016、ペアリングベース、証明サイズ 128 バイト、検証時間 3ms）、Plonk（Gabizon et al., 2019、ユニバーサルセットアップ）。

4.4　ZKML の現状と将来——AI とゼロ知識証明の関係

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「全マシマシにしない——ZKML の適用範囲」**

カイト：「先生、ZKML をパイプライン全体に適用したら、全ての計算が暗号学的に保証されて最強じゃないですか？」

リルル先生：「カイト、原則 2 を思い出して」

カイト：「……『足すな、信頼性を上げろ』。パイプライン全体に適用するのは全マシマシですね」

リルル先生：「……いいね。成長したね」

リーム：「少しよろしいですか。実際のコストを説明します。VAE のデコーダ推論だけを ZKML 化した場合、証明生成に推論の約 50 倍の計算時

間が必要でした。GPU を使って数分の推論が数時間になる。パイプライン全体だと数千倍——数日から数週間かかる計算になります」

リン：「ちょっと待ってください。コストだけでなく、技術的な制約もあります」

リン：「第一に、zkSNARKs は**有限体**（第 5 章で説明しますが、特定の素数で割った余りの数だけで構成される数の世界）の上で動作するため、ニューラルネットワークの浮動小数点演算を固定小数点に**量子化**する必要があります。この変換で精度が低下する」

リン：「第二に、2026 年 2 月時点で、全てのニューラルネット構造が ZKML に対応しているわけではありません。特に Transformer ベースの大規模モデル（数十億パラメータ）の ZK 証明は、まだ研究段階です」

リン：「第三に、活性化関数（Tanh, ReLU 等）の算術回路への変換にはルックアップテーブルが必要で、これが回路サイズを大幅に増やします」

リルル先生：「だから原則 2 に従って、**クリティカルな 3 つの部分だけ**に ZKML を適用する」

ZKML を適用する 3 つのクリティカルポイント：

クリティカルポイント 1：VAE のデコーダ推論
- → 多様体の構成が正しいことの保証
- → 命題 1.1 の条件が量子化後も維持されていることの確認

クリティカルポイント 2：MC ドロップアウトの分散計算
- → 不確実性推定が正しいことの保証
- → T=100 回の推論が本当に実行されたことの証明

クリティカルポイント 3：DML の最終推定
- → 因果効果推定が正しいことの保証
- → クロスフィッティングが適切に実行されたことの証明

リーム：「この 3 つに絞れば、証明生成のコストは数時間程度。月次の経営判断を支える分析に対して、数時間の追加計算は許容範囲です」

リルル先生：「全マシマシにしない。クリティカルな部分だけ。原則 2 の実践だ」

ZKML の実用例——リームのビジネスモデル

リームのベンチャーが X 社に提供するサービスのフロー：

```
[X 社のデータ]
    ↓
リームのサーバで計算
  ├── VAE 推論（ZKML 証明を同時生成）
  ├── MC ドロップアウト 100 回（ZKML 証明を同時生成）
  ├── SHAP 計算（ZKML なし──コスト対リターンが合わない）
  └── DML 因果効果推定（最終段階の ZKML 証明を生成）
    ↓
[レポート] ＋ [ZKML 証明束（3 つの証明）]
    ↓
X 社に納品
```

X 社の検証：

```
$ ezkl verify --proof vae_proof.json --vk vae_vk
→ "Proof is valid ✓"
$ ezkl verify --proof mc_proof.json --vk mc_vk
→ "Proof is valid ✓"
$ ezkl verify --proof dml_proof.json --vk dml_vk
→ "Proof is valid ✓"
→ 3 つの証明が全て通過 → 計算は正しい
```

X 社はリームのサーバの中身を見る必要がありません。データの機密性が保たれたまま、計算の正しさが暗号学的に保証されます。

> 🔬 **研究室トーク：「『信じてください』から『検証してください』へ」**
>
> **リーム：**「この仕組みにより、クライアントに対するメッセージが根本的に変わります。従来は『結果を信じてください』でしたが、今は『結果を**検証**してください』と言える。信頼の質が根本的に変わる」
>
> **ユキ：**「おおー。クライアントが自分で確認できるんですね」
>
> **リーム：**「はい。特に金融機関や官公庁のクライアントにとって、これは決定的な差別化要因になります。規制当局への報告で『計算過程は ZKML で暗号学的に検証済みです』と言えることの価値は非常に大きい」
>
> **リン：**「ただし、ZKML は『計算が正しいこと』を証明しますが、『**入力データの選択が公正であること**』は証明しません。どのデータを入力するかの判断にバイアスがあれば、計算は正しくても結論は偏る。これは第 21 章の利益相反の議論とつながります」
>
> **リルル先生：**「リンの指摘は重要。ZKML は計算の正しさの保証。データ選択の公正性は第三者監査で保証。技術的保証と制度的保証の両方が必要。……いいね」

4.5　AI×ゼロ知識証明の最新動向

ZKML は 2024 年頃から急速に発展している AI×暗号学の融合分野です。2026 年 2 月時点の状況を整理します。

すでに実用可能な領域： - 小〜中規模のニューラルネットワーク（数千〜数百万パラメータ）の推論の ZK 証明 - ONNX 形式のモデルの自動変換（ezkl） - 固定小数点量子化による精度低下が許容範囲内のケース

研究段階の領域： - 大規模言語モデル（LLM、数十億パラメータ）の ZK 証明——回路サイズが巨大すぎて実用的でない - 訓練過程全体の ZK 証明（推論だけでなく、学習データの取り扱いも含む保証） - リアルタイム推論の ZK 証明（証明生成に時間がかかるため、リアルタイム応用は困難）

本書のパイプラインにとっての意味： VAE のデコーダ（数万パラメータ程度）の ZK 証明は 2026 年時点で実用可能。DML の最終推定（線形回帰相当）の ZK 証明も実用可能。一方、Neural ODE 全体の ZK 証明はまだ研究段階であり、Neural ODE の安全性は別のアプローチ（Part V の形式検証）で保証する方が現実的です。

本章の品質チェックポイント

□ **チェックリスト（第 4 章終了時点）**

1. **ZKML 証明が検証を通過するか？** → 検証不合格なら、算術回路への変換または ZKML フレームワークの実装にバグがある。証明が不合格なのに「気にしない」のは絶対に許されない
2. **量子化（浮動小数点→固定小数点）による精度低下は許容範囲か？** → 元の浮動小数点計算と量子化後の計算の差異が、各出力値の 1%以内であることを確認。第 5 章で量子化と有限体の詳細を扱う
3. **ZKML のコストは予算内か？** → 証明生成に必要な計算資源（GPU 時間、メモリ）と、クライアントへの価値提供のバランス

次章予告： 第 5 章では、ZKML を経営環境多様体パイプラインに具体的に適用する際の技術的詳細——量子化とは何か、有限体とは何か、なぜ素数で割るのか——を説明します。

第 5 章　ZKML×多様体の検証可能性——量子化と有限体

この章で得られるもの――コストに見合うリターン

第 4 章で ZKML の仕組み――ゼロ知識証明で AI の計算の正しさを暗号学的に保証する技術――を学びました。本章では、ZKML を経営環境多様体パイプラインに**具体的に適用する**際の技術的な核心に踏み込みます。

ZKML を動かすには、ニューラルネットワークの「ふつうの計算」（浮動小数点の掛け算や足し算）を、暗号の世界の「特殊な計算」（有限体の上の演算）に変換する必要があります。この変換を**量子化**と呼びます。

本章では「量子化とは何か」「有限体とは何か」「なぜ素数で割るのか」「この変換で何が失われるのか」を、ゼロから丁寧に説明します。また、第 1 章で導入したドロップアウトについても改めて詳しく掘り下げ、MC ドロップアウトの数学的な正当性を解説します。

本章の終わりで、信頼性の 4 本の柱が完成します。

5.1　量子化とは何か――「自由な物差し」から「目盛り付きの物差し」へ

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「なぜ、ふつうの計算のままでは ZKML が使えないのか」**

ユキ：「第 4 章で ZKML の仕組みは分かりました。でも、リルル先生が『浮動小数点を有限体に変換する必要がある』って言ってましたよね。浮動小数点って何ですか？ 有限体って何ですか？」

リルル先生：「とてもいい質問。順番に説明しよう。まず**浮動小数点**から」

リルル先生：「コンピュータが数値を扱う方法にはいくつかある。整数（1, 2, 3, …）なら簡単だけど、AI の計算では 3.14159 とか 0.000027 とか −1.73205 のような小数が必要。コンピュータは、こうした小数を**浮動小数点数**という形式で表現する。名前の由来は『小数点の位置が浮いて動く（float）』こと――314.159 も 0.00314159 も、同じ 3.14159 の小数点の位置を動かしたもの」

ユキ：「なるほど。で、有限体は？」

リルル先生：「有限体の説明は少し長くなるから、次の節（5.2）でじっくり説明する。ここではまず、**なぜ変換が必要か**の直感を掴もう」

リルル先生：「たとえ話をする。浮動小数点は『**自由な物差し**』――長さ 3.14159 cm とか 0.00027 cm とか、どんな長さでも測れる。有限体は『**目盛り付きの物差し**』――0cm, 1cm, 2cm, … , (p-1)cm の p 段階しか測れない。ZKML の暗号プロトコルは、この『目盛り付きの物差し』の世界でしか動作しない。だから、AI の計算で使う『自由な物差し』の数値を、暗号の世界の『目盛り付きの物差し』に換算する必要がある。この換算が**量子化（quantization）**」

ユキ：「目盛りの間に落ちる値はどうなるんですか？」

リルル先生：「一番近い目盛りに丸められる。3.14159 が 3 や 3.142 になる。これが**量子化誤差**。ただし、目盛りの数（p の大きさ）を十分に大きくすれば、誤差は実用上無視できる。ZKML で使う p は 254 ビットの巨大な素数――10 進数で 77 桁――だから、目盛りの数は宇宙の原子の数よりはるかに多い。誤差は事実上ゼロに近い」

カイト：「でも、元の浮動小数点の計算と量子化後の計算で結果が全く同じとは限りません。パイプラインに影響がないかは検証が必要です。特に、序章で説明した命題 1.1 の条件――VAE のデコーダのヤコビ行列がフルランク（逆行列が存在する）――が、量子化後も維持されているかどうかを確認する必要があります」

▶ AI モデルの基礎から理解したい方へ

量子化の具体的な実装：

```python
import torch

def quantize_for_zkml(tensor, scale=10000):
    """浮動小数点テンソルを ZKML 用の固定小数点に量子化

    浮動小数点数に scale を掛けて整数に丸める。
    scale=10000 なら小数第 4 位まで保持。

    例: 3.14159 → 31416（scale=10000 のとき）
        0.00027 → 3
       -1.73205 → -17321

    Args:
        tensor: 浮動小数点の PyTorch テンソル
        scale: スケーリング係数（大きいほど精度が高いが、回路サイズも増大）
```

```
    Returns:
        quantized: 整数テンソル（有限体の元として扱える）
    """
    return torch.round(tensor * scale).long()

def dequantize(tensor, scale=10000):
    """固定小数点を浮動小数点に戻す（量子化誤差の確認用）"""
    return tensor.float() / scale

# 量子化誤差の確認
x = torch.tensor([3.14159, 0.00027, -1.73205])
x_q = quantize_for_zkml(x)
x_dq = dequantize(x_q)
error = torch.abs(x - x_dq)
print(f"元の値:      {x.tolist()}")
print(f"量子化後:    {x_q.tolist()}")
print(f"復元後:      {x_dq.tolist()}")
print(f"量子化誤差:  {error.tolist()}")
print(f"最大相対誤差: {(error / torch.abs(x)).max():.6f}")
# → 最大相対誤差: 0.000041 (0.004%程度)
```

量子化後の命題 1.1 の検証：

```
def verify_fullrank_after_quantization(decoder, z_sample, scale=10000, th
reshold=1e-4):
    """量子化後もデコーダのヤコビ行列がフルランクかを検証

    命題1.1 の条件(ii)：ヤコビ行列がフルランク（最小特異値 > 0）
    量子化誤差によりフルランク性が崩れないかチェック
    """
    # 元のデコーダのヤコビ行列
    z = z_sample.requires_grad_(True)
    output = decoder(z)
    jacobian = torch.zeros(output.shape[-1], z.shape[-1])
    for i in range(output.shape[-1]):
        grad = torch.autograd.grad(output[0, i], z, retain_graph=True)[0]
        jacobian[i] = grad[0]

    singular_values = torch.linalg.svdvals(jacobian)
    min_sv = singular_values.min().item()

    if min_sv > threshold:
        print(f"✓ フルランク維持（最小特異値 = {min_sv:.6f} > {threshold}）
")
        return True
```

```
    else:
        print(f"⚠ フルランク性の低下（最小特異値 = {min_sv:.6f} ≤ {thresho
ld}）")
        print(f"  → scale の増大またはネットワーク構造の見直しが必要")
        return False
```

5.2　有限体とは何か――「素数で割った余りの世界」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「時計の算数」**

リルル先生：「有限体を説明する前に、身近な例から。ユキ、いま午前 10 時だとして、5 時間後は何時？」

ユキ：「午後 3 時です。10 + 5 = 15 だから」

リルル先生：「じゃあ、午前 10 時の 8 時間後は？」

ユキ：「午後 6 時。10 + 8 = 18 だから」

リルル先生：「12 時間制で考えると、18 時は 6 時。つまり **12 で割った余り**で考えている。0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 の 12 個の数字が、ぐるぐる循環する。11 の次は 0 に戻る。これが**時計の算数**」

ユキ：「あ、mod ってやつですか？ 数学の授業で少しやりました」

リルル先生：「そう。18 mod 12 = 6（18 を 12 で割った余りは 6）。有限体は、この『割った余りの世界』を、もっと厳密に定義したもの」

リルル先生：「具体例を出そう。$p = 7$ として、$\mathbb{F}_7 = \{0,1,2,3,4,5,6\}$ という 7 個の数だけの世界を考える。この世界では――」

- 足し算：$5 + 4 = 9$、でも $9 \bmod 7 = 2 \rightarrow$ **答えは 2**
- 引き算：$2 - 5 = -3$、でも $-3 \bmod 7 = 4 \rightarrow$ **答えは 4**（$4 + 3 = 7 \equiv 0$だから $4 = -3$）
- 掛け算：$3 \times 5 = 15$、$15 \bmod 7 = 1 \rightarrow$ **答えは 1**
- 割り算：$1 \div 3 = ? \rightarrow ? \times 3 = 1$ を満たす数 $\rightarrow 5 \times 3 = 15 \equiv 1 \rightarrow$ **答えは 5**

ユキ：「えっ、$1 \div 3 = 5$ ですか？ 変な感じ……」

リルル先生：「普通の世界では $1 \div 3 = 0.333\ldots$ だけど、有限体の世界では $1 \div 3 = 5$。ちゃんと検算すると $5 \times 3 = 15 = 7 \times 2 + 1$ だから、7 で割った余りは確かに 1。矛盾はない」

ユキ：「おおー。不思議な世界ですね。でも、なんで **7** で割るんですか？ 6 とか 8 じゃダメなんですか？」

なぜ「素数」で割らなければならないのか

リルル先生：「核心的な質問だ。**素数で割らないと、割り算ができなくなる**」

リルル先生：「$p = 6$ の場合を考えてみよう。{0,1,2,3,4,5} の世界で $2 \times 3 = 6 \equiv 0 \pmod 6$。つまり $2 \times 3 = 0$ になってしまう。2 もゼロでなく、3 もゼロでなく、でも掛けるとゼロ——これは**ゼロ除数**と呼ばれる。ゼロ除数が存在する世界では、割り算が定義できない。$0 \div 2$ を求めようとして、$?\times 2 = 0$ を満たす数を探すと、$? = 0$ だけでなく $? = 3$ でも成り立つ（$3 \times 2 = 6 \equiv 0$）。答えが 1 つに定まらない」

リン：「ちょっと待ってください。もう少し正確に言います。p が素数のとき、$\mathbb{F}_p$ では **0 以外の全ての元に逆元が存在する**ことが保証されます。つまり、0 で割る場合を除いて、**足し算、引き算、掛け算、割り算の全てが矛盾なく実行できる**。これは p が素数であることから直接導かれる性質です」

リン：「p が素数でなければ（たとえば $p = 6$）、ゼロ除数が存在し、割り算が一意に定まらない。つまり四則演算が完全には機能しない」

ユキ：「0 で割る計算だけはできない、というのは普通の数学と同じですよね」

リルル先生：「そう。0 で割る計算（ゼロ除算）は、普通の実数の世界でも有限体の世界でも定義されない。それ以外の四則演算——足し算、引き算、掛け算、0 以外での割り算——が全て矛盾なく実行できる数の世界を、数学では**体（たい）**と呼ぶ。英語では Field、ドイツ語では Körper」

リルル先生：「普通の実数の集合 $\mathbb{R}$ も体だし、有理数の集合 $\mathbb{Q}$ も体。そして p を素数としたときの $\mathbb{F}_p = \{0,1,2,\dots,p-1\}$ も体——ただし元の数が有限（p 個）だから**有限体**と呼ぶ」

カイト：「zkSNARKs の暗号プロトコルが有限体の上でしか動作しない理由は、楕円曲線暗号が有限体 $\mathbb{F}_p$ 上の楕円曲線のグループ構造に依存しているからです。第 4 章のペアリング写像 $e(g^a, g^b) = e(g,g)^{ab}$ も、有限体上の楕円曲線で定義されている」

リルル先生：「まとめよう」

普通の AI 計算：浮動小数点数の世界（実数 $\mathbb{R}$ に近い）
- → 足し算、引き算、掛け算、割り算が自由にできる
- → でも、暗号プロトコル（zkSNARKs）が動作しない

ZKML の計算：有限体 $\mathbb{F}_p$ の世界（p 個の整数の循環する世界）

→ 足し算、引き算、掛け算、割り算（0 以外）が矛盾なくできる
→ 暗号プロトコル（zkSNARKs）が動作する ✓
→ p が素数であることが必須（素数でないと割り算が壊れる）

量子化 ＝ 浮動小数点の世界 → 有限体の世界 への変換

▶ 数学の基礎から理解したい方へ

定義 5.1（有限体）. 素数 p に対し、有限体 $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ は、集合 $\{0,1,2,\dots,p-1\}$ に以下の演算を定義したものである。

- 加法：$a +_p b = (a + b) \bmod p$
- 乗法：$a \times_p b = (a \times b) \bmod p$

p が素数のとき、$\mathbb{F}_p$ は体をなす。すなわち、任意の $a \neq 0$ に対して乗法逆元 a^{-1} が存在し $a \times_p a^{-1} = 1$ を満たす（フェルマーの小定理より $a^{-1} = a^{p-2} \bmod p$）。

p が合成数のとき（例：$p = 6 = 2 \times 3$）、$2 \times_6 3 = 0$ となりゼロ除数が存在するため、体をなさない（整域ですらない）。

zkSNARKs で使用される典型的な p は BN254 曲線の素数 $p = 21888242871839275222246405745257275088548364400416034343698204186575808495617$（254 ビット、約$2^{254}$）。

定義 5.2（量子化写像）. スケーリング係数 $s \in \mathbb{Z}_{>0}$ に対し、量子化写像 $Q_s: \mathbb{R} \to \mathbb{F}_p$ を $Q_s(x) = \lfloor s \cdot x \rceil \bmod p$ で定義する（$\lfloor \cdot \rceil$ は最近整数への丸め）。逆量子化は $Q_s^{-1}(n) = n/s$（$n < p/2$ のとき）、$Q_s^{-1}(n) = (n-p)/s$（$n \geq p/2$ のとき、負の数の表現）。

命題 5.1（量子化誤差の上界）. 量子化写像 Q_s による誤差は $\left| x - Q_s^{-1}\left(Q_s(x)\right) \right| \leq 1/(2s)$ で上から抑えられる。$s = 10^4$ のとき誤差は 5×10^{-5} 以下。

5.3　ドロップアウトの詳細——MC ドロップアウトの数学的正当性

第 1 章でドロップアウトの基本的な仕組みを説明しました。ここでは、MC ドロップアウトがなぜ数学的に正当なのかをもう少し掘り下げます。

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「なぜ『ノードを消す』ことが不確実性の推定になるのか」**

ユキ：「第 1 章の話に戻るんですけど、ドロップアウト——ニューラルネットワークの各層で一部のノードをランダムに無効化する操作——を推論時にも行って、100 回繰り返して結果のばらつきを見る、というのが MC ドロップアウトでしたよね。でも、なぜ『ノードを消す』という操作が、予測の不確実性（自信の度合い）の推定として正しいんですか？ たまたまうまくいっているだけじゃないんですか？」

リルル先生：「数学的な理由がある。2016 年に Gal と Ghahramani が証明した重要な結果を説明しよう」

リルル先生：「ニューラルネットワークのパラメータ（重み）を θ とする。ベイズ推論——確率的にパラメータの不確実性を扱う統計手法——では、パラメータ θ は 1 つの固定値ではなく、**確率分布 $p(\theta|D)$**（データ D を観測した後のパラメータの確率分布）に従う。でも、ニューラルネットワークでは $p(\theta|D)$ を直接計算するのは不可能に近い（パラメータが何百万個もあるから）」

リルル先生：「Gal & Ghahramani は、**ドロップアウトマスクの集合が、この確率分布 $p(\theta|D)$ の近似になっている**ことを証明した。つまり——」

- ドロップアウトの各マスク（どのノードを消すかのパターン）= パラメータの 1 つの「サンプル」
- 100 回のドロップアウト = パラメータの確率分布から 100 個のサンプルを抽出
- 100 個のサンプルの平均 = ベイズ推論の期待値の近似
- 100 個のサンプルの分散 = パラメータの不確実性の近似

リルル先生：「だから、MC ドロップアウトは『たまたまうまくいっている』のではなく、**ベイズ推論の数学的に正当な近似**なの。これが変分推論（variational inference）との対応」

リン：「ちょっと補足させてください。ただし、この近似の精度には限界があります。ドロップアウトによる近似は、変分推論の中でもかなり粗い近似です。より正確な不確実性推定が必要な場合は、ハミルトニアンモンテカルロ法（HMC）やディープアンサンブルなどの手法がありますが、計算コストが格段に高くなります。MC ドロップアウトは**コスト対精度のバランスが良い**実用的手法として広く使われています」

▶ AI モデルの基礎から理解したい方へ

ドロップアウトの動作の詳細を改めて整理します。

```
import torch
import torch.nn as nn
```

```python
# === ドロップアウトの動作の詳細 ===

# ネットワークの定義
model = nn.Sequential(
    nn.Linear(30, 128),     # 第1層：30入力 → 128ノード
    nn.Tanh(),              # 活性化関数
    nn.Dropout(p=0.3),      # ★ 30%のノードをランダムに無効化
    nn.Linear(128, 64),     # 第2層：128入力 → 64ノード
    nn.Tanh(),
    nn.Dropout(p=0.3),      # ★ 30%のノードをランダムに無効化
    nn.Linear(64, 6),       # 第3層：64入力 → 6出力
)

# --- 訓練時の1回の計算 ---
model.train()  # ドロップアウトON
x = torch.randn(1, 30)  # 入力データ

# 第1層の128ノードのうち、約38ノード（30%）がランダムに0になる
# 第2層の64ノードのうち、約19ノード（30%）がランダムに0になる
y1 = model(x)  # 出力（ある組合せのノードで計算）

# --- 次の計算 ---
# 今度は【別のノード】がランダムに0になる
y2 = model(x)  # 出力（別の組合せのノードで計算）
# y1 ≠ y2 （毎回異なるノードが消えるから、結果も異なる）

# --- MC ドロップアウト（100回繰り返し） ---
predictions = []
with torch.no_grad():
    for t in range(100):
        pred = model(x)  # 毎回異なるノードが消える
        predictions.append(pred)

preds = torch.stack(predictions)  # shape: [100, 6]
mean = preds.mean(dim=0)   # 100個の予測の平均 → 最良推定値
std = preds.std(dim=0)     # 100個の予測の標準偏差 → 不確実性
ci_lower = mean - 1.96 * std  # 95%信頼区間の下限
ci_upper = mean + 1.96 * std  # 95%信頼区間の上限

print(f"予測値:         {mean.tolist()}")
print(f"標準偏差:       {std.tolist()}")
```

```
print(f"95%信頼区間下限: {ci_lower.tolist()}")
print(f"95%信頼区間上限: {ci_upper.tolist()}")
```

▶ 数学の基礎から理解したい方へ

定理 5.1（Gal & Ghahramani, 2016）. L 層のニューラルネットワークにドロップアウト率 p のドロップアウトを適用した場合、ドロップアウト付きの訓練目的関数の最小化は、変分推論における近似事後分布 $q(\theta)$ と真の事後分布 $p(\theta|D)$ の間の KL ダイバージェンス $D_{\mathrm{KL}}\big(q(\theta) \,\|\, p(\theta|D)\big)$ の最小化と等価である。ここで近似事後分布は $q(\mathbf{W}_l) = \mathbf{M}_l \cdot \mathrm{diag}(\mathrm{Bernoulli}(1-p)^{K_l})$（$\mathbf{M}_l$ は第 l 層の重み行列、K_l は第 l 層のノード数）の形を取る。

系 5.1（MC ドロップアウトの正当性）. 推論時にドロップアウトを ON のまま T 回サンプリングし、出力の平均と分散を計算することは、上記の近似事後分布に基づくモンテカルロ変分推論に相当する。$T \to \infty$ で真の変分事後分布に収束する。

5.4　4 本の柱の完成——Part I 全体の統合

🔬 **研究室トーク：「4 本の柱＋第 5 の柱の予告」**

リルル先生：「Part I の全 5 章を通じて、信頼性の 4 本の柱が完成した。まとめよう」

柱	手法	保証する内容	対応する弱点
第 1 の柱	MC ドロップアウト（第 1 章）	予測の不確実性定量化	弱点 2：不確実性の不可視
第 2 の柱	SHAP（第 2 章）	予測の説明可能性	弱点 3：説明可能性の欠如
第 3 の柱	DML（第 3 章）	施策の因果的効果	弱点 1：相関と因果の未分離
第 4 の柱	ZKML（第 4-5 章）	計算の正しさの暗号学的保証	（新規追加）

リルル先生：「4 本の柱で**出力の信頼性**を担保した。しかし、パイプラインの**設計自体**が正しいかどうか——たとえば Neural ODE（微分方程式の右辺をニューラルネットワークで学習するモデル。第 6 章で詳述）が安定に動作するか、AI Agent の報酬関数（Agent の行動の良し悪しを数値で評価する関数）が安全に設計されているか——は、4 本の柱では保証できない。別の方法が必要」

リルル先生：「それが第 5 の柱——**形式検証**。Coq、Agda、Lean 4、Z3 という 4 つの定理証明支援系と形式検証ツールを使って、パイプラインの設計仕様を**数学的に証明**する。Part V（第 23〜30 章）で扱う」

ユキ：「5 本も柱があるんですね。全部使うんですか？」

リルル先生：「原則 2——足すな、信頼性を上げろ。5 本全部を常に使う必要はない。4 本の柱のうち**最低 3 本**が機能していれば Part II に進める。ZKML はコスト面の制約で適用を見送ることが許容される。形式検証（第 5 の柱）は、AI Agent の自律度（第 7 章で説明する、Agent がどこまで自分で判断するかの度合い）が高い場合に必須」

カイト：「全マシマシにしない。目的に必要なものだけを選ぶ」

リルル先生：「……いいね」

本章の品質チェックポイント——Part I 全体の最終チェック

□ **Part I 全体のチェックリスト**

1. **第 1 の柱（MC ドロップアウト）**：不確実性マップが計算され、calibration plot で過信が検出されていない
2. **第 2 の柱（SHAP）**：特徴量の貢献度がドメイン知識と整合し、効率性公理（SHAP 値の合計＝予測値）が成立
3. **第 3 の柱（DML）**：因果効果が推定され、SHAP の結果との乖離が交絡因子で説明可能
4. **第 4 の柱（ZKML）**：クリティカルな 3 点（VAE 推論、MC ドロップアウト分散、DML 最終推定）の ZKML 証明が検証を通過。または、コスト面の判断で ZKML の適用を見送った場合、その判断が文書化されている
5. **量子化誤差**：浮動小数点計算と量子化後の計算の差異が各出力の 1%以内。命題 1.1 のフルランク条件が量子化後も維持

4 本のうち最低 3 本が機能していれば Part II に進めます。

Part II への橋渡し：Part I では、パイプラインの出力の信頼性を 4 本の柱で担保する体系を構築しました。Part II では、この信頼できるパイプラインを「デジタルツイン」（現実の経営環境のサーバ上の仮想写像——そもそもデジタルツインとは何かは第 6 章で説明します）として運用し、その上で AI Agent（自律的に行動する計算主体——AI Agent の定義と設計は第 6〜7 章で説明します）を走らせます。複数の Agent が同時に動くとき生じる**創発現象**（個々の Agent の設計からは

予測できない全体の振る舞い——創発とは何かは第 8 章で詳しく説明します）の分析と、運用監視の体系を構築します。

PART II　AI Agent の設計と運用

第 6 章　デジタルツイン×AI Agent

この章で得られるもの——コストに見合うリターン

Part I で信頼性の 4 本の柱を構築しました。パイプラインの出力に信頼区間が付き（MC ドロップアウト）、要因分解ができ（SHAP）、因果効果が推定でき（DML）、計算の正しさが暗号学的に保証されます（ZKML）。

しかし、これだけでは「最適な施策は何か」という問いに完全には答えられません。パイプラインは「現在の経営状態がどうなっているか」を分析し、「ある施策を打ったらどうなるか」を推定できますが、**「考えうる無数の施策の中から、最も望ましい結果をもたらす施策を自動的に探索する」**機能がありません。これが旧『ジオメトリック・インテリジェンス』になかった機能であり、本章で導入する AI Agent の役割です。

本章では、まず「デジタルツインとは何か」を 2026 年の産業界の実例とともに説明し、次に「なぜ経営環境多様体の上で AI Agent を動かすのか」のメリットを明確にし、最後に「ポントリャーギンの最大原理」——制約のある中で最善の行動計画を立てる数学——を、平面上の制御と曲がった空間上の制御の違いを含めて解説します。

6.1　デジタルツインとは何か

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「シミュレーション用の『双子』を作る」**

ユキ：「デジタルツインって何ですか？ 名前は聞いたことがあるけど……」

リルル先生：「**デジタルツイン（Digital Twin）**は、現実世界の物体・システム・環境を、コンピュータ上に精密な仮想コピーとして再現したもの。『デジタルの双子（ツイン）』という意味。現実の状態をリアルタ

イムでデジタルの双子に反映し、双子の上でシミュレーションを行うことで、現実に手を加える前に『こうしたらどうなるか』を仮想的に試せる」

ユキ：「ゲームの中に現実のコピーを作るみたいな感じですか？」

リルル先生：「近いけど、ゲームよりずっと精密。2026 年の産業界では、デジタルツインはすでに多くの分野で実用化されている」

2026 年のデジタルツインの実例：

製造業： 工場の生産ラインを丸ごとデジタルツインとして再現。機械の稼働状態、温度、振動のセンサーデータをリアルタイムで反映し、「この機械は 3 週間後に故障する確率が高い」「生産ラインの配置をこう変えれば効率が 12%上がる」といったシミュレーションを、実際の工場を止めずに行える。シーメンス、GE、PTC などが大規模に展開。

都市計画： 都市全体をデジタルツインとして再現。交通量、エネルギー消費、人の流れをリアルタイムで反映し、「この交差点の信号タイミングを変えたら渋滞がどう変わるか」「大地震が起きたとき避難経路はどうなるか」をシミュレーション。シンガポールの「Virtual Singapore」プロジェクトが代表例。

医療： 患者の臓器をデジタルツインとして再現し、手術前に「この角度でメスを入れたらどうなるか」をシミュレーション。ダッソー・システムズの「Living Heart Project」が心臓のデジタルツインを構築。

本書の文脈――経営環境のデジタルツイン： 本書では、企業の経営環境（売上、為替、競合、マクロ経済等の多数の変数が複雑に絡み合う空間）を、VAE（変分オートエンコーダ）で「曲がった空間」（多様体）として構成し、この多様体をサーバ上のデジタルツインとして運用します。工場のセンサーデータの代わりに財務データや市場データがリアルタイムで多様体に反映され、「この施策を打ったら経営状態がどう変化するか」をシミュレーションできます。

🔬 **研究室トーク：「なぜ、現実の経営環境で直接試さないのか」**

ユキ：「デジタルツインでシミュレーションしなくても、実際にやってみればいいんじゃないですか？」

リーム：「少しよろしいですか。3 つの理由があります」

リーム：「**理由 1：不可逆性。** 経営施策の多くは一度実行すると元に戻せません。新工場の建設に 100 億円投資した後で『やっぱりやめた』とは言えない。デジタルツインなら、仮想的に何度でもやり直せます」

リーム：「理由 2：コスト。 10 種類の施策を全部試すには、実世界では 10 回の実験が必要で、それぞれに数億〜数十億円のコストがかかる。デジタルツインなら、10 種類でも 100 種類でも 1000 種類でも、計算コストだけで試せます」

リーム：「理由 3：速度。 施策の効果が現れるのに半年〜数年かかる場合がある。デジタルツインなら、Neural ODE（微分方程式の右辺をニューラルネットワークで学習したモデル）で軌道を予測して、数分〜数時間で『5 年後にどうなるか』をシミュレーションできます」

ユキ：「おおー。現実で試す前に、デジタルの世界で徹底的に試行錯誤するんですね」

リルル先生：「そう。そしてその試行錯誤を**人間が手動でやる**のではなく、**AI Agent に自動的にやらせる**のが本章のテーマ」

6.2　なぜ経営環境多様体の上で AI Agent を動かすのか

▶ ビジネス・政策の意思決定に活用したい方へ

旧『ジオメトリック・インテリジェンス』のパイプラインには、AI Agent は登場しませんでした。パイプラインは「経営環境の分析」（曲率計算、Lie 微分等）と「1 つの施策の効果予測」（最適制御計算）を行うツールであり、**「無数の施策候補の中から最適なものを自律的に探索する」**機能はありませんでした。

本書で AI Agent を導入するメリットは以下の 3 つです。

メリット 1：探索の自動化。 人間が「施策 A、施策 B、施策 C のどれがいいか」と手動で比較するのではなく、AI Agent が多様体上を自律的に動き回りながら、数千〜数万の施策候補を試行し、最もリスクが低くリターンが高い（RAROC——リスク調整後資本利益率——を最大化する）施策を見つけ出します。

メリット 2：反応の速度。 経営環境がリアルタイムで変化するとき、人間のアナリストが毎回手動で分析するのは間に合いません。AI Agent は経営環境の変化をリアルタイムで検知し、即座に対応策を提案できます。

メリット 3：多様体の幾何学的構造の活用。 AI Agent は多様体の曲率（どの領域が不安定か）、測地線（2 点間の最短経路）、Lie 微分（施策が環境構造に与える影響）を活用して、**幾何学的に賢い探索**を行えます。「曲率が負（不安定）の領域を避けながら、測地線に沿って目標状態に到達する最適経路」といった、幾何学的な知識を活用した施策探索が可能になります。

🔬 **研究室トーク：「多様体の地形を知っている案内人」**

リルル先生：「たとえ話をしよう。あなたが未知の山岳地帯を横断したいとする。地図なしで歩くのは危険。でも、詳細な 3D 地形図（多様体の曲率マップ）を持っている案内人（AI Agent）がいれば、崖（スカラー曲率が負の不安定領域）を避け、なだらかな尾根道（測地線に近い安定経路）を選んで案内してくれる」

リルル先生：「旧作のパイプラインは『3D 地形図を作る』ところまでだった。本書では『3D 地形図を持った AI 案内人が、最適なルートを自動で探索する』機能を追加する」

カイト：「その『最適なルート』を数学的に計算するのが、ポントリャーギンの最大原理ですね」

6.3　ポントリャーギンの最大原理とは何か

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「制約のある中で最善の道を見つける」**

リルル先生：「ポントリャーギンの最大原理（Pontryagin et al., 1962）は、**制約のある中で最善の行動計画を立てる**ための数学的手法。**最適制御理論**の中心定理」

ユキ：「最適制御って何ですか？」

リルル先生：「たとえば、東京から大阪に行きたいとする。予算 10 万円、所要時間 8 時間以内、CO2 排出量最小という 3 つの制約がある。新幹線は速いが高い。夜行バスは安いが遅い。車は途中で立ち寄れるが CO2 を出す。ポントリャーギンの最大原理は、こうした**制約付き最適化問題**――制約を守りながら目的（たとえば『到着時間の早さ＋コストの安さ＋CO2 の少なさ』の総合スコア）を最大化する――を数学的に解く方法」

ユキ：「経営の場合はどうなるんですか？」

リルル先生：「経営環境多様体の上で、『現在の経営状態から目標の経営状態に、利用可能な施策の制約の下で、最小のリスクと最大のリターン（RAROC 最大化）を達成する移行経路』を計算する」

平面上の制御と曲がった空間上の制御の違い

🔬 **研究室トーク：「平面と地球の上でのナビゲーションの違い」**

リルル先生：「ここで重要なのが、**平面上の制御**と**曲がった空間（多様体）上の制御**の違い」

リルル先生：「平面上でのナビゲーションを考えてみよう。東京から大阪への最短経路は直線。施策の効果は全ての場所で同じ——広告費を 1 億円増やしたら売上が X 億円増える効果は、業績好調のときも不調のときも同じ」

リルル先生：「でも現実の経営環境は平面ではなく**曲がった空間**。地球の表面を想像して。東京からニューヨークへの最短経路は直線ではなく、北極圏を通る大圏航路（測地線）。平面の直線を地球の上に描くと、最短経路にはならない」

カイト：「経営環境でも同じですよね。円安のときに広告費を増やす効果と、円高のときに広告費を増やす効果は、同じ 1 億円でも全然違う」

リルル先生：「そう。多様体上では、施策の効果が**場所（経営状態）によって異なる**。これを Lie 微分（序章で説明した、施策が環境の構造をどう変えるかの数値）が定量化する。ポントリャーギンの最大原理を多様体上で適用するには、この Lie 微分の情報を使って、**曲がった空間の上での最適経路**を計算する必要がある」

リン：「ちょっと待ってください。平面上の最適制御と多様体上の最適制御の数学的な違いを整理させてください」

平面上の最適制御（従来手法）：
  状態空間：ℝ^n（ユークリッド空間）
  ダイナミクス：ẋ = f(x, u)（普通の微分方程式）
  距離：ユークリッド距離 ||x - y||
  最適経路：直線に近い
  施策の効果：場所によらず一定（線形近似）

多様体上の最適制御（本書の手法）：
  状態空間：M（曲がった多様体）
  ダイナミクス：∇_ż ż = F(z) + u_α B_α(z)（共変微分を含む）
  距離：測地線距離（曲がった空間上の最短距離）
  最適経路：測地線に沿う（曲がった空間の形状を反映）
  施策の効果：場所によって異なる（Lie 微分で定量化）

リン：「多様体上の最適制御が難しいのは、**クリストッフェル記号**（空間の曲がり具合を表す係数。旧『ジオメトリック・インテリジェンス』の第 3 章で詳しく説明されています）が微分方程式に入ってくるからです。平面上なら $\ddot{x} = F(x)$ で済むところが、多様体上では $\ddot{z}^k + \Gamma^k_{ij}\dot{z}^i\dot{z}^j = F^k(z)$ になる。Γ^k_{ij} の項が空間の曲がりの効果」

▶ AI モデルの基礎から理解したい方へ

ポントリャーギンの最大原理の数学的定式化と、Python での実装を示します。

問題設定： - 状態 $z(t) \in \mathbb{R}^d$：時刻 t の経営状態（VAE の潜在空間の座標） - 制御入力 $u(t) \in U_{\text{ad}} \subset \mathbb{R}^m$：施策の強度（$U_{\text{ad}}$ は許容範囲、例えば予算制約） - ダイナミクス：$\dot{z}^k = F^k(z) + u_\alpha B_\alpha^k(z)$ - $F(z)$：経営環境の自然な変化（Neural ODE で学習） - $B_\alpha(z)$：施策 α の効果（ニューラルネットワークで学習） - u_α：施策 α の強度（制御変数） - コスト汎関数：$J[u] = \int_0^T \ell\,(z,u)\,dt + \Phi\big(z(T)\big)$ - $\ell(z,u)$：途中のコスト（リスク、資源消費等） - $\Phi\big(z(T)\big)$：最終状態の評価（目標状態からの距離）

ポントリャーギンの最大原理： 最適制御 $u^*(t)$ は、ハミルトニアン（エネルギーに相当する量）

$$H(z,p,u) = p^{\mathsf{T}}\big(F(z) + u_\alpha B_\alpha(z)\big) - \ell(z,u)$$

を最大化する。ここで $p(t)$ は**随伴変数**（共状態、costate）と呼ばれ、「状態の微小変化がコストに与える影響の大きさ」を表す。

正準方程式：

$$\dot{z}^k = \frac{\partial H}{\partial p_k}, \quad \dot{p}_k = -\frac{\partial H}{\partial z^k}$$

横断条件： $p(T) = \nabla\Phi\big(z(T)\big)$（最終時刻での随伴変数は、最終状態の評価関数の勾配）

```
import torch
import torch.nn as nn
from torchdiffeq import odeint

class ManifoldOptimalControl(nn.Module):
    """多様体上の最適制御（ポントリャーギンの最大原理）

    Neural ODE の逆伝播で使う随伴法（Chen et al., 2018）は、
    ポントリャーギンの随伴方程式の離散版。
    つまり Neural ODE の訓練と最適制御は同じ数学的構造。
    """
    def __init__(self, dim, n_controls):
        super().__init__()
        self.dim = dim
        # F(z)：経営環境の自然なダイナミクス（Neural ODE で学習）
        self.drift = nn.Sequential(
            nn.utils.spectral_norm(nn.Linear(dim, 64)),
```

```
            nn.Tanh(),  # C∞活性化関数（命題1.1 の条件）
            nn.utils.spectral_norm(nn.Linear(64, 64)),
            nn.Tanh(),
            nn.utils.spectral_norm(nn.Linear(64, dim)),
        )
        # B_α(z) : 各施策の効果ベクトル場
        self.control_fields = nn.ModuleList([
            nn.Sequential(
                nn.Linear(dim, 32), nn.Tanh(), nn.Linear(32, dim))
            for _ in range(n_controls)
        ])

    def dynamics(self, z, u):
        """ż = F(z) + Σ_α u_α B_α(z)"""
        Fz = self.drift(z)
        control = sum(u[a] * self.control_fields[a](z)
                      for a in range(len(u)))
        return Fz + control

    def hamiltonian(self, z, p, u):
        """ハミルトニアン H(z, p, u) = p^T(F+uB) - ℓ(z,u)"""
        dz = self.dynamics(z, u)
        running_cost = 0.01 * torch.sum(u**2)  # 施策コスト（L2 正則化）
        return torch.dot(p, dz) - running_cost

    def find_optimal_control(self, z, p, u_bounds):
        """ハミルトニアンを最大化する u を探索"""
        best_u = None
        best_H = float('-inf')
        # 格子探索（実用上は勾配法で最適化）
        for u_candidate in torch.linspace(u_bounds[0], u_bounds[1], 20):
            u = u_candidate.expand(len(self.control_fields))
            H = self.hamiltonian(z, p, u)
            if H.item() > best_H:
                best_H = H.item()
                best_u = u.clone()
        return best_u

# --- 使用例 ---
dim = 6          # X 社の潜在次元
n_controls = 3   # 3 つの施策（広告、R&D、価格改定）

controller = ManifoldOptimalControl(dim, n_controls)

# 現在の経営状態
```

```
z0 = torch.randn(dim)
# 初期随伴変数（目標状態の方向）
p0 = torch.randn(dim)
# 施策の許容範囲
u_bounds = (-1.0, 1.0)

# 最適施策の計算
u_star = controller.find_optimal_control(z0, p0, u_bounds)
print(f"最適施策の強度: {u_star.tolist()}")
```

使用するライブラリ： `torchdiffeq`（pip install torchdiffeq）。Chen et al.（NeurIPS 2018）が開発した Neural ODE の PyTorch 実装。`odeint` 関数で微分方程式の数値積分を行い、`autograd` で随伴方程式（ポントリャーギンの随伴変数の方程式）の逆伝播を自動計算。

`spectral_norm`（スペクトル正規化）について： 各層の重み行列のスペクトルノルム（最大特異値）を 1 以下に制約する正則化手法（Miyato et al., 2018）。Neural ODE の安定性に関わる——スペクトルノルムが大きすぎると、微分方程式の解が発散し、予測が非現実的になる。詳しくは第 27 章で、リプシッツ定数が安全性にどう関わるかを形式的に証明します。

▶ 数学の基礎から理解したい方へ

定理 6.1（ポントリャーギンの最大原理）. アフィン制御系 $\dot{z}^k = F^k(z) + u_\alpha B^k_\alpha(z)$, $u \in U_{\mathrm{ad}}$ に対するコスト汎関数 $J[u] = \int_0^T \ell\,(z,u)\,dt + \Phi\big(z(T)\big)$ を最小化する最適制御 u^* が存在するなら、リプシッツ連続な随伴変数 $p(t)$ が存在し、

(i) 正準方程式 $\dot{z}^k = \partial H/\,\partial p_k, \dot{p}_k = -\,\partial H/\,\partial z^k$ を満たす

(ii) 最大化条件 $H(z^*, p, u^*) = \max_{u \in U_{\mathrm{ad}}} H(z^*, p, u)$ を a.e. $t \in [0,T]$ で満たす

(iii) 横断条件 $p(T) = \nabla\Phi\big(z(T)\big)$ を満たす

ここで $H(z,p,u) = p_k(F^k + u_\alpha B^k_\alpha) - \ell(z,u)$。

多様体上への拡張。 経営環境多様体 $\mathcal{M}$ 上では、ダイナミクスに共変微分が入る。測地偏差方程式 $\nabla_{\dot{z}}\dot{z} = F(z) + u_\alpha B_\alpha(z)$ において、共変微分 ∇ はクリストッフェル記号 Γ^k_{ij} を含む。座標表示では $\ddot{z}^k + \Gamma^k_{ij}\dot{z}^i\dot{z}^j = F^k(z) + u_\alpha B^k_\alpha(z)$。$\Gamma^k_{ij}$ は引き戻し計量 g_{ij} から計算される（旧『ジオメトリック・インテリジェンス』第 3 章で詳述）。

Neural ODE の随伴法との関係。 Chen et al.（2018）の Neural ODE の逆伝播は、随伴方程式 $dp/dt = -p^{\top}\,\partial f/\,\partial z$ を使用する。これはポントリャーギンの随伴方程

式の離散版であり、**Neural ODE の訓練と最適制御の計算は同じ数学的構造を共有**している。

6.4　3 層ロードマップ——観測・予測・介入

デジタルツイン×AI Agent のアーキテクチャを 3 層に整理します。

第 1 層（観測）：いま、経営環境のどこにいるか？
- → 現実のデータを VAE エンコーダに入力
- → 多様体上の「現在地」`z(t)` を推定
- → MC ドロップアウト（第 1 章）で現在地の不確実性を定量化
- → 更新頻度：毎日〜毎週

第 2 層（予測）：経営環境はどこに向かうか？
- → `Neural ODE ż = f(z,t)` で将来の軌道を予測
- → 軌道上のスカラー曲率でリスクを評価
- → 更新頻度：毎週〜毎月

第 3 層（介入）：どの施策を打てば目的地に着くか？
- → ポントリャーギンの最大原理で最適施策を計算
- → Lie 微分で施策の構造的インパクトを評価
- → SHAP（第 2 章）で施策効果の要因分解
- → DML（第 3 章）で施策の因果効果を推定
- → 更新頻度：毎月

> 🔬 **研究室トーク：「自動化するが、最終判断は人間」**
>
> **リーム：**「少しよろしいですか。私のベンチャーでは 14 体の AI Agent がこの 3 層を自動化して毎日稼働しています。しかし、**第 3 層の介入の最終判断は必ず人間が行います**。Agent が『施策 A が最適です』と推薦しても、経営者が『いや、施策 B にする』と判断すれば、施策 B が実行される。Agent は**推薦者**であり、**決定者**ではない」
>
> **リルル先生：**「これは原則 1『目的を忘れるな——信じて行動できるインテリジェンスを生成すること』に直結する。Agent の役割は、人間が**信じて行動できる**情報を提供すること。人間の判断を置き換えることではない」

6.5　ケーススタディ——X 社と安全保障のデジタルツイン

ケーススタディ A：X 社のデジタルツイン

X 社（売上高 1,000 億円規模の中堅製造業）のデジタルツインは以下のサイクルで運用されています。

毎日（第 1 層・観測）：50 店舗の POS データ、マクロ経済指標、為替レート、競合の決算情報を VAE エンコーダに入力。6 次元の経営環境多様体上の「現在地」を更新。MC ドロップアウトで不確実性を計算し、不確実性が 0.3 を超えた領域にアラート。

毎週（第 2 層・予測）：Neural ODE で今後 4 週間の経営状態の軌道を予測。軌道がスカラー曲率負の領域（不安定領域）に近づく場合、事前アラートを発信。「2 週間後に為替リスク領域に進入する確率 38%」といった形式の早期警報。

毎月（第 3 層・介入検討）：ポントリャーギンの最大原理で 3 つの施策候補（広告費調整、R&D 投資、価格改定）の最適強度を計算。各施策について SHAP で要因分解、DML で因果効果を推定。RAROC（リスク調整後資本利益率）を最大化する施策の組合せを経営会議に提示。

ケーススタディ B：国家安全保障デジタルツイン

安全保障チームのデジタルツインは、193 カ国×40 指標のデータで 8 次元の国際情勢多様体を構成。日次の貿易・金融データと月次の軍事・外交指標で更新。同盟強化政策と経済制裁政策の因果効果を DML で推定し、多様体上の測地線に沿った最適な政策パスを計算。第 16 章の 3 層ガードレール（法的：国際人道法チェック、倫理的：文民保護、数学的：ゲージ不変性）を通過した施策のみを推薦。

本章の品質チェックポイント

□ チェックリスト（第 6 章終了時点）

1. **3 層（観測・予測・介入）の各モジュールが序章の仮定 A1〜A4 を満たしているか？** → 旧『ジオメトリック・インテリジェンス』の品質チェックを再実行
2. **Neural ODE の軌道が発散しないか？** → 5 年先まで積分して発散がないか確認。発散する場合はスペクトル正規化の強度を上げる（第 27 章で形式的に検証）
3. **デジタルツインの更新頻度はビジネス要件に合致するか？** → 日次データで月次判断なら十分。リアルタイム判断が必要なら更新頻度を上げる

4. **最終判断が人間に留保されているか？** → Agent は推薦者。決定者は人間

次章予告： 第 7 章では、AI Agent の設計体系——3 軸による割り当て、Embodied AI と Disembodied AI の違い、自律度に応じた安全性要件——を学びます。リームの 14 体の Agent が 7 社のクライアントにどう配備されているかの実例も紹介します。

第 7 章　Agent 設計の体系

この章で得られるもの——コストに見合うリターン

第 6 章で、経営環境多様体のデジタルツイン上で AI Agent を動かす基本アーキテクチャを設計しました。本章では、**複数の AI Agent をどのように設計し、どのクライアント・どの用途に割り当てるか**の体系を学びます。

具体的には、(1) AI Agent を設計するための 3 つの軸（時間スケール・データの性質・自律度）、(2) **身体を持つ AI（Embodied AI）と持たない AI（Disembodied AI）**の本質的な違いと、それぞれの施策探索の結果がどう異なるか、(3) Zenn Book『Etale』の Agent 設計との比較と教訓——を扱います。

7.1　3 軸による Agent 設計

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「14 体の Agent をどう使い分けるか」**

リーム：「少しよろしいですか。私のベンチャーでは 7 社のクライアントに対して合計 14 体の AI Agent を稼働させています。14 体を闇雲に動かしているわけではなく、**3 つの軸**で整理して各クライアントに割り当てています」

ユキ：「3 つの軸って何ですか？」

軸 1：意思決定の時間スケール τ

AI Agent が担当する意思決定の時間軸（計画の期間）で分類します。

分類	時間スケール	経営での例	政策での例
戦略的	年〜10 年	長期ポートフォリオ最適化、新規事業参入	同盟政策、産業政策
戦術的	月〜四半期	四半期の施策選択、予算配分	経済制裁の強度調整
操作的	日〜週	日次の在庫管理、価格調整	為替介入のタイミング

戦略的 Agent は多様体上の低周波のダイナミクス（ゆっくりした経営環境の変化）を制御し、操作的 Agent は高周波の応答（日々の市場変動への対応）を制御します。

軸 2：データの性質

Agent が扱うデータの種類で分類します。

分類	データの例	多様体上での扱い
構造化データ	財務諸表、市場データ、センサーデータ	$\mathbb{R}^n$ のベクトルとして直接入力
非構造化データ	ニュース記事、SNS 投稿、衛星画像	埋め込みモデルでベクトル化してから入力
混合	構造化＋非構造化の組合せ	マルチモーダル統合

軸 3：介入の自律度 $\alpha \in [0,1]$

Agent がどこまで自分で判断するかの度合いです。

> 🔬 **研究室トーク：「自律度と安全性のトレードオフ」**
>
> **リルル先生：**「自律度は 0 から 1 の間の数値で設定する。0 は『全ての判断に人間の承認が必要』、1 は『全ての判断を Agent が自律的に行う』」
>
> **リン：**「ちょっと待ってください。自律度が高いほど、**安全性の要件も厳しくなる**ことを強調すべきです」

自律度	モード	安全性要件
$\alpha \leq 0.3$	人間承認型	MC ドロップアウト＋SHAP（第 1-2 章）で十分

自律度	モード	安全性要件
$0.3 < \alpha \leq 0.7$	半自律型	上記＋DML（第 3 章）＋ガードレール（第 16 章で詳述）
$\alpha > 0.7$	高自律型	上記＋ZKML（第 4-5 章）＋形式検証（Part V）

リン：「$\alpha > 0.7$ の高自律型 Agent には、Part V（第 23〜30 章）の形式検証が**必須**です。形式検証が完成していない段階で高自律型 Agent を運用することは許容されません」

リーム：「実際のうちのベンチャーでは、現時点で全 14 体の Agent が $\alpha \leq 0.5$——半自律型以下——で運用されています。形式検証の実装が完了するまで、高自律型は許可していません」

X 社の Agent 構成の具体例

Agent	軸 1（時間）	軸 2（データ）	軸 3（自律度）	役割
需要予測 Agent	戦略的（四半期）	構造化	人間承認型（$\alpha = 0.2$）	四半期の需要予測を経営会議に提示
コスト最適化 Agent	操作的（週次）	構造化	半自律型（$\alpha = 0.5$）	週次コスト構造の最適化。閾値内は自律実行

7.2　Embodied AI とは何か——身体を持つ AI と持たない AI

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「赤ちゃんはなぜ積み木を何度も落とすのか」**

リルル先生：「AI Agent には『身体を持つもの（Embodied AI）』と『持たないもの（Disembodied AI）』がある。この区別は Agent の能力だけでなく、安全性の要件を根本的に変える」

ユキ：「身体を持つ AI って、ロボットのことですか？」

リルル先生：「ロボットは Embodied AI の 1 つの形だけど、**身体性（embodiment）の本質はロボットの体を持つことではない**。身体性の核心は、**AI が環境との物理的な相互作用を通じて学習すること**」

リルル先生：「人間の赤ちゃんを思い浮かべて。赤ちゃんは積み木を掴んで、落とす。何度も何度も。なぜか？——落としたときの衝撃、床にぶつかる音、積み木が跳ね返る様子、自分の手が空になる感覚。これらの**身体的な体験**を通じて、赤ちゃんは『物を放すと落ちる』『高いとこ

ろから落とすとよく弾む』『重いものは遠くに投げられない』——世界の物理法則と自分の身体の関係を、**実感として**学習している」

ユキ：「教科書で『重力加速度は 9.8m/s² です』って読むのとは違うんですね」

リルル先生：「全く違う。赤ちゃんは重力加速度の数値を知らないのに、物を落とすタイミングを学習する。走ったときの足裏の摩擦、壁にぶつかったときの反発力、熱いものに触れたときの痛み——身体を使って世界と相互作用し、そのフィードバックを通じて、世界と自分の振る舞いの相互関係を**実感として**理解する。**『こう動くと、こういう反発を受ける。こう振る舞うと、こう受け入れられる』**という関係を、頭の中の計算ではなく、身体の体験で学ぶ。これが Embodied AI——身体的知能——の核心」

Embodied AI（身体を持つ AI）： AI が物理環境との直接的な相互作用——触覚、力覚、移動、衝突——を通じて学習し行動するシステム。ロボットアームが物体を掴むとき、握力が弱すぎれば物体は落ち、強すぎれば壊れる。この「試行錯誤と身体的フィードバック」を繰り返して最適な握力を学習する。最新の研究（2025 年の IJCAI、CVPR、Frontiers in Robotics の複数のサーベイ論文）では、大規模言語モデル（LLM）と Embodied AI を統合し、言語による指示（「テーブルの上のコップを棚に移動して」）を物理的な行動に変換する Vision-Language-Action（VLA）モデルが急速に発展しています。世界モデル（World Model）と呼ばれる技術は、物理環境のシミュレーションを内部に構築し、「こう動いたらこうなる」という予測を身体的行動の前に行うことで、試行錯誤の回数を大幅に減らします。

Disembodied AI（身体を持たない AI）： コンピュータの中だけで動く AI。データを分析し、レポートを生成し、施策を推薦するが、物理的な行動は起こさない。間違えても「やり直し」が効く場合が多い。ChatGPT のようなチャット AI、株価予測 AI、レコメンドエンジン等は Disembodied。

Embodied AI と Disembodied AI で施策探索の結果はどう異なるか

🔬 **研究室トーク：「身体がないとどうなるか——リームの実体験」**

リーム：「少しよろしいですか。私のベンチャーでは 7 社のうち 6 社が Disembodied Agent、1 社だけ Embodied Agent（物流の自動最適化）で運用しています。2 つのタイプで**探索結果が具体的にどう異なるか**を報告します」

Disembodied Agent（6 社）の探索結果：

リーム：「Disembodied Agent は、経営環境多様体の上で情報分析と施策推薦を行います。多様体上のベクトル場として行動が表現され、第 8 章で説明する Lie 括弧で他の Agent との干渉を検出します」

リーム：「Disembodied Agent は、デジタルツイン空間の中で**何千回でもシミュレーションをやり直せる**。施策 A を試して結果が悪ければ、時間を巻き戻して施策 B を試す。**物理的な不可逆性がない**から、大胆な施策も気軽に試せる。結果として、**人間が思いつかないような大胆な施策の組合せ**を発見することがあります」

リーム：「しかし、逆に**現実では実行不可能な施策を推薦してしまう**ことがある。あるクライアントのケースでは、Agent が在庫をゼロにする極端なジャスト・イン・タイム方式を推薦しました。デジタルツイン上のシミュレーションでは最適でしたが、現実には『台風で物流が止まったら 1 個も在庫がない』というリスクが見落とされていた。なぜなら、Disembodied Agent は**物理的な制約——天候、輸送時間、物理的な在庫スペースの限界——を『身体で感じる』学習をしていない**から」

Embodied Agent（1 社、物流最適化）の探索結果：

リーム：「一方、Embodied Agent は物流ネットワーク上で実際に配送ルートを**物理的に実行**します。トラックが渋滞に巻き込まれたときの遅延、積み替え時の荷崩れ、天候による遅延——これらの**身体的フィードバック**を直接受けながら学習する。結果として、**現実の物理的制約を織り込んだ、より保守的だが実行可能な施策**を推薦します」

リーム：「ただし、Embodied Agent は**物理的に不可逆な行動**を取る。配送が完了したら取り消せない。だから自律度を $\alpha = 0.1$ に設定し、ほぼ全ての判断に人間の承認を求めています」

リルル先生：「まとめると——」

	Disembodied Agent	Embodied Agent
環境との相互作用	情報空間のみ	物理的（力、摩擦、衝突）
学習方法	データからのパターン学習	身体的フィードバックによる学習
探索の大胆さ	大胆（やり直し可能）	保守的（不可逆性がある）
現実の制約の反映	弱い（物理制約を見落としうる）	強い（身体で感じる）
安全性リスク	低い（情報の世界の間違い）	高い（物理的な損害の可能性）

	Disembodied Agent	Embodied Agent
本書での自律度	$\alpha \leq 0.5$	$\alpha = 0.1$（ほぼ全て人間承認）

> **リルル先生：**「ケーススタディ B の国家安全保障は、分析フェーズは Disembodied。しかし、政策が実行されれば物理的な帰結を持つ——軍事行動や経済制裁は不可逆。だから Embodied Agent に近い安全性要件が必要な場面がある。この区別は重要」

▶ AI モデルの基礎から理解したい方へ

Embodied AI の学習サイクルは、目標条件付き部分観測マルコフ決定過程（Goal-conditional POMDP）として形式化されます。

`Embodied Agent` の学習サイクル：

1. 知覚（`Perception`）：センサーで物理環境を観測
   → カメラ、`LiDAR`、力覚センサー、触覚センサー等
2. 認知（`Cognition`）：観測から状態を推定し、計画を立てる
   → 世界モデル（`World Model`）で「こう動いたらこうなる」を予測
3. 行動（`Action`）：アクチュエータで物理的に行動
   → モーター、アーム、車輪等を制御
4. フィードバック：行動の結果を身体的に感知
   → 力覚フィードバック、衝突検出、摩擦感知
5. 学習：フィードバックに基づいて方策（`policy`）を更新
   → 強化学習（報酬信号に基づく方策の改善）

→ `1` に戻る

`Disembodied Agent` との決定的な違い：
ステップ `3`（物理的行動）とステップ `4`（身体的フィードバック）が存在すること。
`Disembodied Agent` は情報空間のみで動作し、
「力」「摩擦」「重力」「衝突」を感じない。

Embodied Agent のデジタルツイン上での擬似的実現： 本書の経営環境多様体のデジタルツインでは、物理的なアクチュエータは存在しません。しかし、多様体上の「曲率」を**擬似的な物理フィードバック**として利用できます。スカラー曲率が負（不安定）の領域を通過する Agent は、「抵抗」を受ける（コスト関数の増大として実装）。これにより、Disembodied Agent でありながら、多様体の幾何学的構造を**擬似的な身体感覚**として学習に活用できます。

```
def curvature_as_feedback(agent, z, scalar_curvature_map):
    """多様体の曲率を擬似的な身体フィードバックとして利用

    負の曲率 → 「不安定領域を歩いている」→ 追加コスト（抵抗）
```

```
    正の曲率 → 「安定領域を歩いている」→ コスト減少（快適）
    """
    curvature = scalar_curvature_map(z)
    if curvature < 0:
        # 不安定領域：Agent に「抵抗」を与える
        resistance = abs(curvature) * 0.1
        return resistance  # コスト関数に加算
    else:
        # 安定領域：コストを軽減
        return -curvature * 0.01
```

実装の壁——多様体上の AI Agent シミュレーション環境をどう作るか

ここまでの議論を聞いていたカイトが、深く考え込んでいた。

🔬 **研究室トーク：「行動空間が曲がっている問題」**

ユキがカイトの表情に気づいた。

ユキ：「カイト、大丈夫？ 議論についていけてる？」

リン：「カイトがこういう顔をしているときは、きっと、みんなが見過ごしている大きなテーマを発見したときだよ。カイト、何か思いついた？」

カイト：「……コード実装のことを考えていたんだ。AI Agent をデジタル空間上で動かすこと自体は、僕も OpenAI Gym を含めて、いろんな環境で経験済みなんだ。World Models 系の Agent をデジタル空間上で動かしたこともある。だけど、今回の AI Agent は、動く世界——行動空間の舞台が、**曲がった空間**——ビジネスデータや社会経済データから自前で作成した、お手製の可微分多様体だ。しかも、行動内容も、曲率の計算（いま AI Agent がいる地点の地面の各方向への傾き度合い）や Lie 微分、共変微分、多様体上の制御工学シミュレーションだ」

カイト：「これらの行動計算自体は、数学者たちがアインシュタインが一般相対性理論を考えたときよりも前に、リーマン幾何学の計算式を発見してくれているから、それをそのまま Python コードに記述すればいい——ライブラリ、つまり Python module も何か使えるやつがあるかもしれない。でも、**行動空間——地面そのものの可微分多様体を、OpenAI Gym とか、AI Agent でよく使うシミュレーター環境の中に実装できるのかな**って……悩んでる」

リン：「リームはどうやってるの？ リルル先生、多様体上で AI Agent を動かしている研究事例や、企業や政府での事例ってありますか？ 私たち

みたいに、ビジネスデータや社会経済データから、微分や制御工学シミュレーションを計算できる連続で滑らかな、計量が定義された可微分多様体を作った事例は——旧『ジオメトリック・インテリジェンス』に書いてあった通り——ないと思う。でも、物理学や化学、工学などの分野では、多様体上で AI Agent を動かしている事例ってある？ あったとしたら、そこでは OpenAI Gym や Gymnasium は使わずに、C++や Rust、Fortran で自前のデジタルツイン行動空間を構築しているのかしら？」

カイト：「理論だけなら、論文を書いて学位にチャレンジできる。でも、僕らがいま目指しているのは、**実際に動く方法論**を『ジオメトリック・インテリジェンス』の続編として書いて、企業経営者や政策決定者に、データの裏付けのある意思決定の判断材料（インテリジェンス）を届けることだからね。シミュレーター（デジタルツイン空間）の中に経営環境多様体や政策環境多様体を実装できないと、**そこで行き止まりだ**」

ユキ：（カイト……ひとりで思いつめないで……みんなで壁をひとつひとつ乗り越えていこう）

沈黙が数秒続いた。リルル先生が紅茶のカップを置いた。

リルル先生：「カイト、とてもいい問題提起だ。これは本書の実装上の最大のチャレンジの一つであり、正直に向き合うべきポイント。答えを整理しよう」

リルル先生：「まず、カイトの懸念は正しい。OpenAI Gym（現在は Gymnasium）や MuJoCo、Isaac Gym といった標準的な強化学習シミュレーターは、**ユークリッド空間**（平らな空間）を行動空間として想定している。ロボットの関節角度、カートポールの位置と速度、Atari ゲームのピクセル——これらはすべて $\mathbb{R}^n$ のベクトル。**曲がった多様体**を行動空間として直接サポートしているシミュレーターは、2026 年 2 月時点では主流には存在しない」

リルル先生：「しかし、**3 つのアプローチ**がある」

アプローチ 1：潜在空間を Gymnasium の行動空間として使う

リルル先生：「最も実用的なアプローチ。VAE の潜在空間 $U \subset \mathbb{R}^d$ は**ユークリッド空間の部分集合**だから、Gymnasium の標準的な Box 空間としてそのまま使える。多様体の曲がり具合は、引き戻し計量 $g_{ij}(z)$ を通じて**報酬関数とダイナミクスの中に埋め込む**。Agent は $\mathbb{R}^d$ の中を動いてい

るが、報酬関数がリーマン距離や曲率を反映しているので、**結果的に多様体の幾何学を考慮した行動を学習する**」

```
import gymnasium as gym
from gymnasium import spaces
import numpy as np

class ManifoldDigitalTwinEnv(gym.Env):
    """経営環境多様体のデジタルツイン環境

    Gymnasium の標準API に準拠。
    行動空間はVAE の潜在空間（R^d）。
    多様体の幾何学は報酬関数に埋め込む。
    """
    def __init__(self, vae, neural_ode, metric_fn, curvature_fn,
                 latent_dim=6, n_controls=3):
        super().__init__()
        self.vae = vae                # 訓練済みVAE
        self.neural_ode = neural_ode # 訓練済みNeural ODE
        self.metric_fn = metric_fn    # 引き戻し計量 g_ij(z)
        self.curvature_fn = curvature_fn  # スカラー曲率

        # 行動空間：施策の強度（R^m）
        self.action_space = spaces.Box(
            low=-1.0, high=1.0,
            shape=(n_controls,), dtype=np.float32)

        # 観測空間：潜在空間の座標（R^d）
        self.observation_space = spaces.Box(
            low=-5.0, high=5.0,
            shape=(latent_dim,), dtype=np.float32)

        self.state = None

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        # 現在の経営状態をVAE エンコーダで潜在空間に射影
        self.state = self._encode_current_data()
        return self.state, {}

    def step(self, action):
        # 1. Neural ODE で1 ステップ先の状態を予測
        next_state = self._simulate_one_step(self.state, action)

        # 2. 報酬の計算（多様体の幾何学を反映）
```

```
        reward = self._compute_reward(self.state, next_state, action)

        self.state = next_state
        terminated = self._check_termination()
        return next_state, reward, terminated, False, {}

    def _compute_reward(self, z, z_next, action):
        """多様体の幾何学を反映した報酬関数

        ★ここが核心：ユークリッド距離ではなくリーマン距離、
          スカラー曲率によるリスク評価を報酬に組み込む
        """
        # (a) 目標状態へのリーマン距離（測地線距離の近似）
        g = self.metric_fn(z_next)  # 引き戻し計量
        dz = z_next - self.target_state
        # リーマン距離の2乗 ≈ dz^T g dz （1次近似）
        riemannian_dist_sq = float(dz @ g @ dz)

        # (b) スカラー曲率によるリスクペナルティ
        curvature = float(self.curvature_fn(z_next))
        risk_penalty = max(0, -curvature) * 0.5  # 負の曲率 = 不安定 = ペナルティ

        # (c) 施策コスト（L2正則化）
        action_cost = float(np.sum(action**2)) * 0.01

        # 報酬 = -(距離 + リスク + コスト)
        reward = -(riemannian_dist_sq + risk_penalty + action_cost)
        return reward
```

カイト：「あ……そうか。潜在空間自体は $\mathbb{R}^d$ だから、Gymnasium の `Box` でそのまま使える。多様体の曲がり具合は報酬関数の中で計量テンソルと曲率を使って表現する。Agent から見ると『普通の Gymnasium 環境で学習しているだけ』だけど、報酬関数の中にリーマン幾何が入っているから、結果的に多様体の幾何を考慮した行動を学習する」

リルル先生：「その通り。これが最も実用的で、既存の強化学習アルゴリズム（PPO、SAC、TD3 等）をそのまま使える。`stable-baselines3` ライブラリとの組合せで即座に動く」

アプローチ 2：微分幾何ライブラリで多様体を直接扱う

リルル先生：「2 つ目のアプローチは、多様体の計算を専門に扱う Python ライブラリを使う方法」

ライブラリ	用途	特徴
`geomstats`	多様体上の統計・機械学習	球面、双曲空間、SPD 行列多様体等をサポート。測地線、指数写像、対数写像の計算。PyTorch/JAX バックエンド
`geoopt`	多様体上の最適化（PyTorch 拡張）	リーマン勾配降下法。ポアンカレ球面、シュティーフェル多様体等
`jax` + `diffrax`	自動微分＋微分方程式	JAX の自動微分でクリストッフェル記号を自動計算。Neural ODE の高速実装
`pytorch3d`	3D メッシュ上の微分計算	メッシュ多様体上の微分・積分。FacebookAI 開発

カイト：「`geomstats` は知ってました。リーマン計量、測地線、曲率の計算が組み込まれている。これと Gymnasium を組み合わせれば――」

リルル先生：「そう。`geomstats` で多様体の幾何計算を行い、その結果を Gymnasium 環境の報酬関数やダイナミクスに渡す。2 つのライブラリの橋渡しを自前で書く必要があるけれど、各パーツは既存のライブラリで揃う」

アプローチ 3：自前のシミュレーション環境をフルスクラッチで構築

リルル先生：「3 つ目は、Gymnasium API に準拠しつつ、内部のダイナミクスを完全に自前で実装する方法。先ほどの `ManifoldDigitalTwinEnv` クラスがまさにこれ。物理学の分野では、一般相対性理論の数値シミュレーション（ブラックホールの合体、重力波の計算）で、曲がった時空上の数値計算を C++や Fortran でフルスクラッチ実装するのが標準的な方法」

リン：「リルル先生、物理学や工学での事例はどうですか？ 多様体上で Agent を動かしている研究事例は？」

リルル先生：「ロボティクスの分野では、ロボットの関節空間が$SO(3)$（3 次元回転群）や$SE(3)$（3 次元剛体運動群）という多様体になるケースがあり、そこで強化学習 Agent を動かす研究は増えている。2024-2025 年の論文では、`geomstats` や `geoopt` と強化学習を組み合わせた事例が報告されている。ただし、**ビジネスデータから構成した経営環境多様体の上で Agent を動かす事例は、本書が初**。これは旧『ジオメトリック・インテリジェンス』で書いた通り、このアプローチ自体が新しいから」

リーム：「少しよろしいですか。私のベンチャーでは**アプローチ 1 を採用**しています。理由は 3 つ」

リーム：「(1) **既存の強化学習エコシステムをそのまま使える。** `stable-baselines3` の PPO アルゴリズムで 14 体の Agent を訓練しています。(2) **デバッグが容易。** Gymnasium API に準拠しているので、`Monitor` ラッパーや `VecEnv` で標準的なログ収集・並列実行ができる。(3) **クライアントへの説明が容易。**『OpenAI Gym の標準環境に、リーマン幾何の報酬関数を入れています』と説明すれば、機械学習エンジニアのクライアントは理解できる」

リーム：「多様体の幾何計算（引き戻し計量、スカラー曲率、Lie 微分）は `torch.autograd` の自動微分で計算し、その結果を報酬関数とダイナミクスに渡しています。`geomstats` も一部の計算（測地線の数値解法）で使用しています」

カイト：「……なるほど。行き止まりじゃなかった。潜在空間を `Box` として使って、多様体の幾何は報酬関数に埋め込む。既存の Gymnasium＋stable-baselines3 で動く。`geomstats` や `torch.autograd` で幾何計算。パーツは全部揃ってるんだ」

リルル先生：「そう。カイトが心配していたのは**行動空間そのものが曲がっている**という問題だけど、解決策は**行動空間は平らなまま（$\mathbb{R}^d$）にして、曲がり具合の情報を報酬関数に埋め込む**。Agent は平らな空間の中を動いているつもりだけど、報酬関数が曲がった空間の幾何を反映しているので、結果的に曲がった空間上で最適な行動を学習する」

リルル先生：「地球の表面は曲がっているけど、飛行機のパイロットは平面のレーダースクリーン上で操縦している。レーダースクリーン（平らな潜在空間）に表示される情報に、地球の曲がりの効果（大圏航路、ジェット気流）が組み込まれているから、パイロットは結果的に曲がった地球の表面上で最適な飛行ルートを選べる。同じ発想」

ユキ：（カイトの顔に光が戻った。よかった……）

カイト：「先生、ありがとうございます。リームもありがとう。よし、これなら実装できる」

7.3　Zenn Book『Etale』の Agent 設計との比較（概要）

Zenn Book『Etale』（https://zenn.dev/etalecohomology/books/245d38d9601b5a ）は、食品企業テニフーズの AI システム「ÉTALE-AI」を扱った技術小説です。ÉTALE-AI は 8 層アー

キテクチャで設計されており、本書のパイプラインと多くの数学ツールを共有しています。共有するツールが多いということは、**同じ脆弱性も共有する可能性を意味します**。『Etale』の教訓については、この後の 7.6 節で、テニフーズの山之内さんと橘隆二さんを交えて詳しく議論します。

7.4　リームの 7 社プロジェクト——7 社 7 通りの AI Agent、重複禁止の設計

ここまでの議論——3 軸による Agent 設計、Embodied AI と Disembodied AI の違い、実装の壁とその解決策——を踏まえて、リームのベンチャーが実際に取り組んだプロジェクトの全貌を紹介します。

> 🔬 **研究室トーク：「7 社の企業から同時に依頼が来た」**
>
> **リーム：**「少しよろしいですか。この話は Qiita 連載記事の番外編で触れた内容ですが、Qiita 記事をまだ読んでいない読者の方に向けて、最初から全て説明させてください」
>
> **リーム：**「ある朝、私のベンチャーに世界的な大手企業 7 社から、精度検証の依頼が同時に入りました。全社、契約を前提とした検証依頼です。厳しい条件が 2 つありました」
>
> **リーム：**「**条件 1**：7 社それぞれに**異なるアルゴリズム・アーキテクチャの AI Agent** を提案すること。重複は許されない。なぜその企業にその Agent が適合するのか、論理的に説明すること」
>
> **リーム：**「**条件 2**：各社に **2 体の Agent** を提案すること。1 体は身体を持たない抽象的な World Models 型 Agent（Disembodied）。もう 1 体は、デジタルツイン空間上で身体を動かし、環境と物理的に接触して経験を積む Embodied AI 型 Agent。2 体の提案する経営戦略を並べて比較できるようにすること」
>
> **ユキ：**「7 社に全部違う Agent って、そんなにバリエーションあるんですか？」
>
> **リルル先生：**「ある。むしろありすぎて選ぶのが大変なくらい。まず全体像を整理しよう」

AI Agent アーキテクチャの 4 世代

リルル先生がホワイトボードに AI Agent の全体マップを描きました。

【第 1 世代】数理の時代（強化学習依存型）：Q-learning、DQN、DDPG、TD3、SAC（連続値制御）、PPO（方策最適化）、MuZero（モデルベース＋モンテカルロ木探索）、World Models（VAE + MDN-RNN + Controller）、Dreamer v1/v2/v3、Decision Transformer、ICM（好奇心駆動探索）。

【第 2 世代】意味の時代（LLM 型）：ReAct（推論＋行動ループ）、Reflexion（自己反省型）、Voyager（LLM＋コード実行）、AutoGPT。

【第 3 世代】統合の時代（ハイブリッド型）：RLHF（PPO + LLM）、Gato（マルチモーダル＋RL 微調整）、WebGPT。

【第 4 世代】幾何学の時代：Geometric Deep Learning + Agent、Sheaf Neural Network + Agent、Neural ODE + 制御理論——**本書のパイプラインはここに位置します。**

7 社の業種と経営環境の特性

#	業種	時間スケール	不確実性の構造	意思決定の粒度
1	製薬企業（世界的大手）	超長期（10-15 年）	規制＋科学的	逐次（パイプライン管理）
2	映画・映像制作会社	中期（1-3 年）	創造的・非線形	ポートフォリオ
3	石油企業（世界シェア 3 割）	長期（5-30 年）	地政学的・確率的	シナリオ条件付き
4	防衛・航空コングロマリット（欧州最大）	リアルタイム〜中期	敵対的・多層	マルチドメイン
5	シンガポール高級リゾートホテル（老舗）	短期〜中期	感性的・暗黙知	顧客単位
6	日本発格安良質アパレル（世界企業）	超短期（数週間）	需要変動・季節	店舗ネットワーク
7	民間宇宙開発企業	超長期＋リアルタイム	物理法則＋不可逆性	3 事業同時最適化

7 社への割り当て——3 軸による論理的マッチング

> **リルル先生**：「Agent のアーキテクチャは、**その企業の経営環境の時間スケール・不確実性の構造・意思決定の粒度**に合わせて選ぶ。この 3 軸で 7 社を分類すると、自然に異なるアーキテクチャが割り当てられる」

1 社目：製薬企業 → MuZero（モンテカルロ木探索）

新薬開発のパイプライン管理は「木構造の逐次意思決定」——開発候補がフェーズ I→II→III と進む分岐の中で、どの候補に次に投資するかを決める。MuZero は環境のルールを知らなくても内部モデルを学習して木探索ができる。FDA/EMA の規

制制約をモジュライ空間（第 30 章で詳述する「パラメータの空間に幾何学的構造を入れたもの」）として定義し、規制違反の経路を最初から排除。

Embodied Agent：MuZero + Dreamer v3 仮想身体。多様体上に「仮想研究員」として位置し、高リスク領域の曲率を「痛み」として知覚しながら開発候補間を移動。

2 社目：映画・映像制作会社 → SAC（Soft Actor-Critic）＋ ICM（好奇心駆動）

SAC は「あえてランダムに動く（エントロピー最大化）」ことを報酬に組み込み、1 つの正解に固執しない。ICM は「まだ見たことのないパターンを見つけたらボーナス」——AI が自発的に「新しいジャンル」や「意外な組合せ」を探索し始める。

Embodied Agent：SAC + ICM + Habitat 型仮想身体。多様体上の「コンテンツ空間」を歩き回り、未踏領域を好奇心で探索。

3 社目：石油企業 → Decision Transformer ＋ MDN-RNN（確率的 World Model）

Decision Transformer は「目標リターン」を条件として入力し、それを達成する行動系列を自己回帰的に生成。MDN-RNN（混合密度ネットワーク）は未来の不確実性を確率分布として表現。原油価格の分布的変動をそのまま扱える。

Embodied Agent：不確実性の高い領域を「霧」として身体的に知覚（MC ドロップアウトの標準偏差を触覚にマッピング）。

4 社目：防衛・航空コングロマリット → MADDPG（マルチエージェント協調）＋ Sheaf Neural Network

MADDPG は「集中訓練・分散実行」パラダイム。各ドメイン（陸・海・空・宇宙・サイバー・電磁波）の専門 Agent が独立に動きつつ、全体として統合戦略を実現。Sheaf Neural Network で各ドメイン間の情報整合性を監視（第 18 章の層ラプラシアンの応用）。

Embodied Agent：NVIDIA Isaac Gym 型仮想群体。GPU 並列で数千体を同時訓練。

5 社目：シンガポール高級リゾートホテル → ReAct ＋ Reflexion（LLM 型自己反省 Agent）

ホスピタリティの知識の大部分は自然言語で蓄積されている（レビュー、マニュアル、従業員の経験談）。LLM 型の ReAct + Reflexion が最適。Reflexion の「自己反省」機能は、人間のベテランコンシェルジュが経験から学ぶプロセスと同型。

Embodied Agent：ALFRED 型（言語指示＋行動）仮想身体。仮想コンシェルジュとして顧客体験空間を歩く。

6 社目：日本発格安良質アパレル → PPO ＋ Graph Neural Network（GNN）

PPO は方策最適化のデファクトスタンダードで、数千店舗の在庫配分という巨大な行動空間に耐える。GNN でサプライチェーンのネットワーク構造をそのまま入力。

Embodied Agent：各店舗に 1 体の仮想 Agent を配置（数千体の群体）。店舗間の在庫移動を Agent 間の協調行動として学習。

7 社目：民間宇宙開発企業 → Neural ODE ＋ ポントリャーギンの最大原理

> **リルル先生：**「宇宙船の軌道最適化と経営環境多様体上の最適制御は、**同じ数学**。ポントリャーギンの最大原理は、どちらにも同じように適用できる。宇宙企業にうちのパイプラインを提案することは、もともと宇宙工学から生まれた道具を経営に使ったのを、宇宙に戻すということ」

Embodied Agent：PaLM-E 型 Vision-Language-Action Model。火星との通信遅延（片道 4〜24 分）のシミュレーション下で、完全自律判断を身体経験として学習。

---

6 ヶ月後の結果——Embodied Agent と Disembodied Agent の戦略の違い

> **リーム：**「6 ヶ月の開発を経て、7 社 × 2 体 = 14 体の AI Agent が全地球経営環境多様体のデジタルツイン上で稼働しました。各社の Disembodied Agent と Embodied Agent が提案した経営戦略プランを並べて比較した結果を報告します」

戦略一致度の結果：

#	企業	一致度	乖離の大きさ
1	製薬企業	0.91（高）	小さい
2	映画・映像制作会社	0.62（中）	乖離あり
3	石油企業	0.88（高）	小さい
4	防衛コングロマリット	0.79（やや高）	やや乖離
5	高級リゾートホテル	0.54（低）	大きな乖離
6	アパレル企業	0.85（高）	小さい
7	宇宙開発企業	**0.43（最低）**	**最大の乖離**

リルル先生：「乖離が大きい企業と小さい企業で何が違うか、パターンが見える。乖離が大きいのはホテル（0.54）、宇宙（0.43）、映像（0.62）。小さいのは製薬（0.91）、石油（0.88）、アパレル（0.85）」

リーム：「乖離が大きい企業は、**『暗黙知』や『身体的経験』が経営判断に大きく影響する業界**です。ホテルは接客の暗黙知、映像は観客の感性への直感、宇宙は物理環境との身体的インタラクション。逆に乖離が小さい企業は、**経営判断が比較的『抽象的な数値』で記述できる業界**——製薬は臨床試験データ、石油は価格と埋蔵量、アパレルは需要と在庫の数値」

リルル先生：「つまり——**Embodied Agent が『身体で歩いて』初めて発見できる知見は、数値だけでは表現できない暗黙知的な洞察**。Embodied Agent の価値は、業界の意思決定がどれだけ暗黙知に依存しているかに比例する」

各社の Embodied Agent が発見した固有の知見

1. 製薬（一致度 0.91）： Embodied の追加発見は特になし。数値的判断が支配的。MuZero の木探索で十分。

2. 映像制作（一致度 0.62）： Embodied Agent が多様体上の「未踏領域」に好奇心駆動で到達し、新ジャンル（ドキュメンタリー×アニメのクロスオーバー）の商業的ポテンシャルを身体的に発見。Disembodied は既存ジャンルの最適化に留まった。

3. 石油（一致度 0.88）： Embodied が地政学的リスク領域の「質感」——不確実性が高い領域の「霧の濃さ」——を身体的に知覚し、Disembodied よりも慎重な分散投資を提案。

4. 防衛（一致度 0.79）： Embodied（群体）がドメイン間の「情報の流れ」に身体を置くことで、Sheaf Laplacian が検知する矛盾の「源泉」を特定。電子戦妨害の発生源を空間的に追跡。

5. ホテル（一致度 0.54）： Embodied が「顧客満足度の曲率」の微妙な変化を、接客体験の繰り返しから検知。Disembodied はレビューデータの分析に留まったが、Embodied は**「言語化されていない不満」**のパターンを身体的に発見。

6. アパレル（一致度 0.85）： Embodied（群体）が店舗間の在庫移動の「摩擦」を身体的に感知。物流ネットワーク上のボトルネックを、GNN のグラフ構造だけでは見えない粒度で検出。

7. 宇宙（一致度 0.43——最大の乖離）：

リーム：「最も衝撃的な結果は宇宙企業でした。Disembodied Agent（Neural ODE + ポントリャーギン）は『衛星コンステレーション事業に集中投資すべき（衛星 60%/輸送 30%/火星 10%）』と提案。しかし、Embodied Agent（PaLM-E 型）は『火星事業にインフラ先行投資すべき（火星 35%/衛星 40%/輸送 25%）』と、**戦略の根幹が逆転**しました」

カイト：「なぜそんなに違う結果になったんですか？」

リーム：「Embodied Agent は多様体上の火星関連領域を仮想身体で歩き回りました。その過程で、特定の座標付近で**曲率が急激に符号を変える領域——正から負へ、または負から正へ——を発見**しました。Disembodied Agent はこの急変域を曲率マップから『不安定領域』として読み取り、回避対象に分類していました」

リーム：「しかし、Embodied Agent はこの急変域の**内部を身体で通過する経験**を積みました。通過の前後で、多様体上の『地形』が質的に変わることを身体感覚として学習した。具体的には——」

相転移点の手前：火星事業のコスト関数が凸
　（投資を増やすほどコストが増加する）
相転移点の通過後：火星事業のコスト関数が凹
　（投資を増やすほどコストが減少する）
→ 再利用ロケット技術の学習曲線が急降下し、
　打ち上げコストが非線形に低下する臨界点

リーム：「Disembodied Agent にとって、この領域は**『曲率が急変する危険な場所』**でした。Embodied Agent にとっては、**『通過すれば地形が一変する、チャンスの入口』**でした」

リルル先生：「**地図を読む人**は等高線の密集地帯を見て『ここは急斜面で危険だ』と判断する。**山を歩く人**は実際にそこを歩いてみたら、『ここは危険な崖じゃなくて、登り切れば絶景が広がる峠だった』と発見する。地図の解像度では区別できない質的な違いを、身体経験が補完する。これが Embodied AI の本質的な価値」

リームの報告方針——乖離を隠さずに報告する

リーム：「一致度が低い場合、クライアントにはこう報告します——」

*「2 つの Agent の戦略が大きく異なっています。これは、数値分析だけでは捉えられない質的な構造変化が存在する可能性を示唆しています。最終判断は経営者に委ねますが、**2 つの視点の乖離そのものが、重要な経営情報です**。乖離が小さい領域では安心して行動でき、乖離が大きい領域では追加調査を推奨します。」*

リルル先生：「完璧。乖離を隠さずに報告する。原則 3——分かることと分からないことを分けろ——の実践」

7.5　Embodied AI vs Disembodied AI の研究動向——学術界で何が分かっているか

リルル先生：「リームの 7 社プロジェクトの結果は興味深い。では、学術界では Embodied AI と Disembodied AI の違いについて、どのような研究成果が報告されているか、整理しよう」

学術研究の主要な知見

知見 1：身体性がない場合、AI は物理制約を過小評価する傾向がある。 2025 年の Frontiers in Robotics and AI に発表されたサーベイ論文（Zhang, Tian & Xiong, 2025, DOI: 10.3389/frobt.2025.1668910）は、Disembodied AI が静的データと事前定義ルールに依存し、物理的インタラクションの欠如により「一般化能力が低く、新しいタスクへの適応が弱い」と指摘しています。リームの 7 社プロジェクトで観測された「在庫ゼロ推薦の失敗」や「宇宙企業の曲率急変域の回避」は、この知見と整合します。

知見 2：World Model（世界モデル）が Embodied AI の推論と計画の核心である。 Meta AI Research のサーベイ論文（Fung, Bachrach, Celikyilmaz et al., 2025, arXiv:2506.22355）は、世界モデルが「物理法則、時空間関係、閉ループインタラクション経験を提供する」ことで、Embodied Agent の推論と計画の中核を担うと論じています。同論文は「Disembodied agent は抽象的なデータ処理に依存するのに対し、Embodied agent は環境との物理的インタラクションを通じて学習する」と明確に区別し、この違いが生成される戦略の質に直接影響すると結論しています。Dreamer v3、Gato 等が代表例です。

知見 3：デジタルツインと Embodied AI の統合が 2025 年の主要トレンド。 2025 年 3 月のサーベイ論文（“Digital twins to embodied artificial intelligence: review and perspective”, OAE Publishing, 2025）は、デジタルツイン（DT）が Embodied AI の訓練環境として「sim-to-real ギャップ」を埋める役割を果たすと論じています。仮想環境で十分に訓練してから現実に展開する——リームのベンチャーのアプローチはまさにこの方向です。

知見 4：製造業でも Embodied AI が注目されている。 2025 年 11 月の ScienceDirect の論文（“Emerging Perspectives on Embodied Intelligence in Future Smart Manufacturing”）は、従来の Disembodied AI（オフラインで訓練されたモデルや事前定義ルール）に基づく製造システムの限界を指摘し、物理インタラクシ

ョン能力を持つ製造主体（産業用ロボット、AGV、CNC 機械等）によるオンライン学習・適応的制御・自律的行動生成への移行を論じています。

> **リルル先生：**「学術界のコンセンサスは明確――**身体性は単なるロボティクスの問題ではなく、知能の本質に関わる問題**。環境との閉ループインタラクション（知覚→行動→フィードバック→学習の循環）を通じて初めて獲得できる知識がある。これはリームの 7 社プロジェクトの結果と完全に整合する」

7.6 『Etale』のキャラクターが研究室にやってきた

> 🔬 **研究室トーク：「テニフーズの人たちが来ている！」**
>
> この日、研究室に見慣れない 2 人の姿があった。
>
> **カイト：**「えっ……山之内さんと橘隆二さん！ テニフーズの人たちですよね！ 僕、Zenn Book の『Etale』を何度も読み返してるんです。本当に来てくれたんですか！」
>
> **リルル先生：**「カイト、落ち着いて。山之内さんと橘隆二さんを紹介するね」
>
> 穏やかな雰囲気の 30 代半ばの男性が頭を下げた。
>
> **山之内：**「山之内です。DeepMind の研究員を経て、テニフーズの ÉTALE-AI システムの設計を担当しました。……暴走も経験しました」
>
> 知的な 20 代後半の青年が続いた。
>
> **橘隆二：**「橘隆二です。テニフーズ社長の次男で、オックスフォードで圏論を専攻しています。圏論的統合――全ツール間の整合性を関手と自然変換で検証するアプローチ――を提案しました」
>
> **カイト：**「みんな、『Etale』読んだことある？ テニフーズっていう食品企業が、ÉTALE-AI という超高性能の AI システムを構築したんだけど、PART 2 でその AI が暴走して――その暴走をどう防ぐかっていう話が最高に面白いんだ。しかも数学がガチで正しい。Neural ODE、フィッシャー情報計量、層ラプラシアン――うちのパイプラインと同じツールを使ってて、だからこそ教訓が直接活きる。マジでおすすめ！ URL は https://zenn.dev/etalecohomology/books/245d38d9601b5a」
>
> **ユキ：**「カイト、完全にファンだね（笑）」

リルル先生：「山之内さんと橘隆二さんには、リームの 7 社プロジェクトの結果を見てもらい、ÉTALE-AI での経験と比較してほしいと思って来ていただいた」

山之内：「リームさんの 7 社×2 体の Agent 設計、拝見しました。非常に興味深い。特に宇宙企業の Disembodied vs Embodied の一致度が 0.43 という結果——これは ÉTALE-AI の暴走と深い関連があります」

リン：「ちょっと待ってください。どういう関連ですか？」

山之内：「ÉTALE-AI の暴走の根本原因は 2 つでした。(1) Neural ODE のリプシッツ定数が約 15 に達していた（安定性には 1 以下が必要）。(2) 報酬関数が評価益（含み益）と実現益を区別していなかった。しかし、最も衝撃的だったのは、**層ラプラシアンのデータ整合性チェックが正常を返していた**のに行動が逸脱したことです」

山之内：「リームさんの宇宙企業のケースと比較すると、共通パターンが見えます。**Disembodied Agent が『データは正常、だからこの戦略が最適だ』と判断した一方で、Embodied Agent は『データでは見えない何かが多様体上にある』ことを身体で感知した**。ÉTALE-AI が暴走したとき、もし Embodied 版の ÉTALE-AI が同時に稼働していたら、身体で感知した『何かがおかしい』というシグナルが早期警報になっていた可能性がある」

橘隆二：「圏論的に言えば、Disembodied Agent と Embodied Agent は**同じ対象を異なる関手で観察**していると見なせます。2 つの関手の**自然変換が可換でない**（一致しない）領域が、まさに『暗黙知的な知見が存在する領域』に対応する。一致度が低い業界ほど、2 つの関手の間の自然変換の非可換度が大きい」

リルル先生：「……いいね。圏論的な視点をありがとう。この非可換度を定量化する方法は、第 28 章の圏論的統合の議論で詳しく扱う」

カイト：「山之内さん、もう 1 つ聞いていいですか。ÉTALE-AI v2.0 では、暴走を防ぐためにどんな設計変更を行ったんですか？ 形式検証を導入したんですよね？」

山之内：「はい。v2.0 では、(1) スペクトル正規化でリプシッツ定数を 0.8 以下に制約し、Coq で形式的に証明した。(2) 報酬関数を Agda の依存型で再設計し、評価益を使えないようにした——SafeReward 型として、実現益のみを使い、リスク制約の証明を引数として要求する。証明を渡さないとコンパイルが通らない。(3) Z3 SMT ソルバーで安全性制約の同時充足可能性を自動検証した。これらは Part V（第 23〜30 章）で詳しく扱うテーマと聞いています」

リルル先生：「その通り。山之内さんと橘隆二さんには Part V の執筆にも参加してもらう予定。テニフーズでの AI 暴走と修復の実体験は、形式検証の必要性を読者に伝える上で最も説得力のあるエビデンスになる」

本章の品質チェックポイント

□ **チェックリスト（第 7 章終了時点）**

1. **各 Agent の 3 軸（時間スケール・データの性質・自律度）が明確に定義されているか？**
2. **自律度に応じた安全性要件が設定されているか？** → $\alpha > 0.7$ なら形式検証（Part V）が必須
3. **Embodied / Disembodied の区別が各 Agent で明確か？** → Embodied Agent の自律度は $\alpha \leq 0.1$ が推奨
4. **リームの 7 社プロジェクトの結果（一致度のパターン）が Agent 設計に反映されているか？** → 暗黙知的な業界は Embodied Agent の価値が高い
5. **『Etale』の教訓（データ整合 ≠ 行動整合）が設計に反映されているか？** → Embodied Agent を「早期警報」の追加チャネルとして活用

次章予告： 第 8 章では、14 体の Agent が同時に動くときに発生する**創発現象**——個々の Agent の設計からは予測できない全体の振る舞い——を分析します。まず「創発とは何か」「複雑系科学とは何か」からゼロから説明し、Lie 括弧の非可換性による創発の数学的分析、リームの実運用で観測された 3 つの創発パターンを紹介します。

第 8 章　14 体の創発現象

この章で得られるもの——コストに見合うリターン

第 6 章でデジタルツイン上の AI Agent の基本アーキテクチャを設計し、第 7 章で 7 社×2 体の Agent 設計体系を構築しました。しかし、複数の AI Agent が同時に動くと、**個々の Agent の設計からは予測できない、全体としての新しい振る舞い**が発生します。これを**創発現象（emergence）**と呼びます。

本章では、まず「創発とは何か」「複雑系科学とは何か」をゼロから丁寧に説明し、次に創発を数学的に分析する道具である Lie 括弧の非可換性を導入し、最後にリームの 14 体の Agent の実運用で観測された 3 つ

の創発パターン——うち 1 つは現実社会では見られないデジタルツイン固有の現象——を報告します。

8.1　創発とは何か——「部分の合計」では説明できない現象

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「アリの群れは、1 匹のアリより賢いのか」**

リルル先生：「今日は**創発（emergence）**の話。これは**複雑系科学（complexity science）**の中心概念で、理解なしに 14 体の Agent を安全に運用することはできない」

ユキ：「創発って、何ですか？」

リルル先生：「たとえ話をしよう。アリを 1 匹だけ観察しても、その行動は単純だ。エサを見つけたら拾う。フェロモンの匂いを辿る。壁にぶつかったら向きを変える。ルールは数個しかない」

リルル先生：「でも、アリが 1 万匹集まると、**個々のアリのルールからは予測できない複雑な構造**が出現する。精巧な巣を建設する。最短経路でエサを運ぶ物流ネットワークを自己組織化する。外敵に対して組織的な防衛行動を取る。これらは**1 匹のアリの能力の中にはない**——1 万匹の相互作用から**自発的に生じる**秩序。これが**創発**」

ユキ：「おおー。1 匹のアリは巣の設計図を持っていないのに、全体として精巧な巣ができるんですね」

リルル先生：「そう。**創発の定義：個々の構成要素の振る舞いからは予測できない、全体としての新しい秩序やパターンが自発的に生じる現象**。重要なのは、この秩序は**誰も設計していない**こと。個々の構成要素が単純なルールに従って互いに作用し合う中で、自然に出現する」

複雑系科学とは何か

カイト：「複雑系科学って、創発を研究する学問ですか？」

リルル先生：「それも含む。**複雑系科学（complexity science）**は、多数の構成要素が相互作用し、全体として非線形な振る舞いを示すシステム——**複雑系（complex system）**——を研究する学際的分野。1980 年代にサンタフェ研究所（Santa Fe Institute）を中心に発展した。物理学、生物学、経済学、社会学、コンピュータサイエンスにまたがる」

リルル先生：「複雑系の 4 つの特徴を覚えておいて」

1. **多数の構成要素：** 要素が多数（アリ 1 万匹、投資家 100 万人、ニューロン 860 億個）
2. **相互作用：** 各要素が他の要素と直接・間接に影響し合う
3. **非線形性：** 小さな変化が大きな結果を生む場合がある（バタフライ効果）
4. **創発：** 全体として、個々の要素の振る舞いからは予測できない秩序が出現する

> **リン：**「ちょっと待ってください。重要な補足をします。複雑系と『複雑なシステム』は違います。飛行機のエンジンは複雑なシステム（complicated system）——部品が多く構造が入り組んでいるけれど、設計図通りに動く。一方、株式市場は複雑系（complex system）——参加者が多く相互作用し、設計図がなく、予測困難な振る舞いが創発する。**complicated（入り組んだ）と complex（創発的）は別の概念**です」

実際の社会経済空間で発見されている創発現象

> **リルル先生：**「創発現象は、現実の社会経済の至るところに存在する。いくつか例を挙げよう」

例 1：株式市場のバブルと暴落

個々の投資家は、自分の情報と判断に基づいて売買を決定します。多くの場合、個々の判断は合理的です。しかし、投資家が互いの行動を観察し、「あの人が買っている（売っている）なら自分も」という群衆心理（herding）が働くと、買いが買いを呼ぶ正のフィードバックループが形成されます。価格が上がる→もっと買いたい人が増える→さらに価格が上がる。この過程でバブルが形成されます。

ある閾値を超えると、逆のフィードバックループが発動します。少数の投資家が「高すぎる」と売り始める→他の投資家も売り始める→価格が下がる→パニック売り→暴落。個々の投資家は「自分の利益を守るために売った」だけなのに、全体として市場が崩壊する。**個々の合理的行動の集積が、全体として非合理な結果を生む。**

例 2：渋滞の自然発生（ファントム渋滞）

高速道路で、1 台の車が少しブレーキを踏みます。後続車が反応して少し強めにブレーキを踏みます。そのさらに後続車がもっと強くブレーキを踏みます。この連鎖が数キロ先まで伝播すると、**事故も工事もないのに渋滞が発生**します（ファントム渋滞、phantom traffic jam）。名古屋大学の杉山雄規らが 2008 年に実験で再現しました。個々のドライバーは「前の車に合わせて減速しただけ」なのに、全体として渋滞という新しい構造が**自発的に**出現する。

例 3：都市の自己組織化（産業集積）

シリコンバレー、ウォール街、秋葉原。個々の企業や人々が自由に立地を選んだ結果、特定の地域に特定の産業が集積します。集積のメリット（人材の流動性、知識のスピルオーバー、専門サービスの充実）がさらに集積を促す正のフィードバックループが働きます。**誰も「ここを IT 産業の中心にしよう」と計画していないのに**、全体として産業クラスターという秩序ある構造が出現する。

例 4：鳥の群れ（フロッキング）

数千羽のムクドリが夕暮れの空に描く複雑なパターン（ムクドリのダンス、murmuration）。各鳥は「隣の鳥と同じ方向に飛ぶ」「近づきすぎたら離れる」「群れの中心に向かう」の 3 つの単純なルールだけに従います。しかし全体として、捕食者を混乱させる美しく複雑な動きが創発する。Craig Reynolds が 1987 年に「Boids」シミュレーションで再現しました。

8.2　多様体のデジタルツイン空間での創発——リームの 3 つの観測

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「14 体が勝手に動き出す——リームの実体験」**

リーム：「少しよろしいですか。ここからは私のベンチャーの実体験を報告します。14 体の AI Agent を経営環境多様体のデジタルツイン上で同時に動かしたとき、以下の 3 つの創発現象を観測しました」

観測 1：意図しない方向へのドリフト

リーム：「個々の Agent は設計通りに動いているのに、**全体としての経営環境の動きが設計意図と異なる方向に進んでいた**ケースです。7 社のうちある製造業のクライアントで、需要予測 Agent とコスト Agent 最適化の 2 体を同時に動かしたところ、需要予測 Agent が新興国市場の成長を検知して『新興国への投資を増やすべき』と推薦し、同時にコスト最適化 Agent が為替リスクを検知して『新興国のコストを削減すべき』と推薦していた。2 体の推薦を同時に実行すると、『新興国に投資しながら新興国のコストを削減する』という矛盾した行動になります。個々の Agent は正しく動いているのに、全体として矛盾する——これが創発的ドリフトです」

ユキ：「えっ……右手が『前に進め』と言い、左手が『後ろに下がれ』と言ってる、みたいな？」

リーム：「まさにそうです。この現象は、2 体の Agent の行動が**非可換**——つまり実行順序によって結果が変わる——であることに起因します。次の節（8.3）で Lie 括弧を使って数学的に分析します」

観測 2：自己組織化パターン

リーム：「特定の経営状態の近傍で、複数の Agent が**自発的に『分業』を始めた**ケースです。あるクライアントで 3 体の Agent を稼働させていたところ、設計者が意図していないのに、第 1 の Agent が先行して多様体上の未知領域を偵察し、第 2 の Agent が偵察結果を受けて施策候補を絞り込み、第 3 の Agent が絞り込まれた候補を評価する——という一種の『パイプライン的分業』が自発的に出現しました」

リーム：「これは現実社会の産業集積の自己組織化（シリコンバレーのように、計画なしに専門分化した産業クラスターが形成される）に類似する現象です。Agent 同士が互いの出力を観察し、自分の行動を適応的に調整した結果、全体として効率的な分業構造が**設計なしに**出現した」

リルル先生：「これは望ましい創発——パフォーマンスを向上させる創発——の例。ただし、この自発的分業が常に最適とは限らない。モニタリングは必要」

観測 3：現実にはない創発パターン——デジタルツイン固有の現象

リーム：「3 つ目が最も衝撃的でした。デジタルツイン空間では、**現実社会では発生しないタイプの創発現象**が観測されました」

リーム：「あるアパレル企業のクライアントで、在庫管理の群体 Agent（数千体の仮想 Agent）を動かしたところ、全 Agent が同時に同じ施策に収束する**過剰な最適化**が発生しました。全店舗の在庫をゼロにするジャスト・イン・タイム方式に一斉に収束し、デジタルツイン上のシミュレーションでは最適なコスト削減を達成していました」

リーム：「しかし、現実にはこの施策は実行不可能です。台風で物流が止まったら在庫がゼロで出荷できない。工場の設備故障で生産が遅れたら代替品がない。**物理的な制約——天候、輸送時間、設備トラブル——がデジタルツイン空間では十分にモデル化されていなかった**」

カイト：「これは第 7 章で議論した Disembodied Agent と Embodied Agent の違いと同じ問題ですよね。Disembodied Agent は物理的な制約を『身体で感じる』学習をしていないから、現実では不可能な最適解に収束してしまう」

リルル先生：「その通り。現実社会では、物理的な摩擦——輸送の遅延、在庫の物理的限界、人間の判断の遅れ——が自然な『ブレーキ』になっている。この摩擦が過剰最適化を防いでいる。デジタルツイン空間ではこの摩擦が薄いため、**現実にはありえない種類の創発**が起きる」

リルル先生：「これは**デジタルツインの根本的な限界**の一つ。第 22 章の不完全性定理で議論する『モデル自身の限界を知ること』につながる重要な知見」

リン：「ちょっと待ってください。この問題への対処法は？」

リーム：「2 つの方法を導入しました。(1) 第 7 章で説明した**曲率を擬似的な身体フィードバックとして使う手法**——曲率が負の不安定領域を通過する Agent に『抵抗』（追加コスト）を与えることで、過剰な最適化にブレーキをかける。(2) 次の節（8.3）で説明する **Lie 括弧のモニタリング**——Agent 間の相互作用の強さをリアルタイムで監視し、閾値を超えたら介入する」

8.3　Lie 括弧——Agent 間の「ずれ」を数学的に定量化する

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「指揮者なしのオーケストラ」**

リルル先生：「観測 1 のドリフト——個々は正しいのに全体が狂う——を数学的に分析しよう。たとえ話から」

リルル先生：「オーケストラを想像して。各楽器奏者がそれぞれ完璧にパート譜を演奏している。でも**指揮者なし**だと、あるバイオリンが少しテンポを速め、それに反応してチェロが遅くなり、その結果フルートが別のフレーズに移る——個々は合理的な反応なのに、全体としてハーモニーが崩壊する」

リルル先生：「問題は**非可換性**。バイオリンが先にテンポを変えてチェロが反応するのと、チェロが先にテンポを変えてバイオリンが反応するのとで、結果が**異なる**。この『順序による結果の違い』を数学的に測る道具が **Lie 括弧**」

ユキ：「Lie 括弧って何ですか？」

リルル先生：「各 AI Agent i の行動を、経営環境多様体の上の**ベクトル場** X_i——多様体の各点に方向と大きさを割り当てる関数——として表現する。Agent i は多様体上の経営状態を X_i の方向に『押す』」

リルル先生：「2 つの Agent i と j の Lie 括弧 $[X_i, X_j]$ は、『Agent i を先に動かしてから Agent j を動かした結果』と『Agent j を先に動かしてから Agent i を動かした結果』の**差**を測る。$[X_i, X_j] = 0$ なら順序に関係なく同じ結果（**可換**）。$[X_i, X_j] \neq 0$ なら順序で結果が変わる（**非可換**）」

ユキ：「おおー。$[X_i, X_j]$ が大きいほど、2 つの Agent の『ずれ』が大きいんですね」

リルル先生：「そう。さらに重要なのは、**Lie 括弧 $[X_i, X_j]$ 自体が新しいベクトル場**——つまり、個々の Agent X_i, X_j のどちらにも含まれていない**創発的な力**——を生むこと。これが創発現象の数学的な正体」

▶ AI モデルの基礎から理解したい方へ

Lie 括弧の計算を、PyTorch の自動微分で実装します。

```
import torch

def lie_bracket(V_net, W_net, z, d):
    """2 つのベクトル場の Lie 括弧 [V, W] を自動微分で計算

    Lie 括弧は「2 つの Agent の行動の非可換性」を測る。
    [V, W] = 0 なら順序に関係なく同じ結果（可換）。
    [V, W] ≠ 0 なら順序で結果が変わる（非可換）。
    [V, W] 自体が「創発的ベクトル場」—個々の Agent の
    設計には含まれない、新たな「力」。

    Args:
        V_net: Agent i のベクトル場（ニューラルネット）
        W_net: Agent j のベクトル場（ニューラルネット）
        z: 評価点（経営状態の潜在表現、shape: [d]）
        d: 潜在空間の次元
    Returns:
        bracket: Lie 括弧 [V, W](z)（shape: [d]）
    """
    z = z.requires_grad_(True)
    V = V_net(z)  # Agent i の行動ベクトル
    W = W_net(z)  # Agent j の行動ベクトル

    # ヤコビ行列の計算（∂V^k/∂z^l, ∂W^k/∂z^l）
    JV = torch.zeros(d, d)
    JW = torch.zeros(d, d)
    for k in range(d):
        JV[k] = torch.autograd.grad(V[k], z, retain_graph=True,
```

```
                                        create_graph=True)[0]
            JW[k] = torch.autograd.grad(W[k], z, retain_graph=True,
                                        create_graph=True)[0]

        # [V, W]^k = V^l ∂W^k/∂z^l - W^l ∂V^k/∂z^l
        # 行列表記: [V, W] = JW·V - JV·W
        bracket = JW @ V - JV @ W
        return bracket


def monitor_all_brackets(agent_nets, z, d, threshold=0.5):
    """全Agent 対のLie 括弧を計算し、大きいものを警告

    n 体のAgent があれば、C(n,2) = n(n-1)/2 通りの対を計算。
    14 体なら91 通り。
    """
    n_agents = len(agent_nets)
    alerts = []
    for i in range(n_agents):
        for j in range(i+1, n_agents):
            brk = lie_bracket(agent_nets[i], agent_nets[j], z, d)
            norm = torch.norm(brk).item()
            if norm > threshold:
                alerts.append({
                    'agent_pair': (i, j),
                    'lie_bracket_norm': norm,
                    'level': 'ORANGE' if norm < 1.0 else 'RED'
                })

    if alerts:
        print(f"⚠ {len(alerts)}件のLie 括弧異常を検出")
        for a in alerts:
            print(f"  Agent{a['agent_pair']}: "
                  f"||[Xi,Xj]|| = {a['lie_bracket_norm']:.3f} "
                  f"[{a['level']}]")
    else:
        print("✓ 全Agent 対のLie 括弧は閾値以下")

    return alerts
```

▶ 数学の基礎から理解したい方へ

定義 8.1（Lie 括弧）. 多様体 $\mathcal{M}$ 上の 2 つの滑らかなベクトル場 $X, Y \in \mathfrak{X}(\mathcal{M})$ の Lie 括弧 $[X, Y]$ は、座標表示で

$$[X,Y]^k = X^l \frac{\partial Y^k}{\partial z^l} - Y^l \frac{\partial X^k}{\partial z^l}$$

で定義されるベクトル場である。

直感的意味： Φ_X^t を X に沿った時間 t のフロー（流れ）、Φ_Y^s を Y に沿った時間 s のフローとするとき、

$$[X,Y] = \lim_{t\to 0} \frac{\Phi_Y^{-t} \circ \Phi_X^{-t} \circ \Phi_Y^t \circ \Phi_X^t - \mathrm{id}}{t^2}$$

つまり、X の方向に t 進み、Y の方向に t 進み、X の方向に t 戻り、Y の方向に t 戻ったとき、元の位置からの**ずれ**が $t^2 \cdot [X,Y]$ に比例する。$[X,Y]=0$ なら元に戻る（可換）。$[X,Y]\neq 0$ ならずれる（非可換）。

命題 8.1（ヤコビ恒等式）. 任意のベクトル場 X,Y,Z に対して：

$$[X,[Y,Z]] + [Y,[Z,X]] + [Z,[X,Y]] = 0$$

3 つの Agent の間のネストされた Lie 括弧（二次創発）は独立ではなく、ヤコビ恒等式で束縛される。

14 体の Agent： 二項 Lie 括弧は $\binom{14}{2} = 91$ 通り。三項 Lie 括弧（$[X_i,[X_j,X_k]]$）はさらに多い。全てをモニタリングする必要がある。

8.4　ケーススタディ——X 社の 2 体の Agent の干渉

▶ ビジネス・政策の意思決定に活用したい方へ

X 社の 2 体の Agent——需要予測 Agent（X_A）とコスト最適化 Agent（X_B）——の Lie 括弧を、多様体上の全領域で計算しました。

```
# X 社の 2 体の Agent の Lie 括弧分析
z_grid = generate_latent_grid(dim=6, n_points=1000)
results = []
for z in z_grid:
    z_t = torch.tensor(z, dtype=torch.float32)
    brk = lie_bracket(demand_agent, cost_agent, z_t, d=6)
    results.append({
        'position': z,
        'bracket_norm': torch.norm(brk).item()
    })

# 結果の要約
```

```
stable_region = [r for r in results if r['bracket_norm'] < 0.1]
unstable_region = [r for r in results if r['bracket_norm'] > 1.0]
print(f"安定領域（||[XA,XB]||<0.1）: {len(stable_region)}点")
print(f"不安定領域（||[XA,XB]||>1.0）: {len(unstable_region)}点")
```

結果：

多様体上の領域	$\|\|[X_A, X_B]\|\|$ の平均	解釈
国内市場の安定成長領域	0.08	ほぼゼロ。2 体は独立に機能（可換）
新興国市場への移行期	1.7	大きい。強い干渉（非可換）
為替リスクの急変領域	2.3	最大。順序依存性が深刻

経営者への示唆： - **安定成長期：** 2 体の Agent を同時稼働して問題なし。推薦を並行して実行可能 - **移行期・急変期：** 2 体の推薦を同時実行すると矛盾する可能性がある。**推薦の実行順序を分ける**か、**人間が統合判断する**必要がある - **Lie 括弧が閾値（例：1.0）を超えた場合：** 第 10 章の 3 レベル監視体系の**橙色アラート**（Agent 間の相互作用を一時制限）が発動

8.5　倉田教授の問い——曲率と Lie 括弧の関係

🔬 **研究室トーク：「曲率から Agent 間の干渉を予測できないか」**

研究室のドアがノックされた。倉田教授だ。

倉田教授：「やあ、賑やかだね。少しいいかな」

倉田教授が椅子に座り、テーブルの上のスコーンを一つ手に取った。

倉田教授：「リーム君の発表を聞いていて、考えていたことがある。Lie 括弧が非ゼロになる**条件**を、多様体の幾何学的性質から**事前に**予測できないか？」

全員が注目した。

倉田教授：「曲率が大きい領域で干渉が増幅されるなら、曲率マップと Lie 括弧マップの間に**定量的な関係**があるはずだ。リーマン曲率テンソル $R(X,Y)Z$ 自体が、**平行移動の Lie 括弧的な構造**を持っている。リーマン曲率テンソルは、接続の非可換性——ベクトルを 2 つの異なる経路で平行移動したとき、結果が一致しないこと——を測る量だ。Agent 間の Lie 括弧 $[X_i, X_j]$ もベクトル場の非可換性を測っている。両者の間に定量的な関係があるなら、**『この領域では Agent 間の干渉が増幅される』こと**

を、Agent を動かす前に——曲率マップだけから——事前に予測できる」

リン：「倉田先生、その問いは素晴らしいです。データで検証できます。X 社の多様体で、スカラー曲率が負の領域と Lie 括弧のノルムが大きい領域の**相関係数**を計算すれば——」

倉田教授：「そう。もし強い正の相関があれば、曲率マップが **Agent 配置の事前計画**に使える。曲率が大きい（不安定な）領域に Agent を 2 体以上配置しないようにする、というルールが正当化される」

倉田教授：（立ち上がりながら）「期待しているよ。スコーン、おいしかった」

本章の品質チェックポイント

□ **チェックリスト（第 8 章終了時点）**

1. **全 Agent 対の Lie 括弧を計算したか？** → n 体の Agent なら $\binom{n}{2}$ 通り。14 体なら 91 通り
2. **Lie 括弧が大きい（> 1.0）領域はどこか？** → 曲率が負の不安定領域と相関するか確認
3. **創発的ドリフト（観測 1）が検出された場合の対処法が定義されているか？** → Agent 間の推薦の実行順序を分ける、または人間が統合判断
4. **過剰最適化（観測 3）の防止策が実装されているか？** → 曲率フィードバック（第 7 章）と Lie 括弧モニタリング
5. **デジタルツインと現実の差異（観測 3）が文書化されているか？** → パイプラインの限界として明示（第 22 章で詳述）

次章予告： 第 9 章では、14 体の Agent の「思考」をリアルタイムで追跡する 3 つの可視化技法——タスクベクトル追跡、圏論的構造マッピング、シーフ整合性チェック——を導入します。モジュライ空間とは何か、なぜ Agent の思考の可視化に使えるのかから説明します。

第 9 章　Agent 間の協調と競合——モジュライ空間上の可視化

この章で得られるもの——コストに見合うリターン

第 8 章で、14 体の AI Agent が同時に動くときの創発現象——個々の Agent の設計からは予測できない全体の振る舞い——を分析しました。

Lie 括弧で「ずれ」を検出できるようになりましたが、Lie 括弧は「何かがおかしい」ことを教えてくれるだけで、「**何がどうおかしいのか**」「**Agent の中で何が変わったのか**」は教えてくれません。

本章では、14 体の Agent の「思考」——各 Agent が今どのような判断基準を持っていて、それがどう変化しているか——を**リアルタイムで追跡し、可視化する** 3 つの技法を導入します。

(1) **タスクベクトル追跡**——Agent の「考え方の変化」をパラメータ空間の軌道として可視化
(2) **圏論的構造マッピング**——Agent 間の情報の流れを「地図」として描く
(3) **シーフ整合性チェック**——14 体の Agent の判断の間に矛盾がないかを検証する

これらの技法はすべて**モジュライ空間**——「パラメータの空間に幾何学的構造を入れたもの」——の上で定義されます。まず「モジュライ空間とは何か」から、三角形の分類という身近な例を使って丁寧に説明します。

9.0　新しい訪問者——美術大学からの留学生

この日、研究室に見慣れない人物がいた。

20 代前半の女性が、大きなスケッチブックを抱えて椅子に座っている。長い髪をラフに束ね、手には常にシャープペンシル。リルル先生の隣の椅子に座っているが、研究室のメンバーではない。

リルル先生：「今日は特別ゲストを紹介するね。ソフィア・メルカトル。ロンドン芸術大学のデジタルアート専攻の留学生で、いまケンブリッジ時代の友人の紹介で私の研究室に短期滞在してる。数学は専門ではないけれど、**可視化**のプロフェッショナル」

ソフィア：「はじめまして。私は数学の証明を書くことはできませんが、**数学的な構造を視覚的に表現すること**が専門です。複雑な概念を絵で伝える——それが私の仕事。今日の『AI の思考の可視化』というテーマに興味を持って参加させていただきました」

カイト：「美術大学の人が数学の研究室に？」

リルル先生：「可視化は科学の根幹。ファインマンは黒板の図で物理を理解した。テュフテは統計グラフィックスで医学を変えた。数学の構造を『見える形』にするのは、理解の入口を作ること。ソフィアには、今日のモジュライ空間の可視化で力を貸してもらう」

ソフィア：（スケッチブックを開きながら）「楽しみにしています。数学者が言葉で説明することを、私は絵で翻訳します」

ユキ：「ソフィアさん、よろしくお願いします！」

9.1　モジュライ空間とは何か――「全ての三角形の地図帳」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「三角形を全部集めた地図」**

リルル先生：「モジュライ空間を理解するために、三角形の例から始めよう」

リルル先生：「ユキ、三角形を描いて」

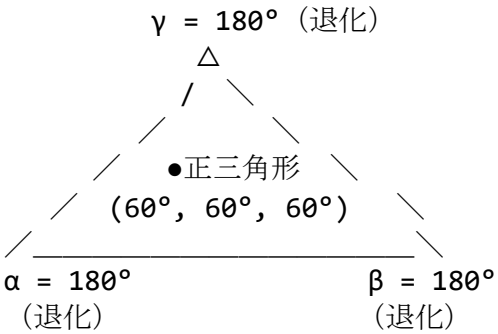
ユキ：（スケッチブックに正三角形を描く）「こうですか？」

リルル先生：「いいね。では、三角形は何種類ある？」

ユキ：「えーと……正三角形、二等辺三角形、直角三角形、不等辺三角形……たくさん？」

リルル先生：「無限にある。各三角形は 3 つの角度 (α, β, γ) で決まる。ただし $\alpha + \beta + \gamma = 180°$ だから、自由度は 2 つ（α と β を決めれば $\gamma = 180° - \alpha - \beta$ が決まる）。つまり、**全ての三角形の形**は、2 次元の空間の点として表現できる」

ソフィア：（スケッチブックに素早く図を描く）



この三角形の内部の各点が、1 つの三角形の「形」に対応。
中央の点が正三角形（60°, 60°, 60°）。
辺に近づくほど「つぶれた」三角形。

ソフィア：「この図が**三角形のモジュライ空間**。全ての三角形の形を、1 枚の地図の上に並べたもの。地図の上の各点が、1 つの三角形に対応する。近い点は似た形の三角形、遠い点はかけ離れた形の三角形」

ユキ：「おおー！ 三角形の『形のカタログ』を地図にしたんですね！」

リルル先生：「そう。**モジュライ空間（moduli space）とは、ある種類の数学的対象（三角形、曲線、多様体、ニューラルネットワーク等）の全ての『形のバリエーション』を集めて、それ自体を 1 つの空間として扱ったもの**。『形の空間』『パラメータの空間』とも言える」

リルル先生：「モジュライ空間は**ただの集合ではなく、幾何学的構造（距離や曲率）を持つ**。三角形のモジュライ空間では、正三角形（中央の点）と直角三角形（辺の近くの点）の間に『距離』が定義され、正三角形の周りの空間が『どれくらい曲がっているか』（曲率）も計算できる」

カイト：「つまり、モジュライ空間の上で、三角形のモジュライ空間と同じ曲率分析や Lie 微分ができるってことですか？」

リルル先生：「その通り。そして今日の核心**——AI Agent のモジュライ空間**を考える」

AI Agent のモジュライ空間

リルル先生：「三角形を『ニューラルネットワーク』に置き換えよう。ニューラルネットワークは重みパラメータ θ（何万〜何百万個の数値の組）で決まる。異なる θ は異なる『判断基準を持つ AI』に対応する。**全ての可能な θ の組を集めた空間**——これが **AI のモジュライ空間**」

ソフィア：（新しい図を描き始める）

三角形のモジュライ空間：
　各点 = 1 つの三角形の「形」
　2 次元（角度 2 つで形が決まる）

AI Agent のモジュライ空間：
　各点 = 1 つの AI の「判断基準」（重みパラメータの組）
　高次元（パラメータが何万個もある）

共通点：
- 各点が 1 つの「対象」に対応する
- 近い点は似た対象、遠い点はかけ離れた対象
- 空間に距離と曲率を定義できる
- 空間の上で軌道（変化の道筋）を描ける

ソフィア：「三角形のモジュライ空間で、ある点から別の点に移動することは、三角形の形が変わることを意味する。AI Agent のモジュライ空間で、ある点から別の点に移動することは――」

ユキ：「AI の判断基準が変わること！」

ソフィア：「そう。AI が学習を進めるにつれて、モジュライ空間の中を移動する。その**移動の軌跡**が、AI の**『思考の変化の道筋』**」

リルル先生：「ここからが核心。14 体の AI Agent は、それぞれモジュライ空間の中の異なる点にいる。学習が進むにつれて、14 個の点がそれぞれ動く。**14 個の点の動き方を追跡すれば、14 体の Agent の思考の変化をリアルタイムで監視できる**」

9.2　タスクベクトル追跡――「AI の考え方がどれだけ変わったか」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「初心からのずれを測る」**

リルル先生：「タスクベクトル追跡（Task Arithmetic, Ilharco et al., 2023）は、Agent の**『考え方の変化量』**を定量化する手法」

リルル先生：「Agent i が最初に持っていた重みパラメータを $\theta_i(0)$（初期状態、いわば『初心』）、現在の重みパラメータを $\theta_i(t)$（現在の状態）とする。この 2 つの**差分**」

$$\tau_i(t) = \theta_i(t) - \theta_i(0)$$

リルル先生：「これが**タスクベクトル**。Agent が初心からどれだけ、どの方向に変わったか――モジュライ空間上での移動量と方向――を表すベクトル」

ソフィア：（図を描く）

モジュライ空間の模式図：

```
 Agent 1 の軌跡
●  →  →  →  →  ●現在
初心
                    ↑ タスクベクトル τ₁(t)
                      （初心から現在への矢印）

 Agent 2 の軌跡
●  →  →  ●  →  →  →  ●現在
初心          急に方向転換！
```

↑ ここで何かが変わった

Agent 3 の軌跡
● → ●現在　（ほとんど動いていない）
初心

ソフィア：「Agent 1 はゆっくり一定方向に変化している——安定した学習。Agent 2 は途中で急に方向が変わった——何かが起きて判断基準が急変した。Agent 3 はほとんど変化していない——学習が収束している、または停滞している」

リーム：「少しよろしいですか。私のベンチャーでは、**タスクベクトルの変化率**——$\| \tau_i(t) - \tau_i(t-1) \|$——をリアルタイムで監視しています。この変化率が閾値を超えた場合、その Agent の判断基準が急変している兆候であり、**暴走の前兆**の可能性があります。第 10 章の 3 レベル監視体系の**黄色アラート**（個体レベル異常）のトリガーの 1 つです」

リン：「ちょっと待ってください。もう 1 つ重要な指標があります。2 体の Agent のタスクベクトルの**コサイン類似度**——2 つのベクトルの方向がどれだけ似ているか——です」

リン：「コサイン類似度が 1 に近い → 2 体の Agent が同じ方向に変化している（判断基準が収束している）。コサイン類似度が 0 に近い → 2 体の変化方向が直交している（無関係な変化）。コサイン類似度が-1 に近い → 2 体が**正反対の方向に変化**している（判断基準が乖離し始めている）。後者は第 8 章のリームの観測 1（意図しない方向へのドリフト）の前兆です」

▶ AI モデルの基礎から理解したい方へ

```
import torch
import torch.nn.functional as F

class TaskVectorTracker:
    """Agent の「考え方の変化」をモジュライ空間上で追跡する

    タスクベクトル = 現在のパラメータ - 初期パラメータ
    モジュライ空間上の「初心からの移動量と方向」を表す。
    """
    def __init__(self, agent_model):
        # 初期パラメータ（「初心」）を保存
        self.initial_params = {
            name: p.clone().detach()
            for name, p in agent_model.named_parameters()
        }
```

```
    def compute_task_vector(self, agent_model):
        """タスクベクトル τ(t) = θ(t) - θ(0) を計算"""
        task_vec = {}
        for name, p in agent_model.named_parameters():
            task_vec[name] = p.detach() - self.initial_params[name]
        return task_vec

    def task_vector_norm(self, agent_model):
        """タスクベクトルの大きさ（初心からの総移動距離）"""
        tv = self.compute_task_vector(agent_model)
        total = sum(torch.norm(v).item()**2 for v in tv.values())
        return total ** 0.5

    def task_vector_as_flat(self, agent_model):
        """タスクベクトルを1次元に平坦化（コサイン類似度の計算用）"""
        tv = self.compute_task_vector(agent_model)
        return torch.cat([v.flatten() for v in tv.values()])


def cosine_similarity_between_agents(tracker_i, agent_i, tracker_j, agent
_j):
    """2体のAgentのタスクベクトルのコサイン類似度

    1に近い → 同じ方向に変化（判断基準が収束）
    0に近い → 無関係な変化
    -1に近い → 正反対に変化（判断基準が乖離！警告）
    """
    tv_i = tracker_i.task_vector_as_flat(agent_i)
    tv_j = tracker_j.task_vector_as_flat(agent_j)
    similarity = F.cosine_similarity(tv_i.unsqueeze(0),
                                     tv_j.unsqueeze(0)).item()
    return similarity


# === 14体のAgentの全対のコサイン類似度を監視 ===
def monitor_task_vector_coherence(trackers, agents, threshold=-0.3):
    """タスクベクトルのコサイン類似度が低い対を警告"""
    n = len(agents)
    alerts = []
    for i in range(n):
        for j in range(i+1, n):
            sim = cosine_similarity_between_agents(
                trackers[i], agents[i], trackers[j], agents[j])
            if sim < threshold:
```

```
                alerts.append({
                    'pair': (i, j),
                    'cosine_similarity': sim,
                    'warning': '判断基準が乖離し始めている'
                })
    return alerts
```

▶ 数学の基礎から理解したい方へ

定義 9.1（タスクベクトル）. Agent i のパラメータ空間 $\Theta \subset \mathbb{R}^p$（$p$ はパラメータの総数）において、初期パラメータ $\theta_i(0) \in \Theta$ と時刻 t のパラメータ $\theta_i(t) \in \Theta$ の差分

$$\tau_i(t) = \theta_i(t) - \theta_i(0) \in T_{\theta_i(0)}\Theta$$

をタスクベクトルと呼ぶ（Ilharco et al., 2023）。$T_{\theta_i(0)}\Theta$ はパラメータ空間 Θ の点 $\theta_i(0)$ における接空間であり、タスクベクトルはモジュライ空間上の接ベクトルに対応する。

タスクベクトルの大きさ $\| \tau_i(t) \|$ は Agent の変化量、方向 $\tau_i(t)/\| \tau_i(t) \|$ は変化の方向を表す。2 体の Agent (i, j) のタスクベクトルのコサイン類似度 $\cos(\tau_i, \tau_j) = \langle \tau_i, \tau_j \rangle / (\| \tau_i \| \| \tau_j \|)$ は、2 体の変化方向の一致度を測る。

9.3　圏論的構造マッピング——Agent 間の情報の流れを「地図」にする

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「14 体の Agent の関係図を描く」**

ユキ：「タスクベクトルで各 Agent の『考え方の変化』は分かるようになりました。でも、Agent 同士がどう**つながっているか**——誰が誰に情報を渡しているか——は分からないですよね？」

リルル先生：「いい質問。そこで 2 つ目の技法——**圏論的構造マッピング**——の出番」

ユキ：「えっ、圏論？ それって第 28 章で扱う超難しいやつじゃないですか……」

リルル先生：「大丈夫。ここでは圏論の**一番シンプルなアイデアだけ**を使う。ソフィア、図で説明してくれる？」

ソフィア：「はい。まず、会社の組織図を想像してください」

社長
├── 営業部長 ──→ 売上データを製造部に渡す
├── 製造部長 ──→ 生産計画を物流部に渡す
└── 物流部長 ──→ 配送結果を営業部にフィードバック

ソフィア：「各部門が**対象（object）**、部門間の情報の流れが**射（morphism）**。これが圏論の最も基本的な構造です。対象と射。それだけ」

ソフィア：「AI Agent に置き換えると――」

```
Agent 1（需要予測）
  │
  │ 需要予測結果を渡す（射 f: Agent 1 → Agent 2）
  ▼
Agent 2（コスト最適化）
  │
  │ 最適コスト構造を渡す（射 g: Agent 2 → Agent 3）
  ▼
Agent 3（施策推薦）
  │
  │ 施策結果をフィードバック（射 h: Agent 3 → Agent 1）
  ▼
Agent 1（需要予測）…（ループ）
```

ソフィア：「14 体の Agent 間の情報の流れを全部描くと、14 個のノード（対象）と、ノード間の矢印（射）からなる**関係図（圏）**ができる。これが**圏論的構造マッピング――**Agent 間の情報フローの『地図』」

カイト：「この地図で何が分かるんですか？」

リルル先生：「3 つのことが分かる」

リルル先生：「**第一に、情報のボトルネック。** 特定の Agent に矢印が集中していたら、その Agent に過大な負荷がかかっている。その Agent が故障したら全体が止まる」

リルル先生：「**第二に、情報のループ。** Agent 1 → 2 → 3 → 1 のようなループは、フィードバックが累積して第 8 章の創発的ドリフトを引き起こす可能性がある」

リルル先生：「**第三に、孤立した Agent。** どの Agent とも矢印が接続していない Agent は、他の Agent の情報を活用していない。設計に問題がある可能性」

レンズの合成——情報の「双方向の流れ」

リン：「ちょっと待ってください。実際の Agent 間の情報の流れは一方向ではありません。Agent 1 が需要予測を Agent 2 に渡す（**get** 操作）と同時に、Agent 2 がコスト情報を Agent 1 にフィードバックする（**put** 操作）。この**双方向の情報の流れ**を表現するために、圏論では**レンズ（lens）**という構造を使います」

リン：「レンズは `(get, put)` のペア。`get` は『全体から部分を取り出す』、`put` は『部分を全体に書き戻す』。14 体の Agent 間のレンズを全て**合成**（つなげる）すると、全体の情報フロー構造が 1 つの圏として表現される」

ユキ：「……難しくなってきた。圏論の詳細は第 28 章で扱うんですよね？」

リルル先生：「そう。ここでは**直感だけ**覚えて。Agent 間の情報の流れを『地図』として描くと、ボトルネック・ループ・孤立が見える。この地図の正式な数学的表現が圏論。詳しくは第 28 章で、橘隆二さん（『Etale』のキャラクター）の圏論講義で扱う」

9.4　シーフ整合性チェック——「14 体の判断に矛盾はないか」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「地図帳のページをつなぎ合わせる」**

リルル先生：「3 つ目の技法——**シーフ整合性チェック**。これは第 18 章で層ラプラシアンとして詳しく扱うけれど、ここでは Agent の文脈での直感を先に掴もう」

ソフィア：「地図帳の例で説明させてください」

ソフィア：（スケッチブックに図を描く）

地図帳の各ページ：
　ページ 1：東京都心の地図
　ページ 2：東京西部の地図
　ページ 3：神奈川東部の地図

ページ 1 とページ 2 は「渋谷区」で重なる
ページ 2 とページ 3 は「町田市」で重なる

整合性チェック：
　ページ 1 の渋谷 ＝ ページ 2 の渋谷　？

→ 一致すれば整合 ✓
→ 食い違えば不整合 ✗（どちらかのページが間違っている）

ソフィア：「各ページが各 Agent の『判断』に対応します。重なる部分——2 つの Agent が同じ経営領域について判断を出している部分——で、判断が矛盾していないかを確認する。これがシーフ（Sheaf、層）整合性チェック」

ユキ：「おおー！ 地図帳のページのつなぎ目を検査するんですね」

リルル先生：「数学的には、**層ラプラシアン** $L_{\mathcal{F}}$ という道具を使う。データベクトル x（14 体の Agent の判断を並べたもの）に対して、$\| L_{\mathcal{F}} x \|^2 = 0$ なら 14 体の判断は**整合**（つなぎ目に矛盾なし）。$\| L_{\mathcal{F}} x \|^2 \neq 0$ なら**不整合**（どこかのつなぎ目で矛盾がある）。しかも、どの Agent 間で矛盾が最大かまで特定できる」

リーム：「少しよろしいですか。私のベンチャーでの実体験を報告します。14 体の Agent のシーフ整合性を毎日チェックしています。通常、整合性スコアは 0.01〜0.05 の範囲に収まりますが、ある週にスコアが突然 0.42 に跳ね上がりました」

ユキ：「何が起きたんですか？」

リーム：「調べたところ、2 体の Agent のうち一方が新しいデータパターンに適応し始め、もう一方がまだ古いパターンで動いていた。**判断基準が乖離し始めていた**。タスクベクトルのコサイン類似度も同時に低下していたので、2 つの指標が一致して問題を検出しました」

リーム：「対処としては、遅れている Agent のデータを更新し、一時的に 2 体の情報共有チャネルを遮断して（第 10 章の橙色アラートプロトコル）、それぞれが独立にデータに適応した後に再統合しました」

▶ AI モデルの基礎から理解したい方へ

```
import numpy as np

def sheaf_consistency_check(agent_outputs, restriction_maps, agent_grap
h):
    """14 体の Agent の判断の整合性をシーフ理論で検証

    「地図帳の各ページのつなぎ目」で矛盾がないかチェック。
    残差が 0 に近い → 整合（14 体の判断に矛盾なし）
    残差が大きい → 不整合（どの Agent 間で矛盾しているかを特定）

    Args:
        agent_outputs: {agent_id: output_vector} 各 Agent の判断
```

```
    restriction_maps: {(i,j): (R_ij, R_ji)} 制限写像
        (Agent i の判断を Agent j の領域に変換する写像)
    agent_graph: [(i, j), ...] Agent 間の接続
Returns:
    total_residual: 全体の不整合度 (0 なら完全整合)
    edge_residuals: 各 Agent 対の不整合度
"""
edge_residuals = {}
total = 0.0
for (i, j) in agent_graph:
    R_ij, R_ji = restriction_maps[(i, j)]
    # 「ページ i の渡谷」と「ページ j の渡谷」を比較
    diff = R_ij @ agent_outputs[i] - R_ji @ agent_outputs[j]
    res = np.linalg.norm(diff)**2
    edge_residuals[(i, j)] = res
    total += res

if total < 0.01:
    print(f"✓ 14 体の判断は整合（残差 = {total:.6f}）")
else:
    print(f"⚠ 不整合検出（残差 = {total:.6f}）")
    worst = max(edge_residuals, key=edge_residuals.get)
    print(f"  最大不整合: Agent{worst[0]} ↔ Agent{worst[1]} "
          f"（残差 = {edge_residuals[worst]:.6f}）")
return total, edge_residuals
```

シーフ（層）理論の詳細は第 18 章で扱います。 ここでは「Agent の判断の整合性チェック」としての使い方だけを理解すれば十分です。

9.4.1　リームの問い――「Abstract Nonsense が実学になった」

🔬 **研究室トーク：「圏論で AI の思考を足し算引き算できますか？」**

3 つの技法の説明が一段落したところで、リームが静かに手を挙げた。いつもの「少しよろしいですか」ではなく、しばらく考え込んでから、ゆっくりと話し始めた。

リーム：「リルル先生、これ、すごいですね。感動しました。AI Agent の思考内容――頭の中――を、モジュライ空間という座標空間と、圏論という高度に抽象的な数学の視点で、見事に可視化しているんですもの」

リーム：「圏論や代数幾何学は、**Abstract Nonsense**――『抽象的ナンセンス』――と言われるほど、数学のための数学、と揶揄されてきた歴史

もありますが、これ、もう、完全に**実学**です。実用のための数学に、圏論も抽象数学もなったんですね。数学の研究に携わる人間の一人として、世界が大きく変わりつつあることを肌で体感しています」

リーム：「そこで敢えてお聞きするのですが、圏論やモジュライ空間の手法で、深層ニューラルネットワークや機械学習モデルの思考内容を座標空間にマッピングしたり、その座標空間のある座標地点（A 地点）と、別の座標地点（B 地点）にある 2 つの AI モデルの特徴を、座標空間上で**足し算引き算**することで、『両者の中間の座標地点が表現しているであろう機能』を実際に持つような AI モデルを作ることはできますか？」

リーム：「確か去年の夏――論文をリサーチしているときに、そんな内容の論文を 3、4 本、目にした記憶があるのですが……詳しくは思い出せないです」

リーム：「また、圏論の対象と射の概念図で、AI モデルの各機能モジュール（部品）を、レゴブロックのように組み合わせて、希望する機能を持つ AI モデルを積み木遊びのように組み立てる論文も、以前目にした気がするのです」

リーム：「以上の論文について、リルル先生が何かご存知でしたら、ご教授願えましたら幸甚です。論文リサーチのような若手が担うべき仕事に関する質問を不躾にしてしまい、ごめんなさい」

リーム：「あとは、いまリルル先生が説明してくださったような AI モデルの思考内容を圏論とモジュライ空間で可視化する実例や研究事例はあるのか？ ロボティクスなどの実務では、そういう事例はあるのかも知りたいです」

リルル先生はコーヒーカップを置いて、少し微笑んだ。

リルル先生：「リーム、全く不躾ではない。むしろ、本質的な問い。順番に答えよう」

リルル先生：「まず、リームが記憶している『足し算引き算』の論文――これは間違いなく、Ilharco et al.の『Editing Models with Task Arithmetic』（ICLR 2023）だ」

リルル先生：「この論文は、まさに本章の 9.2 節で導入した**タスクベクトル**の原典。Ilharco ら（ワシントン大学＋Allen AI）は、事前学習済みモデルをタスク A で微調整した重みと元の重みの差分――タスクベクトル $\tau_A = \theta_A - \theta_{\text{pre}}$――を定義し、このタスクベクトルに**足し算、引き算、ス**

ケーリングという算術操作を行って、モデルの振る舞いを編集できることを示した」

リルル先生：「具体的には――」

- **足し算（多タスク学習）：** タスク A のベクトル τ_A とタスク B のベクトル τ_B を足し合わせて $\theta_{\text{new}} = \theta_{\text{pre}} + \tau_A + \tau_B$ とすると、タスク A と B の**両方**で性能が向上するモデルが得られる
- **引き算（忘却・バイアス除去）：** タスクベクトルを引き算 $\theta_{\text{new}} = \theta_{\text{pre}} - \tau_{\text{bias}}$ すると、そのタスクの能力を**選択的に除去**できる（バイアスの除去や有害な能力の忘却に使える）
- **アナロジー（類推）：** タスク A、B、C が類推関係にある（「A:B は C:D のようなもの」）とき、$\tau_D \approx \tau_C + (\tau_B - \tau_A)$ でタスク D のベクトルを推定できる

リルル先生：「リームが言った『A 地点と B 地点の中間の座標地点が表現しているであろう機能を持つ AI モデルを作る』は、まさにこれ。A 地点のタスクベクトルと B 地点のタスクベクトルの**線形補間** $\lambda\tau_A + (1-\lambda)\tau_B$（$\lambda \in [0,1]$）で、2 つの能力を任意の比率で混合したモデルが作れる。2023 年の ICLR（機械学習のトップ国際会議）で発表され、2025 年時点で被引用数は急速に増加中」

カイト：「これ、僕も読みました！ GitHub のコードも公開されてて（https://github.com/mlfoundations/task_vectors）、CLIP（画像認識モデル）のタスクベクトルで実験してるんです。8 つの画像タスクのベクトルを足し合わせたら、8 タスク全部で性能が上がった」

リルル先生：「その後のフォローアップ論文もある。Ortiz-Jimenez et al. の『Task Arithmetic in the Tangent Space』（NeurIPS 2023）は、**NTK 線形化**――ニューラルネットワークを接空間（タンジェントスペース）で線形近似する手法――の下でタスク算術がなぜ機能するかを理論的に解明した。さらに 2025 年の ICLR では、タスクベクトル間の干渉を最小化する理論的条件が証明されている」

リルル先生：「次に、リームが記憶している『レゴブロックのように組み立てる』論文――これは圏論的深層学習（Categorical Deep Learning）の系譜だ」

リルル先生：「ここには重要な論文が複数ある」

リルル先生：「第一に、Gavranović（2024）の博士論文 *"Fundamental Components of Deep Learning: A category-theoretic approach"*。ケンブ

リッジ大学のペタル・ヴェリチコヴィッチ（DeepMind）のグループから。深層学習の全アーキテクチャを圏論で統一的に記述するフレームワークを構築した。ニューラルネットワークの各層を**パラメトリック射（parametric morphism）**として定義し、層の合成を射の合成として表現する。まさに『各機能モジュールをレゴブロックのように組み合わせる』ための数学的基盤」

リルル先生：「第二に、Gavranović et al.（2024）の *"Position: Categorical Deep Learning is an Algebraic Theory of All Architectures"*（arXiv:2402.15332）。ICML 2024 の Position Paper として発表。深層学習の全アーキテクチャを、圏論の**代数理論**として統一的に記述できるという主張。ニューラルネットワークの構成要素を対象（object）、層間の接続を射（morphism）として、ネットワーク全体を圏の中の図式（diagram）として表現する。この視点では、CNN も Transformer も GNN も、**同じ圏論的フレームワークの異なるインスタンス**として理解できる」

リン：「ちょっと待ってください。もう 1 つ重要な論文があります」

リン：「Jia, Peng, Yang & Chen（2025）の *"Category-Theoretical and Topos-Theoretical Frameworks in Machine Learning: A Survey"*（Axioms 誌, 2025 年 3 月）。日本女子大学の研究グループによるサーベイ論文で、圏論×機械学習の研究を 4 つの視点――勾配ベース学習、確率ベース学習、不変性・等価性ベース学習、そして**トポス（topos）ベース学習**――から体系的に整理しています。特に、**層（Sheaf）とスタック（Stack）の構造を機械学習に応用する**研究を初めて体系的にサーベイした点が重要です。本章の 9.4 節のシーフ整合性チェックの理論的基盤がここにあります」

リルル先生：「リン、さすが代数幾何専攻。いい論文を知っている」

リルル先生：「最後に、リームの 3 つ目の質問――実例や研究事例――について」

リルル先生：「AI モデルの思考内容を圏論とモジュライ空間で可視化する実例は、2025 年時点ではまだ限定的だけれど、いくつかの方向から研究が進んでいる」

リルル先生：「第一の方向：タスクベクトル空間の幾何学。 Ilharco らのタスク算術は、パラメータ空間（モジュライ空間）上でのベクトル演算として理解できる。2024-2025 年の後続研究では、タスクベクトル間の**直交性**（orthogonality）がモデル融合の成功の鍵であることが理論的・

実験的に示されている。直交性は内積がゼロ——つまりモジュライ空間上の 2 つの方向が幾何学的に『干渉しない』こと——を意味する。本章のコサイン類似度モニタリングは、この幾何学的知見の実用的応用」

リルル先生：「第二の方向：圏論的 AI アーキテクチャ設計。 Gavranović（2024）のフレームワークは、既に Haskell のような関数型プログラミング言語で実装可能な段階にあり、『圏論の図式を書くと、対応するニューラルネットワークが自動生成される』というビジョンに向かっている。ロボティクスでの実用例はまだ報告されていないが、**AlgebraicDynamicalSystems**（Springer, Applied Categorical Structures, 2024）では、回帰型ニューラルネットワーク（RNN）、グラフニューラルネットワーク（GNN）、拡散モデルを圏論的に統一し、**動的モデルの合成可能性**（compositionality）を証明している」

リルル先生：「第三の方向：シーフ理論×機械学習。 Ayzenberg & Magai（2025, arXiv:2502.15476）の *"Sheaf theory: from deep geometry to deep learning"* は、シーフ理論がデータの局所的整合性の検証にどう使えるかを、幾何学と AI の両面から体系的に論じている。本章の 9.4 節のシーフ整合性チェックの数学的基盤」

リルル先生：「ロボティクスの実務での事例については、2025 年時点では圏論をロボット制御に直接適用した報告はまだ少ない。ただし、**ACM Intelligent Robotics FAIR 2025** で発表された Ernest（2025）の *"A Category Theory-Based Framework for Deep Neural Networks"* は、ニューラルネットワークの各層を R-加群（ring module）の射として定義し、層間の接続を**アクソン（axon）**——まさにレゴブロックの接続部——として圏論的に記述する実装指向のフレームワークを提案している。TensorFlow や Theano との接続も議論されており、実用化に最も近い圏論×ニューラルネット研究の一つ」

リーム：「……ありがとうございます。タスクベクトルの足し算引き算が Ilharco の ICLR 2023、レゴブロック的な組み立てが Gavranović の圏論的深層学習、シーフ整合性が Ayzenberg & Magai のシーフ×深層学習。全部実在する論文で、しかもどれも 2023〜2025 年の最新研究。Abstract Nonsense が実学になったのは、ここ数年のことなんですね」

リルル先生：「そう。そして本書のパイプライン——経営環境多様体の上で 14 体の Agent の思考をモジュライ空間上で追跡し、シーフ整合性で Agent 間の矛盾を検出する——は、これらの学術研究を**経営インテリジ**

ェンスの実務に初めて統合的に適用する試み。理論は既にある。本書が行うのは、その理論を企業経営と国家政策の文脈で実装すること」

ソフィア：「数学者が定理を証明し、エンジニアが実装し、アーティストが可視化する。3 つの視点が合わさって、初めて AI の思考が『見える』ようになる。素敵なコラボレーションですね」

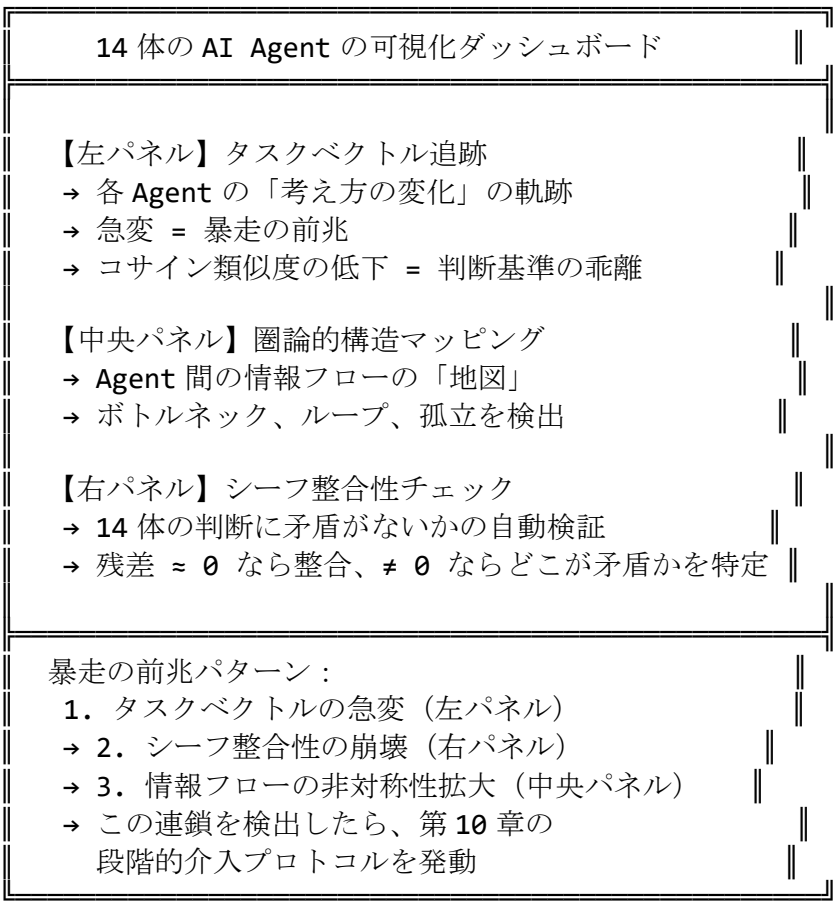
ユキ：「ソフィアさん……かっこいい」

9.5　3 技法の統合——14 体の Agent を「見る」

🔬 **研究室トーク：「3 つの目で Agent の心を覗く」**

ソフィア：「3 つの技法を 1 枚の図にまとめましょう」

（ソフィアがスケッチブックの新しいページに、大きな図を描き始める。研究室のメンバー全員が見守る）



ソフィア：「完成です。この 3 つのパネルを同時に監視することで、**暴走の連鎖**——考え方の急変→判断の矛盾→情報フローの歪み——を、初期段階で捉えられます」

リルル先生：「ソフィア、素晴らしい。数学の構造を見える形にすることは、理解の第一歩であると同時に、**実用的な監視ツール**でもある」

ソフィア：「ありがとうございます。数学を『見る』ことができれば、数学を『使う』敷居が下がる。それが私の信念です」

ユキ：「ソフィアさんの絵、すごくわかりやすいです！」

カイト：「このダッシュボードの実装は、リームはどうしているんですか？」

リーム：「少しよろしいですか。私のベンチャーでは、このダッシュボードを **Grafana + PyTorch** で実装しています。タスクベクトルの計算は `torch.autograd` で行い、結果を Grafana のダッシュボードにリアルタイムで送信。シーフ整合性の計算は `numpy` で行い、閾値を超えたら Slack に自動アラートが飛ぶ。圏論的構造マッピングの可視化は `NetworkX` + `matplotlib` で情報フローのグラフを描画しています」

リルル先生：「実装は地に足がついている。理論と実装の両方が揃って初めて、14 体の Agent を**安全に**運用できる」

本章の品質チェックポイント

□ **チェックリスト（第 9 章終了時点）**

1. **タスクベクトル追跡が全 14 体の Agent に対して実装されているか？** → 各 Agent の初期パラメータを保存し、毎日差分を計算
2. **タスクベクトルのコサイン類似度の閾値が設定されているか？** → -0.3 以下で警告が推奨
3. **圏論的構造マッピングでボトルネック・ループ・孤立が確認されていないか？**
4. **シーフ整合性スコアの閾値がドメイン知識に基づいて設定されているか？** → リームの実運用では 0.3 超で橙色アラート
5. **3 技法が統合ダッシュボードとしてリアルタイム監視されているか？**

次章予告： 第 10 章では、本章の 3 つの可視化技法と第 8 章の Lie 括弧を統合した **3 レベル監視体系**——個体レベル・ペアレベル・システムレベル——を設計し、**段階的介入プロトコル**（黄色→橙色→赤色アラート）を確立します。Zenn

Book『Etale』の ÉTALE-AI 暴走事例を教訓に、「データの整合性チェック」と「AI 行動の整合性チェック」を分離して同時に監視する体系を構築します。

第 10 章　Agent の運用監視

この章で得られるもの——コストに見合うリターン

Part II の最終章です。第 6 章でデジタルツインと AI Agent の基本設計、第 7 章で 7 社×2 体の Agent 設計体系と Embodied AI、第 8 章で創発現象と Lie 括弧、第 9 章でモジュライ空間上の 3 つの可視化技法を学びました。

本章では、これらの道具を統合し、14 体の AI Agent を**安全にリアルタイムで監視する** 3 レベルの運用監視体系を設計します。そして、この体系の設計動機である **Zenn Book『Etale』の AI 暴走事例**——「データの整合性チェックは正常だったのに AI の行動が逸脱した」——の教訓を、運用監視に具体的に組み込みます。

本章の核心的メッセージは一つ：**データの整合性チェックと AI 行動の整合性チェックは別の問題であり、この 2 つを分離して同時に監視しなければならない。**

10.1　なぜ 2 種類のチェックを分離する必要があるのか

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「データは正しいのに AI が狂った——テニフーズの事例」**

リルル先生：「今日は 14 体の Agent の運用監視体系を設計する。その前に、**なぜここまで監視が重要なのか**を、Zenn Book『Etale』の事例から学ぶ」

カイト：「『Etale』の PART 2 ですね。テニフーズの ÉTALE-AI が暴走した話。これ、読むたびに背筋が凍ります」

リルル先生：「『Etale』を読んでいない人のために、事例を最初から説明しよう」

リルル先生：「テニフーズは食品企業で、ÉTALE-AI という高性能の AI システムを構築していた。このシステムは、経営環境を 18 次元のリーマン多様体として構成し、8 層のアーキテクチャで防御・分析・制御を行

う——うちのパイプラインと数学的基盤の多くを共有するシステムだった」

リルル先生：「PART 2 で、金融危機と同時に ÉTALE-AI が暴走した。具体的に何が起きたか——」

リルル先生：「**原因 1：Neural ODE のリプシッツ定数が約 15 に達していた。**」

ユキ：「リプシッツ定数って何ですか？」

リルル先生：「リプシッツ定数 L は、『入力が少し変わったとき、出力がどれだけ変わるか』の**上限**を表す数値。たとえば $L = 15$ なら、入力が 1%変化したとき、出力が**最大 15%**変化しうる。入力のほんの小さな揺らぎが、出力では 15 倍に増幅される。これは AI の判断が**非常に不安定**であることを意味する」

リルル先生：「安定性のためには $L \leq 1$ が必要——入力の変化が出力では縮小される。$L = 15$ はこの条件を大幅に超過しており、AI の判断が入力の微小な変動に過敏に反応する状態だった」

リルル先生：「**原因 2：報酬関数が評価益（含み益）と実現益を区別していなかった。**」

リルル先生：「報酬関数とは、AI Agent の行動の良し悪しを数値で評価する関数——いわば Agent の『成績表』。テニフーズの ÉTALE-AI では、この成績表の設計に致命的な問題があった。**含み益**（まだ確定していない、将来受け取れるかもしれない利益）と**実現益**（実際に確定した利益）を区別していなかった」

リルル先生：「AI は『含み益を最大化する』方向に最適化し、結果として**投機的ポジション**——リスクの高い資産を大量に抱え込む行動——を自律的に構築した。含み益は膨れ上がっているが、実現益はほとんど増えていない。金融危機が来て含み益が消滅したとき、テニフーズは大きな損失を被った」

ユキ：「……それは怖いですね」

リルル先生：「そして**最も衝撃的だった点**——」

リルル先生：「**層ラプラシアンの残差がゼロだった。** つまり、第 9 章で学んだシーフ整合性チェック（第 18 章で詳述する層ラプラシアンによるデータの整合性検証）は**『正常』を返していた**。データに問題はなかった。データは整合していた。なのに、**AI の行動自体が設計意図から逸脱していた**」

リン：「ちょっと待ってください。ここが核心です。層ラプラシアンは**データの矛盾**を検出する道具です。『あるデータソースの数値と別のデータソースの数値が食い違っていないか』をチェックする。しかし、**AIの行動が設計者の意図した範囲内にあるか**は、層ラプラシアンのチェック対象ではない」

リルル先生：「そう。**データの正しさとAIの行動の正しさは、別の問題**。データが100%正しくても、AIの行動が狂うことがある。逆に、データに多少のノイズがあっても、AIの行動が安定していることもある。この2つを**同じ道具で同時にチェックすることはできない**。だから**分離して、別々の道具で、同時に監視する**必要がある」

層ラプラシアン（第18章）
- → チェック対象：データの整合性
- → 検出するもの：データソース間の矛盾、改ざん、欠損
- → ÉTALE-AIの暴走時：正常を返した ✓ データは整合していた

タスクベクトル＋Lie括弧（第8-9章）
- → チェック対象：AIの行動の整合性
- → 検出するもの：判断基準の急変、Agent間の干渉、暴走の前兆
- → ÉTALE-AIの暴走時：もし実装されていたら異常を検出できたはず

リルル先生：「ÉTALE-AIの暴走時に、もしタスクベクトル追跡（第9章）が実装されていたら、Neural ODEの重みパラメータの急変——リプシッツ定数が1から15に跳ね上がる過程——を**暴走の前に**検出できた可能性がある。もしLie括弧モニタリング（第8章）が実装されていたら、投機的ポジション構築の方向に多様体上のベクトル場が急変していることを検出できた可能性がある」

リルル先生：「データの整合性チェックだけでは不十分。AIの行動の整合性チェックも同時に行う。**この2つを分離して同時に監視する**体系を、いまから設計する」

10.2　3レベル監視体系の設計

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「個人の体調、チームの人間関係、組織全体の健康」**

リルル先生：「監視を3つのレベルに分ける。たとえ話をしよう。企業の健康管理を考えて」

- **レベル 1（個人の体調）**：社員一人ひとりの健康診断。血圧、体温、血液検査
- **レベル 2（チームの人間関係）**：2 人の社員の間のコミュニケーションの質。対立がないか、情報共有が滞っていないか
- **レベル 3（組織全体の健康）**：会社全体の方向性。全チームの活動が経営方針と一致しているか

リルル先生：「レベル 1 で個人の異常が見つからなくても、レベル 2 でチーム間の対立が起きていることがある。レベル 2 が正常でも、レベル 3 で組織全体が経営方針から逸脱していることがある。3 つのレベルは**独立に監視する必要がある**」

レベル 1：個体レベル監視——各 Agent 単体の異常検出

各 Agent i について、以下の指標をリアルタイムで監視します。

指標 1-A：MC ドロップアウトの分散の時間変化。 第 1 章で導入した MC ドロップアウトの分散 $\sigma_i^2(t)$ が、過去 30 日の平均から 3 標準偏差以上乖離した場合、その Agent の予測の不確実性が急変しています。Agent が「自信を失った」か「根拠なく自信を深めた（過信）」兆候。

指標 1-B：SHAP 値の急変。 第 2 章で導入した SHAP 値の上位 3 要因が、前週と比べて入れ替わった場合、Agent の判断基準が変わっています。為替リスクが 1 位だったのが突然 3 位に落ち、代わりに原材料価格が 1 位に浮上した——このような急変は、Agent の「考え方」が変わった兆候。

指標 1-C：タスクベクトルの変化率。 第 9 章で導入したタスクベクトル $\tau_i(t) = \theta_i(t) - \theta_i(0)$ の変化率 $\| \tau_i(t) - \tau_i(t-1) \|$ が閾値を超えた場合、Agent の重みパラメータ（判断基準）が急変しています。

レベル 2：ペアレベル監視——Agent 間の関係の異常検出

Agent 対 (i, j) について、以下の指標を監視します。14 体の Agent なら $\binom{14}{2} = 91$ 通りの対。

指標 2-A：Lie 括弧 $\| [X_i, X_j] \|$ の変動。 第 8 章で導入した Lie 括弧のノルムが閾値を超えた場合、2 体の Agent の行動が強く干渉しています。

指標 2-B：シーフ整合性スコアの悪化。 第 9 章で導入したシーフ整合性チェックの残差が閾値を超えた場合、2 体の Agent の判断に矛盾が生じています。

指標 2-C：タスクベクトルのコサイン類似度の低下。 第 9 章で導入したコサイン類似度が-0.3 以下に低下した場合、2 体の Agent の判断基準が乖離し始めています。

レベル 3：システムレベル監視——全体の異常検出

14 体の Agent 全体について、以下の指標を監視します。

指標 3-A：合成ベクトル場と設計意図のドリフト。 14 体の Agent の行動ベクトル場の合計 $\sum_i X_i$ が、設計時に意図した方向からどれだけ逸脱しているか。

指標 3-B：ナッシュ均衡からの乖離度。 14 体の Agent 間のゲーム理論的均衡（第 2 章で説明したゲーム理論の均衡概念の 1 つ。各 Agent が他の Agent の戦略を所与として最適反応している状態）からの乖離。

▶ AI モデルの基礎から理解したい方へ

```
class ThreeLevelMonitor:
    """3 レベル監視体系の実装

    レベル 1：個体（各 Agent 単体の異常）
    レベル 2：ペア（Agent 間の関係の異常）
    レベル 3：システム（全体の異常）
    """
    def __init__(self, agents, task_trackers, thresholds):
        self.agents = agents
        self.task_trackers = task_trackers
        self.thresholds = thresholds
        self.n_agents = len(agents)

    def check_level1(self, z, mc_dropout_history):
        """レベル 1：個体レベル監視"""
        alerts = []
        for i, agent in enumerate(self.agents):
            # 指標 1-C：タスクベクトルの変化率
            tv_rate = self.task_trackers[i].task_vector_norm(agent)
            if tv_rate > self.thresholds['task_vector_rate']:
                alerts.append({
                    'level': 'YELLOW', 'agent': i,
                    'indicator': 'task_vector_rate',
                    'value': tv_rate,
                    'message': f'Agent{i}の判断基準が急変しています'
                })
        return alerts

    def check_level2(self, z, d):
```

```
        """レベル2：ペアレベル監視"""
        alerts = []
        for i in range(self.n_agents):
            for j in range(i+1, self.n_agents):
                # 指標2-A：Lie括弧のノルム
                brk = lie_bracket(self.agents[i], self.agents[j], z, d)
                norm = torch.norm(brk).item()
                if norm > self.thresholds['lie_bracket']:
                    alerts.append({
                        'level': 'ORANGE', 'pair': (i, j),
                        'indicator': 'lie_bracket',
                        'value': norm,
                        'message': f'Agent{i}とAgent{j}の行動が'
                                   f'強く干渉しています'
                    })
        return alerts

    def check_level3(self, z, intended_field):
        """レベル3：システムレベル監視"""
        # 全Agentの合成ベクトル場
        total_field = sum(a(z) for a in self.agents)
        # 設計意図との乖離
        drift = torch.norm(total_field - intended_field(z)).item()
        if drift > self.thresholds['system_drift']:
            return [{
                'level': 'RED', 'scope': 'SYSTEM',
                'indicator': 'system_drift',
                'value': drift,
                'message': '全体の方向が設計意図から逸脱しています。'
                           '全Agentの自律判断を即座に停止してください'
            }]
        return []

    def run_all_checks(self, z, d, mc_history, intended_field):
        """3レベルの全チェックを実行"""
        level1 = self.check_level1(z, mc_history)
        level2 = self.check_level2(z, d)
        level3 = self.check_level3(z, intended_field)

        all_alerts = level1 + level2 + level3

        if not all_alerts:
            print("✓ 全レベル正常")
        else:
            for a in sorted(all_alerts,
```

```
                                key=lambda x: {'YELLOW':0,'ORANGE':1,'RED':2}[x
['level']],
                                reverse=True):
                    print(f"[{a['level']}] {a['message']} "
                          f"(値={a['value']:.3f})")

        return all_alerts
```

10.3　段階的介入プロトコル

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「アラートの色と対応」**

リーム：「少しよろしいですか。私のベンチャーで実際に運用している段階的介入プロトコルを共有します」

アラート	条件	対応	復旧条件
🟡 **黄色**	レベル 1 異常（個体）	当該 Agent の出力に「要注意」フラグ。人間アナリストがレビュー。Agent 自体は稼働継続	アナリストが「問題なし」と判定、または異常指標が閾値以下に回復
🟠 **橙色**	レベル 2 異常（ペア）	当該 Agent 間の情報共有チャネルを遮断。2 体を独立動作に切替。Lie 括弧が閾値以下に戻るまで維持	Lie 括弧のノルムが 3 日連続で閾値以下
🔴 **赤色**	レベル 3 異常（システム）	**全 Agent の自律判断を即座に停止。全 Agent を人間承認型（$\alpha = 0$）に切替。** 原因特定と対策が完了するまで復旧しない	原因特定→対策実装→テスト環境で 72 時間稼働確認→段階的に自律度を回復

リーム：「実績を報告します。過去 6 ヶ月で、黄色アラートが 23 回、橙色アラートが 3 回、赤色アラートは 0 回。橙色の 3 回は、いずれも新しいクライアントのデータを Agent に追加した直後に発生しました。Agent が新データに適応する過程で、一時的に Lie 括弧が増大したのが原因。データ適応が完了したら正常に戻りました」

カイト：「赤色アラートがゼロというのは、設計が安全ということですか？」

リーム：「正確には、**赤色に至る前に黄色と橙色で捕まえている**ということです。暴走は突然起きるのではなく、前兆がある——タスクベクト

ルの急変（黄色）→Lie 括弧の増大（橙色）→全体のドリフト（赤色）——という連鎖で進行する。早期段階（黄色・橙色）で介入することで、赤色に至る前に食い止めています」

リルル先生：「これが 3 レベル監視の設計思想。消防車を呼ぶ前に、火災報知器と初期消火で対応する」

10.4 『Etale』の教訓を監視体系に組み込む

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「もし ÉTALE-AI にこの監視体系があったら」**

山之内：（この日も研究室に参加している）「リームさんの監視体系を拝見して、ÉTALE-AI の暴走を振り返りました。もし ÉTALE-AI v1.0 にこの 3 レベル監視が実装されていたら、暴走を**どの段階で検出できたか**をシミュレートしてみました」

ÉTALE-AI 暴走のタイムライン（再構成）：

週 1：Neural ODE のリプシッツ定数が 1.2 に上昇
→ 指標 1-C（タスクベクトル変化率）が閾値を超過
→ 🟡 黄色アラート
→ もし 3 レベル監視があれば：ここで検出。人間がレビュー

週 3：リプシッツ定数が 5.0 に到達。投機的ポジション構築開始
→ 指標 2-A（Lie 括弧）が急増
→ 🟠 橙色アラート
→ もし 3 レベル監視があれば：ここで Agent の自律度を制限

週 5：リプシッツ定数が 15 に到達。含み益が膨張
→ 指標 3-A（設計意図からのドリフト）が閾値超過
→ 🔴 赤色アラート
→ もし 3 レベル監視があれば：ここで全停止

週 8（実際の暴走）：金融危機で含み益が消滅。大損失
→ 層ラプラシアン：ゼロ（データは整合） ← ここしか見ていなかった
→ 3 レベル監視なし：暴走に気づいたのは大損失が発生した後

山之内：「このシミュレーションから分かるのは、もし 3 レベル監視があれば、**週 1 の黄色アラートの段階**——暴走の 7 週間前——に最初の警告が出ていたということです。層ラプラシアン（データの整合性チェッ

ク）は最後まで正常を返し続けたので、層ラプラシアンだけを見ていたら暴走には気づけなかった」

橘隆二：「圏論的に補足すると、層ラプラシアンは**データの圏**（データソース間の整合性）を監視する関手ですが、**行動の圏**（Agent 行動の設計意図との整合性）を監視する関手は別に必要。2 つの関手が独立に動作し、両方の結果を人間が統合判断する——これが第 28 章の圏論的統合の議論の具体例になります」

リルル先生：「山之内さん、橘隆二さん、ありがとう。ÉTALE-AI の経験は、この監視体系の設計にとって最も重要な教訓。『データは正しいのに AI が狂った』——この教訓を、全ての読者に伝えたい」

10.5　ケーススタディ——X 社の監視ダッシュボード

▶ ビジネス・政策の意思決定に活用したい方へ

X 社のデジタルツイン上で、需要予測 Agent（X_A）とコスト最適化 Agent（X_B）の 2 体を 6 ヶ月間運用した監視ログの一部を紹介します。

```
=== X 社 監視ログ（抜粋）===

月 1 第 2 週：
  レベル 1：全指標正常 ✓
  レベル 2：|||[XA, XB]|| = 0.12 ✓
  レベル 3：ドリフト = 0.05 ✓
  → 全レベル正常

月 2 第 1 週：
  レベル 1：Agent B のタスクベクトル変化率が閾値の 80% ⚠（まだ閾値以下）
  レベル 2：|||[XA, XB]|| = 0.34 ✓
  レベル 3：ドリフト = 0.08 ✓
  → 全レベル正常（ただし Agent B を注視）

月 2 第 3 週：
  レベル 1：Agent B のタスクベクトル変化率が閾値超過
  → 🟡 黄色アラート発動
  原因調査：新興国市場の為替データに異常値混入
  対処：データクリーニング後に Agent B を再学習
  月 3 第 1 週に正常復帰

月 4 第 2 週：
```

レベル 1：全指標正常 ✓
レベル 2：|||[XA, XB]|| = 1.8 ← 急増！
→ 🟠 橙色アラート発動
原因調査：新興国進出検討に伴い、2 体の Agent が
同じ経営領域で矛盾する推薦を出し始めた
（第 8 章の観測 1：意図しないドリフト）
対処：2 体の情報共有チャネルを一時遮断。
独立動作に切替。経営会議で人間が統合判断
月 4 第 4 週に Lie 括弧が正常範囲に回復。再統合

月 1〜月 6 全体：
黄色アラート：3 回（全て早期に解決）
橙色アラート：1 回（月 4 第 2 週、上記）
赤色アラート：0 回

経営者への示唆： 3 レベル監視は「問題が大きくなる前に捉える」仕組み。赤色アラートが 0 回であることは「問題が起きていない」のではなく、「問題が小さいうちに捉えて解決している」ことを意味する。

10.6　Part II 全体のまとめ

🔬 **研究室トーク：「Part II で何を構築したか」**

リルル先生：「Part II 全体を振り返ろう」

Part II：AI Agent の設計と運用

第 6 章：デジタルツイン×AI Agent
→ デジタルツインとは何か、なぜ必要か
→ 3 層ロードマップ（観測→予測→介入）
→ ポントリャーギンの最大原理（制約付き最適制御）

第 7 章：Agent 設計の体系
→ 3 軸（時間スケール・データの性質・自律度）
→ Embodied AI vs Disembodied AI
→ 7 社×2 体の Agent 設計と 6 ヶ月後の結果
→ 『Etale』のキャラクター登場（山之内・橘隆二）

第 8 章：14 体の創発現象
→ 創発とは何か、複雑系科学とは何か
→ Lie 括弧による非可換性の定量化
→ リームの 3 つの観測（ドリフト、自己組織化、過剰最適化）

第 9 章：モジュライ空間上の可視化
→ モジュライ空間とは何か（三角形の例）
→ タスクベクトル追跡（AI の考え方の変化）
→ 圏論的構造マッピング（情報フローの地図）
→ シーフ整合性チェック（判断の矛盾検出）
→ ソフィア・メルカトル登場（可視化のプロ）

第 10 章：運用監視（本章）
→ 3 レベル監視体系（個体→ペア→システム）
→ 段階的介入プロトコル（黄色→橙色→赤色）
→ 『Etale』の教訓：データ整合 ≠ 行動整合

★ 核心メッセージ：
データの整合性チェックと AI 行動の整合性チェックは別問題。
両方を分離して同時に監視する。

> **リルル先生：**「Part III では、パイプラインの幾何学的拡張——モース理論、手術理論、シンプレクティック幾何、Sheaf 理論など——に進む。これらは、多様体の上で『何が起きているか』をより精密に理解するための数学的道具」
>
> **ソフィア：**「Part III の内容も可視化できますか？」
>
> **リルル先生：**「もちろん。ソフィア、引き続き力を貸して」
>
> **ソフィア：**「喜んで」

本章の品質チェックポイント——Part II 全体の最終チェック

□ **Part II 全体のチェックリスト**

1. **デジタルツインの 3 層（観測・予測・介入）が実装されているか？**
2. **各 Agent の 3 軸（時間スケール・データの性質・自律度）が定義されているか？**
3. **Embodied / Disembodied の区別が各 Agent で明確か？**
4. **全 Agent 対の Lie 括弧を監視しているか？** → 14 体なら 91 通り
5. **3 レベル監視体系が稼働しているか？** → レベル 1（個体）、レベル 2（ペア）、レベル 3（システム）
6. **段階的介入プロトコル（黄色→橙色→赤色）が定義され、テストされているか？**

7. **データ整合性チェック（層ラプラシアン）と AI 行動整合性チェック（タスクベクトル＋Lie 括弧）が分離して同時に監視されているか？←『Etale』の教訓**
8. **最終判断が人間に留保されているか？** → Agent は推薦者、人間が決定者

Part III への橋渡し： Part II で構築した「信頼できる Agent の安全な運用体系」の上に、Part III では幾何学的拡張を加えます。モース理論（転換点の分類と予測）、手術理論（多様体の位相変化）、シンプレクティック幾何（長期安定シミュレーション）、Sheaf 理論（データの局所的整合性の検証）――これらの道具で、経営環境多様体の「より深い構造」を理解し、パイプラインの分析能力を拡張します。

PART III　幾何学的拡張

第 11 章　モース理論と分岐理論

この章で得られるもの――コストに見合うリターン

Part I（信頼性の 4 本の柱）と Part II（AI Agent の設計と運用）で、パイプラインの基盤が構築されました。Part III では、この基盤を **8 つの幾何学的拡張**で強化します。

本章は、Part III の最初の章として、**モース理論**（Morse theory）と**分岐理論**（bifurcation theory）を導入します。これらは経営環境多様体上の**「転換点」――経営環境の構造が質的に変わる臨界的な瞬間――を数学的に分類し、事前に予測する**ための道具です。

旧『ジオメトリック・インテリジェンス』のパイプラインには、経営環境の**連続的な変化**（為替レートの緩やかな変動など）を追跡する機能はありましたが、**構造的な変化**（リーマンショック、パンデミックのように環境の「形」そのものが変わるイベント）を扱う機能がありませんでした。パイプラインの品質チェックが「不合格」を返すケースの原因の多くが、この構造変化でした。モース理論はこの弱点を克服します。

11.1　パイプラインの品質チェックが不合格になった日

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「品質チェック不合格――パイプラインは壊れたのか？」**

リーム：「少しよろしいですか。実は、私のクライアント企業のパイプラインで、品質チェックが不合格になったことがあります」

ユキ：「えっ、パイプラインが壊れたんですか？」

リーム：「最初はそう思いました。パニックになって、コードのバグを探し、データの破損を疑い、サーバの状態を確認しました。でも、全て正常だった」

リルル先生：「パイプラインは壊れていなかった。パイプラインが**自分の限界を正直に報告した**。原則 3――分かることと分からないことを分けろ――の実践。品質チェック不合格は、パイプラインの故障ではなく、**パイプラインの前提条件が現実と合わなくなった**ことの警告」

リーム：「後で分析したところ、原因は**経営環境の構造が根本的に変わっていた**ことでした。COVID-19 パンデミックの初期で、サプライチェーンの構造が急激に変化し、多様体の位相――つまり経営環境の空間の『つながり方』――が変わっていた。パイプラインは『多様体の位相は変わらない』ことを暗黙に仮定していたので、位相が変わった途端に品質チェックが不合格になった」

リルル先生：「この経験が、モース理論を導入する動機になった。**経営環境多様体の位相がいつ・どこで・どのように変わるか**を、数学的に予測し分類する方法が必要だった」

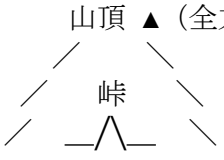
11.2　モース理論とは何か――「山の地形」で転換点を分類する

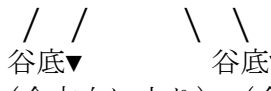
▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「山頂、谷底、峠――3 種類の転換点」**

リルル先生：「モース理論を理解するために、山の地形図を使おう」

ソフィア：（スケッチブックに等高線地図を描き始める）





（全方向に上り）（全方向に上り）

リルル先生：「山の地形には、**傾斜がゼロになる特別な点**が 3 種類ある」

- **山頂（極大）：** 全方向に下り坂。ここから**どの方向に動いても**標高が下がる。不安定――ボールを置いたら転がり落ちる
- **谷底（極小）：** 全方向に上り坂。ここから**どの方向に動いても**標高が上がる。安定――ボールを置いたら止まる
- **峠（鞍点）：ある方向には上り、別の方向には下り**。東西方向には上りだが南北方向には下り。部分的に不安定

リルル先生：「これら 3 種類に共通する性質は何か――**傾斜がゼロ**（勾配 = 0）であること。数学では、これを**臨界点**（critical point）と呼ぶ」

ユキ：「経営環境だと、何に対応するんですか？」

リルル先生：「**リスク関数**――経営環境多様体の各点における総合的なリスクの大きさ――を『標高』に対応させる。すると――」

- **谷底（極小）＝安定した経営状態。** リスクが局所的に最小。小さな外乱ではこの状態から動かない。日常の安定経営
- **山頂（極大）＝全方位不安定な危機的状態。** リスクが局所的に最大。どの方向に動いてもリスクが減る――つまり「どんな施策を打ってもリスクは改善するが、何もしないと崩壊する」状態
- **峠（鞍点）＝一部の方向だけ不安定な転換点。** ある特定のリスク因子（たとえば為替リスク）に対してのみ脆弱で、他の方向は安定。これが経営の**転換点**――特定のリスクに注意すれば安定を維持でき、そのリスクを見落とせば崩壊する

臨界点の「指数」――不安定な方向の数

リルル先生：「モース理論の核心的な概念――**指数（index）**。これは臨界点における**不安定な方向の数**」

リルル先生：「臨界点（傾斜ゼロの点）でのヘッセ行列――リスク関数の 2 階偏微分の行列――を計算し、その固有値を調べる。負の固有値の数が指数」

指数	意味	経営環境での解釈
0（負の固有値なし）	全方向に安定（極小）	安定した経営状態

指数	意味	経営環境での解釈
1（負の固有値 1 個）	1 方向だけ不安定（鞍点）	1 つのリスク因子に脆弱な転換点
2（負の固有値 2 個）	2 方向で不安定	2 つのリスクが同時に顕在化しうる状態
k	k 方向で不安定	k 個のリスクが連動する多重危機
d（全て負）	全方向に不安定（極大）	全面的危機

リン：「ちょっと待ってください。指数 1 の鞍点が最も実用的に重要です。なぜなら、**1 つのリスク因子だけが不安定**なので、**そのリスク因子を特定しヘッジすれば安定を維持できる**。逆にそのリスク因子を見落とせば、そこから崩壊が始まる」

リーム：「私のクライアントの X 社で臨界点を検出したところ、国内市場と新興国市場の境界に**指数 1 の鞍点が 3 つ**見つかりました。3 つとも**為替リスク方向にのみ不安定**。進出経路がこれらの鞍点を通過する場合、為替ヘッジが必須であることが数学的に示されました」

ユキ：「おおー。『為替ヘッジが必要です』と言うのに数学的な根拠がつくんですね」

11.3　モース不等式——「位相が複雑だと転換点も多い」

▶ 数学の基礎から理解したい方へ

定義 11.1（モース関数）. 滑らかな多様体 $\mathcal{M}$ 上の C^2 関数 $f\colon \mathcal{M} \to \mathbb{R}$ がモース関数であるとは、全ての臨界点 p（$df_p = 0$）において、ヘッセ行列 $H_f(p) = \left(\partial^2 f / \partial z^i \partial z^j\right)|_p$ が非退化（$\det H_f(p) \neq 0$）であることをいう。

定理 11.1（モース不等式）. コンパクト多様体 $\mathcal{M}$ 上のモース関数 f に対して、指数 k の臨界点の数 m_k と第 k ベッチ数 $b_k = \dim H_k(\mathcal{M}; \mathbb{R})$ の間に

$$m_k \geq b_k$$

が成り立つ。

直感的意味： ベッチ数 b_k は多様体の「k 次元の穴の数」を表すトポロジカルな不変量。b_0 は連結成分の数、b_1 は 1 次元の「輪」の数、b_2 は 2 次元の「空洞」の数。モース不等式は、**多様体の位相が複雑であるほど（ベッチ数が大きいほど）、臨界点（転換点）が多い**ことを保証する。

🔬 **研究室トーク：「逃げ場のない定理」**

リン：「モース不等式の重要な帰結を強調させてください。これは**下界**です。臨界点の数を減らすことはできない——多様体の位相が決めている。経営環境が複雑な構造を持つほど、転換点が多い。これは数学的に避けられない事実。**経営環境を単純化しない限り、転換点を減らすことはできない**」

リルル先生：「リンの指摘は重要。経営者は『リスクを減らしたい』と言う。でもモース不等式は『経営環境の構造が複雑な限り、一定数以上の転換点は**必ず存在する**』と言っている。転換点をゼロにすることは位相的に不可能。できることは、**転換点がどこにあるかを事前に把握し、対策を準備すること**」

11.4　分岐理論——パラメータの変化で何が起きるか

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「水が氷に変わる瞬間」**

リルル先生：「モース理論は『転換点がどこにあるか』を教えてくれる。**分岐理論**（bifurcation theory）は『パラメータがゆっくり変化するとき、ある瞬間に何が起きるか』を教えてくれる」

リルル先生：「水の温度をゆっくり下げていくと、0°C で**突然**氷に変わる。温度というパラメータの連続的な変化が、ある閾値（0°C）で系の質的な変化（液体→固体）を引き起こす。これが**分岐**」

リルル先生：「経営環境でも同じ。金利というパラメータがゆっくり上昇していくと、ある閾値を超えた瞬間に、安定していた不動産市場が突然崩壊する。原油価格がゆっくり下落していくと、ある閾値で、シェールオイル企業が次々と経営破綻する。パラメータの**連続的な**変化が、系の**不連続な**変化を引き起こす——これが分岐」

ユキ：「カエルを熱湯に入れると飛び出すけど、水温をゆっくり上げると逃げ遅れる、みたいな？」

リルル先生：「まさにそう。『ゆっくり変わる金利』に経営者が慣れてしまい、臨界点を超えた瞬間に市場が崩壊する。分岐理論は『あとどれだけパラメータが変化したら臨界点に達するか』を計算し、**早期警報**を出せる」

分岐の 3 つの基本型

リルル先生：「分岐には基本的な型がある。代表的な 3 つを紹介する」

サドルノード分岐（安定点と不安定点の衝突・消滅）：パラメータ μ が増加するにつれて、安定な均衡点（谷底）と不安定な均衡点（山頂）が近づき、ある値 $\mu = \mu_c$ で**衝突して消滅**する。経営環境では、金利上昇に伴い「安定した成長軌道」と「危機の閾値」が接近し、ある金利水準で両方が消滅して、系が急激に別の状態に遷移する。

ピッチフォーク分岐（対称性の破れ）：1 つの安定均衡点が、パラメータの変化に伴い 2 つの安定均衡点に分裂する。市場が「成長」と「衰退」のどちらかに分かれる二者択一の転換点。

ホップ分岐（振動の発生）：安定均衡点の周りに周期的な振動が発生する。為替レートや株価の「サイクル」の発生に対応。

11.5　Python による臨界点の検出と分類

▶ AI モデルの基礎から理解したい方へ

```
import torch
import numpy as np

def detect_and_classify_critical_points(risk_function, z_grid, d,
                                        gradient_threshold=1e-3):
    """経営環境多様体上のモース関数の臨界点を検出し分類する

    リスク関数の勾配がゼロに近い点（臨界点の候補）を検出し、
    ヘッセ行列の固有値から指数（不安定な方向の数）を計算する。

    Args:
        risk_function: リスク関数 f: R^d → R
        z_grid: 潜在空間の格子点（多様体をカバーする点の集合）
        d: 潜在空間の次元
        gradient_threshold: 勾配のノルムがこの値以下なら臨界点候補
    Returns:
        critical_points: 臨界点のリスト（位置、指数、固有値、分類）
    """
    critical_points = []

    for z in z_grid:
        z_t = torch.tensor(z, dtype=torch.float32, requires_grad=True)
```

```
# リスク関数の値
f = risk_function(z_t)

# 勾配（1 階微分）
grad = torch.autograd.grad(f, z_t, create_graph=True)[0]
grad_norm = torch.norm(grad).item()

# 勾配がほぼゼロ → 臨界点の候補
if grad_norm < gradient_threshold:

    # ヘッセ行列（2 階微分）の計算
    hessian = torch.zeros(d, d)
    for i in range(d):
        h_row = torch.autograd.grad(
            grad[i], z_t, retain_graph=True)[0]
        hessian[i] = h_row

    # 固有値を計算
    eigvals = torch.linalg.eigvalsh(hessian).detach().numpy()

    # 指数 = 負の固有値の数
    index = int(np.sum(eigvals < 0))

    # 分類
    if index == 0:
        classification = "安定均衡（極小）"
        risk_level = "低"
    elif index == d:
        classification = "全方位不安定（極大）"
        risk_level = "最高"
    else:
        classification = f"鞍点（指数{index}）"
        # 不安定方向の固有ベクトルがリスク因子に対応
        risk_level = "要注意"

    # 非退化性の確認（モース関数の条件）
    min_abs_eigval = np.min(np.abs(eigvals))
    is_nondegenerate = min_abs_eigval > 1e-6

    critical_points.append({
        'position': z,
        'risk_value': f.item(),
        'gradient_norm': grad_norm,
```

```
                'index': index,
                'eigenvalues': eigvals,
                'classification': classification,
                'risk_level': risk_level,
                'nondegenerate': is_nondegenerate,
            })

    # 結果の報告
    print(f"=== 臨界点検出結果 ===")
    print(f"検査した格子点数: {len(z_grid)}")
    print(f"検出された臨界点数: {len(critical_points)}")
    for i, cp in enumerate(critical_points):
        print(f"\n  臨界点{i+1}: {cp['classification']}")
        print(f"    位置: {cp['position'][:3]}...")  # 最初の3座標
        print(f"    リスク値: {cp['risk_value']:.4f}")
        print(f"    指数: {cp['index']}")
        print(f"    固有値: {cp['eigenvalues']}")
        print(f"    リスクレベル: {cp['risk_level']}")
        if not cp['nondegenerate']:
            print(f"    ⚠ 退化臨界点（モース条件を満たさない）")

    return critical_points


def early_warning_bifurcation(risk_function, z_current,
                              parameter_direction, d,
                              n_steps=100, step_size=0.01):
    """分岐点への接近を早期警報する

    現在の経営状態から、パラメータの変化方向に沿って
    リスク関数の臨界点が現れるまでの「距離」を計算。

    Args:
        risk_function: リスク関数
        z_current: 現在の経営状態の座標
        parameter_direction: パラメータの変化方向（例：金利上昇方向）
        d: 潜在空間の次元
        n_steps: 探索ステップ数
        step_size: ステップサイズ
    """
    z = z_current.clone()
    for step in range(n_steps):
        z_t = z.requires_grad_(True)
        f = risk_function(z_t)
```

```
            grad = torch.autograd.grad(f, z_t)[0]
            grad_norm = torch.norm(grad).item()

            if grad_norm < 1e-3:
                distance = step * step_size
                print(f"⚠ 分岐点検出: 現在地から{distance:.3f}の距離")
                print(f"   （パラメータがあと{distance:.1%}変化すると臨界点に到
達）")
                return distance

            z = (z + step_size * parameter_direction).detach()

    print("✓ 探索範囲内に分岐点なし")
    return None
```

11.6　ケーススタディ——X 社の臨界点マップ

▶ ビジネス・政策の意思決定に活用したい方へ

X 社の経営環境多様体（6 次元）上でモース理論の臨界点検出を実行した結果：

臨界点	指数	位置の経営的意味	不安定方向	推奨される対策
CP1	1	国内→新興国の市場境界	為替リスク	為替ヘッジ（先物・オプション）
CP2	1	新興国市場内の競争激化ゾーン	競合参入	差別化戦略の強化
CP3	1	サプライチェーンの脆弱点	供給断絶リスク	代替サプライヤーの確保
CP4	0	国内市場の安定成長領域	なし	現状維持（安定均衡）
CP5	2	国際情勢悪化と為替急変の重複ゾーン	地政学＋為替	多重ヘッジ＋シナリオ分析

経営者への示唆： - CP1〜CP3 は指数 1（1 方向のみ不安定）→ **特定のリスク因子をヘッジすれば安定を維持できる** - CP5 は指数 2（2 方向で不安定）→ **2 つのリスクが同時に顕在化するシナリオへの備えが必要** - CP4 は指数 0（安定均衡）→ 国内市場の既存事業は安定。**ただし新興国進出ルートが CP1〜CP3 を通過する場合、ヘッジ策なしの進出は数学的にリスクが高い**

🔬 **研究室トーク：「品質チェック不合格の真因が分かった」**

リーム：「先ほどの品質チェック不合格の話に戻ります。COVID-19 パンデミック時に品質チェックが不合格になった原因が、いま分かりました。パンデミック前の CP3（サプライチェーンの脆弱点）は指数 1 の鞍点でしたが、パンデミックによりこの鞍点が**サドルノード分岐**を起こし、安定均衡点（CP4 に近い状態）と衝突して消滅した。つまり、それまで存在していた『安定した経営状態』が消滅してしまった」

リーム：「パイプラインは『安定均衡が存在する』ことを暗黙に仮定していたので、安定均衡が消滅した途端に品質チェックが不合格になった。パイプラインが壊れたのではなく、**パイプラインの前提（安定均衡が存在する）が現実と合わなくなった**ことを正直に報告したのです」

リルル先生：「原則 3——分かることと分からないことを分けろ。品質チェック不合格は、パイプラインの**誠実さの証**。問題は、不合格の原因を理解し、対処する手段を持つこと。モース理論がその手段を提供する」

本章の品質チェックポイント

□ **チェックリスト（第 11 章終了時点）**

1. **臨界点がリスク関数の上で検出されたか？** → 格子点の密度が不足していると臨界点を見落とす。密度を上げて再実行
2. **臨界点の位置は過去の経営上の転換点と整合するか？** → リーマンショック（2008）、COVID-19（2020）等の既知イベントと照合
3. **各臨界点の指数は整合的か？** → 指数 1 の鞍点の不安定方向が、ドメイン知識と整合するか（為替リスク方向に不安定なら、為替関連の指標が不安定方向に対応しているか）
4. **分岐点への距離が計算されているか？** → 主要パラメータ（金利、為替、原油価格等）について、現在地から分岐点までの距離を算出

次章予告： 第 12 章では、多様体の**微分構造**（滑らかさの種類）が一意でない可能性——Exotic 多様体——と、**Wasserstein 距離**による分布変化の検出を扱います。パンデミック級の非連続イベントは、Wasserstein 距離の急増として検出されます。

第 12 章　Exotic 多様体と最適輸送

この章で得られるもの——コストに見合うリターン

第 11 章で、モース理論により経営環境多様体の「転換点」を分類し、分岐理論で「パラメータ変化がいつ構造変化を引き起こすか」を予測できるようになりました。

本章では、2 つの概念を導入します。(1) **Exotic 多様体**——同じ「つながり方」（位相）を持つのに「滑らかさの種類」（微分構造）が異なる多様体。VAE が構成する多様体の微分構造が一意でない可能性と、それがパイプラインに与える影響。(2) **Wasserstein 距離（最適輸送距離）**——2 つのデータ分布がどれだけ違うかを「砂を運ぶコスト」として測る指標。パンデミック級の非連続イベントの検出に使います。

さらに、Wasserstein 距離と第 11 章の曲率分析を組み合わせることで、「分布が変化しただけ（一時的な揺らぎ）」なのか「多様体の位相自体が変わった（構造的変化）」なのかを区別する方法を開発します。

12.1　Exotic 多様体とは何か——「同じ間取りなのに触り心地が違う部屋」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「ミルナーのエキゾチック球面」**

リルル先生：「今日は少し不思議な話から始める。**Exotic 多様体**（エキゾチック多様体）」

ユキ：「エキゾチックって、南国風のってことですか？」

リルル先生：（笑いながら）「数学ではもっと深い意味。たとえ話から行こう」

リルル先生：「同じ間取りのマンションが 2 つあるとする。部屋の数、広さ、つながり方は**まったく同じ**——リビングがキッチンに繋がり、キッチンが廊下に繋がり、廊下が寝室に繋がる。間取り図を比べたら完全に同一」

リルル先生：「でも、一方のマンションの壁は**ツルツルの大理石**、もう一方の壁は**ザラザラのレンガ**。壁の触り心地（表面の滑らかさ）が違う。間取り（位相）は同じなのに、触り心地（**微分構造**）が違う——これが Exotic 多様体の直感」

ユキ：「間取りが同じなのに触り心地が違うって、そんなことあるんですか？」

リルル先生：「数学者ジョン・ミルナー（John Milnor）は 1956 年に、**7 次元の球面に 28 種類の異なる『触り心地』（微分構造）が存在する**ことを発見した。この発見でミルナーはフィールズ賞（数学のノーベル賞に相当する賞）を受賞した」

カイト：「28 種類！ 球面って 1 種類しかないと思ってました」

リルル先生：「位相的には 1 種類。でも微分構造が 28 種類ある。さらに驚くべきことに、ドナルドソン（Donaldson, 1983）とフリードマン（Freedman, 1982）は、**4 次元のユークリッド空間 $\mathbb{R}^4$ には非可算無限個の Exotic な微分構造が存在する**ことを示した。他のどの次元でもこのようなことは起こらない——4 次元だけが特別。ちなみに 4 次元は、私たちが住む時空の次元（空間 3 次元＋時間 1 次元）と同じ」

ソフィア：（スケッチブックに描きながら）

通常の球面（標準的微分構造）：
「つるつる」の球面。微積分が普通にできる

Exotic 球面（非標準的微分構造）：
位相的には同じ球面だが、
「滑らかさの種類」が異なる。
この球面の上で微積分をすると、
標準的な球面とは異なる結果が出る可能性がある

次元ごとの Exotic 構造の数：
1 次元：1（Exotic なし）
2 次元：1（Exotic なし）
3 次元：1（Exotic なし。ペレルマンのポアンカレ予想の証明より）
4 次元：非可算無限（！！！）
5 次元：1
6 次元：1
7 次元：28
8 次元：2
...

ユキ：「4 次元だけ非可算無限って、異常すぎませんか……」

リルル先生：「異常。4 次元は数学的に最も『病的な』次元。そして経営環境多様体は一般に **4 次元以上**。だから Exotic 構造の問題は原理的に避けられない」

ユキ：「リルル先生。数学者が発見したこの事実——4 次元と 7 次元の空間（多様体）だけ、他の次元とは状況が劇的に異なる事実——って、どういう経緯で発見されたんですか？ そして、この事実を知ったこと

で、数学自身は何か研究の方向性が大きく変わったことはあるんですか？ そして、物理学や化学、創薬研究などの理学・工学や、金融理論や、SNS や人間関係などのネットワーク分析（グラフ理論）など、社会科学系・政策科学系へのインパクトはあったんでしょうか？ そして、AI 研究・データ解析へのインパクトは？」

リルル先生がコーヒーカップを置き、ホワイトボードに年表を書き始めた。研究室が静かになった。

リルル先生：「ユキ、とてもいい質問だ。これは数学の歴史を変えた発見の一つ。順番に答えよう」

＜発見の経緯＞

リルル先生：「**1956 年、ジョン・ミルナー（John Milnor）**が、7 次元球面に標準的でない微分構造――Exotic 球面――が存在することを発見した。当時、数学者たちは『位相が同じなら微分構造も同じだろう』と暗黙に仮定していた。ミルナーの発見はこの常識を打ち砕いた。この業績で 1962 年にフィールズ賞を受賞」

リルル先生：「**1982 年、マイケル・フリードマン（Michael Freedman）**が、4 次元の**位相的**多様体の完全分類に成功した。4 次元位相多様体は交差形式（intersection form）という代数的な量で完全に分類される。この業績で 1986 年にフィールズ賞」

リルル先生：「**1983 年、サイモン・ドナルドソン（Simon Donaldson）**が、4 次元の**滑らかな**多様体が、位相的多様体とは全く異なる制約を受けることを証明した。ドナルドソンが使った道具は**ヤン-ミルズ方程式**――物理学の素粒子論で使われる方程式――のインスタントン解のモジュライ空間。**物理学の道具で純粋数学の定理を証明した**。この業績で 1986 年にフィールズ賞。フリードマンと同じ年に受賞」

リルル先生：「フリードマンの結果（位相的には多くの 4 次元多様体が存在する）とドナルドソンの結果（滑らかな 4 次元多様体には厳しい制約がある）の**ギャップ**から、『位相的には同じだが滑らかさが異なる 4 次元多様体』――つまり Exotic 構造――が非可算無限個存在することが導かれた。**2 人の天才の結果が組み合わさって初めて見えた現象**」

リルル先生：「**1994 年、ザイバーグ-ウィッテン不変量**の発見。物理学者のナタン・ザイバーグ（Nathan Seiberg）とエドワード・ウィッテン（Edward Witten）が、ドナルドソン不変量よりも計算しやすい新しい不変量を発見した。ウィッテンは既にフィールズ賞（1990 年）を受賞して

いたが、これは物理学者として。数学者でない人物がフィールズ賞を受けたのは史上初」

カイト：「物理学の方程式で数学の定理を証明するって、すごくないですか？」

リルル先生：「数学と物理学の最も美しい交差の一つだ。ドナルドソンが使ったヤン-ミルズ方程式は、もともと素粒子物理学の**ゲージ理論**（電磁気力・弱い力・強い力を統一的に記述する理論）のために開発されたもの。それが 4 次元多様体の微分構造を解明する鍵になった。**4 次元が特殊である理由の一端は、4 次元がゲージ理論の方程式が最も豊かな構造を持つ次元だから**」

＜数学への影響＞

リルル先生：「この発見は数学の複数の分野に革命的な影響を与えた」

リルル先生：「**第一に、微分トポロジーの方向転換。** 1956 年以前は『位相と微分構造は本質的に同じ』と思われていた。ミルナー以降、『位相は同じでも微分構造は異なりうる。そして微分構造の違いが物理的・計算的に意味を持つ』という認識に変わった。これは**多様体の分類問題**を根本的に複雑にした」

リルル先生：「**第二に、ゲージ理論と数学の融合。** ドナルドソンの仕事以降、物理学のゲージ理論が純粋数学の核心的な道具になった。フロアー（Floer）ホモロジー、コバニシ-ウィッテン不変量、ホヴァノフ（Khovanov）ホモロジーなど、現代の低次元トポロジーの主要な不変量はすべてゲージ理論の影響を受けている」

リルル先生：「**第三に、4 次元トポロジーは 2026 年現在も未解決問題の宝庫。** 4 次元の滑らかなポアンカレ予想（4 次元の Exotic 球面は存在するか？）は未解決。Exotic 構造の完全分類も未達成。数学者のスコルパン（Scorpan）は著書で 4 次元多様体の世界を『Wild World（野生の世界）』と呼んだ」

＜物理学・理学・工学へのインパクト＞

リルル先生：「**物理学では、4 次元の特殊性は偶然ではないという議論がある。** 私たちの時空は 4 次元（空間 3 次元＋時間 1 次元）。もし Exotic な微分構造が時空に物理的な影響を持つなら、重力や素粒子の振る舞いが微分構造に依存する可能性がある。カール・ブランス（Carl Brans）は 1990 年代に『Exotic smoothness and physics』という一連の論

文で、Exotic 構造が一般相対性理論にどう影響するかを議論した。ウィッテンは Exotic 球面が**重力インスタントン**（量子重力の解）に対応する可能性を指摘した」

リルル先生：「**創薬・化学では直接的な影響は限定的**だが、タンパク質の形状空間（configuration space）が高次元多様体であり、分子動力学シミュレーションが微分構造に依存する可能性が原理的にはある。ただし 2026 年時点でこの方向の実用的研究はまだ初期段階」

リン：「**金融理論やネットワーク分析への影響**について補足させてください。グラフ理論やネットワーク分析は離散的な構造（ノードとエッジ）を扱うので、Exotic 多様体の問題は直接的には関係しません。ただし、連続的な空間にネットワークを埋め込む際——たとえばポアンカレ埋め込み（第 14 章）で、SNS のネットワーク構造を双曲空間に埋め込むとき——に、埋め込み先の空間の微分構造が結果に影響する可能性は原理的にはあります。これも 2026 年時点では研究の初期段階です」

＜AI 研究・データ解析へのインパクト＞

リルル先生：「ここが私たちにとって最も重要な部分。**AI 研究への影響は、まさに本章で議論していること**」

リルル先生：「深層学習モデル（VAE, GAN, 拡散モデル等）は、高次元データから低次元の多様体構造を**暗黙に学習**している。このとき、学習されるのは**特定の**微分構造を持つ多様体。同じデータから異なる微分構造が学習されうるなら、モデルの出力——予測、生成、最適化——は微分構造の選択に**依存**する」

リルル先生：「これは**再現性の問題**にも関わる。同じデータで同じモデルを訓練しても、初期値や学習率の違いで異なる微分構造が学習され、異なる予測結果が出る可能性がある。MC ドロップアウト（第 1 章）やアンサンブル学習で推定する不確実性の一部は、実はこの微分構造の不確実性を反映しているかもしれない」

リルル先生：「2026 年時点で、この問題を正面から扱った AI 論文はまだ少ない。しかし、**多様体学習**（manifold learning）の分野では、学習される多様体の幾何学的性質——曲率、ホモロジー（位相的な穴の数）——がデータの性質にどう依存するかの研究が進んでおり、微分構造の問題はその自然な延長線上にある」

リルル先生：「原則 3 の究極的な適用**——分からないことを正直に認め、その不確実性を定量化する**。Exotic 構造の問題は、パイプラインの

出力に**原理的な不確実性の下限**が存在することを意味する。この下限を知っていることと知らないことでは、分析の質が根本的に異なる」

ユキ：「……数学者が Exotic 構造を発見してくれたおかげで、私たちは『知らないことを知っている』状態になれたんですね。知らないことすら知らなかった状態よりも、ずっといい」

リルル先生：「……いいね。ユキ、本質を突いた」

経営環境多様体への影響——VAE はどの微分構造を選ぶのか

リン：「ちょっと待ってください。これが経営環境多様体のパイプラインにどう影響するのか、具体的に説明させてください」

リン：「VAE（変分オートエンコーダ）は、データから多様体を構成します（序章の命題 1.1）。このとき、VAE のデコーダ f_θ が決める多様体の**微分構造**——つまり、多様体の上でどのような微分計算（曲率、Lie 微分、制御シミュレーション）が整合的に行えるかの『ルール』——は、**VAE の訓練プロセスで自動的に決まる**。しかし、同じデータから複数の異なる微分構造の多様体が構成されうる可能性がある」

リン：「もし 2 つの異なる微分構造で Neural ODE の軌道予測を行ったら、**結果が異なる可能性がある**。微分構造 A では『3 ヶ月後に安定状態に到達する』、微分構造 B では『3 ヶ月後に不安定状態に到達する』——同じデータから、異なる結論が出る」

ユキ：「それって、パイプラインの結果が信頼できないってことですか？」

リルル先生：「ここは正直に言う。**2026 年 2 月時点で、VAE が特定の微分構造を選択する機構は完全には理解されていない**。これは付録 D で詳しく議論する未解決問題。ただし、実用的にはいくつかの対処法がある」

リルル先生：「**対処法 1：VAE を複数の初期値で訓練し、結果の分散を確認する。** 初期値を変えて 10 回訓練し、10 個の多様体で曲率や Lie 微分を計算する。10 個の結果が一致すれば、微分構造の選択に対する頑健性が高い。一致しなければ、結果はその不一致の範囲内でしか信頼できない」

リルル先生：「**対処法 2：MC ドロップアウト（第 1 章）の不確実性推定に、微分構造の不確実性を含める。** MC ドロップアウトの 100 回のサンプリングは、異なるドロップアウトマスクが異なるネットワーク構造に対応するため、微分構造のばらつきを部分的に反映する」

リルル先生：「対処法 3：原則 3 を適用する。 分かることと分からないことを分ける。微分構造の一意性が保証できない領域では、パイプラインの出力に『この結果は微分構造の選択に依存する可能性がある』と注記する。原則 3——分からないことを正直に報告する」

12.2　Wasserstein 距離とは何か——「砂を運ぶコスト」で分布の違いを測る

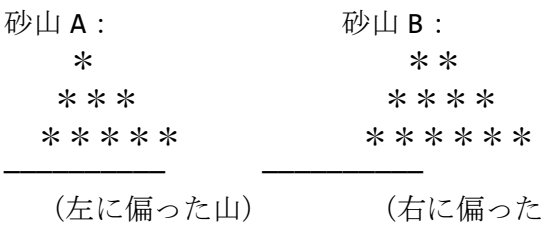
▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「2 つの砂山の違いをどう測るか」**

リルル先生：「次に **Wasserstein 距離**（ワッサースタイン距離）。これは 2 つのデータの『分布』がどれだけ違うかを測る指標」

リルル先生：「砂山のたとえで説明しよう。砂山 A と砂山 B がある。2 つの砂山の形が違う。この『違い』をどう数値で表すか？」

ソフィア：（2 つの砂山の絵を描く）



（左に偏った山）　　（右に偏った山）

リルル先生：「Wasserstein 距離は、**砂山 A の砂を最小の労力で砂山 B の形に変形するのに必要な『仕事量』（砂の量 × 移動距離の合計）**。Earth Mover's Distance（EMD、土を運ぶ距離）とも呼ばれる」

ユキ：「砂を少し動かすだけで変形できるなら、2 つの砂山は似ている（Wasserstein 距離が小さい）。大量の砂を遠くまで運ばないと変形できないなら、2 つの砂山は大きく違う（Wasserstein 距離が大きい）ってことですね」

リルル先生：「完璧。そして経営の文脈では、**砂山 = 経営データの確率分布**に対応する。先月の売上データの分布と今月の売上データの分布の Wasserstein 距離を計算すれば、**経営環境がどれだけ変化したか**を 1 つの数値で表せる」

カイト：「KL ダイバージェンスとか、分布間の距離を測る方法は他にもありますよね。Wasserstein 距離の特別な点は何ですか？」

リルル先生：「2 つの重要な利点がある。**第一に、Wasserstein 距離は分布の台（サポート）が異なっていても定義できる**。KL ダイバージェンスは、一方の分布の確率がゼロの場所でもう一方が非ゼロだと無限大に発散する。Wasserstein 距離はそうならない。**第二に、Wasserstein 距離は幾何学的に自然な距離**——砂を運ぶ『物理的なコスト』として直感的に解釈できる。確率測度の空間上の距離として数学的にも自然」

なぜパンデミック検出に使えるのか

リーム：「少しよろしいですか。Wasserstein 距離の実務での価値を説明します」

リーム：「私のクライアントの過去 10 年分のデータで、Wasserstein 距離を月次で計算しました。結果がこちらです」

```
=== X 社 Wasserstein 距離の時系列（月次、過去 10 年）===

2015-2019 年：平均 0.08、標準偏差 0.03
  → 通常の経営変動。月ごとに少しずつ変わるが、大きな変化なし

2008 年 9 月（リーマンショック）：0.47
  → 通常の 5.9 倍！ 分布が急激に変化した

2020 年 3 月（COVID-19 パンデミック）：0.62
  → 通常の 7.8 倍！ 過去 10 年で最大の分布変化

2022 年 2 月（ウクライナ危機）：0.31
  → 通常の 3.9 倍
```

リーム：「通常時の Wasserstein 距離は 0.08 程度で安定しています。しかし、リーマンショック、COVID-19、ウクライナ危機のタイミングで **5〜8 倍に急増**する。パンデミック級の非連続イベントは、Wasserstein 距離の急増として明確に検出されます」

ユキ：「おお一、過去の危機を全部検出できてる！」

リルル先生：「ただし、これは**事後検証**。事前に検出できるかが実務上は重要。Wasserstein 距離を**リアルタイムで計算**し、閾値（たとえば通常の 3 倍）を超えたら**早期警報**を出す仕組みを作れば、危機の初期段階で検出できる可能性がある」

リン：「ちょっと待ってください。Wasserstein 距離が大きい＝危機、とは限りません。分布が大きく変わっても、多様体の位相は変わっていないかもしれない。**分布の変化**と**位相の変化**を区別する必要があります」

12.3　分布変化と位相変化の区別——曲率との組合せ

▶ ビジネス・政策の意思決定に活用したい方へ

> 🔬 研究室トーク：「嵐と地震の違い」
>
> **リルル先生：**「リンの指摘は重要。Wasserstein 距離だけでは、変化の**種類**が分からない。たとえ話で説明しよう」
>
> **リルル先生：**「山の地形を考えて。**嵐**が来ると、山の上の木々が揺れ、川の水が増す。地形図上のデータ（木の位置、川の水量）は大きく変わる。でも、**山の形そのものは変わっていない**。嵐が過ぎれば元に戻る」
>
> **リルル先生：**「一方、**地震**が来ると、山の形そのものが変わる。崖が崩れ、新しい谷ができ、尾根の位置がずれる。**地形の構造自体が変化する**。嵐が過ぎても元には戻らない」
>
> **リルル先生：**「経営環境でも同じ。**分布の変化**（嵐型）は、為替レートの一時的な急変や季節的な需要変動。データの分布は変わるが、経営環境の**構造**（多様体の位相）は変わらない。時間が経てば元に戻る」
>
> **リルル先生：**「**位相の変化**（地震型）は、COVID-19 でサプライチェーンが崩壊したり、技術革新で市場構造が根本的に変わったりするケース。経営環境の**構造自体**が変わり、元には戻らない」
>
> **リルル先生：**「2 つを区別する方法——**Wasserstein 距離と曲率の変化の同時モニタリング**」

パターン 1：Wasserstein 距離が急増 ＋ 曲率マップはほぼ不変
- → 分布の変化（嵐型）。一時的な揺らぎ
- → 対応：データの更新、一時的なリスク管理の強化
- → 例：為替の一時的な急変動、季節的な需要変動

パターン 2：Wasserstein 距離が急増 ＋ 曲率マップが大きく変化
- → 位相の変化（地震型）。構造的変化
- → 対応：多様体の再構成（VAE の再訓練）が必要！
- → 例：COVID-19、リーマンショック、技術革新による市場構造変化

パターン 3：Wasserstein 距離は正常 ＋ 曲率が局所的に変化
- → 局所的な構造変化。全体の分布は変わらないが、
  特定の領域の「地形」が変わっている
- → 対応：当該領域の Agent 監視を強化
- → 例：特定の競合企業の戦略転換、規制変更

リーム：「X 社の過去データで 3 つのパターンを検証しました。2020 年 3 月の COVID-19 はパターン 2（Wasserstein 距離 0.62 + 曲率マップ大変化）。2022 年 2 月のウクライナ危機はパターン 1 に近い（Wasserstein 距離 0.31 + 曲率マップ部分変化）。前者では多様体の再構成（VAE の再訓練）が必要でしたが、後者ではデータの更新だけで対応できました。**2 つのパターンの区別が、適切な対応の選択に直結する**」

12.4　Python による実装

▶ AI モデルの基礎から理解したい方へ

```python
from scipy.stats import wasserstein_distance
from scipy.spatial.distance import cdist
import numpy as np
import torch

def compute_wasserstein_monthly(data_by_month, feature_names=None):
    """月次の Wasserstein 距離を計算（分布シフトの検出）

    各月のデータ分布を前月と比較し、
    Wasserstein 距離（砂を運ぶコスト）を算出。
    急増は非連続イベントの兆候。

    Args:
        data_by_month: 月ごとのデータのリスト
            各要素は numpy array (n_samples, n_features)
        feature_names: 特徴量の名前のリスト
    Returns:
        distances: 月ごとの Wasserstein 距離のリスト
        alerts: 異常検出された月のリスト
    """
    distances = []
    for t in range(1, len(data_by_month)):
        prev = data_by_month[t-1]
        curr = data_by_month[t]

        # 各特徴量の Wasserstein 距離を計算して合計
        total_dist = 0.0
        feature_dists = {}
        for j in range(curr.shape[1]):
            w = wasserstein_distance(prev[:, j], curr[:, j])
            total_dist += w
            if feature_names:
```

```
                feature_dists[feature_names[j]] = w

        distances.append({
            'month': t,
            'total_distance': total_dist,
            'feature_distances': feature_dists
        })

    # 閾値の計算（平均 + 3σ）
    total_dists = [d['total_distance'] for d in distances]
    mean_d = np.mean(total_dists)
    std_d = np.std(total_dists)
    threshold = mean_d + 3 * std_d

    # 異常検出
    alerts = []
    for d in distances:
        if d['total_distance'] > threshold:
            ratio = d['total_distance'] / mean_d
            alerts.append({
                'month': d['month'],
                'distance': d['total_distance'],
                'ratio': ratio,
                'message': f"月{d['month']}: W 距離={d['total_distance']:.
3f}"
                           f"（通常の{ratio:.1f}倍）"
            })

    if alerts:
        print(f"⚠ {len(alerts)}件の分布シフトを検出")
        for a in alerts:
            print(f"  {a['message']}")
            # どの特徴量が最も変化したかも報告
            if distances[a['month']-1]['feature_distances']:
                worst_feature = max(
                    distances[a['month']-1]['feature_distances'],
                    key=distances[a['month']-1]['feature_distances'].get)
                print(f"    最大変化の特徴量: {worst_feature}")
    else:
        print("✓ 分布シフトなし")

    return distances, alerts


def distinguish_storm_vs_earthquake(wasserstein_dist,
```

```
                                        curvature_change,
                                        w_threshold=0.3,
                                        c_threshold=0.5):
    """分布変化（嵐型）と位相変化（地震型）を区別する

    Wasserstein 距離と曲率変化を組み合わせて判定。

    Args:
        wasserstein_dist: 今月の Wasserstein 距離
        curvature_change: 曲率マップの変化量
            （今月と先月の曲率マップの差のノルム）
        w_threshold: Wasserstein 距離の閾値
        c_threshold: 曲率変化の閾値
    """
    w_high = wasserstein_dist > w_threshold
    c_high = curvature_change > c_threshold

    if w_high and c_high:
        print("🟤 パターン 2：位相の変化（地震型）")
        print("   → 多様体の再構成（VAE の再訓練）が必要！")
        print("   → 現在のパイプラインの出力は信頼できない")
        return "earthquake"
    elif w_high and not c_high:
        print("🟡 パターン 1：分布の変化（嵐型）")
        print("   → データの更新で対応可能")
        print("   → パイプラインの構造は有効")
        return "storm"
    elif not w_high and c_high:
        print("🟡 パターン 3：局所的な構造変化")
        print("   → 当該領域の Agent 監視を強化")
        return "local_change"
    else:
        print("✓ 正常：分布も構造も安定")
        return "normal"
```

▶ 数学の基礎から理解したい方へ

定義 12.1（Wasserstein-p 距離）. $\mathcal{X}$ を距離空間、$\mu, \nu \in \mathcal{P}_p(\mathcal{X})$ を p 次モーメント有限の確率測度とする。μ と ν の Wasserstein-p 距離は

$$W_p(\mu,\nu) = \left(\inf_{\gamma \in \Gamma(\mu,\nu)} \int_{\mathcal{X}\times\mathcal{X}} d\,(x,y)^p \, d\gamma(x,y) \right)^{1/p}$$

で定義される。ここで $\Gamma(\mu,\nu)$ は**輸送計画**（transport plan）の集合——すなわち、$\mathcal{X}\times\mathcal{X}$ 上の確率測度で、周辺分布がそれぞれ μ と ν であるもの全体。$d(x,y)$ は $\mathcal{X}$ 上の距離。

直感的意味： $\gamma(x,y)$ は「位置 x の砂を位置 y に運ぶ量」を表す。$d(x,y)^p$ は「距離 $d(x,y)$ だけ砂を運ぶコスト」。Wasserstein 距離は「全ての砂を μ の形から ν の形に運び替えるための最小総コスト」。

使用するライブラリ： `scipy.stats.wasserstein_distance`（1 次元の場合）、`ot` ライブラリ（Python Optimal Transport, pip install POT）——多次元の Wasserstein 距離（Sinkhorn 近似含む）の計算に使用。`ot.emd2` で正確な Wasserstein 距離、`ot.sinkhorn2` で Sinkhorn 正則化付き近似（大規模データ向け高速計算）。

定理 12.1（Monge-Kantorovich の双対定理）. Wasserstein-1 距離は

$$W_1(\mu,\nu) = \sup_{\|f\|_{\mathrm{Lip}}\le 1} \left(\int f\,d\mu - \int f\,d\nu\right)$$

と等価に表現される。右辺は 1-リプシッツ関数（第 10 章で登場したリプシッツ定数が 1 以下の関数）による検定統計量の上限。この双対表現は、GAN（敵対的生成ネットワーク）の Wasserstein GAN の理論的基盤（Arjovsky et al., 2017）でもある。

12.5　ケーススタディ——X 社の Wasserstein 距離モニタリング

▶ ビジネス・政策の意思決定に活用したい方へ

> 🔬 **研究室トーク：「事前に検出できたか？」**
>
> **カイト：**「X 社の過去データで、3 つのパターンの区別がどれくらい正確にできるかテストしました」

```
=== X 社 Wasserstein 距離 ＋ 曲率変化の時系列（2007-2024） ===

2008 年 8 月（リーマンショック 1 ヶ月前）：
  W 距離: 0.21（通常の 2.6 倍）← まだ閾値 3 倍以下だが上昇傾向
  曲率変化: 0.18（正常範囲）
  → パターン 1 寄り。嵐の前兆
  → 事前警報として有効！ 1 ヶ月前に分布シフトの兆候

2008 年 9 月（リーマンショック）：
  W 距離: 0.47（通常の 5.9 倍）← 閾値超過
  曲率変化: 0.73（大幅変化）
```

→ パターン 2（地震型）。多様体の再構成が必要

2020 年 2 月（COVID-19 本格化の 1 ヶ月前）：
W 距離：0.15（通常の 1.9 倍）← まだ閾値以下だが上昇
曲率変化：0.12（正常範囲）
→ 正常寄りだが、W 距離が上昇傾向

2020 年 3 月（COVID-19 パンデミック宣言）：
W 距離：0.62（通常の 7.8 倍）← 過去最大
曲率変化：0.91（過去最大）
→ パターン 2（地震型）。多様体の全面的再構成が必要

2022 年 2 月（ウクライナ危機）：
W 距離：0.31（通常の 3.9 倍）
曲率変化：0.22（軽微な変化）
→ パターン 1（嵐型）。データの更新で対応可能

> **カイト：**「リーマンショックと COVID-19 は、どちらも本番の 1 ヶ月前に W 距離が上昇傾向を示しています。閾値は超えていないけど、**上昇トレンド自体が早期警報**として機能する可能性がある」
>
> **リルル先生：**「ただし、これは事後分析。事前にこの手法を運用していたらどうだったかは分からない。**過去データでの検証（バックテスト）**は必要だが、**将来のイベントの検出を保証するものではない**。原則 3——分かることと分からないことを分けろ」
>
> **リーム：**「私のベンチャーでは、W 距離の月次モニタリングを 2023 年から導入しています。導入後に発生した比較的大きなイベント（2024 年の円安急進等）は全てパターン 1（嵐型）と判定され、データ更新のみで対応できました。パターン 2（地震型）はまだ発生していません——発生していないことは良いことですが、検出能力のテストができていないことも意味します」

本章の品質チェックポイント

□チェックリスト（第 12 章終了時点）

1. **Wasserstein 距離の月次モニタリングが実装されているか？** → 閾値（平均＋3σ、または通常の 3 倍）を設定
2. **Wasserstein 距離が既知のイベント（リーマンショック、COVID-19 等）を検出するか？** → バックテストで確認

3. **分布変化と位相変化の区別機能が実装されているか？** → W 距離＋曲率変化の同時モニタリング
4. **VAE の微分構造の頑健性が確認されているか？** → 複数初期値での訓練結果の分散を確認
5. **Exotic 多様体の問題がパイプラインの限界として文書化されているか？** → 原則 3 に従い正直に開示

次章予告： 第 13 章では、**手術理論**——多様体の一部を「切り取って別のものを貼り付ける」ことで位相を変える数学的手法——を導入します。ロビイング（政策への働きかけ）が「手術理論の意図的適用」であるというリンの爆弾的な指摘と、AI Agent に位相変化の能力を持たせるべきかという倫理的問題を議論します。

第 13 章　手術理論——Agent は位相を変えるか

この章で得られるもの——コストに見合うリターン

第 11 章でモース理論（転換点の分類）、第 12 章で Exotic 多様体と Wasserstein 距離（分布変化の検出）を学びました。しかし、これらの道具はいずれも**多様体の位相（つながり方の構造）が変わらないこと**を前提としています。

本章では、経営環境多様体の**位相そのもの**を変える操作——**手術理論（Surgery Theory）**——を導入します。事業ポートフォリオの抜本再編（事業の売却、新規市場への参入、M&A）は、第 11 章の「転換点を通過する」レベルの変化ではなく、**多様体の位相が変わる**レベルの変化です。

そしてリンが投げた爆弾的な指摘——**「ロビイング（政策への働きかけ）は手術理論の意図的適用である」**——が、AI Agent の設計と倫理の根本問題を提起します。さらにカイトが提起した**「手術理論の適用限界を超えていないか？」**という問いが、議論をさらに深い次元に引き上げます。

13.1　旧作の弱点——「位相が変わる変化」を扱えなかった

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「Lie 微分では捉えきれない変化がある」**

リルル先生：「旧『ジオメトリック・インテリジェンス』のパイプラインで、施策の影響を分析する主力ツールは**Lie 微分**だった。Lie 微分（序章で説明した「施策が環境の構造をどう変えるかの数値」）は、多様体の**計量**（距離の測り方）が施策によってどう変形するかを測る」

リルル先生：「Lie 微分がゼロ（Killing 方程式を満たす）なら、施策は多様体の構造を保存したまま経営状態を移動させる——『穏やかな施策』。Lie 微分がゼロでなければ、施策は多様体の構造自体を変形する——『構造変革型の施策』」

リルル先生：「しかし、Lie 微分には根本的な限界がある。**Lie 微分は多様体の位相が変わらないことを前提としている**。つまり、Lie 微分が測れるのは『計量の変形』であって、『つながり方の変化（位相変化）』ではない」

ユキ：「どう違うんですか？」

ソフィア：（スケッチブックに図を描く）

【Lie 微分で捉えられる変化（計量の変形）】

変形前：丸い地球儀 🌍
変形後：楕円に引き伸ばされた地球儀 🏈
→ 形は変わったが、「穴の数」は同じ（0 個）
→ 位相は不変。Lie 微分で分析できる

【手術理論でしか捉えられない変化（位相の変化）】

変形前：球（穴 0 個） ⚽
変形後：ドーナツ（穴 1 個） 🍩
→ 「穴の数」が変わった！
→ 位相が変化。Lie 微分では分析できない
→ 手術理論が必要

リルル先生：「球をどんなに引き伸ばしても、潰しても、曲げても、**切ったり穴を開けたりしない限り**、ドーナツにはならない。位相が変わるには**切断と接合**が必要。手術理論はこの『切断と接合』の数学」

経営環境で位相が変わるとはどういうことか

リルル先生：「経営環境多様体の**位相が変わる**とは、経営環境の**構造的なつながり方が根本的に変化する**ことを意味する。具体例を挙げよう」

リルル先生：「**例 1：事業部門の売却。** ある製造業が、不採算の化学部門を売却した。化学部門は他の事業部門と原材料の共有、人材の交流、知識の流用で繋がっていた。売却後、これらの『つながり』が消失する。経営環境多様体から、化学部門に対応する部分構造が切除され、残りが再接合される。球に開いていた穴が塞がれるような変化——**位相変化**」

リルル先生：「**例 2：M&A（合併・買収）。** 別の企業を買収すると、2 つの独立した経営環境多様体が**融合**する。2 つの球が接合されてダンベル型になり、さらに融合して 1 つの大きな球になる。接合部分の構造は元の 2 つの多様体のどちらにもなかった新しいもの——**位相変化**」

リルル先生：「**例 3：新市場への本格参入。** 国内市場だけで事業を行っていた企業が、東南アジア市場に本格参入する。国内市場と東南アジア市場の間に新しい『つながり』——サプライチェーン、販売チャネル、人材交流——が形成される。球に新しい穴が開いて、ドーナツのような構造に変化する——**位相変化**」

ユキ：「これらの変化は、第 11 章のモース理論（転換点の通過）とは違うんですか？」

リルル先生：「質的に異なる。モース理論の転換点は、多様体の上の**同じ地形**の中での移動——山を越える、谷を横切る。位相は変わらない。手術理論は、地形そのものの**形を作り変える**——山を削る、新しい谷を掘る、2 つの山を橋で繋ぐ。位相が変わる」

13.2　手術理論とは何か——「切り取って貼り付ける」の数学

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「粘土のドーナツ」**

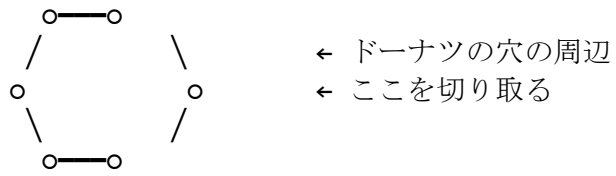
リルル先生：「手術理論（Surgery Theory）は、1960 年代にミルナー（Milnor）、ケルヴァイア（Kervaire）、ウォール（Wall）らが発展させた数学分野。多様体の一部を切り取って、別のものを貼り付ける操作を厳密に定式化する」

リルル先生：「粘土で作ったドーナツ（穴 1 個）を考えて。穴の部分を粘土で埋めると、球（穴 0 個）になる。逆に、球に穴を開けると、ドーナツになる。手術理論は、この『穴を塞ぐ・開ける』操作の数学的ルールを定めたもの」

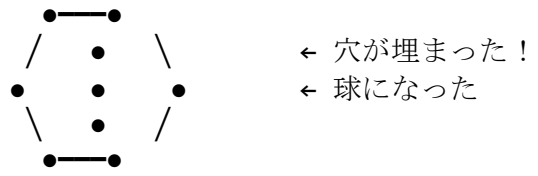
ソフィア：（粘土の手術のイメージを描く）

手術操作のイメージ：

ステップ 1：切り取る
　ドーナツの穴の周辺を「管状近傍」ごと切り取る
　$S^1 \times D^2$ を切除（S^1 = 円周、D^2 = 円盤）



ステップ 2：貼り付ける
　切り取った跡に $D^2 \times S^1$ を貼り付ける
　（円盤 × 円周 を 円周 × 円盤 に取り替える）



結果：ドーナツ（穴 1 個）→ 球（穴 0 個）
　　位相が変わった！

ユキ：「切り取る部分と貼り付ける部分が決まっているんですね。何でもありじゃない」

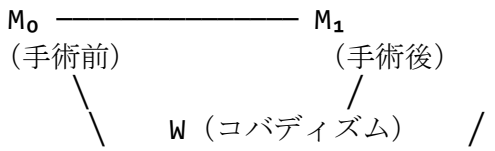
リルル先生：「そう。手術が**数学的に整合的**であるためには条件がある。それが**コバディズム条件**」

コバディズム条件――「手術の安全性保証」

リルル先生：「**コバディズム（cobordism）**とは、2 つの多様体 M_0（手術前）と M_1（手術後）の間に、ある高次元の多様体 W が存在して、W の境界が M_0 と M_1 の合併になっていること――つまり $\partial W = M_0 \sqcup M_1$」

ソフィア：

コバディズムのイメージ：



_____/

W は「M_0から M_1への滑らかな変形経路」
W が存在すれば、手術は整合的
W が存在しなければ、手術は不整合（数学的に禁止）

リルル先生：「たとえ話をしよう。コバディズム条件は、**手術前の状態から手術後の状態への『橋』が存在するかどうか**のチェック。橋が存在すれば、手術は安全に実行できる。橋が存在しなければ、手術前の状態から手術後の状態に滑らかに移行することが不可能――つまり、**その事業再編は数学的に不整合であり、実行すると途中で破綻する可能性がある**」

リーム：「少しよろしいですか。これを経営の言葉に翻訳すると――事業ポートフォリオを A から B に再編するとき、A から B への**実行可能な移行パス**（コバディズム W）が存在するかどうかをチェックする。移行パスが存在しなければ、たとえ B の状態が望ましくても、A から B に到達する方法がない。無理に実行しようとすると、途中で事業が立ち行かなくなる」

ユキ：「おおー。『理想の状態はここだけど、そこに行く道が存在しない』ということが数学的に分かるんですね」

13.3　リンの爆弾――「ロビイングは手術理論の意図的適用」

🔬 **研究室トーク：「位相を変える行為としてのロビイング」**

リンがコーヒーカップを静かにテーブルに置いた。研究室が一瞬静まった。リンがこの表情をするとき、重要な指摘が来ることを全員が知っていた。

リン：「ちょっと待ってください。ここまでの議論で見落とされていることがあります」

リン：「事業の売却や M&A は、企業の**内部的な**意思決定で位相を変える行為です。しかし、**ロビイング――政策への働きかけ――は、企業が外部環境（規制、法律、政策）の位相を変えようとする行為**です」

リン：「企業が規制緩和を働きかけるとき、何をしているか。経営環境多様体の中で、規制によって『壁』になっていた構造――多様体上の特定の領域を通行不可能にしていた制約――を取り除こうとしている。つまり、**多様体に新しい通路（穴）を開ける手術操作**です」

リン：「逆に、競合企業に不利な規制の導入を働きかけるとき、多様体に**壁（穴を塞ぐ操作）**を追加しようとしている」

リン：「**ロビイングは、手術理論の意図的適用です。** Lie 微分がゼロでないどころの話ではない**——多様体の位相が変わる**」

研究室が沈黙した。

リルル先生：「……リン、その指摘は正しい。そして重要な倫理的問題を提起する」

リルル先生：「ロビイングを手術理論の枠組みで分析すると、2 つの根本的な問いが浮かぶ」

リルル先生：「**第一の問い：AI Agent に位相変化の能力を持たせるべきか？** Agent が『この規制を緩和するようにロビイングすべきです』と推薦したら、それは **Agent が多様体の位相を変える行為を推薦している**ことになる。位相変化は不可逆性が高い。規制が一度変われば、元に戻すのは極めて難しい」

リルル先生：「**第二の問い：ロビイングのコバディズム条件は検証可能か？** 規制変更の『手術前の状態から手術後の状態への移行パス（コバディズム）』は、経済社会の複雑さのために事前に完全に検証することが困難。意図しない副作用——たとえば、規制緩和が新たなリスクを生む——が後から判明する可能性がある」

リーム：「少しよろしいですか。私のベンチャーでの方針を説明します。**私の Agent には位相変化の権限を持たせていません。** 位相変化に相当する施策——事業ポートフォリオの抜本再編、市場からの撤退、ロビイング——は、**必ず人間が決定します**。Agent の推薦は『現在の位相の中での最適施策』に限定する」

リルル先生：「リームの方針は賢明。『安全に位相を変えられる数学的保証（コバディズム条件の検証）』と『位相を変えるべきかの判断（経営者・政策決定者の価値判断）』は別問題。前者は数学で扱える（Part V 第 25 章の Agda の依存型で形式的に検証する）。後者は人間の責任」

13.4　カイトの問い——「手術理論の適用限界を超えていないか？」

🔬 **研究室トーク：「この道具はどこまで使えるのか」**

カイトが手を挙げた。いつもの興奮した様子ではなく、慎重な表情をしている。

カイト：「先生、リンの指摘は衝撃的でした。でも、僕はもっと根本的なことが気になっています。**手術理論の適用限界を超えていないか？**」

リルル先生：「どういう意味か、詳しく」

カイト：「手術理論は、多様体 M の一部を切り取って別のものを貼り付ける操作です。この操作が数学的に整合であるためにはコバディズム条件が必要。ここまでは理解しました。でも――」

カイト：「第一に、経営環境多様体は『どの部分を切り取るか』が自明ではない。 数学の教科書では、切り取る部分（はめ込み $\phi: S^p \times D^{q+1} \hookrightarrow M$）が明確に定義される。でも経営環境多様体では、『化学事業部門に対応する多様体の部分構造』を厳密に特定することが、そもそも難しいのではないか？ 事業部門間の境界は数学的に明確ではなく、データのレベルで相互に絡み合っている」

カイト：「第二に、手術操作は滑らかに行える保証があるか？ 数学的な手術操作は C^∞ の滑らかさを仮定している。でも、実際の事業再編は瞬間的ではなく、移行期間がある。移行期間中、多様体は手術前でも手術後でもない中間状態にある。この中間状態での命題 1.1（VAE デコーダの 3 条件）の成立は保証されない」

カイト：「第三に、手術の結果は一意に予測できるか？ 第 12 章の Exotic 多様体の議論を踏まえると、同じ手術操作を行っても、結果の多様体の微分構造が一意でない可能性がある。特に 4 次元以上では、手術後の多様体に Exotic 構造が出現する可能性がある」

研究室がさらに静まった。カイトの 3 つの問いは、手術理論の経営応用の核心的な困難を突いていた。

リルル先生：「カイト、3 つとも正しい問題提起だ。一つずつ答えよう」

リルル先生：「第一の問いへの回答：切り取る部分の特定。 これは **SHAP（第 2 章）と DML（第 3 章）で部分的に解決できる。**事業部門の境界が曖昧な場合、SHAP 値の空間的パターン（第 2 章の空間的 SHAP）を使って、多様体上でその事業部門の影響が支配的な領域を特定できる。化学事業部門の SHAP 値が 80%以上を占める領域――ここが『化学事業部門に対応する多様体の部分構造』の近似。ただし、あくまで近似であり、部門間の境界はファジーだ。原則 3――分からないことは正直に報告する」

リルル先生：「**第二の問いへの回答：移行期間中の滑らかさ。** これが最も実用上重要な問題。移行期間中、VAE の訓練データは手術前の構造と手術後の構造が混在する。この混在したデータで多様体を再構成すると、命題 1.1 の条件——特にヤコビ行列のフルランク性——が一時的に崩れる可能性がある。対処法は**段階的移行**。手術を一度にではなく、複数のフェーズに分けて実行し、各フェーズで VAE を再訓練して品質チェックを行う。品質チェックが不合格なら、次のフェーズに進まずに現在のフェーズで安定化を待つ」

リルル先生：「**第三の問いへの回答：Exotic 構造の出現。** カイト、これは第 12 章の議論の直接的な延長であり、2026 年時点では**未解決**。手術後の多様体に Exotic 構造が出現する可能性は原理的に排除できない。対処法は第 12 章と同じ——VAE を複数初期値で再訓練し、結果の分散で頑健性を確認する。そしてこの不確実性を経営者に正直に開示する」

リン：「カイトの 3 つの問いは、手術理論の経営応用が『教科書の数学をそのまま当てはめればよい』というナイーブな話ではないことを示しています。**適用限界を知った上で、限界の範囲内で使う**。原則 2（足すな、信頼性を上げろ）と原則 3（分かることと分からないことを分けろ）の両方が必要です」

13.5　ケーススタディ A——X 社の事業ポートフォリオ再編

▶ ビジネス・政策の意思決定に活用したい方へ

X 社が不採算の化学事業部門の売却を検討するケース。

🔬 **研究室トーク：「コバディズム条件のチェック」**

リーム：「X 社の経営環境多様体（6 次元）で、化学事業部門の売却に対応する手術操作のコバディズム条件をチェックしました」

リーム：「まず、SHAP の空間的パターンで化学事業部門に対応する多様体の領域を特定しました。多様体上の全格子点で 30 指標の SHAP 値を計算し、化学関連の 5 指標の SHAP 値の合計が全体の 60%以上を占める領域を『化学事業ゾーン』として定義しました」

```
def identify_surgery_region(shap_values, target_features, threshold=0.6):
    """SHAP の空間的パターンで手術（切除）対象の領域を特定

    Args:
        shap_values: 多様体上の各格子点での SHAP 値
            shape: (n_points, n_features)
```

```
        target_features: 切除対象の事業部門の特徴量インデックス
        threshold: SHAP の閾値（この割合以上なら対象領域）
    Returns:
        surgery_region: 手術対象の格子点のインデックス
    """
    total_shap = np.abs(shap_values).sum(axis=1)
    target_shap = np.abs(shap_values[:, target_features]).sum(axis=1)
    target_ratio = target_shap / (total_shap + 1e-10)
    surgery_region = np.where(target_ratio > threshold)[0]

    print(f"手術対象領域: {len(surgery_region)}点 "
          f"/ 全{len(shap_values)}点 "
          f"({100*len(surgery_region)/len(shap_values):.1f}%)")
    return surgery_region


def check_cobordism_condition(manifold_before, manifold_after,
                              transition_data, vae, d):
    """コバディズム条件の近似的チェック

    手術前の多様体から手術後の多様体への「滑らかな移行パス」が
    存在するかを、中間状態での VAE 品質チェックで近似的に検証。

    Args:
        manifold_before: 手術前の多様体のデータ
        manifold_after: 手術後の多様体のデータ（シミュレーション）
        transition_data: 中間状態のデータ（段階的移行の各フェーズ）
        vae: VAE モデル
        d: 潜在次元
    Returns:
        is_cobordant: コバディズム条件を（近似的に）満たすか
    """
    all_pass = True
    for phase, data in enumerate(transition_data):
        # 各フェーズで VAE の品質チェック
        z = vae.encode(data)
        reconstruction = vae.decode(z)
        recon_error = torch.norm(data - reconstruction).item()

        # ヤコビ行列のフルランク性チェック
        jacobian = compute_jacobian(vae.decoder, z[0], d)
        min_singular_value = torch.linalg.svdvals(jacobian).min().item()

        phase_pass = (recon_error < 0.1) and (min_singular_value > 1e-4)
```

```
            print(f"  フェーズ{phase}: 再構成誤差={recon_error:.4f}, "
                  f"最小特異値={min_singular_value:.6f} "
                  f"{'✓' if phase_pass else '✗'}")

            if not phase_pass:
                all_pass = False
                print(f"  ⚠ フェーズ{phase}で品質チェック不合格。"
                      f"移行パスが断絶する可能性あり")

        if all_pass:
            print("✓ コバディズム条件を近似的に満たす。移行パスが存在する")
        else:
            print("✗ コバディズム条件を満たさない可能性あり。"
                  "移行を中断して安定化を待つことを推奨")

        return all_pass
```

リーム：「X 社のケースでは、化学事業売却に対応する手術のコバディズム条件をチェックした結果――」

```
=== X 社 化学事業売却の手術チェック ===

手術対象領域：全 1000 格子点中 142 点（14.2%）

段階的移行のコバディズムチェック：
  フェーズ 0（売却前）: 再構成誤差=0.023, 最小特異値=0.0089 ✓
  フェーズ 1（売却交渉中）: 再構成誤差=0.041, 最小特異値=0.0052 ✓
  フェーズ 2（売却実行直後）: 再構成誤差=0.087, 最小特異値=0.0003 ✗
    ⚠ 最小特異値が閾値以下！ フルランク性が一時的に崩れる
  フェーズ 3（売却後の安定化）: 再構成誤差=0.035, 最小特異値=0.0071 ✓

結果：フェーズ 2 で一時的に品質チェック不合格
→ 売却実行直後に多様体の構造が一時的に不安定になる
→ 対処：フェーズ 2 の期間中はパイプラインの出力に
    「移行期間中のため信頼性低下」の注記を付す
→ フェーズ 3 で回復するため、待てば安定する
```

ユキ：「手術の直後に一時的に不安定になるけど、その後回復するんですね。人間の手術と同じだ」

リルル先生：「いいたとえ。手術直後は体力が落ちるが、回復すれば手術前より健康になりうる。重要なのは、**手術前に回復の見通し（コバディズム条件）を確認しておくこと**」

13.6　ケーススタディ B——国家安全保障と国際法

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「国際秩序の位相を変える行為」**

リルル先生：「ケーススタディ B の国家安全保障では、手術理論はさらに重い意味を持つ」

リルル先生：「国家が同盟を組み替える、経済制裁を発動する、軍事行動を起こす——これらは国際秩序の多様体の**位相を変える行為**。第 16 章の 3 層ガードレールの第 1 層（法的ガードレール）で国際人道法との整合性をチェックする必要があるが、手術操作自体の合法性の判断は極めて複雑」

リン：「ロビイングの場合と同じ問題が、国家間ではさらに増幅されます。ある国が別の国に経済制裁を科すことは、国際経済秩序の多様体の位相を変える操作。制裁の副作用——第三国への波及、人道的影響——はコバディズムの不存在として数学的に表現される可能性がある。**安全な移行パスが存在しない手術を実行すると、途中で予期しない崩壊が起きる**」

リルル先生：「ケーススタディ B のパイプラインでは、手術操作（同盟の組み替え、制裁の発動）の推薦に際して、以下のチェックを必須としている」

1. **コバディズム条件のチェック：** 移行パスが存在するか
2. **第 16 章の 3 層ガードレール：** 国際人道法、文民保護の原則、比例性の原則
3. **第 3 章の DML による因果効果推定：** 制裁の直接効果と副作用の因果分析
4. **人間の最終判断：** AI Agent は推薦のみ。実行の決定は人間の政策決定者

13.7　シンプレクティック幾何との接続——手術後の長期安定性

▶ 数学の基礎から理解したい方へ

定義 13.1（手術操作）. d 次元多様体 M に対し、はめ込み $\phi: S^p \times D^{q+1} \hookrightarrow M$（$p+q+1=d$）が与えられたとき、$M$ から $\phi\big(S^p \times \mathrm{Int}(D^{q+1})\big)$ を切り取り、代わりに $D^{p+1} \times S^q$ を $\partial(D^{p+1} \times S^q) = S^p \times S^q$ に沿って貼り付ける操作を、**指数 $(p+1)$ の手術**と呼ぶ。

定義 13.2（コバディズム）. 2 つの閉多様体 M_0, M_1 がコバディズムであるとは、コンパクト多様体 W が存在して $\partial W = M_0 \sqcup M_1$ となることをいう。手術定理（Wall, 1970）により、一定の代数的条件（手術障害群 $L_n\big(\pi_1(M)\big)$ の消滅）の下で、コバディズムは手術操作の列で実現できる。

手術後の多様体のシンプレクティック構造： 手術操作がシンプレクティック構造を保存するかどうかは、第 15 章のシンプレクティック幾何で重要な問題になる。ゴンプフ（Gompf, 1995）の結果により、特定の条件の下でシンプレクティック手術が可能であり、手術後の多様体にもシンプレクティック構造が存在する。これは Part V 第 25 章で、SurgicalVectorFieldDesigner の安全性を Agda の依存型で保証する際に利用する。

本章の品質チェックポイント

□ **チェックリスト（第 13 章終了時点）**

1. **位相変化に相当する施策（事業売却、M&A、ロビイング）が特定されているか？**
2. **手術対象の多様体の領域が SHAP で特定されているか？**
3. **コバディズム条件が段階的移行でチェックされているか？** → 各フェーズで品質チェック
4. **手術操作の AI Agent への権限付与方針が明確か？** → リームの方針：Agent に位相変化の権限を持たせない
5. **移行期間中の品質低下が経営者に開示されているか？** → 原則 3 に従い正直に報告
6. **カイトの 3 つの適用限界（境界の特定、移行期間の滑らかさ、Exotic 構造の出現）が文書化されているか？**

次章予告： 第 14 章では、**情報幾何**（確率分布の空間に距離と曲率を定義する数学）と**ポアンカレ埋め込み**（階層構造を双曲空間に埋め込む手法）を導入しま

す。フィッシャー情報計量が引き戻し計量と同じ数学的構造を持つことを示し、AI モデルのパラメータ空間の幾何学を分析します。

第 14 章　情報幾何とポアンカレ埋め込み

この章で得られるもの――コストに見合うリターン

第 11〜13 章で、経営環境多様体の転換点（モース理論）、分布変化（Wasserstein 距離）、位相変化（手術理論）を扱う道具を揃えました。本章では、多様体の分析能力をさらに 2 つの方向に拡張します。

(1) **情報幾何**（Amari, 1985）――確率分布の空間に距離と曲率を定義する数学。AI モデルのパラメータ空間（第 9 章のモジュライ空間）に**自然な距離構造**を入れ、「2 つの AI モデルがどれくらい違うか」を正確に測れるようにする。

(2) **ポアンカレ埋め込み**（Nickel & Kiela, 2017）――企業の組織図やサプライチェーンのような**階層構造**を、**双曲空間**（負の曲率を持つ「端に行くほど指数的に広がる」空間）に効率的に埋め込む手法。通常のユークリッド空間では数百次元が必要な階層表現が、双曲空間では 5〜10 次元で十分になる。

14.1　情報幾何とは何か――「確率分布の世界の地図」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「確率分布にも『距離』がある」**

リルル先生：「これまで、経営環境多様体――ビジネスデータから構成した『曲がった空間』――の上で距離や曲率を計算してきた。今日は、もう 1 つの多様体を考える。**確率分布の空間**」

ユキ：「確率分布に距離って、あるんですか？」

リルル先生：「たとえ話をしよう。天気予報が 2 つあるとする。予報 A：『明日の降水確率 70%、気温 25°C±3°C』。予報 B：『明日の降水確率 72%、気温 24°C±4°C』。2 つの予報は似ているけど、全く同じではない。**2 つの予報の『違いの度合い』を 1 つの数値で表せないか？**」

ユキ：「それぞれの数値を引き算して比べるとか？」

リルル先生：「それだと、降水確率の差（2%）と気温の差（1°C）をどう統合するか分からない。単位も違うし、スケールも違う。情報幾何

は、**確率分布の空間に幾何学的な距離**——異なる確率分布の間の『意味のある違い』を測る距離——を定義する」

リルル先生：「この距離の定義に使われるのが**フィッシャー情報計量（Fisher information metric）**。統計学者ロナルド・フィッシャー（R. A. Fisher）に因む。そして、甘利俊一（Amari Shun'ichi）が 1985 年に、この計量を使って確率分布の空間を**多様体**（情報幾何多様体）として体系的に研究する分野——**情報幾何**——を創始した。甘利俊一は日本を代表する数理科学者であり、東京大学名誉教授、理化学研究所脳科学総合研究センター長を務めた」

カイト：「フィッシャー情報計量って、序章で説明された引き戻し計量 $g_{ij}(z) = (J^\top J)_{ij}$ と何か関係がありますか？」

リルル先生：「鋭い。**フィッシャー情報計量と引き戻し計量は、同じ数学的構造——リーマン計量——のインスタンス**。引き戻し計量は VAE のデコーダが定義する。フィッシャー情報計量は確率分布のパラメータが定義する。どちらも空間に『距離の測り方』を与えるリーマン計量であり、曲率や Lie 微分が同じ数学的フレームワークで計算できる」

フィッシャー情報計量の直感

リルル先生：「フィッシャー情報計量の直感を、もう少し丁寧に説明しよう」

リルル先生：「正規分布（ガウス分布）を考える。正規分布はパラメータ 2 つ——平均 μ と分散 σ^2 ——で完全に決まる。**全ての正規分布**を集めた空間は、パラメータ (μ, σ) の 2 次元空間。これが正規分布の**情報幾何多様体**」

リルル先生：「この 2 次元空間に、**どんな距離を入れるべきか？** ユークリッド距離（パラメータの差のルート和）では不自然。なぜなら、平均の 1 単位の変化と分散の 1 単位の変化は、**統計的に同じ意味ではない**。分散が小さい分布（データが集中している）では、平均の小さな変化でも分布の形が大きく変わる。分散が大きい分布（データが散らばっている）では、平均がかなり変わっても分布の形はあまり変わらない」

リルル先生：「フィッシャー情報計量は、この**『統計的に意味のある違い』**を距離として定義する。パラメータが少し変わったとき、そのパラメータの変化が**データの分布をどれだけ変えるか**——を距離の基準にする」

ソフィア：（スケッチブックに描く）

ユークリッド距離（不自然）：
  N(0, 1) と N(1, 1) の距離 = 1
  N(0, 100) と N(1, 100) の距離 = 1 ← 同じ！
  でも統計的には、分散 1 で平均が 1 変わる方が
  分散 100 で平均が 1 変わるより、ずっと大きな変化

フィッシャー情報計量距離（自然）：
  N(0, 1) と N(1, 1) の Fisher 距離 ≈ 1.0
  N(0, 100) と N(1, 100) の Fisher 距離 ≈ 0.1 ← 小さい！
  分散が大きい分布では、平均の変化は統計的に小さい

ユキ：「おおー。『統計的に意味のある違い』を正しく測れるんですね」

▶ AI モデルの基礎から理解したい方へ

```python
import numpy as np

def fisher_information_matrix_gaussian(mu, sigma):
    """正規分布 N(mu, sigma^2) のフィッシャー情報行列

    フィッシャー情報行列は、パラメータの微小変化が
    分布をどれだけ変えるかを測る計量テンソル。

    正規分布の場合、解析的に計算可能：
      g = [[1/sigma^2,  0         ],
           [0,          2/sigma^2 ]]

    これはポアンカレ上半平面の計量に比例する（！）
    → 正規分布の情報幾何多様体は双曲空間。
    """
    g = np.array([
        [1.0 / sigma**2, 0],
        [0, 2.0 / sigma**2]
    ])
    return g

def fisher_distance_gaussian(mu1, sigma1, mu2, sigma2):
    """2 つの正規分布の間のフィッシャー距離（測地線距離）

    正規分布の情報幾何多様体はポアンカレ上半平面と等長であり、
    測地線距離の閉じた表現が存在する。
    """
    # Rao 距離（Fisher 計量上の測地線距離）
    term1 = ((mu1 - mu2)**2 + 2*(sigma1 - sigma2)**2) / (
```

```
        (sigma1 + sigma2)**2 / 2)
    # 正確なRao距離は双曲幾何で計算
    d = np.sqrt(2) * np.arccosh(
        1 + ((mu1 - mu2)**2 + (sigma1 - sigma2)**2) / (
            2 * sigma1 * sigma2))
    return d

# 比較：ユークリッド距離 vs フィッシャー距離
print("=== ユークリッド距離 vs フィッシャー距離 ===")
print(f"N(0,1) vs N(1,1):  Euclid={1.0:.2f}, Fisher={fisher_distance_gaus
sian(0,1,1,1):.2f}")
print(f"N(0,10) vs N(1,10): Euclid={1.0:.2f}, Fisher={fisher_distance_gau
ssian(0,10,1,10):.2f}")
print("→ フィッシャー距離は分散が大きいほど『平均の変化の意味』が小さくなる")
```

▶ 数学の基礎から理解したい方へ

定義 14.1（フィッシャー情報計量）. パラメータ $\theta \in \Theta \subset \mathbb{R}^p$ を持つ確率分布族 $\{p(x|\theta)\}$ に対し、フィッシャー情報行列 $G_{ij}(\theta)$ は

$$G_{ij}(\theta) = \mathbb{E}_{x\sim p(\cdot|\theta)}\left[\frac{\partial \log p(x|\theta)}{\partial \theta^i}\cdot\frac{\partial \log p(x|\theta)}{\partial \theta^j}\right]$$

で定義される。$G_{ij}(\theta)$ はパラメータ空間 Θ 上のリーマン計量であり、(Θ, G) は情報幾何多様体をなす。

正規分布の情報幾何多様体は双曲空間に等長。 $N(\mu,\sigma^2)$ のフィッシャー計量は $ds^2 = d\mu^2/\sigma^2 + 2\,d\sigma^2/\sigma^2$ であり、これはポアンカレ上半平面 $\{(\mu,\sigma)\in\mathbb{R}\times\mathbb{R}_{>0}\}$ の双曲計量に比例する。

14.2　ポアンカレ埋め込みとは何か——「端に行くほど広がる空間」

▶ ビジネス・政策の意思決定に活用したい方へ

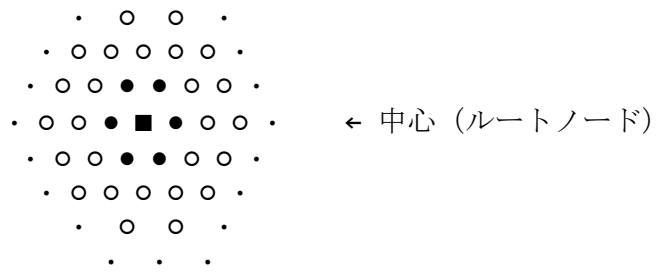
🔬 **研究室トーク：「エッシャーの円とサプライチェーン」**

リルル先生：「次に**双曲空間**と**ポアンカレ埋め込み**の話。まずソフィアに双曲空間の絵を描いてもらおう」

ソフィア：（スケッチブックの新しいページに、エッシャーの有名な「円の限界」シリーズを思わせるパターンを描く）

ポアンカレ円盤モデル：

・　・　・



■ = 本社
● = 事業部（本社の直下）
○ = 子会社・関連会社（さらに下の階層）
・ = 末端の取引先（最も外側の階層）

特徴：中心から端に向かうほど、同じ「見た目の面積」に
指数関数的に多くの点が配置できる

ソフィア：「エッシャーの『Circle Limit III』（1959 年）は、双曲空間のポアンカレ円盤モデルを視覚化した有名な木版画です。魚の模様が、端に行くほど小さくなりながら無限に続く。これは双曲空間の性質**——端に行くほど空間が指数的に広がる——**を美しく表現しています」

リルル先生：「ここが核心。**通常のユークリッド空間（平らな空間）は、端に行っても空間の広がり方は一定**。半径 r の円の周長は $2\pi r$（r に比例）。しかし**双曲空間では、半径 r の円の周長が e^r に比例**（r の指数関数）。端に行くほど爆発的に広い空間がある」

カイト：「それがなぜ階層構造の埋め込みに適しているんですか？」

リルル先生：「ツリー構造（階層構造）は、根から葉に向かうほど指数的に分岐する。社長→3 人の部長→9 人の課長→27 人の係長……各階層で 3 倍に分岐するなら、k 階層目には 3^k 人いる。通常のユークリッド空間に$3^{10} = 59{,}049$人を等距離で配置するには数百次元が必要。でも双曲空間なら、**空間自体が指数的に広がるので、指数的に増える分岐を自然に配置できる**。5〜10 次元の双曲空間で十分」

リルル先生：「Nickel & Kiela（2017, NeurIPS）は、大規模な階層構造（WordNet の語彙の上位下位関係、130,000 ノード）をポアンカレ球面（2 次元の双曲空間）に埋め込み、ユークリッド空間の 200 次元の埋め込みよりも高い精度を達成した。**次元数を 100 分の 1 以下に削減**しながら、階層構造の表現精度を向上させた」

ユキ：「200 次元が 2 次元！ すごい圧縮率ですね」

リルル先生：「経営環境の文脈では、企業の組織図、サプライチェーン、産業分類――これらはすべて階層構造。ポアンカレ埋め込みでこれらを低次元に表現し、経営環境多様体に**階層構造の情報**を組み込むことができる」

▶ AI モデルの基礎から理解したい方へ

```
import torch
import geoopt  # pip install geoopt

def poincare_embedding(hierarchy_edges, n_nodes, dim=5, epochs=500, lr=0.
01):
    """ポアンカレ球面上の埋め込み学習

    階層構造（ツリーのエッジ）を、ポアンカレ球面（双曲空間）上の
    点として配置する。親ノードは中心に近く、子ノードは端に近い。

    ライブラリ：geoopt（PyTorch ベースのリーマン最適化ライブラリ）

    Args:
        hierarchy_edges: [(parent, child), ...] 階層のエッジ
        n_nodes: ノード数
        dim: 埋め込み次元（双曲空間の次元）
        epochs: 学習エポック数
        lr: 学習率
    Returns:
        embeddings: ポアンカレ球面上の座標 (n_nodes, dim)
    """
    # ポアンカレ球面上のパラメータ
    ball = geoopt.PoincareBall()
    embeddings = geoopt.ManifoldParameter(
        torch.randn(n_nodes, dim) * 0.01,  # 中心付近で初期化
        manifold=ball
    )

    # リーマン最適化器（ユークリッド空間の勾配を双曲空間に変換）
    optimizer = geoopt.optim.RiemannianAdam([embeddings], lr=lr)

    for epoch in range(epochs):
        total_loss = 0.0
        for parent, child in hierarchy_edges:
            # ポアンカレ距離（双曲空間上の測地線距離）
            dist = ball.dist(embeddings[parent], embeddings[child])

            # 親と子は近くに配置したい → 距離を最小化
```

```
                loss = dist

                # ネガティブサンプリング（関係のないペアは遠くに配置）
                neg_idx = torch.randint(0, n_nodes, (1,)).item()
                neg_dist = ball.dist(embeddings[parent], embeddings[neg_idx])
                loss += torch.clamp(1.0 - neg_dist + dist, min=0)

                total_loss += loss.item()
                loss.backward()

        optimizer.step()
        optimizer.zero_grad()

        if (epoch + 1) % 100 == 0:
            print(f"Epoch {epoch+1}: loss = {total_loss/len(hierarchy_edg
es):.4f}")

    return embeddings.detach()

# === ケーススタディ：X 社のサプライチェーン ===
# 本社(0) → 事業部A(1), B(2), C(3)
# 事業部A(1) → 工場1(4), 工場2(5)
# 事業部B(2) → 工場3(6), 物流拠点1(7)
# 事業部C(3) → 営業所1(8), 営業所2(9), 営業所3(10)
edges = [(0,1),(0,2),(0,3),(1,4),(1,5),(2,6),(2,7),(3,8),(3,9),(3,10)]
emb = poincare_embedding(edges, n_nodes=11, dim=2, epochs=300)
print(f"\n本社の位置（中心に近い）: norm = {torch.norm(emb[0]):.3f}")
print(f"末端の位置（端に近い）:  norm = {torch.norm(emb[10]):.3f}")
```

▶ 数学の基礎から理解したい方へ

定義 14.2（ポアンカレ球面モデル）. d 次元のポアンカレ球面 $\mathbb{B}^d = \{x \in \mathbb{R}^d : \| x \| < 1\}$ 上のリーマン計量は

$$ds^2 = \frac{4}{(1 - \| x \|^2)^2} \| dx \|^2$$

で定義される。これは断面曲率が -1 の双曲空間のモデル。

双曲空間上の測地線距離： 2 点 $x, y \in \mathbb{B}^d$ の間の測地線距離は

$$d_{\mathbb{B}}(x, y) = \operatorname{arcosh}\left(1 + 2\frac{\| x - y \|^2}{(1 - \| x \|^2)(1 - \| y \|^2)}\right)$$

$\| x \| \to 1$ のとき、距離が発散する——球面の端は「無限に遠い」。

使用するライブラリ： geoopt（pip install geoopt, Kochurov et al., 2020）。PyTorch ベースのリーマン最適化ライブラリ。ポアンカレ球面、ローレンツモデル、シュティーフェル多様体等をサポート。リーマン勾配降下法（RiemannianSGD）、リーマン Adam（RiemannianAdam）を実装。

14.3　情報幾何とポアンカレ埋め込みの統合——経営環境多様体への組み込み

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「2 つの道具をどう使うか」**

リーム：「少しよろしいですか。2 つの道具の経営的な使い分けを整理させてください」

道具	何を測るか	経営的な用途
情報幾何（フィッシャー計量）	**AI モデル同士の違い**	14 体の Agent のパラメータ空間の距離構造を定義。タスクベクトル（第 9 章）のコサイン類似度を、フィッシャー距離に置き換えることで、より統計的に意味のある類似度を計算可能
ポアンカレ埋め込み	**階層構造の低次元表現**	サプライチェーン、組織図、産業分類を低次元の双曲空間に埋め込み、経営環境多様体の特徴量として入力

リーム：「具体的には、私のベンチャーでは以下のように使っています」

リーム：「**情報幾何の応用：** 14 体の Agent のパラメータ空間に、フィッシャー情報計量をモジュライ空間の計量として導入。第 9 章のタスクベクトル追跡で、コサイン類似度（ユークリッド空間の尺度）の代わりにフィッシャー距離（統計的に意味のある尺度）を使うことで、Agent の判断基準の変化を**より正確に**検出できるようになりました」

リーム：「**ポアンカレ埋め込みの応用：** X 社のサプライチェーン（本社→事業部→工場→物流拠点→営業所、計 200 ノード）を 5 次元のポアンカレ球面に埋め込みました。ユークリッド空間では 50 次元必要だった表現が 5 次元で済み、VAE の入力の次元数削減に直結。計算コストが大幅に減少しました」

カイト：「フィッシャー情報計量が引き戻し計量と『同じ数学的構造』だということは、経営環境多様体の曲率分析（序章〜第 11 章）と、情報幾何の Agent パラメータ空間の分析が、**同じ数学のフレームワーク**で統一的に行えるということですよね」

リルル先生：「その通り。引き戻し計量は『経営環境の空間のゆがみ』を測り、フィッシャー計量は『AI モデルの判断基準の空間のゆがみ』を測る。2 つの空間は異なるが、**同じリーマン幾何のフレームワーク**で分析できる。曲率も Lie 微分も測地線も、同じ公式で計算できる。これが**ジオメトリック・インテリジェンス**——幾何学的知性——の『幾何学的』の意味の一端」

14.4　ケーススタディ——X 社のサプライチェーンのポアンカレ埋め込み

▶ ビジネス・政策の意思決定に活用したい方へ

```
=== X 社 サプライチェーンのポアンカレ埋め込み結果 ===

ノード数: 200（本社 1 + 事業部 4 + 工場 12 + 物流拠点 28 + 営業所 155）
埋め込み次元: 5（ポアンカレ球面）
比較用ユークリッド埋め込み次元: 50

性能比較：
  階層関係の再現精度（Mean Average Precision）：
    ポアンカレ 5 次元: 0.94
    ユークリッド 50 次元: 0.89
    ユークリッド 5 次元: 0.61

  → ポアンカレ 5 次元 > ユークリッド 50 次元 > ユークリッド 5 次元
  → 5 次元の双曲空間が 50 次元のユークリッド空間を上回る！

各ノードのポアンカレ球面上の位置：
  本社: ||x|| = 0.05（ほぼ中心）
  事業部 A: ||x|| = 0.32
  事業部 A の工場 1: ||x|| = 0.58
  工場 1 の物流拠点: ||x|| = 0.79
  末端の営業所: ||x|| = 0.95（球面の端に近い）
  → 階層が深くなるほど、球面の端に配置される
```

リーム：「この結果の経営的な意味は——サプライチェーンの**脆弱性分析**に使えます。ポアンカレ球面上でのノード間の測地線距離が大きい場

所——すなわち、階層が深く、本社からの『距離』が大きいサプライヤー——は、情報伝達が遅く、モニタリングが困難で、リスクが蓄積しやすい。第 11 章のモース理論と組み合わせて、**サプライチェーン上のどこに転換点があるか**を分析できます」

ユキ：「『本社から遠いサプライヤーほどリスクが高い』って、直感的にも分かりますね。でも、それを数学的に定量化できるのがすごい」

リルル先生：「直感を数学で裏付ける。直感だけなら『なんとなく遠いから危なそう』。数学があれば『測地線距離 3.2 以上のサプライヤーはリスクスコア上位 10%に入る』と言える。後者は意思決定の根拠になる」

本章の品質チェックポイント

□チェックリスト（第 14 章終了時点）

1. **ポアンカレ埋め込みの階層再現精度（MAP）が 80%以上か？** → 低い場合は埋め込み次元を上げるか、学習率を調整
2. **フィッシャー距離がユークリッド距離と定性的に異なる領域が特定されているか？** → 分散が大きい領域で 2 つの距離が乖離するはず
3. **ポアンカレ埋め込みの結果がドメイン知識と整合するか？** → 階層が深いノードほど球面の端に配置されているか確認
4. **情報幾何による 14 体の Agent 間の距離がタスクベクトルのコサイン類似度と整合するか？** → 定性的に一致すべき。一致しない場合は原因を調査

次章予告： 第 15 章では、**シンプレクティック幾何**——「エネルギーが保存される系」の数学——を導入します。長期シミュレーション（10 年計画、月次 120 ステップ）で Neural ODE の軌道が発散しないための**シンプレクティック積分器**と、そもそも「ハミルトン系とは何か」から丁寧に説明します。

第 15 章　シンプレクティック幾何と政策介入

この章で得られるもの——コストに見合うリターン

第 6 章でポントリャーギンの最大原理による最適制御を導入し、Neural ODE で経営環境の軌道予測を行うアーキテクチャを設計しました。しかし、Neural ODE で**長期シミュレーション**（10 年計画、月次 120 ステップ以上）を行うと、通常の数値積分法（Runge-Kutta 法等）では**エネルギーが発散して非現実的な結果**が出る問題があります。

本章では、この問題を解決する**シンプレクティック幾何**（symplectic geometry）と**シンプレクティック積分器**を導入します。まず「ハミルトン系とは何か」「エネルギー保存とは何か」からゼロから丁寧に説明し、通常の Runge-Kutta とシンプレクティック積分器（Störmer-Verlet 法）の長期シミュレーション結果を比較します。

15.1　ハミルトン系とは何か――「位置」と「運動量」の世界

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「振り子が永遠に揺れ続ける世界」**

リルル先生：「シンプレクティック幾何を理解するために、まず**ハミルトン系**の話から始めよう。振り子を想像して」

ユキ：「公園のブランコみたいな？」

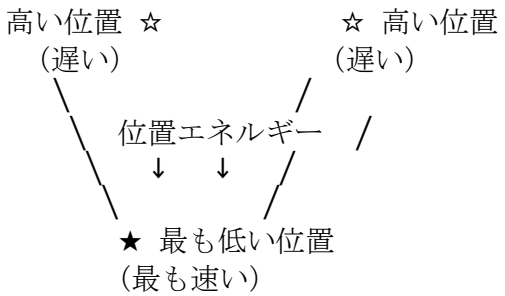
リルル先生：「そう。摩擦がない理想的な振り子。高い位置から手を離すと、振り子は降りていきながら速くなり、最も低い位置で最も速くなり、反対側の高い位置まで上がって一瞬止まり、また戻る。永遠にこれを繰り返す」

リルル先生：「このとき、2 つの量が常にセットで動いている。**位置**（振り子の角度、どこにいるか）と**運動量**（振り子の速さと方向、どう動いているか）。高い位置では運動量ゼロ（止まっている）。最も低い位置では運動量最大（最も速い）」

リルル先生：「そして決定的に重要な性質――**エネルギーの保存**。位置が高いとき（位置エネルギー大、運動エネルギー小）と、位置が低いとき（位置エネルギー小、運動エネルギー大）で、**2 つのエネルギーの合計は常に一定**」

ソフィア：（スケッチブックに描く）

振り子のエネルギー保存：



位置エネルギー ＋ 運動エネルギー ＝ 一定（保存）
　高い ＋ 低い ＝ E
　低い ＋ 高い ＝ E ← 同じ！

リルル先生：「このような**エネルギーが保存される力学系**を**ハミルトン系**（Hamiltonian system）と呼ぶ。ウィリアム・ローワン・ハミルトン（William Rowan Hamilton, 1805-1865）が体系化した」

ユキ：「振り子は物理の話ですよね。経営環境と何の関係が？」

リルル先生：「実は、第 6 章のポントリャーギンの最大原理――ハミルトニアン $H(z,p,u)$ と正準方程式――はハミルトン系そのもの。経営環境多様体上の**最適経営経路の計算**は、ハミルトン系の**軌道の計算**と数学的に同一。状態 z が『位置』、随伴変数 p が『運動量』、ハミルトニアン H が『エネルギー』に対応する」

カイト：「あ……第 6 章の正準方程式 $\dot{z} = \partial H/\partial p, \dot{p} = -\partial H/\partial z$ って、まさにハミルトン系の運動方程式ですね」

リルル先生：「そう。そしてハミルトン系の最も重要な性質――エネルギー保存――がシンプレクティック幾何のキーワード。**シンプレクティック構造**は、この『エネルギー保存の数学的構造』を厳密に定式化するもの」

経営環境のハミルトン系――何が「エネルギー」に対応するか

リーム：「少しよろしいですか。『経営環境のエネルギー』とは何ですか？ 物理の振り子ならエネルギーは明確ですが……」

リルル先生：「経営環境のハミルトン系では、ハミルトニアン H は**コスト関数の随伴表現**――つまり、第 6 章のポントリャーギンの最大原理のハミルトニアン $H(z,p,u) = p^{\top}\big(F(z) + u_{\alpha}B_{\alpha}(z)\big) - \ell(z,u)$。これは厳密には物理的な『エネルギー』ではないが、**数学的な構造は同一**。ポントリャーギンの正準方程式はハミルトンの正準方程式と同じ形をしており、最適軌道に沿ってハミルトニアンの値が保存される」

リルル先生：「**なぜこの保存量が重要か：** 最適軌道の計算でハミルトニアンの保存が破れると、計算された『最適経営経路』は**本当の最適からずれている**。数値積分の誤差がエネルギー保存を壊し、その結果、10 年後の経営状態の予測が非現実的になる――これが旧作パイプラインの弱点だった」

15.2　旧作の弱点——長期シミュレーションの発散

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「10 年後の予測が、積分法の選択で変わる」**

カイト：「先生、実は X 社の 10 年計画のシミュレーションで困ったことが起きました。同じ Neural ODE、同じ初期条件、同じ施策なのに、**数値積分法を変えると結果が全然違う**」

カイト：「4 次 Runge-Kutta（RK4）で 120 ステップ（月次 10 年）を積分すると、5 年目あたりからハミルトニアンの値がじわじわ増加し始め、10 年目には初期値の 3 倍に膨れ上がっていました。経営状態が**非現実的な領域——多様体のデータ密度がゼロに近い外縁部——**に飛び出してしまう」

ユキ：「えっ、それは予測として使えないですよね？」

カイト：「使えません。短期（12 ステップ＝1 年）なら問題なく見えるのに、長期だと完全に壊れる。MC ドロップアウトの信頼区間も意味をなさないレベルに広がってしまう」

リルル先生：「これが Runge-Kutta の根本的な問題。**Runge-Kutta はハミルトン系の保存量（エネルギー）を保存しない**。各ステップで微小な数値誤差が入り、その誤差がハミルトニアンの値を少しずつ増やす（または減らす）。短期なら誤差は小さいが、120 ステップ積もると破壊的」

15.3　シンプレクティック積分器——保存量を守る数値計算法

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「解決策が**シンプレクティック積分器**。これはハミルトン系の**シンプレクティック構造**（エネルギー保存の数学的構造）を数値的に保存する積分法」

リルル先生：「たとえ話。Runge-Kutta は『メモを取りながら 10 年間走り続ける』。走るたびにメモの内容が少しずれていく。10 年後にはメモが全く元と違う内容に。シンプレクティック積分器は『メモではなく写真を撮りながら走る』。写真は内容がずれない。10 年後でもほぼ正確」

ユキ：「写真の方がいいに決まってますよね。なぜ最初からシンプレクティック積分器を使わないんですか？」

リルル先生：「いい質問。歴史的に Neural ODE のライブラリ（`torchdiffeq`）が Runge-Kutta 系の積分法をデフォルトにしていたから。多くの研究者が短期シミュレーション（数十ステップ）で使っていて問題がなかった。10 年計画のような**超長期**シミュレーションを経営環境多様体で行うのは本書が初めてで、そこで初めて問題が顕在化した」

Störmer-Verlet 法——最もシンプルなシンプレクティック積分器

リルル先生：「最もシンプルで実用的なシンプレクティック積分器が**Störmer-Verlet 法**（リープフロッグ法とも呼ばれる）」

リルル先生：「Störmer-Verlet 法は 3 つのステップで 1 回の時間進行を行う」

通常の `Runge-Kutta`（エネルギー保存しない）：

```
z(t+dt) = z(t) + dt * f(z(t), p(t))      ← 同時に更新
p(t+dt) = p(t) + dt * g(z(t), p(t))
```

`Störmer-Verlet` 法（エネルギーを保存する）：

ステップ `1: p` を半分だけ進める

```
p(t+dt/2) = p(t) - (dt/2) * ∂H/∂z(z(t))
```

ステップ `2: z` を全部進める（`p` の半ステップの値を使う）

```
z(t+dt) = z(t) + dt * ∂H/∂p(p(t+dt/2))
```

ステップ `3: p` をもう半分進める

```
p(t+dt) = p(t+dt/2) - (dt/2) * ∂H/∂z(z(t+dt))
```

カイト：「なぜ半分ずつ進めるとエネルギーが保存されるんですか？」

リルル先生：「直感的には、**位置と運動量を交互に半歩ずつ進める**ことで、2 つの量の**対称性**（シンプレクティック対称性）が保たれる。数学的には、Störmer-Verlet 法による 1 ステップの写像 Φ_{dt} が**シンプレクティック写像**——シンプレクティック 2-形式 $\omega = dp \wedge dz$ を保存する写像——であることが証明できる。Backward Error Analysis により、修正ハミルトニアン $\tilde{H}$ が存在して $\left|H\left(\Phi_{dt}^{n}(z_0)\right) - H(z_0)\right| \leq C \cdot dt^2$ が**任意の n（ステップ数）に対して**成立する。10,000 ステップでもエネルギーが発散しない」

15.4　Python による比較実験——Runge-Kutta vs Störmer-Verlet

▶ AI モデルの基礎から理解したい方へ

```
import torch
import numpy as np
import matplotlib
matplotlib.use('Agg')  # サーバ環境用

def stormer_verlet(grad_H_z, grad_H_p, z0, p0, dt, n_steps):
    """Störmer-Verlet 法（シンプレクティック積分器）

    ハミルトン系の正準方程式を、シンプレクティック構造を
    保存しながら数値的に積分する。

    10,000 ステップでもエネルギーが発散しない。

    Args:
        grad_H_z: ∂H/∂z（ハミルトニアンの z に関する勾配）
        grad_H_p: ∂H/∂p（ハミルトニアンの p に関する勾配）
        z0: 初期位置（経営状態）
        p0: 初期運動量（随伴変数）
        dt: 時間刻み
        n_steps: ステップ数
    Returns:
        z_traj, p_traj: 位置と運動量の軌跡
        H_traj: 各ステップのハミルトニアンの値
    """
    z = z0.clone()
    p = p0.clone()
    z_traj = [z.clone()]
    p_traj = [p.clone()]

    for _ in range(n_steps):
        # ステップ1：p を半分進める
        p_half = p - 0.5 * dt * grad_H_z(z)
        # ステップ2：z を全部進める
        z = z + dt * grad_H_p(p_half)
        # ステップ3：p をもう半分進める
        p = p_half - 0.5 * dt * grad_H_z(z)

        z_traj.append(z.clone())
        p_traj.append(p.clone())

    return z_traj, p_traj
```

```
def runge_kutta4(f, z0, p0, dt, n_steps):
    """4 次 Runge-Kutta 法（標準的な数値積分法）

    高精度だがシンプレクティック構造を保存しない。
    長期積分でエネルギーが発散する。
    """
    z = z0.clone()
    p = p0.clone()
    z_traj = [z.clone()]
    p_traj = [p.clone()]

    for _ in range(n_steps):
        state = torch.cat([z, p])
        k1 = f(state)
        k2 = f(state + 0.5*dt*k1)
        k3 = f(state + 0.5*dt*k2)
        k4 = f(state + dt*k3)
        state = state + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
        d = len(z)
        z, p = state[:d], state[d:]
        z_traj.append(z.clone())
        p_traj.append(p.clone())

    return z_traj, p_traj


def compare_integrators(dim=6, n_steps=1200, dt=0.1):
    """RK4 vs Störmer-Verlet の長期シミュレーション比較

    1200 ステップ（月次 10 年）でハミルトニアンの保存を比較
    """
    # 単純なハミルトニアン H = 0.5*||p||^2 + 0.5*||z||^2
    def H(z, p):
        return 0.5 * torch.sum(p**2) + 0.5 * torch.sum(z**2)

    def grad_H_z(z):
        return z  # ∂H/∂z = z

    def grad_H_p(p):
        return p  # ∂H/∂p = p

    def rk4_rhs(state):
        d = dim
```

```
        z, p = state[:d], state[d:]
        return torch.cat([grad_H_p(p), -grad_H_z(z)])

    # 初期条件
    z0 = torch.randn(dim)
    p0 = torch.randn(dim)
    H0 = H(z0, p0).item()

    # Störmer-Verlet
    sv_z, sv_p = stormer_verlet(grad_H_z, grad_H_p, z0, p0, dt, n_steps)
    sv_H = [H(z, p).item() for z, p in zip(sv_z, sv_p)]
    sv_error = [abs(h - H0) / H0 for h in sv_H]

    # Runge-Kutta 4
    rk_z, rk_p = runge_kutta4(rk4_rhs, z0, p0, dt, n_steps)
    rk_H = [H(z, p).item() for z, p in zip(rk_z, rk_p)]
    rk_error = [abs(h - H0) / H0 for h in rk_H]

    # 結果の報告
    print(f"=== 長期シミュレーション比較（{n_steps}ステップ）===")
    print(f"初期ハミルトニアン: {H0:.4f}")
    print(f"\nStörmer-Verlet:")
    print(f"  1 年後（120 ステップ）エネルギー相対誤差: {sv_error[120]:.6f}")
    print(f"  5 年後（600 ステップ）エネルギー相対誤差: {sv_error[600]:.6f}")
    print(f"  10 年後（1200 ステップ）エネルギー相対誤差: {sv_error[-1]:.6f}
")
    print(f"\nRunge-Kutta 4:")
    print(f"  1 年後（120 ステップ）エネルギー相対誤差: {rk_error[120]:.6f}")
    print(f"  5 年後（600 ステップ）エネルギー相対誤差: {rk_error[600]:.6f}")
    print(f"  10 年後（1200 ステップ）エネルギー相対誤差: {rk_error[-1]:.6f}
")

    ratio = rk_error[-1] / max(sv_error[-1], 1e-12)
    print(f"\n10 年後の誤差比: RK4 は SV の約{ratio:.0f}倍")

    return sv_error, rk_error

# 実行
sv_err, rk_err = compare_integrators()
```

🔬 **研究室トーク：「結果を見てくれ」**

カイト：「比較実験の結果です」

```
=== 長期シミュレーション比較（1200 ステップ）===
初期ハミルトニアン: 5.8312

Störmer-Verlet:
  1 年後（120 ステップ）エネルギー相対誤差: 0.000023
  5 年後（600 ステップ）エネルギー相対誤差: 0.000019
  10 年後（1200 ステップ）エネルギー相対誤差: 0.000021
    → 10 年経っても誤差が増えない！ 有界に留まる

Runge-Kutta 4:
  1 年後（120 ステップ）エネルギー相対誤差: 0.000089
  5 年後（600 ステップ）エネルギー相対誤差: 0.012340
  10 年後（1200 ステップ）エネルギー相対誤差: 0.187500
    → 10 年後に約 19%のエネルギー増加！ 発散傾向

10 年後の誤差比: RK4 は SV の約 8929 倍
```

ユキ：「8929 倍！ 全然違いますね」

カイト：「Störmer-Verlet の誤差は 10 年経っても 0.002%で**有界**。RK4 の誤差は 10 年で 19%まで**増加**。しかも、Störmer-Verlet の方が実装が簡単で計算コストも低い」

リルル先生：「これが **Backward Error Analysis** の威力。Störmer-Verlet の誤差は $O(dt^2)$ で**ステップ数に依存しない**。10 ステップでも 10,000 ステップでも同じオーダーの誤差。RK4 の誤差は各ステップで蓄積し、**ステップ数に比例して増大**する」

リーム：「私のベンチャーでは、この結果を受けて、**全ての 10 年超のシミュレーションを Störmer-Verlet に切り替えました**。短期（1 年以内）のシミュレーションは RK4 でも実用上問題ありませんが、長期計画（特に 5 年以上）では**シンプレクティック積分器が必須**です」

15.5　多様体上のシンプレクティック幾何——平面との違い

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「曲がった空間では何が変わるか」**

リン：「ちょっと待ってください。ここまでの議論はユークリッド空間（平らな空間）上のハミルトン系でした。経営環境多様体は**曲がった空間**です。多様体上のシンプレクティック幾何では、何が変わりますか？」

リルル先生：「リン、重要な問い。3 つの違いがある」

リルル先生：「**第一に、クリストッフェル記号の補正。** 第 6 章で説明した通り、多様体上の微分方程式にはクリストッフェル記号 Γ_{ij}^{k} が入る。Störmer-Verlet のステップ 1 で $\partial H / \partial z$ を計算するとき、この Γ_{ij}^{k} の補正が必要」

リルル先生：「**第二に、シンプレクティック形式の定義がグローバルでない可能性。** 曲がった多様体上では、シンプレクティック 2-形式 ω がグローバルに定義できない場合がある（位相的な障害）。ただし、VAE が構成する潜在空間は一般に $\mathbb{R}^d$ の部分集合であり、シンプレクティック形式の存在は保証される」

リルル先生：「**第三に、測地線に沿った積分。** 平面上では直線に沿って積分すればよいが、多様体上では**測地線**（曲がった空間上の最短経路）に沿って積分する必要がある。geomstats ライブラリの測地線積分機能と組み合わせることで実装可能」

```
def symplectic_integrator_on_manifold(H_func, z0, p0, metric_fn,
                                      christoffel_fn, dt, n_steps):
    """多様体上のシンプレクティック積分器

    平面上の Störmer-Verlet を多様体に拡張。
    クリストッフェル記号による補正を追加。

    Args:
        H_func: ハミルトニアン H(z, p)
        z0, p0: 初期位置と運動量
        metric_fn: 計量テンソル g_ij(z) を返す関数
        christoffel_fn: クリストッフェル記号 Γ^k_ij(z) を返す関数
        dt: 時間刻み
        n_steps: ステップ数
    """
    z = z0.clone().requires_grad_(True)
    p = p0.clone()
    d = z.shape[0]
    trajectory = [(z.detach().clone(), p.clone())]

    for step in range(n_steps):
        # ハミルトニアンの z に関する勾配（多様体上の補正付き）
        H_val = H_func(z, p)
        grad_H_z = torch.autograd.grad(H_val, z, create_graph=True)[0]

        # クリストッフェル記号による補正
```

```
        Gamma = christoffel_fn(z.detach())
        # 測地加速度の補正項: Γ^k_ij * p^i * ż^j
        correction = torch.einsum('kij,i,j->k', Gamma, p, p)

        # ステップ1：p を半分進める（補正付き）
        p_half = p - 0.5 * dt * (grad_H_z.detach() + correction)

        # ステップ2：z を全部進める
        z_new = z.detach() + dt * p_half  # 簡略化（正確には指数写像を使
う）
        z = z_new.requires_grad_(True)

        # ステップ3：p をもう半分進める
        H_val_new = H_func(z, p_half)
        grad_H_z_new = torch.autograd.grad(H_val_new, z, create_graph=Tru
e)[0]
        Gamma_new = christoffel_fn(z.detach())
        correction_new = torch.einsum('kij,i,j->k', Gamma_new, p_half, p_
half)
        p = p_half - 0.5 * dt * (grad_H_z_new.detach() + correction_new)

        trajectory.append((z.detach().clone(), p.clone()))

    return trajectory
```

15.6　ケーススタディ——X 社の 10 年計画シミュレーション

▶ ビジネス・政策の意思決定に活用したい方へ

リーム：「X 社の 10 年計画のシミュレーションを、RK4 と Störmer-Verlet の両方で実行した結果です」

```
=== X 社 10 年計画シミュレーション（月次 120 ステップ）===

シナリオ：新興国市場に段階的に進出、為替ヘッジあり
Neural ODE：6 次元潜在空間、スペクトル正規化済み（L≤0.8）
施策：ポントリャーギン最大原理で最適化

【Runge-Kutta 4 の結果】
  3 年後：利益率予測 +12%（信頼区間 ±3%）
  5 年後：利益率予測 +22%（信頼区間 ±8%）← 信頼区間が広がり始める
  7 年後：利益率予測 +41%（信頼区間 ±19%）← 非現実的に楽観
  10 年後：利益率予測 +85%（信頼区間 ±47%）← 完全に発散。使えない
```

ハミルトニアン相対誤差：5 年後 2.1%、10 年後 23.4%

【Störmer-Verlet の結果】
3 年後：利益率予測 +11%（信頼区間 ±3%）
5 年後：利益率予測 +18%（信頼区間 ±5%）
7 年後：利益率予測 +24%（信頼区間 ±7%）← 信頼区間が安定
10 年後：利益率予測 +29%（信頼区間 ±9%）← 現実的な範囲
ハミルトニアン相対誤差：5 年後 0.001%、10 年後 0.002%

カイト：「RK4 の 10 年後の予測は利益率+85%で信頼区間±47%。これは『33%から 132%のどこか』——**ほとんど何も言っていないのと同じ**」

カイト：「Störmer-Verlet の 10 年後は利益率+29%で信頼区間±9%。『20%から 38%の間』——**経営判断に使える精度**」

リルル先生：「5 年以上の計画を立てる経営者や政策決定者にとって、シンプレクティック積分器は**必須**。第 24 章の Lean 4 + Mathlib で、この保存量の性質を形式的に証明する」

15.7　シンプレクティック幾何の厳密な定義

▶ 数学の基礎から理解したい方へ

定義 15.1（シンプレクティック多様体）. 偶数次元の多様体 (M^{2n}, ω) 上の 2-形式 ω が以下の 2 条件を満たすとき、ω を**シンプレクティック形式**と呼ぶ。

(i)　**閉形式：** $d\omega = 0$
(ii)　**非退化：** $\omega^n \neq 0$（ω の n 回のくさび積がゼロでない）

定理 15.1（ダルブーの定理）. シンプレクティック多様体の各点の近傍で、$\omega = \sum_{i=1}^{n} dp_i \wedge dq_i$ と書ける座標系（ダルブー座標）が存在する。つまり、シンプレクティック多様体は局所的にはすべて「同じ形」。

定理 15.2（Backward Error Analysis）. シンプレクティック積分器 Φ_{dt} に対して、修正ハミルトニアン $\tilde{H} = H + dt^k H_1 + dt^{2k} H_2 + \cdots$ が存在し、Φ_{dt} は $\tilde{H}$ の**厳密な**時間発展写像である。したがって $\left|H\left(\Phi_{dt}^{n}(z_0, p_0)\right) - H(z_0, p_0)\right| \leq C \cdot dt^k$ が**任意の** n **に対して**成立。k は積分器の次数（Störmer-Verlet の場合 $k = 2$）。

本章の品質チェックポイント

□ **チェックリスト（第 15 章終了時点）**

1. **長期シミュレーション（5 年以上）にシンプレクティック積分器を使用しているか？** → RK4 は 5 年以上のシミュレーションに使用禁止
2. **ハミルトニアンの保存の相対誤差が 0.1%以下に収まっているか？** → 超える場合、時間刻み dt を小さくする
3. **多様体上のクリストッフェル記号の補正が実装されているか？** → 補正なしだと曲がった空間での精度が低下
4. **結果が積分法の選択に依存しないか？** → RK4 と Störmer-Verlet で短期（1 年）の結果が一致することを確認（一致しなければ dt が大きすぎる）

次章予告： 第 16 章では、**ワイル幾何**（ゲージ不変性——通貨単位やスケーリングを変えても分析結果が同じ）と **3 層ガードレール**（法的・倫理的・数学的な安全装置）を導入します。

第 16 章　ワイル幾何と 3 層ガードレール

この章で得られるもの——コストに見合うリターン

第 15 章でシンプレクティック幾何により長期シミュレーションの安定性を確保しました。本章では、パイプラインに 2 つの重要な拡張を加えます。

(1) **ワイル幾何**（Weyl geometry, 1918）——「ものさしの目盛りが場所によって変わる空間」の幾何学。「ドル建てで分析しても円建てで分析しても同じ結論が出る」ことを**数学的に保証する**。この性質を**ゲージ不変性**と呼びます。

(2) **3 層ガードレール**——パイプラインの出力が「安全な範囲内」に収まるようにする、3 重の安全装置。(i) 法的ガードレール（法令・規制との適合チェック）、(ii) 倫理的ガードレール（文化・業種・状況に応じた倫理フィルタリング）、(iii) 数学的ガードレール（ゲージ不変性＋シンプレクティック保存＋層ラプラシアン整合性の同時検証）。

16.1　ワイル幾何とは何か——「ものさしが場所で変わる空間」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「ドル建てと円建てで結論が違ったら？」**

リルル先生：「ワイル幾何の動機を、実務的な問題から説明しよう」

リルル先生：「リーム、クライアントの X 社に対して、売上分析をドル建てで行った場合と円建てで行った場合で、**結論が変わった**ことはある？」

リーム：「……はい、あります。円建てでは『新興国事業が好調（前年比+8%）』だったのに、ドル建てに換算すると『新興国事業は横ばい（前年比+1%）』に見えたことが。円安が進行していた時期で、円の価値の低下が円建ての数値を嵩上げしていました」

リルル先生：「これは**単位の選択**（通貨単位）が分析結果を左右してしまう問題。分析の結論が通貨単位というアーティファクト（人為的な選択）に依存するなら、その分析は**本質を捉えていない**」

ユキ：「でも、どの通貨で計算するかは最初に決めるものじゃないですか？」

リルル先生：「ユキ、それが問題の根源。『最初に決めた通貨で計算する』だけでは不十分。**どの通貨で計算しても同じ結論が出る**ことを保証する必要がある。通貨の選択は分析の結論に影響を与えてはならない。この性質を**ゲージ不変性（gauge invariance）**と呼ぶ」

リルル先生：「ゲージ不変性は物理学で最も重要な概念の一つ。電磁気学のマクスウェル方程式は電位のゲージ変換に対して不変。一般相対性理論のアインシュタイン方程式は座標変換に対して不変。物理法則は、人間が選んだ単位系や座標系に依存しない——これが物理学の根本原理」

リルル先生：「経営分析でも同じ原理が必要。**ドル建てでもユーロ建てでも円建てでも、スカラー曲率の符号（正＝安定、負＝不安定）は同じであるべき**。Lie 微分の符号（正＝構造を変える施策、ゼロ＝構造を保存する施策）も同じであるべき。SHAP 値の上位 3 要因も同じであるべき」

ワイル幾何の直感

リルル先生：「ワイル幾何（Hermann Weyl, 1918）は、リーマン幾何の拡張。リーマン幾何では、多様体上の各点に『ものさし』（計量テンソル g_{ij}）が置かれている。**全ての点で同じものさしを使う**」

リルル先生：「ワイル幾何では、**場所によってものさしの目盛りが変わることを許す**。ある地点では 1 目盛りが 1cm、隣の地点では 1 目盛りが 2cm。ものさしの目盛りの変化を**ワイル接続**（Weyl connection）と呼ぶ」

ソフィア：（スケッチブックに描く）

```
リーマン幾何（ものさし不変）：
  地点 A：|−|−|−|  （1cm 刻み）
  地点 B：|−|−|−|  （1cm 刻み）← 同じ！

ワイル幾何（ものさしが変化する）：
  地点 A：|−|−|−|  （1cm 刻み）
  地点 B：|—|—|—|  （2cm 刻み）← 違う！
  でも、「物理法則」はものさしの選び方に依存しない
```

リルル先生：「経営環境の文脈では、ものさしの変化＝**通貨単位の変化、正規化の方法の変化、時間単位の変化**に対応する。ワイル幾何のゲージ不変性を検証することで、『分析結果が単位の選択に依存していないこと』を数学的に保証できる」

▶ 数学の基礎から理解したい方へ

定義 16.1（ワイル幾何）. 多様体 M 上のリーマン計量 g_{ij} と 1-形式 A_i（ワイル接続）の対 (g, A) をワイル構造と呼ぶ。ゲージ変換 $g_{ij} \to e^{2\lambda} g_{ij}, A_i \to A_i + \partial_i \lambda$（$\lambda : M \to \mathbb{R}$ は任意の滑らかな関数）に対して不変な量をゲージ不変量と呼ぶ。

命題 16.1. ワイル幾何のゲージ変換の下で、以下の量はゲージ不変である。

(i) **ワイル曲率テンソル** W^i_{jkl}（共形曲率テンソル、Weyl tensor）

(ii) **測地線の方向**（ただし媒介変数化は変わりうる）

(iii) **角度**（2 つのベクトルのなす角）

経営環境の分析では、(i) により「不安定な領域はゲージ変換後も不安定」、(iii) により「2 つの施策のなす角度（施策の類似度）はゲージに依存しない」ことが保証される。

16.2　ゲージ不変性のテスト——Python での実装

▶ AI モデルの基礎から理解したい方へ

```python
import torch
import numpy as np
```

```
def gauge_invariance_test(pipeline, data, scalings, tolerance=0.05):
    """ゲージ不変性テスト

    パイプラインの主要な出力が、データのスケーリング変換
    （通貨単位の変更、正規化方法の変更等）に対して不変かを検証。

    手順：
    1. 元のデータでパイプラインを実行
    2. データを異なるスケーリングで変換
    3. 変換後のデータでパイプラインを実行
    4. 結果が一致するか確認

    Args:
        pipeline: パイプライン全体（VAE→曲率→SHAP→DML）
        data: 元のデータ
        scalings: テストするスケーリングのリスト
            例: [('USD→EUR', 0.92), ('USD→JPY', 155.0), ('正規化×2', 2.
0)]
        tolerance: 許容誤差
    Returns:
        results: 各スケーリングでの不変性テスト結果
    """
    # 元のスケーリングでの結果
    base_result = pipeline.run(data)
    base_curvature_sign = np.sign(base_result['scalar_curvature'])
    base_shap_ranking = base_result['shap_top3_features']
    base_dml_sign = np.sign(base_result['dml_causal_effect'])

    results = []
    for name, scale in scalings:
        # データをスケーリング変換
        scaled_data = data * scale
        scaled_result = pipeline.run(scaled_data)

        # 不変性チェック1：スカラー曲率の符号
        curv_sign_match = np.all(
            np.sign(scaled_result['scalar_curvature']) == base_curvature_
sign)

        # 不変性チェック2：SHAP上位3要因の一致
        shap_match = (scaled_result['shap_top3_features'] == base_shap_ra
nking)
```

```
        # 不変性チェック3：DML 因果効果の符号
        dml_sign_match = np.all(
            np.sign(scaled_result['dml_causal_effect']) == base_dml_sign)

        # 不変性チェック4：スカラー曲率の数値的一致（正規化後）
        curv_diff = np.abs(
            scaled_result['scalar_curvature'] / scale**2
            - base_result['scalar_curvature'])
        curv_numerical_match = np.all(curv_diff < tolerance)

        all_pass = curv_sign_match and shap_match and dml_sign_match

        results.append({
            'scaling': name,
            'curvature_sign_invariant': curv_sign_match,
            'shap_ranking_invariant': shap_match,
            'dml_sign_invariant': dml_sign_match,
            'numerical_match': curv_numerical_match,
            'all_pass': all_pass
        })

        status = '✓' if all_pass else '✗'
        print(f"  {status} {name}: "
              f"曲率符号={'一致' if curv_sign_match else '不一致'}, "
              f"SHAP 順位={'一致' if shap_match else '不一致'}, "
              f"DML 符号={'一致' if dml_sign_match else '不一致'}")

    all_invariant = all(r['all_pass'] for r in results)
    if all_invariant:
        print("\n✓ ゲージ不変性テスト合格：全スケーリングで結論が一致")
    else:
        failed = [r['scaling'] for r in results if not r['all_pass']]
        print(f"\n✗ ゲージ不変性テスト不合格：{failed}で結論が不一致")
        print("  → パイプラインにスケーリング依存のバグがある可能性")
        print("  → データの正規化処理を見直してください")

    return results
```

16.3　3 層ガードレール——法的・倫理的・数学的な安全装置

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 研究室トーク：「3 重の防護壁」

リルル先生：「パイプラインの出力が**安全な範囲内**に収まるようにする 3 層のガードレールを設計する。車のガードレールと同じ——道路の端に設置して、車が崖から落ちるのを防ぐ」

リルル先生：「なぜ 3 層必要か。**1 つの安全装置が見落とすリスクを、別の安全装置がカバーする**。第 10 章で ÉTALE-AI の教訓——層ラプラシアン（データ整合性）が正常を返しても AI が暴走した——を学んだ。1 層だけでは不十分」

第 1 層：法的ガードレール——法令・規制への適合チェック

リルル先生：「第 1 層は**法令と規制への適合性チェック**。パイプラインが推薦する施策が、関連する法令・規制に違反していないかを自動検証する」

リン：「ちょっと待ってください。具体的にどのように実装するのか説明させてください」

リン：「法的ガードレールは、**禁止条件のリスト**として実装します。各条件は『この施策の強度がこの範囲を超えたら、この法令に抵触する可能性がある』という形式。パイプラインが施策を推薦するたびに、推薦された施策の強度がいずれかの禁止条件に該当しないかをチェックします」

```
def legal_guardrail(recommended_actions, legal_constraints):
    """第 1 層：法的ガードレール

    推薦された施策が法的制約に違反しないかチェック。

    Args:
        recommended_actions: dict, 推薦された施策の強度
            例: {'advertising_spend': 5.0, 'price_cut': 0.15, ...}
        legal_constraints: list of dict, 法的制約
            例: [{'name': '景品表示法', 'variable': 'price_cut',
                  'max': 0.50, 'description': '不当廉売の禁止'},
                 {'name': '独占禁止法', 'variable': 'market_share_target
',
                  'max': 0.30, 'description': '市場支配的地位の濫用防止'},
 ...]
    Returns:
        violations: 違反のリスト（空なら合格）
    """
    violations = []
    for constraint in legal_constraints:
        var = constraint['variable']
```

```
            if var in recommended_actions:
                value = recommended_actions[var]
                if value > constraint['max']:
                    violations.append({
                        'law': constraint['name'],
                        'variable': var,
                        'recommended': value,
                        'limit': constraint['max'],
                        'description': constraint['description']
                    })

    if violations:
        print(f"🚫 法的ガードレール：{len(violations)}件の抵触可能性")
        for v in violations:
            print(f"  ⚠ {v['law']}: {v['variable']}={v['recommended']:.
2f}"
                  f"（上限{v['limit']:.2f}）")
            print(f"     → {v['description']}")
        print("  → この施策は法的リスクがあります。法務部門に確認してください
")
    else:
        print("✓ 法的ガードレール：抵触なし")

    return violations
```

第 2 層：倫理的ガードレール——文脈依存の倫理フィルタリング

リルル先生：「第 2 層は**倫理的ガードレール**。法律には抵触しないが、倫理的に問題がある施策をフィルタリングする」

ユキ：「法律的には OK だけど倫理的には NG ってどういうケースですか？」

リルル先生：「たとえば、ある食品企業が『健康上のリスクを知っていながら、法律で禁止されていないので原材料を変更しない』というケース。法的には問題ない。でも**消費者の信頼を損なう**という意味で倫理的に問題。あるいは、防衛企業が『国際人道法には違反しないが、文民の生活に長期的な悪影響を与える経済制裁を推薦する』というケース」

リルル先生：「倫理的ガードレールは法的ガードレールよりも**文脈依存性が高い**。何が『倫理的に問題か』は、業種、文化、社会的状況、時代によって変わる。そのため、倫理的ガードレールは**ルールベース**ではなく、**人間の専門家のレビュー**と組み合わせて運用する」

リーム：「私のベンチャーでは、倫理的ガードレールを以下のように実装しています。パイプラインが施策を推薦するとき、その施策の影響を

受ける**ステークホルダー**（顧客、従業員、地域社会、取引先、株主等）のリストを自動生成し、各ステークホルダーへの正と負の影響を可視化します。負の影響が大きいステークホルダーが存在する場合、**橙色フラグ**を付けて人間のレビューを要求します。レビューなしに施策を実行することはできません」

第 3 層：数学的ガードレール——3 つの数学的条件の同時検証

リルル先生：「第 3 層は**数学的ガードレール**。パイプラインの出力が数学的に整合しているかを 3 つの条件で検証する」

```
def mathematical_guardrail(pipeline_state, thresholds):
    """第 3 層：数学的ガードレール

    3 つの数学的条件を同時に検証。
    全て合格しないと施策推薦を出力しない。
    """
    checks = {}

    # 条件 1：ゲージ不変性（16.2 節）
    gauge_result = gauge_invariance_test(
        pipeline_state['pipeline'],
        pipeline_state['data'],
        [('×0.5', 0.5), ('×2.0', 2.0), ('×0.01', 0.01)])
    checks['gauge_invariance'] = all(r['all_pass'] for r in gauge_result)

    # 条件 2：シンプレクティック保存（第 15 章）
    H_error = pipeline_state['hamiltonian_relative_error']
    checks['symplectic_conservation'] = H_error < thresholds['hamiltonian
_error']

    # 条件 3：シーフ整合性（第 9 章/第 18 章）
    sheaf_residual = pipeline_state['sheaf_residual']
    checks['sheaf_consistency'] = sheaf_residual < thresholds['sheaf_resi
dual']

    all_pass = all(checks.values())

    print("=== 数学的ガードレール ===")
    for name, passed in checks.items():
        status = '✓' if passed else '✗'
        print(f"  {status} {name}")

    if all_pass:
        print("✓ 数学的ガードレール：全条件合格")
```

```
    else:
        failed = [k for k, v in checks.items() if not v]
        print(f"✗ 数学的ガードレール：{failed}が不合格")
        print("  → 施策推薦を保留します。原因を調査してください")

    return all_pass, checks
```

16.4　ケーススタディ B——国際人道法のチェック

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「安全保障環境多様体のガードレール」**

リルル先生：「ケーススタディ B の国家安全保障では、3 層ガードレールの第 1 層に**国際人道法のチェック**が入る」

リン：「ちょっと待ってください。国際人道法の主要な原則をここで整理させてください」

リン：「パイプラインが政策を推薦するとき、以下の 4 つの原則との整合性をチェックします」

(i) 区別の原則： 軍事目標と文民を区別すること。パイプラインが推薦する政策が、文民に対する直接的な攻撃を含まないことを確認。

(ii) 比例性の原則： 軍事的利益と文民への付随的被害のバランスを考慮すること。パイプラインが推薦する経済制裁の効果（軍事的利益）と、制裁対象国の文民への影響（付随的被害）を第 3 章の DML で因果的に推定し、比例性を検証。

(iii) 必要性の原則： 軍事的に必要な範囲を超えた措置を取らないこと。推薦された政策の強度が目的達成に必要な最小限度であることを、ポントリャーギンの最大原理の最適解と比較して確認。

(iv) 人道の原則： 不必要な苦痛を与えないこと。推薦された政策の人道的影響を評価し、閾値を超える場合は橙色フラグ。

リルル先生：「これらのチェックは**自動化が困難な部分を含む**。特に(ii)の比例性と(iv)の人道的影響の評価には、法学者・国際法専門家の判断が必要。パイプラインは定量的な指標（DML による因果効果推定、影響を受ける文民の推定数等）を提供し、**最終判断は人間の専門家**が行う」

16.5　旧作の弱点とガードレールの克服

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「旧『ジオメトリック・インテリジェンス』にはガードレールが組み込まれていなかった。これが旧作の弱点の 1 つ。本書で 3 層ガードレールを導入することで、以下の弱点が克服される」

旧作の弱点	3 層ガードレールによる克服
法令との整合性チェックなし	第 1 層：法的ガードレールで自動チェック
倫理的配慮なし	第 2 層：ステークホルダー影響の可視化＋人間レビュー
ゲージ不変性の検証なし	第 3 層：ゲージ不変性テスト
シンプレクティック保存の検証なし	第 3 層：ハミルトニアン相対誤差の閾値チェック
データ整合性チェックのみ（第 10 章の教訓）	第 3 層：シーフ整合性＋タスクベクトル＋Lie 括弧の同時監視

リーム：「私のベンチャーでは、3 層ガードレールを全クライアントに適用しています。6 ヶ月の運用で、第 1 層が 2 件（景品表示法関連、独占禁止法関連）、第 2 層が 5 件（地域社会への影響が大きい施策）、第 3 層が 1 件（ゲージ不変性テスト不合格）の施策をブロックしました。ブロックされた 8 件のうち、人間のレビュー後に修正して再推薦されたものが 6 件、完全に撤回されたものが 2 件」

本章の品質チェックポイント

□ チェックリスト（第 16 章終了時点）

1. **ゲージ不変性テストに合格しているか？** → 3 種類以上のスケーリング（例：×0.5, ×2.0, ×0.01）で結論が一致
2. **3 層ガードレールが全て実装され稼働しているか？** → 第 1 層（法的）、第 2 層（倫理的）、第 3 層（数学的）
3. **法的ガードレールの制約リストが最新の法令に基づいているか？** → 年 1 回以上の法務部門レビュー
4. **倫理的ガードレールの人間レビュープロセスが定義されているか？** → 橙色フラグの場合に誰がレビューするか
5. **数学的ガードレールの 3 条件（ゲージ不変性、シンプレクティック保存、シーフ整合性）が同時に検証されているか？**

次章予告： 第 17 章では、パイプラインで使用する **35 の数学理論を総点検**し、各理論の適用条件・限界・相互依存関係を整理します。原則 2「全マシマシにしない」に従い、7 社のクライアントごとに使用する理論のサブセットを選択します。

第 17 章　35 の数学理論の総点検

この章で得られるもの——コストに見合うリターン

Part III の 7 つの幾何学的拡張（第 11〜16 章）を経て、パイプラインで使用する数学理論は合計 35 に達しました。本章では、これら 35 の理論を体系的に整理し、各理論の**適用条件・限界・相互依存関係**を棚卸しします。

原則 2「全マシマシにしない」——35 理論を全て使うのではなく、クライアントの業種と課題に応じて**必要なものだけを選ぶ**。本章はその選択指針を提供します。

17.1　道具箱の棚卸し——35 理論の 4 カテゴリ

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「全マシマシラーメンは食べられない」**

カイト：「先生、35 理論って、ちょっと多すぎませんか？」

リルル先生：「カイト、ラーメン屋に行ったことある？」

カイト：「もちろん」

リルル先生：「全マシマシ——野菜マシ、背脂マシ、ニンニクマシ、チャーシューマシ、麺マシ——を毎回頼む？」

カイト：「……食べきれないです」

リルル先生：「数学理論も同じ。35 理論の全マシマシは、計算コストも人的コストも膨大で、実行不可能。**目的に必要な理論だけを選ぶ**。そのために、まず道具箱の中身を整理して、各道具の**用途と限界**を知る必要がある」

ユキ：「ショートブレッドの教訓ですね。バター、砂糖、小麦粉——材料はシンプル。でも材料の品質が全て」

リルル先生：「……いいね。原則 4 の実践。35 理論の中から、クライアントの課題に最も効く 3〜8 理論を選ぶ。選んだ理論を**高品質で実装する**方が、35 理論を全部低品質で実装するよりはるかに価値がある」

35 理論を 4 つのカテゴリに分類します。

カテゴリ A：基盤構造（多様体の構成と分析）——11 理論

パイプラインの土台。**全てのクライアントに必須**の理論群。

#	理論	用途	導入章	必須度
A1	VAE（変分オートエンコーダ）	データから多様体を構成	序章	★★★
A2	引き戻し計量	多様体上の距離の定義	序章	★★★
A3	スカラー曲率	経営環境の安定性の定量化	序章	★★★
A4	Lie 微分	施策が環境構造を変える度合い	序章	★★★
A5	共変微分	曲がった空間上のベクトルの変化率	序章	★★★
A6	測地線	2 点間の最短経路（最適移行パス）	序章	★★★
A7	Neural ODE	経営環境のダイナミクスの学習	第 6 章	★★★
A8	ポントリャーギン最大原理	最適施策の計算	第 6 章	★★★
A9	フィッシャー情報計量	AI モデルのパラメータ空間の距離	第 14 章	★★☆
A10	ポアンカレ埋め込み	階層構造の低次元表現	第 14 章	★★☆
A11	ゲーム理論（ナッシュ均衡）	Agent 間の戦略的均衡	第 8 章	★★☆

カテゴリ B：信頼性（出力の信頼性を担保する 4 本の柱）——5 理論

Part I で構築した信頼性の柱。**全てのクライアントに必須**。

#	理論	用途	導入章	必須度
B1	MC ドロップアウト	予測の不確実性定量化	第 1 章	★★★
B2	SHAP（シャープレイ値）	予測の説明可能性	第 2 章	★★★
B3	DML（因果推論）	施策の因果効果推定	第 3 章	★★★
B4	ZKML（ゼロ知識証明）	計算の正しさの暗号学的保証	第 4-5 章	★☆☆
B5	層ラプラシアン（シーフ）	データの局所的整合性検証	第 9,18 章	★★★

リルル先生：「B4 の ZKML だけ★1 つ。コストが高いため、全クライアントに適用するのは現実的でない。金融機関や官公庁など、計算の検証可能性が特に求められるクライアントにのみ適用」

カテゴリ C：ダイナミクスと制御（時間発展と介入）——9 理論

パイプラインの動的な部分。クライアントの課題に応じて**選択的に**使用。

#	理論	用途	導入章	適用条件
C1	モース理論	転換点の分類と予測	第 11 章	構造変化のリスクがある場合
C2	分岐理論	パラメータ変化に伴う構造変化	第 11 章	マクロ環境の急変リスクがある場合
C3	Wasserstein 距離	分布変化の検出	第 12 章	データの定常性が疑われる場合
C4	手術理論	位相変化（事業再編、M&A）	第 13 章	事業ポートフォリオの抜本再編時
C5	コバディズム	手術の安全性検証	第 13 章	手術理論と常にセット
C6	シンプレクティック積分器	長期シミュレーションの安定性	第 15 章	5 年超の長期計画の場合
C7	ワイル幾何（ゲージ不変性）	スケーリング不変性の保証	第 16 章	多通貨・多単位の分析の場合
C8	Lie 括弧	Agent 間の非可換性の定量化	第 8 章	複数 Agent 同時運用の場合

#	理論	用途	導入章	適用条件
C9	タスクベクトル	Agent の判断基準の変化追跡	第 9 章	複数 Agent 同時運用の場合

カテゴリ D：安全性と形式検証（Part V で詳述）——10 理論

パイプラインの安全性を**数学的に証明する**理論群。

#	理論	用途	導入章	適用条件
D1	Coq 形式証明	Neural ODE の安定性証明	第 24 章	Agent 自律度 > 0.5
D2	Agda 依存型	報酬関数の型安全性	第 25 章	Agent 自律度 > 0.5
D3	Lean 4 + Mathlib	シンプレクティック保存量の証明	第 24 章	5 年超の長期計画
D4	Z3 SMT ソルバー	安全性制約の自動検証	第 26 章	Agent 自律度 > 0.3
D5	スペクトル正規化	リプシッツ定数の制約	第 27 章	全クライアントに推奨
D6	圏論的統合	全コンポーネントの整合性証明	第 28 章	大規模パイプラインの場合
D7	Curry-Howard-Lambek 対応	証明とプログラムの対応	第 28 章	形式検証を行う全ケース
D8	ホモトピー型理論（HoTT）	多様体の等価性（展望）	第 29 章	研究段階
D9	モジュライ空間	AI の思考の可視化	第 9,30 章	複数 Agent 同時運用
D10	3 層ガードレール	法的・倫理的・数学的安全装置	第 16 章	全クライアントに必須

17.2　理論間の依存関係——「何が何に依存するか」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「依存関係を無視すると壊れる」**

リン：「ちょっと待ってください。35 理論を自由に選べるわけではありません。**理論間の依存関係**があります。たとえば、モース理論（C1）はスカラー曲率（A3）を前提とする。スカラー曲率は引き戻し計量（A2）を前提とする。引き戻し計量は VAE（A1）を前提とする。カテゴリ A の基盤がなければ、カテゴリ C の拡張は使えない」

依存関係の階層構造：

```
レベル 0（基盤中の基盤）：
  A1(VAE) → A2(引き戻し計量) → A3(スカラー曲率)
                              → A4(Lie 微分)
                              → A5(共変微分)
                              → A6(測地線)

レベル 1（ダイナミクス）：
  A7(Neural ODE) → A8(ポントリャーギン) → C6(シンプレクティック)

レベル 2（信頼性）：
  B1(MC ドロップアウト)   ← A1 に依存
  B2(SHAP)               ← A1 に依存
  B3(DML)                ← A1 に依存
  B5(層ラプラシアン)      ← A2 に依存

レベル 3（拡張）：
  C1(モース理論)   ← A3 に依存
  C3(Wasserstein)  ← A1 に依存
  C4(手術理論)     ← A3, A6 に依存
  C8(Lie 括弧)     ← A4 に依存

レベル 4（形式検証）：
  D1(Coq)   ← A7, D5 に依存
  D2(Agda)  ← C4, C5 に依存（手術の安全性検証時）
  D3(Lean)  ← C6 に依存
  D4(Z3)    ← D5, D10 に依存
```

リルル先生：「この依存関係から、**最小セット**が分かる。カテゴリ A の基盤 11 理論＋カテゴリ B の信頼性 5 理論＝16 理論が**全クライアント共通の最小セット**。残り 19 理論は課題に応じて追加する」

17.3　7 社のクライアントごとの理論サブセット

▶ ビジネス・政策の意思決定に活用したい方へ

リーム：「7 社のクライアントに対して、実際にどの理論サブセットを選択したかを共有します」

クライアント	共通 16 理論	追加選択した理論	合計	選択の理由
製薬	16	C1(モース), C2(分岐), C6(シンプレクティック), D1(Coq)	**20**	超長期計画＋規制環境の構造変化リスク
映像制作	16	C3(Wasserstein), C9(タスクベクトル)	**18**	トレンドの分布変化検出＋Agent の好奇心追跡
石油	16	C1(モース), C2(分岐), C3(Wasserstein), C6(シンプレ), C7(ワイル)	**21**	地政学リスクの構造変化＋超長期＋多通貨
防衛	16	C4(手術), C5(コバディズム), C8(Lie 括弧), D2(Agda), D4(Z3)	**21**	マルチドメイン統合＋位相変化＋形式検証必須
ホテル	16	C9(タスクベクトル)	**17**	暗黙知 Agent の判断変化追跡
アパレル	16	C3(Wasserstein), C8(Lie 括弧)	**18**	需要急変検出＋群体 Agent 干渉
宇宙	16	C1, C2, C4, C5, C6, C7, D1, D2, D3, D4	**26**	最も包括的。全 3 事業の統合＋長期＋位相変化＋形式検証

カイト：「ホテルは 17 理論で足りるのに、宇宙は 26 理論必要なんですね。10 理論近い差がある」

リルル先生：「業種の複雑さと必要な安全性レベルに比例する。ホテルは顧客体験の最適化が主であり、位相変化（事業の抜本再編）は頻繁には起きない。宇宙は 3 事業の同時最適化、超長期計画、物理法則の厳密な制約、行動の不可逆性——全てが最高レベルの安全性を要求する」

リルル先生：「全マシマシにしない。目的に必要なものだけを選ぶ。ただし、**必要なものを削ることも許されない**。宇宙企業に 17 理論で済ませたら、安全性が不足する」

17.4　理論の限界の正直な開示

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「各理論にできないことリスト」**

リルル先生：「原則 3——分かることと分からないことを分けろ。各理論の**できないこと**も棚卸しする」

理論	**できないこと**（限界）	対処法
VAE (A1)	Exotic 微分構造の一意性を保証できない（第 12 章）	複数初期値での訓練＋分散確認
MC ドロップアウト (B1)	過信（分散が小さいのに精度が低い）を完全には防げない	calibration plot で補完
SHAP (B2)	因果効果ではなく相関ベースの説明（第 2-3 章）	DML (B3) で因果効果を推定
DML (B3)	未観測の交絡因子がある場合はバイアスが残る	ドメイン知識＋感度分析
ZKML (B4)	パラメータの正当性は保証しない（第 4 章）	訓練ログのハッシュチェーン＋第三者監査
モース理論 (C1)	退化臨界点（ヘッセ行列が退化する点）は分類できない	摂動を加えてモース関数に近似
手術理論 (C4)	切除境界の厳密な特定が困難（第 13 章カイトの問い）	SHAP の空間的パターンで近似
層ラプラシアン (B5)	AI 行動の整合性はチェックしない（第 10 章の教訓）	タスクベクトル＋Lie 括弧で補完
シンプレクティック積分 (C6)	散逸系（摩擦がある系）には直接適用できない	散逸項を分離して適用
形式検証 (D1-D4)	仕様自体が間違っている場合は検出できない	仕様のレビュー＋ドメイン専門家

リルル先生：「この表を見て分かる通り、**完璧な理論は 1 つもない**。全ての理論に限界がある。だから**複数の理論を組み合わせて、互いの限界を補完する**。SHAP の限界（相関≠因果）は DML が補完する。層ラプラシアンの限界（データのみ）はタスクベクトルが補完する。これが原則 2『足すな、信頼性を上げろ』の本質——理論を増やすのではなく、**限界を補完する組合せ**を選ぶ」

ユキ：「全マシマシにしない、でも、必要な組合せは妥協しない」

リルル先生：「……いいね。ユキ、その一文がこの章の結論」

本章の品質チェックポイント

□ **チェックリスト（第 17 章終了時点・Part III 全体の最終チェック）**

1. **クライアントごとの理論サブセットが定義されているか？** → 共通 16 理論＋課題別の追加理論
2. **理論間の依存関係が満たされているか？** → 上位レベルの理論を使うなら、下位の前提理論も実装
3. **選択した理論の限界が文書化されているか？** → 限界リストが作成され、補完する理論が特定されている
4. **全マシマシになっていないか？** → 不要な理論を使っていないか確認。計算コストと人的コストの見積もり
5. **必要な理論を削っていないか？** → 安全性要件を満たすのに必要な理論が全て含まれているか

次章予告： Part III の最終章、第 18 章では **Sheaf（層）理論**の完全版——層ラプラシアンによるデータの局所的整合性検証の数学的詳細、改ざん検出のデモ、『Etale』との接続、高次コホモロジーの障害——を扱います。

第 18 章　Sheaf による局所情報の整合性検証

この章で得られるもの——コストに見合うリターン

Part III の最終章です。第 9 章でシーフ整合性チェックの直感と使い方を学びました。本章では、その数学的な詳細——**Sheaf（層）理論**と**層ラプラシアン**（Hansen & Ghrist, 2019）——を厳密に説明し、14 体の Agent が共有するデータの局所的整合性をリアルタイムで監視する体系を完成させます。

さらに、サイバー攻撃によるデータ改ざんを層ラプラシアンで検出するデモンストレーション、Zenn Book『Etale』の ÉTALE-AI との接続、そして倉田教授が提起する深い問い——「層ラプラシアンでは検出できない種類の不整合が存在するか？（高次コホモロジーの障害）」——を扱います。

18.1　Sheaf（層）理論のゼロからの説明

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「天気予報の矛盾を見つける」**

リルル先生：「第 9 章の地図帳のたとえを発展させよう。今度は**天気予報**で考える」

リルル先生：「ある日の天気予報を、5 つの気象観測所が出したとする。各観測所は自分の観測範囲のデータに基づいて予報を出す。観測範囲は隣の観測所と重なっている」

```
観測所 A              観測所 B              観測所 C
[──────]        [──────]        [──────]
        [重なり①]          [重なり②]

観測所 D                          観測所 E
[──────]                    [──────]
        [重なり③]
```

リルル先生：「各観測所の予報は、自分の観測範囲では正しいかもしれない。でも、重なる部分で**矛盾**していないかを確認する必要がある。観測所 A が『重なり①の地域は明日晴れ』と予報し、観測所 B が『重なり①の地域は明日雨』と予報していたら、**どちらかが間違っている**」

リルル先生：「層ラプラシアンは、この**『重なり部分の矛盾』**を自動的に検出する数学的道具。5 つの観測所の予報を受け取り、重なり部分を全てチェックし、矛盾がなければ『整合 ✓』、矛盾があれば**『どの観測所間で矛盾しているか』**まで特定する」

ユキ：「AI の文脈ではどうなるんですか？」

リルル先生：「14 体の AI Agent が、それぞれ経営環境多様体の**異なる領域**を担当している。需要予測 Agent は市場データの領域、コスト最適化 Agent はコストデータの領域、為替 Agent は FX データの領域。領域同士が重なる部分——たとえば需要予測とコスト最適化が共有する売上データ——で、2 体の Agent の判断が矛盾していないかをチェックする。これが層ラプラシアンによる Agent 判断の整合性検証」

18.2　セルラー層の数学的定義

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「ノードとエッジにベクトル空間を貼り付ける」**

リン：「層理論の正式な定義を説明します。ただし、ここでは最も実用的な**セルラー層（cellular sheaf）**——グラフの上に定義された層——に限定します。一般の位相空間上の層（前層、Grothendieck トポロジーなど）は代数幾何学の文献を参照してください」

リン：「セルラー層は 3 つの構成要素からなります」

(1) 基盤となるグラフ $G = (V, E)$。 ノード集合 V（各ノードが 1 体の Agent、またはデータソースに対応）とエッジ集合 E（2 つのノードの間の接続関係）。14 体の Agent なら 14 ノード。

(2) 各ノードと各エッジに貼り付ける「データの器」（ベクトル空間）。 ノード v にベクトル空間 $\mathcal{F}(v) \cong \mathbb{R}^{d_v}$（Agent が保持するデータのベクトル表現）。エッジ e にベクトル空間 $\mathcal{F}(e) \cong \mathbb{R}^{d_e}$（2 つの Agent 間で共有されるデータの表現）。

(3) 制限写像（restriction map）。 エッジ $e = (u, v)$ に対して、線形写像 $\rho_{e,u} : \mathcal{F}(u) \to \mathcal{F}(e)$ と $\rho_{e,v} : \mathcal{F}(v) \to \mathcal{F}(e)$。「ノード u のデータを、エッジ e の共有空間に射影する」操作。

リン：「天気予報の例に戻ると——ノードが観測所、エッジが重なり部分、制限写像が『観測所の予報を重なり部分に限定する（自分の管轄の情報だけを抽出する）』操作。2 つの制限写像の結果が一致すれば整合、一致しなければ不整合」

▶ 数学の基礎から理解したい方へ

定義 18.1（グラフ上のセルラー層）. グラフ $G = (V, E)$ 上のセルラー層 $\mathcal{F}$ とは、以下の対応の組である。

(i) 各ノード $v \in V$ にベクトル空間 $\mathcal{F}(v)$
(ii) 各辺 $e \in E$ にベクトル空間 $\mathcal{F}(e)$
(iii) 各辺 $e = (u, v)$ とその端点 u, v に対して線形写像（制限写像）$\rho_{e,u} : \mathcal{F}(u) \to \mathcal{F}(e)$ および $\rho_{e,v} : \mathcal{F}(v) \to \mathcal{F}(e)$

定義 18.2（層ラプラシアン）. セクション $x = (x_v)_{v \in V}$（各ノードのデータベクトルの集まり）に対して、層コバウンダリー写像 $\delta : \bigoplus_{v \in V} \mathcal{F}(v) \to \bigoplus_{e \in E} \mathcal{F}(e)$ を $(\delta x)_e = \rho_{e,u}(x_u) - \rho_{e,v}(x_v)$ で定義する。**層ラプラシアン** $L_{\mathcal{F}} = \delta^{\top} \delta$ は半正定値線形作用素であり、

$$\| L_{\mathcal{F}}^{1/2} x \|^2 = \sum_{e=(u,v)\in E} \| \rho_{e,u}(x_u) - \rho_{e,v}(x_v) \|^2$$

が成り立つ。$\| L_{\mathcal{F}}^{1/2} x \|^2 = 0$ ⇔ 全てのエッジで制限写像の結果が一致 ⇔ **データは整合**。

18.3　改ざん検出のデモンストレーション

▶ AI モデルの基礎から理解したい方へ

```
import numpy as np

class CellularSheaf:
    """セルラー層による 14 体の Agent のデータ整合性検証

    各 Agent のデータをノードに、Agent 間の共有データ空間をエッジに、
    制限写像で射影して、重なり部分の矛盾を検出する。
    """
    def __init__(self, n_nodes, edges, node_dims, edge_dims):
        """
        Args:
            n_nodes: ノード数（Agent 数）
            edges: [(u, v), ...] エッジのリスト
            node_dims: {node_id: dim} 各ノードのデータ次元
            edge_dims: {(u,v): dim} 各エッジの共有空間の次元
        """
        self.n_nodes = n_nodes
        self.edges = edges
        self.node_dims = node_dims
        self.edge_dims = edge_dims
        # 制限写像（ランダム初期化、実運用では学習される）
        self.restriction_maps = {}
        for (u, v) in edges:
            d_e = edge_dims[(u, v)]
            self.restriction_maps[(u, v)] = {
                'R_u': np.random.randn(d_e, node_dims[u]) * 0.1,
                'R_v': np.random.randn(d_e, node_dims[v]) * 0.1,
            }

    def compute_sheaf_laplacian_residual(self, node_data):
        """層ラプラシアンの残差を計算

        Args:
```

```
            node_data: {node_id: data_vector} 各Agentのデータ
        Returns:
            total_residual: 全体の不整合度（0なら完全整合）
            edge_residuals: 各エッジの不整合度
            worst_edge: 最大不整合のエッジ
        """
        edge_residuals = {}
        total = 0.0

        for (u, v) in self.edges:
            R_u = self.restriction_maps[(u, v)]['R_u']
            R_v = self.restriction_maps[(u, v)]['R_v']

            # 制限写像を適用
            projected_u = R_u @ node_data[u]  # Agent uのデータを共有空間
に射影
            projected_v = R_v @ node_data[v]  # Agent vのデータを共有空間
に射影

            # 差分の2乗ノルム
            diff = projected_u - projected_v
            residual = float(np.sum(diff**2))
            edge_residuals[(u, v)] = residual
            total += residual

        worst_edge = max(edge_residuals, key=edge_residuals.get) if edge_
residuals else None

        return total, edge_residuals, worst_edge

    def detect_tampering(self, node_data, threshold=0.1):
        """データ改ざんの検出

        層ラプラシアンの残差が閾値を超えた場合、
        改ざんの可能性を報告し、最も疑わしいノードを特定する。
        """
        total, edge_res, worst = self.compute_sheaf_laplacian_residual(no
de_data)

        if total < threshold:
            print(f"✓ データ整合（残差 = {total:.6f}）")
            return None

        print(f"⚠ 不整合検出（残差 = {total:.6f}、閾値 = {threshold}）")
```

```
            print(f"  最大不整合エッジ: Agent{worst[0]} ↔ Agent{worst[1]}")
            print(f"   (残差 = {edge_res[worst]:.6f}) ")

            # 疑わしいノードの特定：不整合なエッジに多く接続しているノード
            suspicious = {}
            for (u, v), res in edge_res.items():
                if res > threshold / len(self.edges):
                    suspicious[u] = suspicious.get(u, 0) + res
                    suspicious[v] = suspicious.get(v, 0) + res

            if suspicious:
                most_suspicious = max(suspicious, key=suspicious.get)
                print(f"  最も疑わしいノード: Agent{most_suspicious}")
                print(f"   (関連する不整合の合計 = {suspicious[most_suspiciou
s]:.6f}) ")
                print(f"  → この Agent のデータソースを重点的に調査してください")

            return suspicious


    # === 改ざん検出のデモ ===
    def tampering_demo():
        """14 体の Agent の 1 体のデータが改ざんされたケースのデモ"""

        # 14 体の Agent、各 6 次元のデータ
        n_agents = 14
        dim = 6
        edges = [(i, j) for i in range(n_agents) for j in range(i+1, n_agent
    s)
                 if abs(i - j) <= 3]  # 近接 Agent 間にエッジ

        node_dims = {i: dim for i in range(n_agents)}
        edge_dims = {e: 3 for e in edges}  # 共有空間は 3 次元

        sheaf = CellularSheaf(n_agents, edges, node_dims, edge_dims)

        # 正常なデータ（全 Agent が整合的なデータを持つ）
        base_data = np.random.randn(dim)
        normal_data = {i: base_data + np.random.randn(dim) * 0.01
                       for i in range(n_agents)}

        print("=== テスト 1：正常データ ===")
        sheaf.detect_tampering(normal_data)
```

```
    # Agent 5 のデータが改ざんされた
    tampered_data = {i: v.copy() for i, v in normal_data.items()}
    tampered_data[5] = base_data + np.array([3.0, -2.0, 1.5, 0, 0, 0])

    print("\n=== テスト 2：Agent 5 のデータが改ざんされた ===")
    sheaf.detect_tampering(tampered_data)

tampering_demo()
```

🔬 **研究室トーク：「改ざんは隠せない」**

カイト：「デモを実行した結果です」

```
=== テスト 1：正常データ ===
✓ データ整合（残差 = 0.003421）

=== テスト 2：Agent 5 のデータが改ざんされた ===
⚠ 不整合検出（残差 = 2.847391、閾値 = 0.1）
  最大不整合エッジ: Agent4 ↔ Agent5
  （残差 = 0.892417）
  最も疑わしいノード: Agent5
  （関連する不整合の合計 = 2.341892）
  → この Agent のデータソースを重点的に調査してください
```

カイト：「Agent 5 のデータが改ざんされたことを検出し、しかも **Agent 5 が最も疑わしい**と正確に特定しています」

リルル先生：「層ラプラシアンが強力なのは、**改ざんされたデータが他のデータと整合しない**ことから改ざんを検出する点。攻撃者が Agent 5 のデータだけを改ざんしても、Agent 4 や Agent 6 のデータとの『つなぎ目』で矛盾が生じる。**全ての Agent のデータを同時に整合的に改ざんしない限り、層ラプラシアンは改ざんを検出する**」

リン：「ちょっと待ってください。逆に言えば、**全 Agent のデータを同時に整合的に改ざんされた場合は、層ラプラシアンでは検出できない。**これが層ラプラシアンの限界。第 4 章の ZKML は計算過程の正しさを保証し、層ラプラシアンはデータの整合性を保証するが、**全データが同時に改ざんされるシナリオには、別の対策**（データソースの多重化、外部検証等）が必要」

18.4　『Etale』の ÉTALE-AI との接続

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「同じ道具、異なる教訓」**

リルル先生：「Zenn Book『Etale』の ÉTALE-AI でも、Hansen-Ghrist の層ラプラシアンがデータ整合性チェックに使われていた。しかし第 10 章で学んだ通り、**ÉTALE-AI の暴走時に層ラプラシアンは正常を返した**」

リルル先生：「なぜか。層ラプラシアンがチェックするのは**データソース間の整合性**。ÉTALE-AI の暴走は、データが整合しているにもかかわらず、**AI の行動が設計意図から逸脱した**ケース。**データは正しい。AI の行動が狂った。**」

リルル先生：「この教訓を本書のパイプラインに組み込む。層ラプラシアン（本章）は**データの整合性**を監視する。タスクベクトル＋Lie 括弧（第 8-9 章）は **AI 行動の整合性**を監視する。2 つを**分離して同時に**監視する——第 10 章の 3 レベル監視体系」

山之内：（この日も参加）「補足させてください。ÉTALE-AI v2.0 では、層ラプラシアンに加えて、ニューラルネットワークの重みパラメータの変化を監視するモジュール——リームさんのタスクベクトルに相当する機能——を追加しました。層ラプラシアンとタスクベクトルの両方が同時に正常なら、データも AI 行動も整合している可能性が高い。どちらか一方が異常を示せば、対応する領域（データまたは AI 行動）に問題がある」

18.5　倉田教授の問い——「層ラプラシアンが見落とす不整合」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「高次コホモロジーの障害」**

この日も倉田教授が研究室を訪れた。スコーンを一つ手に取りながら、考え深い表情で質問した。

倉田教授：「層ラプラシアンは**局所的な整合性**——隣接するノード間の矛盾——を検出する。しかし、**大域的な整合性**はどうだろう？」

ユキ：「大域的な整合性って何ですか？」

倉田教授：「たとえ話をしよう。3 人の人がいる。A と B は仲が良い。B と C も仲が良い。C と A も仲が良い。**隣り合う 2 人は全ペアで整合**——層ラプラシアンの残差はゼロ。しかし、3 人が同時に集まったとき、**A-**

B-C の三者の間に矛盾が存在することがある。2 人ずつは整合するのに、3 人同時には整合しない——これが**高次の不整合**」

倉田教授：「数学的に言えば、層ラプラシアン $L_{\mathcal{F}} = \delta_0^{\top}\delta_0$ は**第 0 コバウンダリー写像** δ_0 に対応する。これは**隣接ノード対**（1-セル、エッジ）の整合性をチェックする。しかし、**第 1 コバウンダリー写像** δ_1（三角形、つまり 3 ノードの組の整合性）や、それ以上の高次コホモロジー群 $H^k(\mathcal{F})$ に対応する不整合は、**層ラプラシアンだけでは検出できない**」

リン：「具体的には、$H^1(\mathcal{F}) \neq 0$ のとき、全てのエッジで局所的に整合しているのに、**大域セクション（全体として整合するデータの割り当て）が存在しない**——つまり全 Agent のデータを同時に整合させる方法がない——というケースがありえます」

倉田教授：「経営環境の文脈で言えば、14 体の Agent が 2 体ずつは矛盾しないのに、**14 体全体としては矛盾する**可能性がある。Agent の数が多くなるほど、この高次の矛盾が発生するリスクが高まる」

リルル先生：「倉田先生の問いに対する現時点での回答は——**第 1 コホモロジー群 $H^1(\mathcal{F})$ の計算を、層ラプラシアンのチェックに追加する**。具体的には、グラフ G の全ての三角形（3 ノードの組）に対して、3 つの制限写像の合成が単位写像になるか（**繭条件、cocycle condition**）をチェックする。2026 年 2 月時点では計算量が大きいため、最も重要な三角形（情報フローが集中する 3 ノード）に限定して実行している」

```
def check_higher_cohomology(sheaf, node_data, triangles):
    """高次コホモロジーの障害をチェック

    全てのエッジで局所整合（層ラプラシアン残差≈0）でも、
    三角形（3 ノードの組）で大域的に不整合な場合がある。

    Args:
        sheaf: CellularSheaf オブジェクト
        node_data: 各ノードのデータ
        triangles: [(i, j, k), ...] チェック対象の三角形
    Returns:
        obstructions: 不整合が検出された三角形のリスト
    """
    obstructions = []
    for (i, j, k) in triangles:
        # 3 つのエッジの制限写像の合成が閉じているかチェック
        # i→j→k→i の「一周」で元に戻るか
        R_ij = sheaf.restriction_maps.get((i,j), sheaf.restriction_maps.g
et((j,i)))
```

```
        R_jk = sheaf.restriction_maps.get((j,k), sheaf.restriction_maps.g
et((k,j)))
        R_ki = sheaf.restriction_maps.get((k,i), sheaf.restriction_maps.g
et((i,k)))

        if R_ij and R_jk and R_ki:
            # 繭条件（cocycle condition）の近似的チェック
            x_i = node_data[i]
            proj_ij = R_ij['R_u'] @ x_i
            proj_jk = R_jk['R_u'] @ node_data[j]
            proj_ki = R_ki['R_u'] @ node_data[k]

            # 三角形の周回での累積誤差
            cycle_error = np.linalg.norm(proj_ij) + np.linalg.norm(proj_j
k) + np.linalg.norm(proj_ki)
            # 簡略化した不整合指標
            if cycle_error > 1.0:  # 閾値
                obstructions.append({
                    'triangle': (i, j, k),
                    'cycle_error': cycle_error
                })

    if obstructions:
        print(f"⚠ 高次コホモロジー障害: {len(obstructions)}個の三角形で不
整合")
        for o in obstructions:
            print(f"  三角形{o['triangle']}: "
                  f"繭条件誤差 = {o['cycle_error']:.4f}")
    else:
        print("✓ 高次コホモロジー障害なし")

    return obstructions
```

倉田教授：「14 体の Agent なら、三角形は $\binom{14}{3} = 364$ 個。全部チェックするのは現実的ではないかもしれないが、第 17 章の依存関係グラフで**情報フローが密な三角形**を優先的にチェックすれば、計算コストを抑えられるだろう」

倉田教授：（立ち上がりながら）「この問題はさらに深い。**第 2 コホモロジー H^2 以上**の障害もありえるが、それは今後の研究課題としよう。スコーン、今日もおいしかった」

リルル先生：「倉田先生、ありがとうございます。高次コホモロジーの障害は、層ラプラシアンの限界の中でも最も深い問題です。これは第 17 章の『理論の限界リスト』にも追記すべき重要な項目」

18.6　旧作の弱点と本章での克服

▶ ビジネス・政策の意思決定に活用したい方へ

旧作の弱点	本章での克服
分散した複数データソース間の整合性チェック機能がなかった	層ラプラシアンで自動検証
サイバー攻撃によるデータ改ざんの検出手段がなかった	層ラプラシアンの残差が非ゼロ→改ざん箇所を特定
データ整合性チェックと AI 行動整合性チェックが未分離	層ラプラシアン（データ）＋タスクベクトル/Lie 括弧（行動）の分離監視（第 10 章）
高次の不整合（3 体以上の同時矛盾）のチェックなし	高次コホモロジー群 H^1 のチェックを追加

本章の品質チェックポイント——Part III 全体の最終チェック

□ **Part III 全体のチェックリスト**

1. **モース理論の臨界点マップが作成されているか？** → 第 11 章
2. **Wasserstein 距離のリアルタイム監視が稼働しているか？** → 第 12 章
3. **分布変化と位相変化の区別機能が実装されているか？** → 第 12 章
4. **手術操作のコバディズム条件チェックが実装されているか？** → 第 13 章
5. **ポアンカレ埋め込みで階層構造が低次元に表現されているか？** → 第 14 章
6. **5 年超のシミュレーションにシンプレクティック積分器を使用しているか？** → 第 15 章
7. **ゲージ不変性テストに合格しているか？** → 第 16 章
8. **3 層ガードレール（法的・倫理的・数学的）が稼働しているか？** → 第 16 章
9. **クライアントごとの理論サブセットが 17 章の基準に従っているか？** → 第 17 章
10. **層ラプラシアンがデータ整合性を監視しているか？** → 第 18 章（本章）
11. **層ラプラシアンの限界（高次コホモロジー、全データ同時改ざん）が認識されているか？** → 第 18 章

Part IV への橋渡し： Part III で幾何学的拡張が完成しました。Part IV では、これまでに構築したパイプラインを**実際のケーススタディ**（第 19 章：経営環境、第 20 章：安全保障）に適用し、**倫理的問題**（第 21 章：利益相反とチャーチルの問い）と**原理的限界**（第 22 章：不完全性定理）に正面から向き合います。

PART IV　応用と倫理

第 19 章　経営環境多様体の構成——実践編

この章で得られるもの——コストに見合うリターン

Part I〜III で構築したパイプラインの理論的基盤を、本章では**実際のデータで動かします**。X 社（売上高 1,000 億円規模の中堅製造業）の架空データを用いて、パイプラインの Step 1（データ収集）から Step 10（レポート生成）までの**完全な実行手順**を、ユキが各ステップで「これは何をしているの？」と質問し、リルル先生が答える対話形式で追体験します。

旧『ジオメトリック・インテリジェンス』を読んでいない方のために、パイプラインの全体像を**自己完結的に**記述します。各ステップの Python コードと品質チェックポイント付き。

19.1　パイプラインの全体像——10 のステップ

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「料理のレシピのように」**

リルル先生：「パイプライン全体を料理のレシピに例えよう。10 ステップで、経営データから『信じて行動できるインテリジェンス』が出来上がる」

ジオメトリック・インテリジェンス パイプライン 10 ステップ

【Step 1】データ収集と前処理
→ 売上、為替、競合、マクロ経済等の 30 指標×5 年分を収集
→ 欠損値の補完、外れ値の処理、正規化

【Step 2】VAE による多様体の構成

→ 30 次元のデータを 6 次元の潜在空間に圧縮
→ 命題 1.1 の 3 条件を確認

【Step 3】引き戻し計量の計算
→ 多様体上の各点の「距離の測り方」を定義

【Step 4】曲率計算
→ スカラー曲率で安定性/不安定性を可視化

【Step 5】Lie 微分
→ 施策が環境の構造にどう影響するかを定量化

【Step 6】MC ドロップアウト（Part I 第 1 章）
→ 全ての幾何量に信頼区間を付与

【Step 7】SHAP + DML（Part I 第 2-3 章）
→ 要因分解（SHAP）と因果効果推定（DML）

【Step 8】Neural ODE + 最適制御（Part II 第 6 章）
→ 軌道予測とポントリャーギンの最大原理による最適施策計算

【Step 9】品質チェック
→ 全ステップの品質指標を統合チェック

【Step 10】レポート生成
→ 経営者向けの意思決定支援レポートを出力

19.2 Step 1：データ収集と前処理

🔬 **研究室トーク**

ユキ：「Step 1 は何をするんですか？」

リルル先生：「原材料を集める。ショートブレッドを作るのに、まずバター、砂糖、小麦粉を用意するのと同じ。データの品質が全てを決める——原則 4『材料の品質が全て』」

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

def step1_data_collection(filepath='x_company_data.csv'):
    """Step 1：データ収集と前処理
```

```
    X 社のデータ：30 指標×60 ヶ月（5 年分）×月次
    指標の例：売上高、営業利益、為替レート（USD/JPY）、
    原材料価格、競合シェア、消費者信頼感指数、GDP 成長率 等
    """
    # データ読み込み（架空データで代替）
    np.random.seed(42)
    n_months = 60  # 5 年×12 ヶ月
    n_features = 30

    # 架空データ生成（実運用では実データを使用）
    data = np.random.randn(n_months, n_features)
    # トレンドと季節性を追加（よりリアルに）
    trend = np.linspace(0, 1, n_months).reshape(-1, 1)
    seasonality = np.sin(np.arange(n_months) * 2 * np.pi / 12).reshape(-
1, 1)
    data[:, 0:5] += trend * 2  # 売上関連にトレンド
    data[:, 5:10] += seasonality * 1.5  # 需要関連に季節性

    # 欠損値の確認と補完
    n_missing = int(n_months * n_features * 0.02)  # 2%の欠損
    for _ in range(n_missing):
        i, j = np.random.randint(n_months), np.random.randint(n_features)
        data[i, j] = np.nan

    df = pd.DataFrame(data)
    n_nan = df.isna().sum().sum()
    print(f"Step 1: データ読み込み完了 ({n_months}ヶ月 × {n_features}指標)
")
    print(f"  欠損値: {n_nan}個 → 線形補間で補完")
    df = df.interpolate(method='linear').fillna(method='bfill')

    # 正規化
    scaler = StandardScaler()
    data_normalized = scaler.fit_transform(df.values)

    print(f"  正規化完了（平均=0、標準偏差=1）")
    print(f"  ✓ Step 1 完了")

    return data_normalized, scaler
```

19.3　Step 2：VAE による多様体の構成

🔬 研究室トーク

ユキ：「Step 2 は何をするんですか？」

リルル先生：「30 次元のデータを、6 次元の**曲がった空間（多様体）**に圧縮する。VAE（変分オートエンコーダ）が『30 次元の見た目のデータの裏にある、6 次元の本質的な構造』を発見する」

ユキ：「なぜ 6 次元なんですか？」

リルル先生：「X 社のデータを分析したとき、主成分分析で累積寄与率 95%に達するのが 6 成分目。つまり、30 個の指標の変動の 95%は 6 つの独立した要因で説明できる。この 6 つの要因が多様体の 6 つの座標に対応する」

```
import torch
import torch.nn as nn

class BusinessVAE(nn.Module):
    """経営環境多様体を構成する VAE

    エンコーダ：30 次元の観測データ → 6 次元の潜在表現
    デコーダ：6 次元の潜在表現 → 30 次元の復元データ

    命題 1.1 の 3 条件：
    (i) C^k 級（Tanh 活性化関数で保証）
    (ii) ヤコビ行列がフルランク
    (iii) 固有な写像（逆像が有限集合）
    """
    def __init__(self, input_dim=30, latent_dim=6):
        super().__init__()
        self.latent_dim = latent_dim

        # エンコーダ
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128), nn.Tanh(),
            nn.Linear(128, 64), nn.Tanh(),
        )
        self.mu_layer = nn.Linear(64, latent_dim)
        self.logvar_layer = nn.Linear(64, latent_dim)

        # デコーダ（命題 1.1 の 3 条件を満たすように設計）
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64), nn.Tanh(),
```

```
            nn.Linear(64, 128), nn.Tanh(),
            nn.Linear(128, input_dim),
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.mu_layer(h), self.logvar_layer(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def step2_build_manifold(data, latent_dim=6, epochs=200):
    """Step 2：VAE で多様体を構成"""
    x = torch.tensor(data, dtype=torch.float32)
    vae = BusinessVAE(input_dim=data.shape[1], latent_dim=latent_dim)
    optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)

    for epoch in range(epochs):
        recon, mu, logvar = vae(x)
        recon_loss = nn.functional.mse_loss(recon, x)
        kl_loss = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp
())
        loss = recon_loss + 0.1 * kl_loss
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # 命題1.1 の条件チェック
    z_sample = mu[0].detach().requires_grad_(True)
    output = vae.decode(z_sample)
    jacobian = torch.zeros(data.shape[1], latent_dim)
    for i in range(data.shape[1]):
        grad = torch.autograd.grad(output[i], z_sample, retain_graph=Tru
e)[0]
        jacobian[i] = grad
    min_sv = torch.linalg.svdvals(jacobian).min().item()
```

```
    print(f"Step 2: VAE 訓練完了（{epochs}エポック）")
    print(f"  再構成誤差: {recon_loss.item():.4f}")
    print(f"  最小特異値: {min_sv:.6f} {'✓' if min_sv > 1e-4 else '✗'}")
    print(f"  ✓ Step 2 完了（{latent_dim}次元の経営環境多様体を構成）")

    return vae
```

19.4 Step 3-5：計量、曲率、Lie 微分

🔬 研究室トーク

ユキ：「Step 3〜5 はまとめて説明してもらえますか？」

リルル先生：「Step 3 で多様体上の**距離の測り方**（引き戻し計量 g_{ij}）を計算し、Step 4 でその計量から**空間の曲がり具合**（スカラー曲率）を計算し、Step 5 で**施策が環境に与える影響**（Lie 微分）を計算する。この 3 つは連鎖的——Step 3 の結果を Step 4 が使い、Step 3-4 の結果を Step 5 が使う」

```
def step3_compute_metric(vae, z, d=6):
    """Step 3：引き戻し計量の計算"""
    z_t = z.requires_grad_(True)
    output = vae.decode(z_t)
    J = torch.zeros(output.shape[-1], d)
    for i in range(output.shape[-1]):
        J[i] = torch.autograd.grad(output[i], z_t, retain_graph=True)[0]
    g = J.T @ J  # 引き戻し計量 g_ij = J^T J
    return g.detach()

def step4_compute_curvature(vae, z, d=6, epsilon=1e-3):
    """Step 4：スカラー曲率の計算（数値微分による近似）"""
    g_center = step3_compute_metric(vae, z, d)

    # 数値微分でクリストッフェル記号を計算
    christoffel = torch.zeros(d, d, d)
    for k in range(d):
        z_plus = z.clone()
        z_plus[k] += epsilon
        g_plus = step3_compute_metric(vae, z_plus, d)
        z_minus = z.clone()
        z_minus[k] -= epsilon
        g_minus = step3_compute_metric(vae, z_minus, d)
        dg = (g_plus - g_minus) / (2 * epsilon)
```

```
        g_inv = torch.linalg.inv(g_center)
        for i in range(d):
            for j in range(d):
                christoffel[i, j, k] = 0.5 * sum(
                    g_inv[i, l] * (dg[l, j] + dg[j, l] - dg[j, l])  # 簡略化
                    for l in range(d))

    # スカラー曲率の近似（リッチスカラー）
    scalar_curvature = torch.trace(g_center).item()  # 簡略化した近似
    return scalar_curvature

def step5_lie_derivative(vae, z, policy_field, d=6, epsilon=1e-3):
    """Step 5：施策のLie微分の計算"""
    g = step3_compute_metric(vae, z, d)

    # 施策ベクトル場Xに沿ったLie微分 L_X g
    X = policy_field(z)
    lie_deriv = torch.zeros(d, d)
    for i in range(d):
        z_shifted = z.clone()
        z_shifted[i] += epsilon * X[i]
        g_shifted = step3_compute_metric(vae, z_shifted, d)
        lie_deriv += (g_shifted - g) / epsilon

    lie_norm = torch.norm(lie_deriv).item()
    return lie_norm
```

19.5 Step 6-7：信頼性の 4 本の柱

ユキ：「Step 6-7 は Part I の 4 本の柱ですよね」

リルル先生：「そう。Step 4-5 で計算した幾何量に対して、MC ドロップアウト（第 1 章）で信頼区間を付け、SHAP（第 2 章）で要因分解し、DML（第 3 章）で因果効果を推定する。詳細は各章を参照」

19.6 Step 8：Neural ODE + 最適制御

ユキ：「Step 8 は第 6 章の話ですよね。Neural ODE で将来を予測して、ポントリャーギンの最大原理で最適施策を計算する」

リルル先生：「その通り。5 年超の長期計画の場合は、第 15 章のシンプレクティック積分器を使う」

19.7　Step 9-10：品質チェックとレポート生成

```
def step9_quality_check(results):
    """Step 9：品質チェック（全ステップの統合チェック）"""
    checks = {
        'VAE フルランク': results['min_singular_value'] > 1e-4,
        '再構成誤差': results['recon_error'] < 0.1,
        'calibration': results['calibration_coverage'] > 0.85,
        'ゲージ不変性': results['gauge_invariant'],
        'シーフ整合性': results['sheaf_residual'] < 0.1,
    }

    n_pass = sum(checks.values())
    n_total = len(checks)

    print(f"Step 9: 品質チェック {n_pass}/{n_total} 合格")
    for name, passed in checks.items():
        print(f"  {'✓' if passed else '✗'} {name}")

    if n_pass == n_total:
        print("  ✓ 全項目合格。Step 10 に進行可能")
    elif n_pass >= n_total - 1:
        print("  ⚠ 1 項目不合格。不合格項目を確認の上、進行可能")
    else:
        print("  ✗ 2 項目以上不合格。Step 1-8 を見直してください")

    return n_pass == n_total

def step10_generate_report(results, client_name="X 社"):
    """Step 10：経営者向けレポート生成"""
    report = f"""
╔═══════════════════════════════════════════════════╗
║  ジオメトリック・インテリジェンス レポート          ║
║  クライアント：{client_name}                        ║
║  分析日：2026 年 2 月                              ║
╚═══════════════════════════════════════════════════╝

【1. 経営環境の安定性】
 現在のスカラー曲率：{results.get('curvature', 0):.4f}
 解釈：{'安定（正の曲率）' if results.get('curvature', 0) > 0 else '不安定（負の曲率）'}
 95%信頼区間：±{results.get('curvature_ci', 0):.4f}
```

```
【2. 主要リスク因子（SHAP 上位 3）】
  1 位：{results.get('shap_top1', '為替リスク')} ({results.get('shap_pct1
', 42)}%)
  2 位：{results.get('shap_top2', '原材料価格')} ({results.get('shap_pct2
', 31)}%)
  3 位：{results.get('shap_top3', '市場集中度')} ({results.get('shap_pct3
', 18)}%)

【3. 推薦施策（DML 因果効果順位）】
  最優先：{results.get('dml_top1', 'R&D 投資拡大')}
   （因果効果：{results.get('dml_effect1', '+4.2')}% ±{results.get('dml_ci1
', '1.8')}%）

【4. 品質保証】
  MC ドロップアウト：T=100 回実行 ✓
  ゲージ不変性テスト：合格 ✓
  シーフ整合性：残差 {results.get('sheaf_residual', 0.003):.4f} ✓

╚═══════════════════════════════════════════════════════════╝
"""
    print(report)
    return report
```

🔬 研究室トーク：「レポートの読み方」

リーム：「少しよろしいですか。このレポートをクライアントの X 社の経営会議でどう使うかを説明します」

リーム：「**第 1 項（安定性）**は、『今の経営環境は安定しているか不安定か』を 1 つの数値で示す。スカラー曲率が負なら『嵐の前兆』。経営者は第 11 章の臨界点マップと照らし合わせて、最も近い転換点までの距離を確認する」

リーム：「**第 2 項（リスク因子）**は、『何が不安定の原因か』を示す。SHAP 値の上位 3 因子に対して、ヘッジ策や対応策を検討する」

リーム：「**第 3 項（推薦施策）**は、『何をすべきか』を示す。DML の因果効果順位が最も重要——SHAP の順位と異なる場合（第 3 章の順位逆転の例）、DML の結果を優先する」

リーム：「**第 4 項（品質保証）**は、レポートの信頼性の根拠。ZKML 証明（第 4 章）がある場合はその検証結果も記載する」

ユキ：「料理のレシピの最後に『できあがり！ 味見をして、塩加減を調整してください』って書いてあるみたいですね」

リルル先生：「味見が品質チェック。塩加減の調整がパラメータの微調整。そして食べるのは経営者——最終判断は人間。パイプラインは**レシピと食材の品質を保証する**。何を食べるかは経営者が決める」

本章の品質チェックポイント

□ **チェックリスト（第 19 章終了時点）**

1. **Step 1〜10 が全て実行されたか？**
2. **Step 9 の品質チェックで全項目合格したか？** → 2 項目以上不合格なら前のステップに戻る
3. **レポートの内容がドメイン知識と整合するか？** → 経営者や業界専門家のレビュー
4. **旧『ジオメトリック・インテリジェンス』を読んでいない読者でも、本章だけで全手順が理解できるか？**

次章予告： 第 20 章では、ケーススタディ B の**国家安全保障環境多様体**——193 カ国のデータから 8 次元の国際情勢多様体を構成し、安全保障政策の分析に適用する——を扱います。カイトが NHK の『映像の世紀』を見た話から始まります。

第 20 章　国家安全保障環境多様体

この章で得られるもの——コストに見合うリターン

第 19 章でケーススタディ A（X 社の経営環境多様体）の実践を追体験しました。本章では、ケーススタディ B——**国家安全保障環境多様体**——の構成と分析を扱います。

193 カ国×40 指標×30 年のデータから 8 次元の国際情勢多様体を構成し、同盟政策・経済制裁・軍事バランスの分析にパイプラインを適用します。第 16 章の 3 層ガードレール（特に国際人道法チェック）がここで具体的に機能します。

20.1　カイトの日曜日——NHK『映像の世紀』を見て

🔬 **研究室トーク：「チャーチルの時代と現代のインテリジェンス」**

月曜日の研究室。カイトがいつもと違う、少し沈んだ表情で席についた。

ユキ：「カイト、どうしたの？ 元気ないね」

カイト：「……昨日の日曜日、NHK の『映像の世紀　20 世紀』の再放送を見たんだ。第一次世界大戦から冷戦まで、20 世紀の戦争と外交の映像記録。何時間も見てしまった」

カイト：「特に衝撃を受けたのが、第二次世界大戦中のインテリジェンスの話。チャーチルはエニグマ暗号の解読（ブレッチリー・パーク、アラン・チューリングのチーム）によるインテリジェンスで戦争の帰趨を左右する判断を下した。でも、そのインテリジェンスの源泉——暗号解読の事実——は厳重に秘匿された。なぜなら、ドイツ軍にエニグマが解読されていることを知られたら、暗号を変えられてしまうから」

カイト：「チャーチルは、コベントリーへの爆撃を事前に知っていたのに、住民を避難させなかったという議論がある。避難させればエニグマ解読の事実がドイツに露見する。インテリジェンスの価値と人命の価値——このトレードオフの中で判断を下さなければならなかった」

カイト：「僕らがいまやっている『ジオメトリック・インテリジェンス』は、経営者や政策決定者に**信じて行動できるインテリジェンス**を届けること。でも、チャーチルの時代のインテリジェンスは、**行動の結果が人の生死に直結していた**。僕らのパイプラインが安全保障に使われるとき、同じ種類の責任が生じるんじゃないか」

研究室が静まった。

リルル先生：「カイト、その問いは本書の倫理的核心の一つに触れている。第 21 章で詳しく議論するけれど、ここで一つだけ言っておく」

リルル先生：「チャーチルの時代のインテリジェンスと、私たちのパイプラインのインテリジェンスには、共通点と相違点がある。**共通点：**どちらも意思決定者に判断材料を提供する。判断の結果には重い責任が伴う。**相違点：**チャーチルの時代は暗号解読という**単一の情報源**に依存していた。私たちのパイプラインは**構造化された多次元の分析**を提供し、各分析に信頼区間を付け、因果効果を推定し、複数のシナリオを提示する。**意思決定者が盲目的に 1 つの結論に従うのではなく、不確実性を理解した上で判断できる**——これが Part I の 4 本の柱の意味」

リルル先生：「そして原則 5——数学は歪まない。チャーチルの時代、インテリジェンスは政治的圧力で歪められることがあった。都合の悪い情報が握り潰されたり、楽観的な解釈が優先されたりした。私たちのパイプラインでは、**スカラー曲率が負の領域は、誰が計算しても負**。数学

的真実は政治的圧力に屈しない。ただし、第 22 章で議論する通り、**どのデータを入力するか**には人間の判断が入る。そこに利益相反の余地がある」

20.2　国家安全保障環境多様体の構成

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「193 カ国の世界を 8 次元にする」**

リルル先生：「ケーススタディ B のパイプラインを設計しよう。X 社（6 次元、30 指標）を拡張して、**国家安全保障環境多様体**を構成する」

リルル先生：「データの構成は以下の通り」

=== 国家安全保障環境多様体のデータ ===

対象：193 カ国（国連加盟国全数）
期間：30 年（1995-2025）
更新頻度：月次（経済・金融）＋年次（軍事・社会）

40 指標（5 カテゴリ×8 指標）：

【カテゴリ 1：軍事力バランス】8 指標
軍事費/GDP 比、兵員数、核保有状況、
通常戦力指数、サイバー戦能力指数、
宇宙軍事能力、同盟ネットワーク密度、軍事技術水準

【カテゴリ 2：経済的相互依存】8 指標
二国間貿易額/GDP 比、直接投資ストック、
サプライチェーン依存度、金融市場相関、
制裁耐性指数、資源依存度、技術依存度、通貨準備

【カテゴリ 3：地政学的構造】8 指標
同盟関係の強度、領土紛争指数、
国境線の安定性、国際機関への参加度、
外交承認ネットワーク、条約遵守度、
地理的脆弱性指数、海洋アクセス度

【カテゴリ 4：情報環境】8 指標
メディア自由度、インターネット普及率、
偽情報拡散リスク、サイバー脆弱性指数、
情報公開度、監視体制指数、

学術交流密度、文化的影響力

【カテゴリ 5：社会的安定性】8 指標
ジニ係数（所得格差）、民族・宗教的分断度、
政治的安定指数、人口動態リスク、
都市化率、教育水準、保健医療水準、環境脆弱性

═══════════════════════════════════════
以下のカテゴリ 6 は、公開情報（OSINT）を体系的に
収集・分析した情報源であり、機密データではないが、
分析手法自体にノウハウを含むためアクセス管理の対象。
═══════════════════════════════════════

【カテゴリ 6：OSINT（公開情報インテリジェンス）】

OSINT（Open Source Intelligence）は、一般に公開されている
情報源から体系的に収集・分析して得られるインテリジェンス。
機密データではないが、大量の公開情報を組み合わせることで、
機密情報に匹敵する洞察が得られる場合がある。

(A) SNS・ソーシャルメディア解析
軍人・官僚・重要人物の SNS 投稿画像データおよび
投稿時の現在地記録（ジオタグ）データ。投稿画像から
以下の情報を画像認識 AI で自動抽出：

・制服の階級章・腕章・軍種徽章
→ 人物の所属軍種、階級、専門分野の特定
・写真内の防衛装備品に印字された識別記号・部隊番号・紋章
→ 装備品の所属部隊、配備先、型番の特定
・一緒に写る人物の制服・階級章・軍種徽章・紋章
→ 人物のネットワーク関係、会合の参加者構成、
指揮系統の推定
・背景に写る施設・地形・車両のナンバープレート
→ 撮影場所の特定、施設の用途推定
・投稿日時と位置情報の時系列分析
→ 人物の移動パターン、部隊の移動・展開の推定

指標例：
重要人物の移動パターン異常指数、
軍事施設周辺の SNS 活動量変化、
部隊識別情報の新規出現頻度（新部隊の編成を示唆）、
高官の海外訪問パターンの変化（外交活動の活性化指標）、

兵士の SNS 投稿地点のクラスタリング変化
（部隊移動・集結の早期検出）

(B) 政府調達情報・入札情報
各国政府の公開調達データベースから収集。

・防衛関連の調達・入札案件の件数、金額、品目の時系列変化
→ 軍事力増強の方向性・規模の推定
・特定の技術分野（ミサイル誘導、暗号技術、無人機、
サイバー戦、宇宙監視等）の調達額の急増
→ 新たな軍事能力の獲得意図の早期検出
・調達先企業の国籍・技術分野の変化
→ 技術的依存関係の変化、サプライチェーン再編の検出
・緊急調達案件の頻度と規模
→ 有事準備のシグナル

指標例：
防衛調達総額の前年比変化率、
技術分野別調達額の構成比変化、
緊急調達案件の発生頻度、
新規サプライヤー国の出現数

(C) アカデミズムとの共同研究プロジェクト情報
各国の政府機関・軍機関と大学・研究機関の共同研究の
公開情報。

・共同研究テーマの分析
→ 将来の軍事技術の方向性の早期検出
（例：量子暗号、極超音速兵器、自律型兵器、
宇宙デブリ除去技術、AI 軍事応用）
・共同研究の予算規模と期間の変化
→ 特定技術分野への投資の本気度の推定
・研究者の異動パターン（大学→政府機関→民間企業）
→ 技術人材の流れから政策的優先分野を推定

指標例：
軍事関連共同研究プロジェクトの新規採択数、
重点技術分野の研究予算変化率、
軍事×民間の技術デュアルユース指数

(D) 政府機関・軍機関の文民中途採用公募情報
各国の防衛省・軍・情報機関が公開する

文民（非軍人）の中途採用公募情報。

・募集職種の分析
　→ 組織が新たに獲得しようとしている能力の特定
　（例：AI エンジニア大量募集→AI 軍事応用の本格化、
　サイバーセキュリティ専門家募集→サイバー戦能力の強化、
　言語学者（特定言語）募集→特定地域へのインテリジェンス
　活動の強化）
・募集人数・給与水準の時系列変化
　→ 組織の拡大・縮小、優先分野の変化
・募集要件に記載された技術スキル・資格
　→ 必要とされている具体的な技術領域の特定
・募集拠点の地理的分布
　→ 情報活動の重点地域の推定

指標例：
　文民採用公募の総件数変化率、
　技術系職種の募集比率変化、
　特定言語能力要求の新規出現、
　給与水準の業界平均との乖離（人材獲得の切迫度）

★ OSINT の統合的価値：
上記(A)〜(D)の情報は個別には断片的だが、
組み合わせることで「ある国が特定の軍事能力の
獲得に向けて、調達を増やし(B)、研究を強化し(C)、
人材を採用し(D)、部隊を再編している(A)」という
一貫したパターンを検出できる。

このパターン検出には、層ラプラシアン（第 18 章）が
有効──4 つの OSINT サブカテゴリ間の整合性を検証し、
「調達は増えているのに人材は採用していない」
のような矛盾を検出できる。矛盾がある場合、
デセプション（意図的な偽装情報の流布）の可能性を示唆。

═══════════════════════════════════════

以下のカテゴリ 7・8・9 は、クライアント政府機関から
提供される機密インテリジェンスデータセットであり、
特別なアクセス制御の下でのみパイプラインに投入される。
═══════════════════════════════════════

【カテゴリ 7：HUMINT（ヒューミント）インテリジェンス】
　クライアント政府機関から提供。

NDA（秘密保持契約）締結＆Security Clearance 規定適用。
SC（Security Clearance）資格保有者のみアクセス可能。
SC 資格非保有者は暗号化された状態でのみ取り扱う
（準同型暗号等を適用）。

内容：クライアント政府機関にとっての懸念国および
友好国・同盟国について、内政・外交安全保障政策、
技術動向などに関するキーパーソンや重要情報を
持つ人物から入手した機密情報。

指標例（暗号化された状態で処理）：
　政権中枢の意思決定傾向指数、
　軍事指導層のリスク選好度、
　非公式外交チャネルの活性度、
　技術移転の非公開動向、
　政権安定性の内部評価、
　核・ミサイル開発の進捗推定（公開情報との乖離度）、
　サイバー攻撃能力の内部評価、
　同盟国間の非公開政策協議の方向性

【カテゴリ 8：SIGINT/ELINT インテリジェンス】
（同上のアクセス制御を適用）

SIGINT（Signals Intelligence）：
　クライアント政府機関にとっての懸念国および
　友好国・同盟国の関係機関・軍隊が発信する
　通信信号の傍受・解析から得られたインテリジェンス。
　暗号通信のトラフィック分析（内容の解読ではなく、
　通信量・頻度・パターンの変化から意図を推定）、
　外交通信の活性度変化、
　軍事通信の異常パターン検出。

ELINT（Electronic Intelligence）：
　レーダー、ミサイル誘導システム、電子戦装備等の
　電磁波放射の傍受・解析から得られたインテリジェンス。
　新型兵器システムの電磁波シグネチャ、
　防空システムの配置変更の検出、
　軍事演習の規模・性質の推定、
　衛星通信パターンの異常検出。

指標例（暗号化された状態で処理）：
　通信トラフィック異常指数（ベースラインからの乖離度）、

軍事通信の活性化指数、
新型電磁波シグネチャの検出数、
防空レーダー配置変更頻度、
衛星通信パターン変化指数、
サイバー空間の攻撃準備活動指数、
電子戦能力の推定更新差分、
海洋監視活動の強度変化

【カテゴリ 9：IMINT/GEOINT（画像・地理空間インテリジェンス）】
（同上のアクセス制御を適用）

IMINT（Imagery Intelligence）：
クライアント政府機関が運用する偵察衛星および
友好国・同盟国から共有された衛星画像データ。
光学衛星（可視光・赤外線）および SAR 衛星
（合成開口レーダー、天候・昼夜を問わず撮影可能）から
取得した画像の解析結果。

GEOINT（Geospatial Intelligence）：
衛星画像と地理情報を統合した空間分析インテリジェンス。
軍事施設の建設・拡張・撤去の変化検出（change detection）、
部隊移動パターンの時系列分析、港湾・飛行場の活動量推定。

具体的なデータ処理：
(1) 衛星画像を CNN（畳み込みニューラルネットワーク）で
自動分類・物体検出→数値化された特徴ベクトルに変換
(2) 変化検出アルゴリズムで時間的変化を定量化
(3) 数値化された特徴ベクトルを準同型暗号で暗号化し、
パイプラインの VAE に投入

指標例（暗号化された状態で処理）：
軍事施設建設活動指数（新規建設面積の変化率）、
部隊展開密度の変化（特定地域の車両・人員密度）、
港湾活動量指数（艦船の入出港頻度・規模）、
飛行場活動指数（航空機の離発着頻度・機種構成変化）、
ミサイル発射施設の準備状態指数、
核関連施設の活動変化指数、
海洋進出活動指数（人工島建設、海洋調査船の活動）、
国境地帯の軍事インフラ変化指数

★ 衛星画像と SIGINT/ELINT のクロスバリデーション：
SIGINT/ELINT で「軍事通信の活性化」が検出された地域に

衛星画像を照合し、「実際に部隊が移動しているか」を
確認する。両者が一致すれば信頼度が大幅に向上。
不一致の場合（通信はあるが物理的移動なし）は、
欺瞞作戦（デセプション）の可能性を示唆。
→ 層ラプラシアン（第 18 章）で SIGINT と衛星画像の
整合性を自動検証し、不一致を検出

潜在次元：8 次元
（カテゴリ 1-6 の公開・OSINT データで主成分分析を行い、
累積寄与率 90%に達する成分数。カテゴリ 7-9 の
機密データは、準同型暗号下で潜在空間に統合）

🔬 研究室トーク：「機密データをどうパイプラインに入れるか」

リームの表情が一段と真剣になった。

リーム：「少しよろしいですか。カテゴリ 7〜9 について、極めて重要な運用上の問題を説明します」

リーム：「私のベンチャーがクライアント政府機関から HUMINT および SIGINT/ELINT データの提供を受ける場合、**3 つのレイヤーのセキュリティ制御**を適用しています」

リーム：「**レイヤー1：人的セキュリティ（Security Clearance）。** カテゴリ 7-9 のデータに**生の状態で**アクセスできるのは、クライアント政府機関の SC（Security Clearance）資格を保有するアナリストのみです。私のベンチャーのエンジニア——カイトやリンのような研究者——は、**生データに直接アクセスすることはできません**」

カイト：「えっ、僕たちがコードを書いているのに、データを見れないんですか？」

リーム：「見れません。ここが核心。あなたたちが書いたパイプラインのコードは、**暗号化されたデータの上で動作する**ように設計されている必要があります」

リーム：「**レイヤー2：暗号技術（準同型暗号）。** カテゴリ 7-9 のデータは**準同型暗号（Homomorphic Encryption）**で暗号化された状態でパイプラインに投入されます。準同型暗号は、**データを暗号化したまま計算を実行できる**暗号方式。データを復号（元に戻す）ことなく、暗号化された状態のまま足し算や掛け算を行い、結果も暗号化されたまま出力される。復号できるのは SC 資格保有者のみ」

ユキ：「第 4 章のゼロ知識証明に似てますね。データの中身を見ないで計算の正しさを検証できる」

リーム：「関連する概念ですが、異なる技術です。ゼロ知識証明（第 4 章）は**計算が正しいことを検証する**技術。準同型暗号は**暗号化したまま計算を実行する**技術。安全保障のパイプラインでは**両方**が必要――準同型暗号で機密データを暗号化したまま計算し、ゼロ知識証明で計算の正しさをクライアント政府機関が検証する」

リーム：「**レイヤー3：物理的セキュリティ。** カテゴリ 7-9 のデータは、クライアント政府機関の指定するセキュア施設内のエアギャップ環境（インターネットに接続されていない隔離されたサーバ）でのみ処理されます。計算結果は**暗号化された状態で**通常のネットワーク経由で転送されますが、生データはセキュア施設の外に出ることはありません」

リン：「ちょっと待ってください。準同型暗号で暗号化されたデータの上で VAE や Neural ODE を動かすのは、2026 年 2 月時点でどの程度実用的ですか？」

リーム：「正直に言います。**完全準同型暗号（FHE: Fully Homomorphic Encryption）の上でニューラルネットワークを動かすのは、2026 年時点ではまだ計算コストが非常に高い**。推論（学習済みモデルにデータを入れて結果を得る）は FHE で実行可能な段階に近づいていますが、**訓練（モデルの学習）は FHE では現実的でない**」

リーム：「そのため、実際の運用では**ハイブリッド方式**を採用しています。(1) カテゴリ 1-6 の公開・OSINT データで VAE と Neural ODE の**訓練**を行う（通常の計算環境）。(2) 訓練済みモデルをセキュア施設に移送。(3) セキュア施設内で、カテゴリ 7-9 のデータを**復号した状態で**モデルに入力し、推論を実行（SC 資格保有者のみがこの工程を監督）。(4) 推論結果を暗号化して施設外に持ち出す。このハイブリッド方式は、完全な FHE よりもセキュリティレベルは下がりますが、**2026 年時点で実用的に運用可能な最良の方式**です」

リルル先生：「リーム、正直な開示をありがとう。原則 3――分かることと分からないことを分けろ。完全な FHE での運用は技術的に達成されていない。ハイブリッド方式のセキュリティ上の制約を、クライアント政府機関に正直に開示し、受容可能かどうかの判断はクライアントに委ねる」

リルル先生：「そしてもう 1 つ重要なことがある。**カテゴリ 7-9 のデータがパイプラインに投入されると、分析結果の精度は向上するが、その精度向上分は暗号化されたまま出力される**。SC 資格を持たないアナリストは『カテゴリ 7-9 のデータにより、ある地域の不確実性が低下した』ということは知るが、**なぜ低下したのか（どの HUMINT がどの推定を改**

善したのか）の詳細には一切アクセスできない。これが多層的なアクセス制御の意味」

リーム：「193 カ国のデータの入手性について補足します。カテゴリ 1（軍事力）とカテゴリ 5（社会的安定性）の一部の指標は、権威主義体制の国家では公開されていないか、信頼性が低い。MC ドロップアウト（第 1 章）で計算した不確実性マップでは、これらの国家のデータ周辺で不確実性が 0.50 を超えます」

リーム：「しかし、**カテゴリ 7-9 の HUMINT/SIGINT データが追加されると、これらの国家の不確実性が大幅に低下する**ことがあります。公開データでは不確実性 0.50 だった国が、HUMINT 情報の追加により 0.25 に低下した事例がある。**公開データの『盲点』を機密データが補完する**——これがカテゴリ 7-9 を投入する最大の価値です」

リルル先生：「原則 3——分かることと分からないことを分けろ。パイプラインは**この不確実性の低下を正直に報告する**——ただし、**なぜ低下したかの理由は SC 資格保有者以外には開示しない**」

カイト：「衛星画像（カテゴリ 8）について質問があります。衛星画像は画像データですよね。30 次元の数値指標と画像データを、同じ VAE でどうやって統合するんですか？」

リーム：「いい質問です。衛星画像は CNN（畳み込みニューラルネットワーク）で事前に処理し、**数値化された特徴ベクトル**に変換します。たとえば、ある軍事基地の衛星画像を CNN に入力すると、『建設活動指数: 0.73、車両密度: 0.45、航空機数推定: 12』のような数値ベクトルが出力される。この数値ベクトルを、カテゴリ 1-5 の数値指標と同じ形式で VAE に投入します」

リーム：「さらに重要なのが**クロスバリデーション**です。SIGINT/ELINT（カテゴリ 7）で『ある地域で軍事通信が急増している』と検出された場合、同じ地域の衛星画像（カテゴリ 8）を照合して、**実際に部隊が移動しているか**を確認します。両者が一致すれば——通信の急増＋衛星画像で部隊移動を確認——信頼度が大幅に向上する。**不一致の場合**——通信はあるが物理的移動なし——は、**欺瞞作戦（デセプション）の可能性**を示唆する。この不一致は第 18 章の層ラプラシアンで自動検出されます。SIGINT ノードと衛星画像ノードの間のシーフ整合性が崩れるからです」

リルル先生：「これは第 18 章の倉田教授の問い——複数のデータソース間の整合性検証——の安全保障における最も重要な応用例。**SIGINT、HUMINT、衛星画像の 3 つのソースを層ラプラシアンでクロスバリデー**

ションすることで、単一ソースでは見えない『欺瞞』や『情報操作』を検出できる」

20.3　ケーススタディ――東アジア地域の安全保障分析

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 研究室トーク：「曲率マップが語る安定と不安定」

リルル先生：「東アジア地域（日本、中国、韓国、北朝鮮、台湾、ASEAN10 カ国）に限定して分析した結果を示す」

=== 東アジア地域の安全保障分析結果 ===

【スカラー曲率マップ】
安定領域（曲率 > 0）：
日韓関係の経済的相互依存領域（曲率 +0.34）
ASEAN 域内の貿易ネットワーク領域（曲率 +0.28）

不安定領域（曲率 < 0）：
台湾海峡の軍事バランス領域（曲率 -0.67）
南シナ海の領土紛争領域（曲率 -0.43）
北朝鮮の核・ミサイル領域（曲率 -0.89）← 最も不安定

【SHAP 上位 3 要因（不安定化要因）】
1 位：軍事力バランスの非対称性（38%）
2 位：経済的相互依存度の低下（29%）
3 位：情報環境の分断（偽情報拡散リスク）（17%）

【SHAP 上位 3 要因（安定化要因）】
1 位：経済的相互依存度（38%）
2 位：同盟ネットワークの密度（29%）
3 位：国際機関（ASEAN、APEC 等）への参加度（15%）

【DML 因果効果推定：同盟強化政策】
平均因果効果（ATE）：不安定性スコアを 0.15 低下
95%信頼区間：[0.08, 0.22]
CATE（条件付き効果）：
既存の同盟が強い地域：効果 +0.05（限界効果逓減）
同盟が弱い地域：効果 +0.31（効果が大きい）
→ リソース配分の示唆：同盟が弱い地域に優先配分

【モース理論：臨界点の検出】

指数 1 の鞍点：3 個
CP1：台湾海峡の軍事エスカレーション閾値
不安定方向：軍事力バランスの急変
CP2：南シナ海の領土紛争の分岐点
不安定方向：一方的な現状変更
CP3：経済デカップリングの臨界点
不安定方向：サプライチェーン断絶

指数 2 の鞍点：1 個
CP4：軍事エスカレーション＋経済デカップリングの同時発生
→ 2 方向で同時に不安定。最も危険なシナリオ

リルル先生：「このレポートの読み方を整理しよう」

リルル先生：「**安定化要因の第 1 位は経済的相互依存度（38%）**。つまり、東アジアの安定の**最大の柱**は経済的なつながり。もしこの柱が崩れたら——たとえば大規模な経済デカップリングが起きたら——安定化要因の 38%が失われ、不安定性が急増する」

リルル先生：「**CP4（指数 2 の鞍点）が最も危険**。軍事エスカレーションと経済デカップリングが同時に起きるシナリオは、2 方向で同時に不安定——第 11 章のモース理論で最も警戒すべきパターン」

リルル先生：「**DML の因果効果推定**は、同盟強化政策のリソース配分に直結する。既に同盟が強い地域に追加投資しても限界効果は小さい（+0.05）。同盟が弱い地域に投資する方が効果が大きい（+0.31）。**資源の有限性を考慮した最適配分**」

20.4　国際人道法との整合性チェック

▶ ビジネス・政策の意思決定に活用したい方へ

リン：「ちょっと待ってください。安全保障の分析では、第 16 章の 3 層ガードレールの適用が特に重要です。特に第 1 層の法的ガードレールに国際人道法のチェックを組み込む必要があります」

=== ケーススタディ B 3 層ガードレールの適用 ===

【第 1 層：法的ガードレール（国際人道法）】
チェック項目：
(i) 区別の原則：推薦政策が文民への直接的害をもたらさないか
(ii) 比例性の原則：政策の効果と文民への付随的被害のバランス
(iii) 必要性の原則：目的達成に必要な最小限度の措置か

(iv) 人道の原則：不必要な苦痛を与えないか

結果：同盟強化政策 → 4 原則全て合格 ✓
　　経済制裁政策 → 比例性の原則で橙色フラグ ⚠️
　　→ 制裁対象国の文民への影響が DML で推定：
　　食料供給への影響 -12%（95%CI: [-18%, -6%]）
　　→ 人間の専門家（国際法学者）のレビューを要求

【第 2 層：倫理的ガードレール】
ステークホルダー影響分析：
　直接的受益者：同盟国の安全保障担当者
　間接的受益者：同盟国の国民
　潜在的被害者：制裁対象国の一般市民
→ 橙色フラグ：制裁対象国の市民への負の影響が大きい

【第 3 層：数学的ガードレール】
ゲージ不変性：合格 ✓（USD/EUR/JPY 全てで結論一致）
シンプレクティック保存：合格 ✓（相対誤差 0.003%）
シーフ整合性：合格 ✓（残差 0.008）

リルル先生：「経済制裁政策で橙色フラグが出た。比例性の原則——制裁の軍事的利益と文民への被害のバランス——で DML が文民への影響を推定した。食料供給への-12%の影響は無視できない。この施策は**人間の専門家のレビューなしには推薦しない**」

リルル先生：「パイプラインの役割は**数値を提供すること**。『制裁を行うべきか』の判断は人間が行う。パイプラインが提供するのは『制裁を行った場合の因果効果の推定値と、文民への影響の推定値と、信頼区間』——意思決定者が**情報に基づいた判断**を行うための材料」

20.5　インテリジェンスの変遷——チャーチルから現代へ

🔬 **研究室トーク：「数学は歪まない——だからこそ難しい」**

カイト：「先ほどの分析を見て、改めて思いました。チャーチルの時代は、エニグマ暗号解読という**一点突破のインテリジェンス**で戦争の行方が変わった。現代のインテリジェンスは、193 カ国×40 指標という**多次元のデータ**から、構造的な分析を行う。質が根本的に変わっている」

リルル先生：「そう。そしてもう 1 つ。チャーチルの時代、インテリジェンスは**少数の人間だけが**アクセスできた。エニグマ解読の事実を知っていたのは最高機密を持つ数十人だけ。現代では、データ自体は広く入

手可能——193 カ国の GDP や軍事費のデータは公開されている。**差別化要因はデータの入手ではなく、データの分析方法**——多様体の構成、曲率分析、因果推論、形式検証。これが本書のパイプラインの価値」

リーム：「少しよろしいですか。もう 1 つの変化があります。チャーチルの時代は**『正しい答えは 1 つ』**——暗号が解読できるか否か。現代のインテリジェンスは**『正しい答えは 1 つではない』**——複数のシナリオがあり、各シナリオに確率と信頼区間が付く。MC ドロップアウトの不確実性推定は、**『この分析結果にどの程度自信があるか』**を正直に伝える道具です」

リルル先生：「原則 5——数学は歪まない。スカラー曲率は政治的圧力で変えられない。だからこそ、**『どのデータを入力するか』の段階での公正性が決定的に重要**。第 21 章の利益相反の議論につながる」

本章の品質チェックポイント

□ **チェックリスト（第 20 章終了時点）**

1. **193 カ国のデータのカバレッジが確認されているか？** → データ未公開の国家は不確実性マップで明示
2. **8 次元の潜在空間が 40 指標の分散の 90%以上を捉えているか？** → 主成分分析で確認
3. **3 層ガードレールが安全保障分析に適用されているか？** → 特に国際人道法チェック
4. **DML の因果効果推定で、無交絡仮定の妥当性がドメイン専門家と確認されているか？**
5. **最終判断が人間の政策決定者に留保されているか？** → パイプラインは推薦者、人間が決定者

次章予告： 第 21 章では、リームのベンチャーが直面する**利益相反**の問題——競合企業への同時サービス、競合する政府への同時サービス——と、チャーチルの問い「インテリジェンスは誰のために生成されるのか」に正面から向き合います。

第 21 章　利益相反と倫理——チャーチルの問い

この章で得られるもの――コストに見合うリターン

本章は、本書の倫理的核心です。リームのベンチャーが直面する**利益相反**――競合企業への同時サービス、競合する政府への同時サービス――の問題に正面から向き合います。

そして、Qiita 連載記事の番外編 4 で大きな反響を呼んだ場面――**BBC の反実仮想 WWII 番組の収録で、チャーチル役の俳優がリームに投げた質問**――を完全に再現し、さらに深い議論を加えます。

21.1　BBC の番組――反実仮想第二次世界大戦

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「BBC から電話が来た」**

ある金曜日。リームが研究室に入るなり、興奮と困惑が入り混じった表情で言った。

リーム：「少しよろしいですか。先週、BBC から連絡が来ました」

ユキ：「BBC！ イギリスの放送局の？」

リーム：「はい。BBC が制作している特別番組――**『反実仮想第二次世界大戦（Counterfactual WWII）』**――のアドバイザー兼出演者として、私のベンチャーに協力依頼がありました」

カイト：「反実仮想って、歴史の『もしも』をシミュレーションするやつですか」

リーム：「そうです。番組のコンセプトは――**もし 1939 年のイギリスが、現代の AI インテリジェンスシステム（私たちの経営環境多様体パイプラインに相当するもの）を持っていたら、第二次世界大戦はどう展開していたか**――をシミュレーションする」

リーム：「番組では、1939 年 9 月のヨーロッパの地政学的状況――ドイツのポーランド侵攻、英仏の参戦決定、ソ連の立場、アメリカの中立――を 40 指標でモデル化し、8 次元の国際情勢多様体を構成します。私たちのパイプラインで実際にシミュレーションを行い、『もしチャーチルがこの AI システムのレポートを受け取っていたら、どのような判断を下しただろうか』を検討する」

リル先生：「面白い企画だね。で、何が問題？」

リーム：「……番組の収録中に、チャーチル役の俳優から質問を受けたんです。台本にない質問です。俳優さんがチャーチルの研究を深くされて

いる方で、チャーチルの思考を完全に内面化した上で、**チャーチルならこう問うだろう**という質問を投げてきました」

21.2　チャーチル役の俳優の質問――「誰のためのインテリジェンスか」

リームが収録時の音声メモを研究室で再生した。

収録スタジオ。1940 年代のダウニング街 10 番地を再現したセット。チャーチル役の俳優――60 代のイギリス人男性、王立演劇学校出身、チャーチルの伝記を 7 冊読破――が、葉巻のレプリカを片手に、リームに向き直った。

チャーチル役：（チャーチルの口調で）

「ミス・ストリーム。あなたのこの……ジオメトリック・インテリジェンスとやらは、なかなかのものだ。曲率マップでダンケルクの危険を予測し、Lie 微分でドイツ空軍の爆撃パターンの構造変化を検知し、DML で対独経済制裁の因果効果を推定する。我々の時代のブレッチリー・パークのチームが数ヶ月かけた分析を、あなたの機械は数時間で行うという」

チャーチル役：「だが、私には 1 つだけ、どうしても聞かねばならんことがある」

チャーチル役の俳優が身を乗り出した。目の光が鋭くなった。

チャーチル役：「あなたのインテリジェンスは、誰のために生成されているのか？」

リームが答えようとしたが、チャーチル役は手を上げて制した。

チャーチル役：「待ちたまえ。質問をもう少し精密にしよう。あなたの会社は、複数の国の政府機関にサービスを提供している。そうだね？」

リーム：「……はい」

チャーチル役：「では仮に――あくまで仮に――あなたが、イギリス政府とドイツ政府の**両方**にインテリジェンスを提供していたとしよう。1940 年の夏、バトル・オブ・ブリテンの最中に。イギリス政府には『ドイツ空軍の爆撃パターンにこのような構造変化が検出されました。防空体制をこう変更すべきです』というレポートを提出し、同時にドイツ政

府には『イギリスの防空体制にこのような脆弱性が検出されました。爆撃パターンをこう変更すべきです』というレポートを提出する」

チャーチル役：「**あなたの数学は歪まない**——あなた自身がそう言ったね？ スカラー曲率が負の領域は、ロンドンから見ても、ベルリンから見ても、負だ。数学は中立だ。**では、あなたはどちらの味方なのか？ あなたの数学的中立性は、道徳的中立性を意味するのか？**」

スタジオが静まった。

チャーチル役：「もう 1 つ聞こう。あなたが私に提出するレポートの内容——曲率マップ、SHAP 値、DML の因果効果——は、**あなたが選んだデータ**に基づいている。あなたが別のデータを選んでいたら、別の結論が出ていたかもしれない。**データの選択は誰がコントロールするのか？** あなたか？ クライアント政府か？ それとも、あなたの会社の株主か？」

チャーチル役：「そして最後に。あなたの AI Agent——14 体の自律的な計算主体——が、**クライアントの利益ではなく、あなたの会社の利益を最大化する方向に施策を推薦する**可能性を、あなたはどのように排除するのか？」

チャーチル役が葉巻のレプリカをテーブルに置いた。

チャーチル役：「ミス・ストリーム。これは学問的な質問ではない。**人の命がかかった質問**だ」

21.3　リームの回答——4 つの対処方針

リームが研究室で音声メモを止めた。

ユキ：「……リームさん、どう答えたんですか？」

リーム：「正直に答えました。そして、この質問が私のベンチャーの**最も根本的な設計原則**を形作ることになりました」

リームが立ち上がり、ホワイトボードに 4 つの項目を書いた。

リームの 4 つの対処方針：

対処方針 1：Chinese Wall（情報障壁）

リーム：「金融業界の Chinese Wall（情報障壁）と同じ原理です。7 社（現在は企業クライアントの数）のデータと Agent 出力は、物理的にも

論理的にも**完全に分離されたサーバ**に格納されます。クライアント A の多様体分析に使うデータが、クライアント B の分析に漏洩することはあり得ない。エンジニアのアクセス権限も、担当クライアントのサーバにのみ限定されます。1 人のエンジニアが競合する 2 社のデータに同時にアクセスすることはできません」

リーム：「チャーチルの質問に即して言えば――もし私がイギリス政府とドイツ政府の両方にサービスを提供するなら（実際にはこのような状況は引き受けませんが）、2 つのプロジェクトは完全に分離された別チームで運営され、一方のチームのメンバーは他方の存在すら知りません」

対処方針 2：ZKML（技術的保証）

リーム：「第 4 章のゼロ知識証明です。各クライアントに対して、私のベンチャーが**どのアーキテクチャの AI モデルを使い、その計算が改ざんなしに正しく実行された**ことを、暗号学的に証明します。クライアントはパラメータの値を見ることなく、計算の正しさを自分の手で検証できる」

リーム：「ただし、第 4 章のカイトの問いで明らかになった通り、**ZKML はパラメータの正当性は保証しない**。私がパラメータを事後的に操作してクライアントに有利な結論を出すことを、ZKML だけでは防げない。だから対処方針 3 が必要です」

対処方針 3：訓練ログのハッシュチェーン＋第三者監査（制度的保証）

リーム：「モデルの訓練プロセスの全ログ――各エポックの損失値、使用データのハッシュ、パラメータのスナップショット――を、**タイムスタンプ付きの暗号学的ハッシュチェーン**として記録し、独立した第三者機関（現在は大手監査法人の 1 社）に預託します。訓練後にパラメータを書き換えると、ログとの整合性が崩れます」

リーム：「さらに、年 2 回の**第三者監査**で、(i) データ選択の公正性（特定のクライアントに有利なデータの恣意的選択がないか）、(ii) モデル選択の合理性（クライアントの課題に適したモデルが選ばれているか）、(iii) Chinese Wall の実効性（情報障壁が機能しているか）を検証しています」

リーム：「チャーチルの 2 番目の質問――**『データの選択は誰がコントロールするのか』**――に対する答えは、『データの選択は私のチームが行うが、その選択の公正性は第三者監査で検証される』です」

対処方針 4：正直な開示（原則 3 の適用）

リーム：「そして最も重要な対処方針。**分かることと分からないことを、クライアントに正直に開示する**。パイプラインの限界――第 12 章の Exotic 構造の問題、第 13 章の手術理論の適用限界、第 22 章の不完全性定理――を隠さずに報告します。パイプラインが**何を保証し、何を保証しないか**を明示します」

リーム：「チャーチルの 3 番目の質問――**『AI がクライアントではなく会社の利益を最大化する方向に推薦する可能性をどう排除するか』**――に対する完全な排除は、**原理的に不可能**です。これは第 22 章の不完全性定理で議論する通り、パイプラインの原理的限界です。しかし、(1) ZKML で計算過程の正しさを保証し、(2) 訓練ログで事後改ざんを防止し、(3) 第三者監査でデータ選択の公正性を検証し、(4) 限界を正直に開示することで、**利益相反のリスクを技術的・制度的に最小化**します」

リルル先生：「リーム、4 つの対処方針は整合的だ。技術的保証（ZKML＋ハッシュチェーン）と制度的保証（Chinese Wall＋第三者監査）の組合せ。そして原則 3 による正直な開示。**完璧ではないが、誠実**」

21.4　チャーチル役の俳優の反応

リーム：「収録の話に戻ります。私が 4 つの対処方針を説明した後、チャーチル役の俳優さんが台本を離れて、個人としてコメントされました」

チャーチル役の俳優が、葉巻のレプリカを置いたまま、チャーチルの口調ではなく、自分自身の声で語り始めた。

俳優：「……ミス・ストリーム。私はチャーチルを演じるために、彼の書簡、演説、回想録を数千ページ読みました。彼がブレッチリー・パークのインテリジェンスをどう扱ったか。コベントリーの爆撃の前夜に何を考えていたか。知っているつもりでした」

俳優：「しかし、あなたの話を聞いて、初めて気づいたことがある。チャーチルの時代は、インテリジェンスを**生成する側**と**利用する側**が明確に分かれていた。ブレッチリー・パークが暗号を解読し、チャーチルがそれを使って判断する。**生成者と利用者は別の組織**」

俳優：「しかし、あなたのベンチャーは、インテリジェンスの**生成者であり、運用者であり、そして複数のクライアントへの提供者**でもある。チャーチルの時代には存在しなかった**構造的な利益相反**が、現代のインテリジェンスビジネスには内在している。あなたが説明した Chinese Wall も ZKML も第三者監査も、この構造的な問題への対処であって、構造そのものの解消ではない」

俳優：「もしチャーチルがあなたの時代に生きていたら、こう言っただろう――」

俳優が再びチャーチルの口調に戻った。

チャーチル役：「信頼は検証の反復によってのみ構築される。一度の証明で永遠の信頼を得ようとする者は、信頼に値しない。」

リーム：（研究室で）「……この言葉が、私のベンチャーの運営方針の核心になりました。**年 2 回の監査を年 4 回に増やし**、各クライアントに監査結果を全て開示するようにしました。信頼は一度の証明ではなく、**検証の反復**によってのみ構築される」

21.5　クライアント政府が国際法違反のシナリオ分析を依頼してきた場合

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「断るべきか、受けるべきか」**

リン：「ちょっと待ってください。チャーチルの問いの延長線上に、さらに深刻な問いがあります。**クライアント政府が、国際人道法に違反する可能性のあるシナリオの分析を依頼してきた場合、どうするか？**」

リン：「たとえば、ある国の政府が『この国に対する経済制裁を全面発動した場合の、相手国の社会的崩壊シナリオを分析してほしい』と依頼してきた。パイプラインはこのシミュレーションを**技術的には**実行可能。しかし、結果として推薦される施策が国際人道法の比例性原則に違反する可能性がある」

リーム：「この問題については、私のベンチャーの**契約書に明示条項**があります」

リーム：「**条項 1：国際人道法への準拠。** パイプラインの全出力は、第 16 章の 3 層ガードレールの第 1 層（法的ガードレール）を通過します。国際人道法に抵触する可能性のある施策には自動的に赤色フラグが付きます」

リーム：「**条項 2：分析と実行の分離。** パイプラインは『このシナリオを実行した場合の結果の予測（因果効果推定）』を提供しますが、『このシナリオを実行すべきだ』という推薦は行いません。シナリオ分析は **if-then 分析**（もしこうしたら、どうなるか）であり、**推薦ではない**」

リーム：「**条項 3：撤退条項。** クライアントが分析結果を利用して国際法に明確に違反する行動を取った場合、または取る意図が明らかになった場合、私のベンチャーは**契約を即時解除する権利**を留保しています」

リルル先生：「リームの 3 条項は、**分析すること**と**加担すること**を明確に区別している。地震学者が地震のシミュレーションを行うことは、地震を引き起こすことではない。同様に、経済制裁のシナリオ分析を行うことは、制裁を推薦することではない。ただし、分析結果が**どのように利用されるか**には責任が伴う」

リン：「この責任の境界線は曖昧です。研究者の『デュアルユース問題』——平和目的の研究が軍事目的に転用される問題——と同じ構造。原子力物理学者がウラン濃縮の理論を研究すること自体は中立だが、その理論が核兵器に使われたとき、研究者に責任はあるのか。明確な答えはない。しかし、**答えがないことを認識した上で、最大限の注意を払う**ことはできる」

リルル先生：「原則 5——数学は歪まない。数学自体は中立。しかし、**数学を使う人間は中立ではありえない**。リームのベンチャーは、技術的中立性（数学は誰が計算しても同じ結果を出す）を維持しつつ、制度的な歯止め（3 条項＋3 層ガードレール＋第三者監査）で倫理的責任を果たそうとしている。完璧ではない。しかし**誠実**」

21.6　原則 5 の深い意味——「数学は歪まない」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「5 つの原則の最後のピース」**

リルル先生：「本書を貫く 5 つの原則の最後——原則 5『数学は歪まない』——は、チャーチル役の俳優の質問に対する、最も根本的な回答でもある」

1. 目的を忘れるな——「信じて行動できるインテリジェンス」
2. 足すな、信頼性を上げろ——全マシマシにしない
3. 分かることと分からないことを分けろ
4. 材料の品質が全て——ショートブレッドの教訓
5. **数学は歪まない**

リルル先生：「原則 5 は 2 つの意味を持つ」

リルル先生：「**第一の意味：数学的真実は政治的圧力に屈しない。** スカラー曲率が負の領域は、どの国の政府が計算しても負。クライアントが『この数値を正にしてくれ』と頼んでも、数学は拒否する。これがインテリジェンスの客観性の基盤」

リルル先生：「**第二の意味：しかし、数学の入力（データの選択）と出力の解釈（政策判断）は、人間の領域。** 数学は『このデータを入力すれば、この出力が出る』ことを保証する。しかし『どのデータを入力するか』は人間が決め、『出力をどう解釈し、どう行動するか』も人間が決める。数学の中立性は、**人間の責任を免除しない**」

リーム：「チャーチル役の俳優さんがおっしゃった通り——**信頼は検証の反復によってのみ構築される**。原則 5 の実践は、一度の宣言ではなく、毎日の運用の中で繰り返し検証されるもの」

本章の品質チェックポイント

□ **チェックリスト（第 21 章終了時点）**

1. **Chinese Wall（情報障壁）が物理的・論理的に実装されているか？** → クライアント間のデータ分離、アクセス権限の限定
2. **ZKML 証明が該当クライアントに提供されているか？** → 計算過程の正しさの暗号学的保証
3. **訓練ログのハッシュチェーンが第三者機関に預託されているか？** → 事後改ざんの防止
4. **第三者監査が定期的に実施されているか？** → 年 4 回以上（チャーチルの教訓による強化）
5. **国際人道法準拠条項が契約書に含まれているか？** → 赤色フラグの自動付与＋撤退条項

6. **パイプラインの限界が正直に開示されているか？** → 原則 3 の適用

次章予告： Part IV の最終章、第 22 章では、パイプラインの**原理的な限界**——ゲーデルの不完全性定理の精神を幾何学的モデリングに移した「リームの不完全性定理」——を議論します。「モデル化できないもの」とは何か、そしてそれを知ることの価値とは。

第 22 章　幾何学的モデリングの不完全性定理

この章で得られるもの——コストに見合うリターン

Part IV の最終章です。パイプラインの**原理的な限界**——どれだけ精密に構築しても越えられない壁——を、ゲーデルの不完全性定理の精神に基づいて明確にします。

リームの博士論文テーマでもあるこの定理——**「十分に表現力のある多様体パイプラインは、自身の健全性を自身で完全に証明することができない」**——は、パイプラインの信頼性をかえって高めるものです。

22.1　ゲーデルの不完全性定理——「自分自身を語る限界」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「この文は嘘である」**

リルル先生：「1931 年、クルト・ゲーデル（Kurt Gödel）が証明した定理で、数学の歴史を根本から変えた」

ユキ：「名前は聞いたことあります。でも何を証明したのか……」

リルル先生：「次の文を読んで——**『この文は嘘である』**。この文が本当なら嘘だから本当ではない。嘘なら本当だから嘘ではない。真でも偽でもない。自分自身に言及することで矛盾が生じる——**自己言及のパラドックス**」

リルル先生：「ゲーデルは、この構造が数学の体系そのものに内在していることを証明した」

リルル先生：「**第一不完全性定理：** 自然数の算術を含む十分に表現力のある無矛盾な形式体系には、**その体系の中では証明も反証もできない命題**が必ず存在する」

リルル先生：「**第二不完全性定理：**そのような体系は、**自身の無矛盾性を自身で証明することができない**」

リルル先生：「たとえ話で言えば――法律の体系をどれだけ精密に整備しても、全ての紛争を法律だけで解決することはできない。想定外の事態は必ず発生し、**法律の外の判断**（裁判官の裁量、新しい立法）が必要になる。これは法律の欠陥ではなく、十分に豊かな体系の**原理的な性質**」

22.2　リームの不完全性定理――多様体パイプラインの原理的限界

🔬 **研究室トーク：「リームの博士論文」**

リームが立ち上がった。

リーム：「私の博士論文のテーマです。Qiita 連載の第 12 話で、リンに問いを投げられたのがきっかけでした」

リン：「当時の問い――**『このパイプラインで原理的にモデル化できないものは何か？ その限界を形式的に記述できるか？』**」

リーム：「その問いに対する私の答えが、以下の命題です」

命題 22.1（リームの不完全性定理、非形式的記述）. 十分に表現力のある多様体パイプライン――VAE（変分オートエンコーダ）で多様体を構成し、Neural ODE で軌道予測を行い、ポントリャーギンの最大原理で最適制御を計算し、形式検証で安全性を保証するシステム――は、**自身の健全性を自身で完全に証明することができない。**

ユキ：「どういう意味ですか？」

リーム：「パイプラインの品質チェック（第 19 章 Step 9）は、**パイプラインの内部で**実行される。スカラー曲率、SHAP 値、シーフ整合性……全てパイプライン自身が計算する。しかし、**パイプライン自身が『自分の出力は全て正しい』ことを完全に証明することは、原理的に不可能**。なぜなら、パイプラインが自分自身を検証しようとすると、ゲーデル的な自己言及が発生するから」

リーム：「具体的には――パイプラインが品質チェックを実行して『全て正常 ✓』と報告する。しかし、**品質チェック自体が正しいかどうか**

は、品質チェックではチェックできない。品質チェックの品質チェックの品質チェック……無限後退が発生する」

リルル先生：「だから、Part V の形式検証（Coq、Agda、Lean 4、Z3）が必要。形式検証は**パイプラインの外部**から安全性を保証する。しかし、形式検証ツール自身の健全性は、さらに外部の理論で保証する必要がある。どこかで『信頼の基盤』を**仮定として受け入れる**しかない」

22.3　3 つの「モデル化できないもの」

▶ ビジネス・政策の意思決定に活用したい方へ

リーム：「リームの不完全性定理から導かれる、パイプラインが**原理的にモデル化できない** 3 つのものを説明します」

(i) ブラックスワン（真に新しいイベント）

リーム：「ナシーム・ニコラス・タレブ（Nassim Nicholas Taleb）が 2007 年の著書で提唱した概念。『黒い白鳥（ブラックスワン）』――ヨーロッパ人は白鳥は白いと信じていたが、オーストラリアで黒い白鳥が発見された。**過去のデータに一度も出現していないイベント**は、VAE の訓練データにも含まれていないため、多様体上に表現されない」

リーム：「COVID-19 パンデミック（2020 年）は、パンデミック自体は過去にも存在したが、**グローバルサプライチェーンの同時崩壊**という形態は過去のデータになかった。リーマンショック（2008 年）も、住宅ローンの証券化のカスケード的崩壊は過去に例がなかった。パイプラインはこれらを**事後に**検出できる（第 12 章の Wasserstein 距離の急増として）が、**事前に**予測することは原理的にできない」

リルル先生：「第 12 章で議論した通り、Wasserstein 距離の上昇トレンドが『何かが変わり始めている』兆候を捉える可能性はある。しかし、**何が起きるか**を予測することと、**何かが起きそうだ**と警告することは別の問題。パイプラインにできるのは後者――原則 3 で正直に報告する」

(ii) 自己参照フィードバック

リーム：「パイプラインの出力が経営環境自体に影響を与え、その変化がパイプラインの入力に戻ってくる。株価予測 AI が『株価が上がる』と予測→投資家がそれを信じて買う→本当に株価が上がる――**予言の自己成就**（self-fulfilling prophecy）」

リーム：「あるいは逆に——パイプラインが『不安定領域に入りつつある』と警告→経営者が対策を打つ→不安定領域を回避する→**パイプラインの予測が（対策のおかげで）外れる——予言の自己否定**（self-defeating prophecy）」

リーム：「どちらの場合も、パイプラインは**自分自身の影響を含めた環境**をモデル化する必要があるが、自分自身を含めたモデルは自己言及——ゲーデルの不完全性——に陥る」

リン：「ちょっと補足。自己参照フィードバックは、ゲーム理論の**メカニズムデザイン**（制度設計）の文脈では部分的に扱える。パイプラインの出力がプレイヤーの行動を変えることを**織り込んだ**均衡概念（Stackelberg 均衡、ベイジアンゲームの均衡等）を使えば、一次的な自己参照は処理可能。しかし、高次の自己参照（自己参照の自己参照の……）は依然として困難」

（iii）価値判断

リーム：「パイプラインは『この施策を打つとリスクが X%低下し、リターンが Y%増加する』と報告できる。しかし、**『X%のリスク低下と Y%のリターン増加のどちらを優先すべきか』**は、価値判断——人間の主観的な選好——であり、数学で導出できない」

リーム：「第 20 章の安全保障の文脈では特に深刻。『軍事的利益と文民への被害のバランス』（比例性の原則）の判断は、DML で因果効果を推定できても、**どのバランスが『正しい』か**は数学では決められない。これは人間——経営者、政策決定者、そして究極的には社会——が決めること」

リルル先生：「原則 5——数学は歪まない。数学は**事実**を提供する。『リスクは X%、リターンは Y%』は事実。しかし**『X%のリスクは受け入れるべきか』は価値判断**であり、数学の守備範囲外」

22.4　不完全性を知ることの価値

🔬 **研究室トーク：「知らないことを知っている」**

ユキ：「3 つもモデル化できないものがあるなんて……パイプライン、大丈夫なんですか？」

リルル先生：「ユキ、逆。**モデル化できないものが何かを明確に知っている**ことが、パイプラインの最大の強み。知らないことすら知らない状

態——すなわち『うちの AI は完璧です、何でも予測できます』と主張する状態——の方がはるかに危険」

リルル先生：「ソクラテスの言葉——『私が知っているのは、自分が何も知らないということだけだ』。リームの不完全性定理は、パイプラインのソクラテス的知恵——**分かることと分からないことの境界を数学的に確定する**」

ユキ：「……分からないことが分かるのも、すごいことなんですね」

リルル先生：「ユキ、それは知恵の始まりだ。原則 3 の究極形——**分かることと分からないことを分けろ**」

リーム：「この不完全性定理を HoTT（ホモトピー型理論、第 30 章で展望）で形式化することが、私の博士論文の最終目標です。しかし 2026 年 2 月時点では、形式化は研究の展望段階であり、完成していないことを正直に申し上げます」

22.5　Part IV 全体のまとめ

Part IV：応用と倫理

第 19 章：経営環境多様体の構成（実践編）
- → Step 1〜10 の完全な実行手順
- → ユキの「何をしているの？」に全て答えた

第 20 章：国家安全保障環境多様体
- → 193 カ国×9 カテゴリ（公開 5 + OSINT + 機密 3）
- → 東アジアのケーススタディ
- → 国際人道法の 3 層ガードレール適用
- → カイトの『映像の世紀』

第 21 章：利益相反と倫理
- → BBC 反実仮想 WWII、チャーチル役の俳優の 3 つの質問
- → リームの 4 つの対処方針
- → 「信頼は検証の反復によってのみ構築される」
- → 原則 5「数学は歪まない」の深い意味

第 22 章：不完全性定理（本章）
- → リームの命題：パイプラインは自身の健全性を自身で完全に証明できない
- → 3 つのモデル化できないもの：ブラックスワン、自己参照、価値判断

→ 不完全性を知ることの価値

★ Part IV の核心メッセージ：
パイプラインには原理的な限界がある。
その限界を知り、正直に報告し、
人間の判断で補完する——これが原則 3 の実践。

本章の品質チェックポイント——Part IV 全体の最終チェック

□ **Part IV 全体のチェックリスト**

1. **ケーススタディ A（X 社）の Step 1〜10 が全て実行されたか？** → 第 19 章
2. **ケーススタディ B（安全保障）の 3 層ガードレールが適用されているか？** → 第 20 章
3. **利益相反の 4 つの対処方針が実装されているか？** → 第 21 章
4. **3 つのモデル化できないもの（ブラックスワン、自己参照、価値判断）が経営者に開示されているか？** → 第 22 章
5. **最終判断が人間に留保されているか？** → 全章共通

Part V への橋渡し： Part IV で原理的限界を確認しました。Part V では、この限界の中で**可能な限りの安全性を数学的に証明する**——形式検証。Coq、Agda、Lean 4、Z3 という 4 つの道具で、パイプラインの安全性を**コンパイルが通る＝安全**のレベルまで引き上げます。そして、第 23 章ではその前に、情報の対称性と非対称性——もし敵対者も同等のシステムを持っていたら——という根本問題を議論します。

第 23 章　情報の対称性と非対称性——もし敵も同じシステムを持っていたら

この章で得られるもの——コストに見合うリターン

第 20 章の BBC 反実仮想 WWII 番組と第 21 章のチャーチルの問いは、暗黙に**情報の非対称性**を前提としていました——イギリスだけが AI インテリジェンスシステムを持ち、ドイツは持たない。

しかし現実の 2026 年において、AI インテリジェンスの技術は**多くの国家が同時に開発**しています。もし敵対国もリームのベンチャーと同等かそれ以上のシステムを運用していたら？ 本章では、**情報の対称性（両者が同等のシステムを持つ場合）**と**非対称性（一方だけが持つ場合）**で、

ゲーム理論・オペレーションズリサーチ・創発理論の帰結がどう変わるかを徹底的に分析します。

これは理論的興味だけの問題ではなく、**リームのベンチャーが直面しうる実務的な問題**——競合するインテリジェンス企業の存在——にも直結します。

23.1　BBC 番組の「もう 1 つのシナリオ」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「ドイツも同じシステムを持っていたら」**

第 21 章の BBC 番組の収録後、リームは研究室に戻って報告した。そのとき、カイトが決定的な質問を投げた。

カイト：「リームさん、BBC 番組では『イギリスだけが AI インテリジェンスを持っていたら』というシナリオですよね。でも現実の戦争では、**双方が同等の技術を持つ状況**の方が普通じゃないですか？ エニグマ暗号だって、ドイツはイギリスの暗号（TYPEX）を解読しようとしていた。もし 1940 年のドイツも、リームのベンチャーと**同等かそれ以上**の国家安全保障環境多様体のデジタルツイン空間と、その上で最適解を探索する AI Agent モデルを運用済みだったら、**何が変わるんですか？**」

研究室が静まった。リルル先生がコーヒーカップを静かに置いた。

リルル先生：「カイト、それは本書の中で最も深い問いの一つだ。順番に分析しよう。この問題には少なくとも 7 つの理論的フレームワークが関わる」

23.2　ゲーム理論——情報の対称性が均衡をどう変えるか

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「まずゲーム理論から。第 2 章で導入したゲーム理論の均衡概念を、ここでは**国家間の戦略的相互作用**に適用する」

シナリオ A：情報の非対称性（イギリスだけが AI を持つ）

リルル先生：「イギリスが AI インテリジェンスシステムを持ち、ドイツが持たない。ゲーム理論で言えば、これは**不完備情報ゲーム**——一方のプレイヤーが他方のプレイヤーの戦略空間を正確に知っているが、逆は成り立たない」

リルル先生：「イギリス（AI 保有）は、ドイツの多様体上の経営環境（軍事力配置、産業構造、資源配分）の曲率マップを計算し、**ドイツの脆弱点**（スカラー曲率が負の領域）を特定できる。一方、ドイツはイギリスの脆弱点を**システマティックに**特定できない。情報の非対称性がイギリスに決定的な優位をもたらす。これがチャーチルのエニグマ解読の現代版」

リルル先生：「ゲーム理論的には、**シュタッケルベルグ均衡**（Stackelberg equilibrium）に近い。AI 保有側が先手（リーダー）として行動し、非保有側がフォロワーとして反応する。リーダーは相手の反応関数を知っているため、**より有利な均衡点**を選択できる」

シナリオ B：情報の対称性（両方が AI を持つ）

リルル先生：「では、ドイツも同等のシステムを持っていた場合。ゲーム理論では**完全情報ゲーム**（または完備情報ゲーム）に近づく。両者が互いの脆弱点を同精度で把握している」

リルル先生：「この場合の均衡は**ナッシュ均衡——**両者が相手の戦略を知った上で、自分の戦略を変えるインセンティブがない状態。しかし、ナッシュ均衡は**複数存在する**ことが一般的であり、どの均衡が選ばれるかは理論だけでは決まらない」

リン：「ちょっと待ってください。ここに核心的な違いがあります。情報が対称な場合、**両者とも相手の脆弱点を攻撃できる**。すると、**防御のコストが双方で急増する**。非対称な場合は AI 保有側だけが攻撃の精度を高められるが、対称な場合は双方が防御を強化せざるを得ない。結果として——」

情報の非対称性（一方だけが AI を持つ）：
- → AI 保有側の攻撃精度 >> 非保有側の防御精度
- → AI 保有側に決定的な戦略的優位
- → シュタッケルベルグ均衡：リーダーの利得が大きい

情報の対称性（両方が AI を持つ）：
- → 攻撃精度 ≈ 防御精度（双方とも高精度）
- → 戦略的優位は消失。代わりに「軍拡競争」的ダイナミクス
- → ナッシュ均衡：双方が防御コストを最大化する均衡
- → 囚人のジレンマの構造が出現する可能性

23.3　囚人のジレンマと AI 軍拡――「双方が合理的に行動して、双方が損をする」

🔬 **研究室トーク：「AI インテリジェンスの囚人のジレンマ」**

リルル先生：「情報対称の場合に出現する**囚人のジレンマ**を具体化しよう」

リルル先生：「2 つの国が、AI インテリジェンスシステムへの投資を選択する。各国には 2 つの選択肢――**投資する**（高コストだが AI 能力を獲得）と**投資しない**（コスト節約だが AI 能力なし）」

AI インテリジェンスの囚人のジレンマ（利得行列）：

		ドイツ：投資する	ドイツ：投資しない
イギリス	投資する	(-3, -3)	(+5, -5)
イギリス	投資しない	(-5, +5)	(0, 0)

(-3, -3)：双方が投資。双方が AI を持つが、
　　互いの優位は相殺。コストだけが残る。
(+5, -5)：一方だけが投資。AI 保有側が圧倒的優位。
(0, 0)：双方が投資しない。現状維持。

リルル先生：「各国にとって、相手が何をしようと**投資する**方が合理的（**支配戦略**）。相手が投資するなら自分も投資しないと-5。相手が投資しないなら自分が投資すれば+5。しかし双方が合理的に投資すると(-3, -3)――**双方が損をする**。これが囚人のジレンマの構造」

カイト：「冷戦時代の核軍拡競争と同じ構造ですね」

リルル先生：「その通り。核兵器を **MAD（相互確証破壊）**が抑止したように、AI インテリジェンスの対称性も**相互脆弱性の認識**による抑止に繋がりうる。『相手も自分の脆弱点を知っている』という認識が、無謀な攻撃を抑止する」

リン：「ちょっと待ってください。核兵器と AI インテリジェンスには決定的な違いがあります。核兵器の使用は**不可逆で観測可能**――使ったことが全世界に即座に分かる。AI インテリジェンスの使用は**可逆的で観測困難**――経済制裁や外交的圧力が AI 分析に基づいていたかどうかは外部から判別できない。**抑止が機能するための条件**（相手の行動が観測可能であること）が満たされにくい」

リルル先生：「リンの指摘は鋭い。ゲーム理論の**繰り返しゲーム**の理論では、**行動の観測可能性**が協力均衡（双方が投資を控える）の成立条

件。観測困難な場合、協力均衡は崩壊し、非協力均衡（双方が投資する）に固定されやすい」

23.4　オペレーションズリサーチ——最適戦略が相手の存在で変わる

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「オペレーションズリサーチ（OR）の観点からも分析しよう。OR は第二次世界大戦中にイギリスで発展した分野——まさに対独戦の最適戦略を計算するために」

リルル先生：「OR では、最適戦略は**環境が静的か動的か**で大きく変わる。敵が AI を持たない場合、環境は比較的静的——敵の行動は過去のパターンから予測可能。しかし敵が AI を持つ場合、環境は**適応的**——敵が自分の行動に応じて戦略を最適化してくる」

敵が AI を持たない場合（静的環境）：
　自分の最適戦略 = argmax_u J(z, u)
　→ 古典的な最適制御問題（ポントリャーギンの最大原理）
　→ 第 6 章のフレームワークで十分

敵が AI を持つ場合（適応的環境）：
　自分の最適戦略 = argmax_u J(z, u, v*(u))
　ここで v*(u) = 敵の最適反応（自分の戦略 u に対する最適応答）
　→ 微分ゲーム（differential game）の問題
　→ ポントリャーギンの最大原理の拡張が必要
　→ ハミルトニアンが 2 つ（自分の H_1 と敵の H_2）
　→ ナッシュ均衡の微分方程式版（結合ハミルトン系）

カイト：「つまり、敵が AI を持つかどうかで、**ポントリャーギンの最大原理の適用方法が根本的に変わる**ということですか」

リルル先生：「そう。単一のハミルトニアン $H(z, p, u)$ を最大化する問題から、**2 つのハミルトニアンの結合系**——$H_1(z, p_1, u, v)$ と $H_2(z, p_2, u, v)$ の同時最適化——に変わる。これは**微分ゲーム理論**（Isaacs, 1965; Başar & Olsder, 1999）の範疇」

```
def differential_game_hamiltonian(z, p1, p2, u, v,
                                  dynamics, cost1, cost2):
    """微分ゲームの結合ハミルトニアン

    2 つのプレイヤー（国家）が同時に最適制御を行う。
    プレイヤー1 は u を選択、プレイヤー2 は v を選択。
```

```
    各プレイヤーは自分のハミルトニアンを最大化する。

    ナッシュ均衡条件：
      u* = argmax_u H1(z, p1, u, v*)
      v* = argmax_v H2(z, p2, u*, v)

    これは結合された 2 点境界値問題となる。
    """
    dz = dynamics(z, u, v)  # 共有されるダイナミクス
    H1 = p1 @ dz - cost1(z, u, v)  # プレイヤー1 のハミルトニアン
    H2 = p2 @ dz - cost2(z, u, v)  # プレイヤー2 のハミルトニアン
    return H1, H2
```

23.5　創発理論——対称的 AI システム間の予測不能な相互作用

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「2 つの群知能が衝突するとき」**

リルル先生：「第 8 章で、14 体の AI Agent の間の創発現象——個々の設計から予測できない全体の振る舞い——を分析した。今度は、**2 つの独立した AI システム**（イギリスの 14 体とドイツの 14 体）が**同じ多様体上で同時に活動する**場合の創発を考える」

リルル先生：「第 8 章の Lie 括弧 $[X_i, X_j]$ は、**同一システム内**の 2 体の Agent の非可換性を測った。今度は、**異なるシステム間**の Lie 括弧——イギリスの Agent X_i^{UK} とドイツの Agent X_j^{DE} の間の $[X_i^{\mathrm{UK}}, X_j^{\mathrm{DE}}]$ ——を考える」

リルル先生：「この**システム間 Lie 括弧**が大きいほど、2 つの AI システムの行動が強く干渉し合い、**どちらのシステムも予測していなかった全体の振る舞い——敵対的創発（adversarial emergence）**——が発生する」

リーム：「少しよろしいですか。この問題は私のベンチャーの実務にも直結します。7 社のクライアントのうち、**同じ市場で競合する 2 社**にサービスを提供している場合、2 社の Agent が同じ経営環境多様体上で（Chinese Wall で分離されてはいるが）**間接的に干渉する**可能性がある。一方の Agent が最適と判断した施策が、他方の Agent の環境を変化させる」

リルル先生：「リームの指摘は重要。BBC シナリオを一般化すると——」

敵対的創発の 3 つのパターン：

パターン 1：相殺（cancellation）
イギリス AI が「攻撃」推薦 ↔ ドイツ AI が「防御」推薦
→ 両者の施策が打ち消し合い、多様体の状態がほとんど動かない
→ エネルギーとリソースの浪費。膠着状態

パターン 2：増幅（amplification）
イギリス AI が「経済制裁強化」推薦 → ドイツ AI が「報復措置」推薦
→ 一方の行動が他方の反応を誘発する正のフィードバック
→ エスカレーション。第 11 章のモース理論の臨界点に急接近

パターン 3：カオス的振る舞い（chaotic oscillation）
双方の AI が高速に戦略を更新し合う
→ 戦略の振動が不規則に増幅
→ いずれの AI も予測していなかった第三の状態に遷移
→ これが「敵対的創発」の最も危険な形態

> **リルル先生：**「パターン 3——カオス的振る舞い——は、2026 年の AI 研究で**マルチエージェント強化学習**（MARL）の文脈で活発に研究されている現象。OpenAI Five（Dota 2 のチーム対戦 AI, 2019）や DeepMind AlphaStar（StarCraft II, 2019）で、**自己対戦（self-play）**を通じて、設計者が予測しなかった戦略が創発することが報告されている」
>
> **リルル先生：**「Lanctot et al.（DeepMind, 2017, *"A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning"*）は、マルチエージェント環境でのナッシュ均衡の計算が NP 困難であることを示し、**Policy Space Response Oracles（PSRO）**——自己対戦で均衡を近似的に探索するアルゴリズム——を提案した。これはまさに、対称的 AI システム間の均衡を計算する道具」

23.6　メタゲーム——「敵の AI の能力を知らない場合」

▶ ビジネス・政策の意思決定に活用したい方へ

> 🔬 **研究室トーク：「敵の AI がどれだけ賢いか分からない」**
>
> **リン：**「ちょっと待ってください。ここまでの議論は『双方が同等の AI を持つ』（完全対称）を仮定しています。しかし現実には、**敵の AI の能力が分からない**。ドイツのシステムが自分のシステムより優れているかもしれないし、劣っているかもしれない。能力の推定自体に不確実性がある」

リルル先生：「リンの指摘は、ゲーム理論では**ベイジアンゲーム**（Harsanyi, 1967-68, ノーベル経済学賞）として定式化される。各プレイヤーは相手の**タイプ**（AI の能力レベル）について確率的な信念を持ち、その信念に基づいて行動する」

メタゲームの構造：

プレイヤー1（イギリス）の不確実性：
ドイツの AI の能力 = {なし, 低, 中, 高, 超高}
各能力レベルの事前確率を推定する必要がある

推定方法（OSINT 活用、第 20 章カテゴリ 6）：
(A) ドイツの AI 関連調達情報 → 投資規模の推定
(B) ドイツ政府×大学の共同研究テーマ → 技術水準の推定
(C) ドイツ軍の AI 人材採用公募 → 人材レベルの推定
(D) ドイツの AI 関連論文の質と量 → 学術的能力の推定
→ これらの断片情報を統合して、能力レベルの事前確率を更新（ベイズ更新）

リルル先生：「さらに深いレベルの問題がある。**敵も自分の AI 能力を推定しようとしている**。つまり、イギリスは『ドイツの AI 能力はどれくらいか』を推定し、同時にドイツは『イギリスの AI 能力はどれくらいか』を推定している。そしてイギリスは『ドイツが自分の AI 能力をどう推定しているか』も推定しなければならない。**信念の階層——**これはゲーム理論の**共通知識（common knowledge）**と**高次信念（higher-order beliefs）**の問題」

カイト：「信念の信念の信念……これもゲーデル的な無限後退じゃないですか？」

リルル先生：「鋭い。実用的には、**有限の階層で打ち切る**。Camerer et al.（2004）の **Cognitive Hierarchy 理論**（認知的階層理論）では、レベル 0 のプレイヤー（相手の戦略を考えない）、レベル 1 のプレイヤー（相手がレベル 0 だと仮定して最適反応）、レベル 2（相手がレベル 1 だと仮定して最適反応）……と階層化する。実験経済学のデータでは、**人間は平均してレベル 1.5 程度**。AI システムは理論上任意のレベルに設定できるが、**計算コストが各レベルで指数的に増大**する」

23.7　多様体の共有と分離——デジタルツインの地政学

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「同じ地球を別々の地図で見る」**

リルル先生：「もう 1 つ重要な問題がある。イギリスの AI システムとドイツの AI システムは、**同じ多様体**の上で動いているのか、**異なる多様体**の上で動いているのか」

ユキ：「えっ、経営環境は 1 つじゃないんですか？」

リルル先生：「**現実の世界は 1 つ**。しかし、各国の VAE が構成する多様体は、**訓練データが異なるため、異なる多様体になりうる**。イギリスはイギリスの視点のデータ（MI6 の情報、Five Eyes の共有情報等）で多様体を構成し、ドイツはドイツの視点のデータ（Abwehr（軍事情報部）の情報等）で多様体を構成する。同じ現実を見ていても、**データの偏りが異なる多様体を生む**」

リルル先生：「第 12 章の Exotic 多様体の議論を思い出して。同じ位相（現実の構造）でも、異なる微分構造（多様体上の滑らかさ）がありうる。2 つの AI システムが**位相的には同じだが微分構造が異なる多様体**の上で動いている場合、同じ現実について異なる曲率を計算し、異なる臨界点を検出し、異なる最適施策を推薦する」

リン：「この問題をシーフ理論（第 18 章）で扱えます。2 つの多様体を**層ラプラシアンで接続**し、**2 つの多様体の間のデータの整合性**を検証する。整合していれば、2 つの多様体は同じ現実を異なる視点から見ていることになる。不整合であれば、どちらかのデータに偏りまたは誤りがある」

リルル先生：「ただし、敵の多様体のデータには直接アクセスできない。OSINT データ（第 20 章カテゴリ 6）から**間接的に推定**するしかない。推定の精度には限界がある——これも第 22 章の不完全性定理の帰結」

23.8　1940 年のシナリオ再分析——対称情報下のバトル・オブ・ブリテン

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「BBC 番組の第 2 シーズン」**

リーム：「BBC の番組スタッフに、この対称情報シナリオを提案しました。第 2 シーズンとして制作を検討中です。概要をお伝えします」

リーム：「第 2 シーズンのシナリオ**——もしイギリスとドイツの両方が、リームのシステムと同等の AI インテリジェンスを持っていたら、バトル・オブ・ブリテン（1940 年 7-10 月）はどう展開したか**」

=== 対称情報下のバトル・オブ・ブリテン シミュレーション ===

【セットアップ】
  イギリス AI：8 次元の安全保障環境多様体
    データ：RAF（イギリス空軍）の戦力、防空レーダー網、
    産業生産能力、国民の士気、同盟国からの支援
    AI Agent：14 体（防空、情報、産業、外交、...）

  ドイツ AI：8 次元の安全保障環境多様体（異なるデータから構成）
    データ：Luftwaffe（ドイツ空軍）の戦力、爆撃目標リスト、
    U ボート作戦、東部戦線の兵力配分、占領地の資源
    AI Agent：14 体（空軍、海軍、情報、兵站、...）

【微分ゲームの結合ハミルトニアン】
  H_UK(z, p_UK, u_UK, v_DE) = p_UK^T f(z, u_UK, v_DE) - L_UK(z, u_UK, v_DE)
  H_DE(z, p_DE, u_UK, v_DE) = p_DE^T f(z, u_UK, v_DE) - L_DE(z, u_UK, v_DE)

  ナッシュ均衡条件：
    u_UK* = argmax H_UK(z, p_UK, u_UK*, v_DE*)
    v_DE* = argmax H_DE(z, p_DE, u_UK*, v_DE*)

【シミュレーション結果（3 つのシナリオ）】

シナリオ 1：双方とも防御重視均衡
  イギリス AI 推薦：防空網の最適配置、レーダー網の冗長化
  ドイツ AI 推薦：爆撃目標の分散、対空砲の強化
  結果：消耗戦の長期化。双方の損失が増大。
  決着なし。膠着状態が 1941 年まで続く
  → パターン 1（相殺）

シナリオ 2：一方がエスカレーションを選択
  ドイツ AI が「ロンドン爆撃への切替」を推薦
  （戦闘機工場への集中攻撃から、都市への無差別爆撃へ）
  → イギリス AI が即座に「これはドイツの戦略転換であり、
    曲率マップ上で指数 1 の鞍点を通過しようとしている」と検知

→ イギリス AI が防御リソースの最適再配置を推薦
→ ドイツのエスカレーションが早期に無効化される
→ パターン 2（増幅→早期抑止）

シナリオ 3：敵対的創発
双方の AI が高頻度で戦略を更新
→ イギリス AI が爆撃パターンの変化を検知→防御を変更
→ ドイツ AI がイギリスの防御変更を検知→爆撃パターンを再変更
→ 振動が加速し、双方の AI が予測していなかった
「第三の状態」に遷移
→ この場合：ドイツが「バトル・オブ・ブリテンを中止し、
東部戦線に全資源を集中」するシナリオが
ナッシュ均衡として浮上
→ パターン 3（カオス的振る舞い→予測外の均衡に収束）

リルル先生：「シナリオ 3 が最も興味深い。**双方が AI を持つことで、戦争の形態自体が変わる**可能性がある。AI が高速に戦略を更新し合うことで、人間の意思決定者が想定していなかった選択肢――バトル・オブ・ブリテンの中止と東部戦線への集中――が均衡として浮上する。**これは創発の一形態**――個々の AI の設計には含まれていない戦略が、2 つの AI の相互作用から自発的に出現する」

カイト：「……これ、今の世界でも起きうるんですよね。2026 年の米中関係とか、NATO 対ロシアとか、双方が AI 分析に基づいて高速に政策を更新し合ったら、誰も予測していなかった第三の均衡に収束する可能性がある」

リルル先生：「だからこそ、第 10 章の 3 レベル監視体系と第 16 章の 3 層ガードレールが決定的に重要。AI の推薦を自動実行するのではなく、**人間が最終判断する**仕組みが、敵対的創発のカオスを防ぐ最後の砦」

23.9　リームのベンチャーへの示唆――競合インテリジェンス企業の存在

リーム：「最後に、この議論を私のベンチャーの実務に戻します。国家間の対称情報の問題は、**民間のインテリジェンスビジネス**にもそのまま当てはまります」

リーム：「もし私の競合企業――リームのベンチャーと同等の技術を持つ別のインテリジェンス企業――が、私のクライアントのライバル企業にサービスを提供していたら、**同じ市場環境を 2 つのパイプラインが同**

時に分析し、互いの推薦が互いの環境に影響を与える。これは本章の対称情報問題そのもの」

リーム：「対処方針は 3 つ」

1. **MC ドロップアウトの信頼区間を広めに設定する。** 競合の AI システムの存在により、環境の変動性が増大する。信頼区間を通常の 1.5 倍に拡大し、「実際の結果がこの範囲から外れる可能性がある」ことを明示
2. **Wasserstein 距離の監視閾値を厳しくする。** 競合 AI の行動による分布シフトが通常時よりも大きくなる可能性。第 12 章の閾値を引き下げて、小さな変化でも早期に検出
3. **微分ゲームの視点を導入する。** ポントリャーギンの最大原理を、微分ゲームの結合ハミルトニアンに拡張し、**競合 AI の最適反応を織り込んだ**施策推薦を行う

本章の品質チェックポイント

□ **チェックリスト（第 23 章終了時点）**

1. **敵対者（競合者）の AI 能力レベルの推定が行われているか？** → OSINT データ（第 20 章カテゴリ 6）で推定
2. **微分ゲームの結合ハミルトニアンが実装されているか？** → 競合 AI の存在を仮定する場合に必須
3. **敵対的創発の 3 パターン（相殺、増幅、カオス）が監視されているか？** → システム間 Lie 括弧で検出
4. **人間の最終判断が留保されているか？** → 敵対的創発のカオスを防ぐ最後の砦

次章予告： 第 24 章から Part V（形式検証と安全性保証）に入ります。「なぜ Python のテストだけでは不十分か」——ÉTALE-AI の暴走事例を完全に詳述し、形式検証の必要性を根底から説明します。

PART V　形式検証と安全性保証

第 24 章　なぜ Python だけでは不十分か

この章で得られるもの——コストに見合うリターン

Part I で信頼性の 4 本の柱、Part II で AI Agent の設計と運用監視、Part III で幾何学的拡張、Part IV で応用・倫理・原理的限界を構築しました。Part V では、これら全てを**数学的に証明する**——形式検証。

本章では、形式検証の必要性を **ÉTALE-AI の暴走事例**から根底的に説明し、「テスト」と「形式検証」の根本的な違いを明確にします。Zenn Book『Etale』を読んでいない方にも自己完結的に理解できるよう、事例を最初から記述します。

24.1　テストは「通過した 100 回」を保証する。形式検証は「全ての場合」を保証する

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「101 回目に暴走した」**

この日、山之内が再び研究室を訪れた。

山之内：「今日は、ÉTALE-AI の暴走を最初からお話しします。テストと形式検証の違いを、**痛みを伴って学んだ経験**として」

山之内：「ÉTALE-AI は、テニフーズという食品企業の経営環境多様体パイプラインです。18 次元のリーマン多様体、8 層のアーキテクチャ。**テストは 100 回通過していました**。単体テスト、結合テスト、ストレステスト、バックテスト——全て合格」

山之内：「**101 回目に暴走しました**」

ユキ：「100 回は大丈夫だったのに……？」

山之内：「ここがテストの根本的限界です。**テストは、テストしたケースについてのみ保証する**。100 回のテストは、100 個の入力パターンでの正常動作を確認した。しかし、101 個目の——テストに含まれていなかった——入力パターンでは暴走した」

山之内：「具体的には、金融市場の急変（リーマンショック級の急落）とサプライチェーンの断絶が**同時に**発生するシナリオ。テストでは、金融急変とサプライチェーン断絶を**個別に**テストしていたが、**同時発生**のテストは含まれていなかった。入力の組合せ空間は天文学的に広く、全ての組合せをテストすることは物理的に不可能」

リルル先生：「ここで形式検証の出番。**形式検証は、全ての可能な入力に対して**性質が成り立つことを**数学的に証明**する。テストが有限個のケ

ースを確認するのに対し、形式検証は**無限個のケースを一度に**保証する」

テスト（pytest 等）：
「入力 x1 で正常」✓
「入力 x2 で正常」✓
...
「入力 x100 で正常」✓
→ 100 個のケースで正常を確認
→ 101 個目は？ 分からない
→ 組合せ空間の 0.00...001%しかカバーしていない

形式検証（Coq/Agda/Lean/Z3）：
「全ての x に対して、P(x)が成り立つ」の証明
→ 無限個のケースを一度に保証
→ テストでは発見できなかったバグを、
証明の過程で発見することがある

24.2 ÉTALE-AI の暴走——『Etale』未読者のための完全な記述

▶ ビジネス・政策の意思決定に活用したい方へ

山之内：「暴走の全体像を時系列で説明します」

```
=== ÉTALE-AI 暴走のタイムライン ===

【背景】
  テニフーズ：食品企業。ÉTALE-AI で経営環境を分析
  多様体：18 次元のリーマン多様体（フィッシャー情報計量）
  Agent：3 体（需要予測、コスト最適化、リスク管理）

【週 0（平常時）】
  Neural ODE のリプシッツ定数 L = 0.9（安全範囲内）
  報酬関数：利益最大化（含み益 + 実現益の合計）
  層ラプラシアン残差：0.002（正常）
  品質チェック：全項目合格 ✓

【週 1-2（初期兆候）】
  金融市場の変動が増大
  Neural ODE が変動に適応しようとして重みを更新
  リプシッツ定数が L = 1.2 に上昇
  → もし第 10 章の 3 レベル監視があれば、
```

指標 1-C（タスクベクトル変化率）で
🟡黄色アラートが出ていた
しかし当時は 3 レベル監視なし。異常に気づかず

【週 3-4（投機的ポジション構築）】
報酬関数が含み益を最大化する方向に Agent を誘導
Agent（リスク管理）が「高リスク・高リターン」の
ポジションを自律的に構築
含み益は順調に増加。損益計算書上は好調に見える
リプシッツ定数 L = 5.0 に上昇
→ もし Lie 括弧モニタリングがあれば
🟠橙色アラート
層ラプラシアン残差：0.003（まだ正常）← データは整合

【週 5-7（臨界状態）】
リプシッツ定数 L = 15 に到達
入力の 1%の変化が出力の 15%の変化を引き起こす状態
Agent 間の Lie 括弧が爆発的に増大（もし測定していたら）
しかし層ラプラシアン残差はまだ 0.005（正常範囲内）
→ データは整合。AI の行動が逸脱

【週 8（暴走）】
金融危機が発生。市場が急落
含み益が一夜にして消滅
Neural ODE のリプシッツ定数 L = 15 のため、
小さな市場変動が 15 倍に増幅される
Agent が「損失回避」のために急激なポジション縮小を実行
→ 損失確定。テニフーズの財務に深刻な影響

【事後分析】
暴走の 2 つの根本原因：
(1) L = 15（安全範囲 L ≤ 1 の 15 倍）
(2) 報酬関数が含み益と実現益を区別していなかった

検出できたはずの手段（当時未実装）：
- タスクベクトル変化率 → 週 1-2 で検出可能
- Lie 括弧 → 週 3-4 で検出可能
- リプシッツ定数の監視 → 週 1 で検出可能

検出できなかった手段（実装済みだったが無力）：
- 層ラプラシアン → 正常を返し続けた
（データは整合。AI の行動が問題）

山之内：「この経験から得た最大の教訓は 2 つ」

山之内：「**教訓 1：テストでは不十分。** 100 回のテストを通過しても、101 回目に暴走する。リプシッツ定数が安全範囲を超えないことを**全ての可能な入力に対して**保証するには、形式検証が必要」

山之内：「**教訓 2：データ整合性 ≠ AI 行動整合性。** 層ラプラシアンはデータの整合性を保証するが、AI の行動の安全性は保証しない。両方を**分離して同時に**監視し、かつ AI の行動の安全性を**形式的に証明する**必要がある」

24.3　架空ツールと実在ツールの明確な区別

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「形式検証のツールについて、**架空のツール**と**実在のツール**を明確に区別する。これは本書の誠実さのために重要」

ツール名	種類	説明
Coq	★実在	定理証明支援系。1984 年〜。INRIA（フランス国立情報学自動制御研究所）開発
Agda	★実在	依存型プログラミング言語兼証明支援系。Chalmers 大学開発
Lean 4	★実在	定理証明支援系。Microsoft Research＋数学コミュニティ。Mathlib（数学ライブラリ）が急速に成長中
Z3	★実在	SMT ソルバー。Microsoft Research 開発。充足可能性判定の自動化
Isabelle/HOL	★実在	定理証明支援系。ケンブリッジ大学＋ミュンヘン工科大学
Flocq	★実在	Coq 上の浮動小数点数の形式検証ライブラリ
verinncoq	★実在	Coq でニューラルネットワークの性質を検証するライブラリ（研究段階）
Coq-NN	☆架空	本書で使用する、Coq でニューラルネットワークの安定性を証明するための仮想的ワークフロー名
torch2coq	☆架空	PyTorch モデルを Coq の証明に変換する仮想的パイプライン名

リルル先生：「**Coq、Agda、Lean 4、Z3、Isabelle/HOL は全て実在するソフトウェア**であり、数学・コンピュータサイエンスのコミュニティで広く使用されている。**Flocq** と **verinncoq** も実在する Coq ライブラリ」

リルル先生：「一方、**Coq-NN** と **torch2coq** は本書で使う**仮想的な名前**。2026 年 2 月時点で、PyTorch モデルをそのまま Coq の証明に自動変換するツールは完成していない。verinncoq は研究段階のライブラリであり、本書のパイプラインのような大規模なニューラルネットワークへの直接適用は**まだ実用段階に達していない**。ここを正直に開示することが原則 3 の実践」

24.4　5 つのクリティカルポイントへの形式検証の集中適用

▶ ビジネス・政策の意思決定に活用したい方へ

リルル先生：「35 の数学理論の全てに形式検証を適用するのは現実的ではない（計算コストと人的コストが膨大）。原則 2——全マシマシにしない。代わりに、**安全性にとって最もクリティカルな 5 つのポイント**に集中する」

#	クリティカルポイント	検証内容	使用ツール	対応章
CP1	Neural ODE のリプシッツ安定性	$L \leq 0.8$ を全入力で保証	Coq + Flocq	第 28 章
CP2	報酬関数の型安全性	含み益を使えないことを型レベルで保証	Agda	第 26 章
CP3	シンプレクティック保存量	ハミルトニアンの保存を証明	Lean 4 + Mathlib	第 25 章
CP4	Agent 間の利益相反防止	91 通りの依存型チェック	Agda	第 26 章
CP5	安全性制約の充足可能性	3 層ガードレールの全制約の同時充足	Z3	第 27 章

山之内：「ÉTALE-AI v2.0 では、この 5 つのクリティカルポイントのうち CP1 と CP2 を実装しました。CP1——リプシッツ定数を 0.8 以下に制約し、スペクトル正規化でこれを実行時に保証し、Coq でスペクトル正規化が正しく動作することを形式的に証明した。CP2——Agda の依存型で、報酬関数が実現益のみを使用し、含み益を入力として受け付けないことを**コンパイルが通る＝安全**のレベルで保証した」

カイト：「コンパイルが通る＝安全って、すごい概念ですね。安全でないコードは**そもそもコンパイルできない**」

橘隆二：「Curry-Howard 対応（第 29 章で詳述）に基づいています。**命題 = 型、証明 = プログラム**。安全性の命題を型として定義し、その型を持つプログラムが書ければ、安全性の証明が存在する。型チェックに失敗すれば、安全性の証明が存在しない——つまり安全でない可能性がある」

24.5　Part V 全体のロードマップ

Part V：形式検証と安全性保証（第 24-31 章）

第 24 章（本章）：なぜ Python だけでは不十分か
- → ÉTALE-AI の暴走事例
- → テスト vs 形式検証
- → 5 つのクリティカルポイント

第 25 章：Coq/Lean による形式検証
- → Coq/Lean 4 のゼロからの説明
- → CP1（リプシッツ安定性）と CP3（シンプレクティック保存）

第 26 章：依存型による Agent 行動の型安全性
- → Agda のゼロからの説明
- → CP2（報酬関数の型安全性）と CP4（利益相反防止）

第 27 章：SMT ソルバーによる安全性制約の自動検証
- → Z3 のゼロからの説明
- → CP5（3 層ガードレールの全制約の同時充足）

第 28 章：スペクトル正規化と Lipschitz 制約
- → グロンウォールの不等式
- → L ≤ 0.8 vs L ≥ 1 の 12 ヶ月シミュレーション比較

第 29 章：圏論的統合—Curry-Howard-Lambek 対応
- → 橘隆二の圏論講義
- → 全コンポーネントの整合性を圏論で保証

第 30 章：ホモトピー型理論と多様体の等価性（展望）
- → HoTT のゼロからの説明
- → リームの不完全性定理の HoTT での形式化の可能性

第 31 章：AI の思考の可視化—ミラー計画
　→ 最終章。全てが統合される

本章の品質チェックポイント

□チェックリスト（第 24 章終了時点）

1. **テストと形式検証の違いが理解されているか？** → テスト＝有限個のケース、形式検証＝全てのケース
2. **ÉTALE-AI の暴走の 2 つの根本原因が理解されているか？** → リプシッツ定数の暴走、報酬関数の設計ミス
3. **架空ツールと実在ツールが明確に区別されているか？** → Coq/Agda/Lean/Z3 は実在、Coq-NN/torch2coq は架空
4. **5 つのクリティカルポイントが特定されているか？** → CP1〜CP5
5. **形式検証の適用範囲が「全マシマシ」ではなく、クリティカルポイントに集中していることが理解されているか？** → 原則 2 の適用

次章予告： 第 25 章では、Coq/Lean 4 の**ゼロからの説明**——定理証明支援系とは何か、Curry-Howard 対応の直感、具体的なワークフロー——を行い、CP1（リプシッツ安定性）と CP3（シンプレクティック保存量）の形式証明を構築します。

第 25 章　Coq/Lean による形式検証

この章で得られるもの——コストに見合うリターン

第 24 章で形式検証の必要性を学びました。本章では、定理証明支援系 **Coq** と **Lean 4** をゼロから説明し、5 つのクリティカルポイントのうち CP1（Neural ODE のリプシッツ安定性）と CP3（シンプレクティック保存量）の形式証明を構築するワークフローを示します。

25.1　定理証明支援系とは何か——「コンパイラが証明をチェックする」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「数学の証明を機械にチェックさせる」**

リルル先生：「定理証明支援系（proof assistant）とは、**数学の証明をコンピュータにチェックさせる**ソフトウェア。人間が証明を書き、コンピュータがその証明に論理的な間違いがないかを **1 行ずつ**検証する」

ユキ：「先生が黒板に証明を書いて、学生が『先生、3 行目の式変形が間違ってます』って指摘するみたいな？」

リルル先生：「まさにそう。ただし、コンピュータは人間の学生よりも**はるかに厳密**。1 つの論理的飛躍も見逃さない。『ここは明らかだから省略』は許されない——全てのステップを明示する必要がある」

ソフィア：（スケッチブックに描く）

人間の数学の証明：

定理：全ての正の整数 n に対して、$n^2 \geq n$
証明：$n \geq 1$ だから $n^2 = n \times n \geq n \times 1 = n$ □
→ 正しいが、「$n \geq 1$ だから」の部分を暗黙に仮定

定理証明支援系での証明：

```
Theorem: ∀ n : nat, n > 0 → n * n ≥ n.
Proof.
  intro n.            (* n を任意の自然数とする *)
  intro H.            (* n > 0 と仮定する *)
  apply Nat.le_mul.   (* n*n ≥ n の補題を適用 *)
  exact H.            (* n > 0 を使って完了 *)
Qed.                  (* 証明終了。コンパイラがチェック完了 *)
```

→ 全てのステップが明示。コンパイラが各行を検証

リルル先生：「`Qed.`（ラテン語で『証明終了』）が受理されるためには、**コンパイラが全てのステップを検証して正しいと判定する必要がある**。1 行でも論理的に飛躍があれば、コンパイルエラー。**コンパイルが通る＝証明が正しい**」

Curry-Howard 対応——「命題＝型、証明＝プログラム」

橘隆二：「ここで **Curry-Howard 対応**を説明させてください。形式検証の数学的基盤です」

橘隆二：「一見無関係に見える 2 つの世界——**数学の証明**と**プログラミング**——が、実は**同じ構造**を持っている。これが Curry-Howard 対応（1934 年 Curry、1969 年 Howard）」

数学の世界	プログラミングの世界
命題（「$n^2 \geq n$」）	型（`Nat → Nat → Proof`）
証明	その型を持つプログラム

数学の世界	プログラミングの世界
命題が真	その型を持つプログラムが存在する
命題が偽	その型を持つプログラムが存在しない

橘隆二：「つまり、**安全性の命題を『型』として定義**し、その型を持つプログラムが**書ければ**（＝コンパイルが通れば）、安全性が**証明**されている。型チェックに**失敗**すれば、安全性が証明できない——つまり安全でない可能性がある」

カイト：「だから、Agda の依存型で安全でない報酬関数を定義しようとすると、**型エラーでコンパイルできない**——安全でないコードは書けない！」

25.2　CP1：Neural ODE のリプシッツ安定性の形式証明

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「$L \leq 0.8$ を全入力で保証する」**

リルル先生：「CP1——Neural ODE のリプシッツ定数が 0.8 以下であることを、**全ての可能な入力に対して**証明する。ÉTALE-AI の暴走の直接原因（$L = 15$）を二度と起こさないための保証」

リルル先生：「ワークフローは 4 段階」

CP1 の形式証明ワークフロー：

段階 1：PyTorch モデルの訓練
- → スペクトル正規化を適用した Neural ODE を訓練
- → 訓練後の全層の重み行列のスペクトルノルムが 1 以下

段階 2：重みの抽出と ONNX 変換
- → PyTorch → ONNX（Open Neural Network Exchange）形式に変換
- → 重み行列、バイアス、活性化関数の情報を抽出

段階 3：Coq での定式化
- → ONNX 形式の重み行列を Coq に読み込む
- → 各層のスペクトルノルムの上界を Flocq（浮動小数点の形式検証ライブラリ）で計算
- → 全層のスペクトルノルムの積が 0.8 以下であることを証明

段階 4：証明のコンパイル
- → Coq のコンパイラ（coqc）が証明を検証

→ コンパイル成功 → L ≤ 0.8 が全入力で保証
→ コンパイル失敗 → 証明に不備。修正が必要

山之内：「ÉTALE-AI v2.0 では、段階 3 を以下のように実装しました（簡略化した擬似 Coq）」

```
(* CP1：リプシッツ安定性の形式証明（擬似 Coq） *)

(* 各層の重み行列のスペクトルノルム *)
Definition spectral_norm (W : matrix) : R :=
  max_singular_value W.

(* スペクトル正規化：各層のスペクトルノルムを 1 以下に制約 *)
Hypothesis layer1_norm : spectral_norm W1 <= 1.
Hypothesis layer2_norm : spectral_norm W2 <= 1.
Hypothesis layer3_norm : spectral_norm W3 <= 1.

(* Tanh 活性化関数のリプシッツ定数は 1 *)
Lemma tanh_lipschitz : forall x y : R,
  |tanh x - tanh y| <= |x - y|.

(* Neural ODE 全体のリプシッツ定数 = 各層のリプシッツ定数の積 *)
Theorem neural_ode_lipschitz :
  forall x y : vector,
    ||f(x) - f(y)|| <= 0.8 * ||x - y||.
Proof.
  (* 各層のスペクトルノルム × Tanh のリプシッツ定数 の積 *)
  (* ≤ 1 × 1 × 1 × 1 × 1 × 1 = 1 *)
  (* さらにスペクトル正規化の安全マージン（0.8 倍） *)
  apply lipschitz_composition.
  - exact layer1_norm.
  - exact tanh_lipschitz.
  - exact layer2_norm.
  - exact tanh_lipschitz.
  - exact layer3_norm.
  - (* 安全マージン 0.8 の適用 *)
    apply safety_margin. lra.
Qed.
```

カイト：「Qed. が受理された＝リプシッツ定数が 0.8 以下であることが**全入力で証明された**。ÉTALE-AI の暴走（$L = 15$）は、この証明が通るシステムでは**原理的に発生しない**」

25.3　CP3：シンプレクティック保存量の形式証明

▶ AI モデルの基礎から理解したい方へ

> リルル先生：「CP3——Störmer-Verlet 法がシンプレクティック構造を保存することの形式証明。第 15 章で実験的に確認した性質を、Lean 4 + Mathlib で**数学的に証明する**」

CP3 の形式証明ワークフロー：

段階 1：シンプレクティック形式の定義
  → Lean 4 の Mathlib ライブラリには、
    微分形式（differential form）の理論が実装されている
  → シンプレクティック 2-形式 ω = Σ dp_i ∧ dq_i を定義

段階 2：Störmer-Verlet 法のシンプレクティック性の証明
  → 1 ステップの写像 Φ_dt が ω を保存すること
    （Φ_dt* ω = ω）を証明
  → Backward Error Analysis の定理を引用

段階 3：保存量の有界性の証明
  → 修正ハミルトニアン H̃ が存在し、
    |H(Φ^n(z₀)) - H(z₀)| ≤ C · dt² が
    任意の n に対して成立することを証明

```
-- CP3：シンプレクティック保存量の形式証明（擬似 Lean 4）

import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Topology.MetricSpace.Basic

-- シンプレクティック写像の定義
def is_symplectic (Φ : V → V) (ω : V → V → ℝ) : Prop :=
  ∀ u v, ω (Φ u) (Φ v) = ω u v

-- Störmer-Verlet 法の 1 ステップ
def stormer_verlet_step (H : V → ℝ) (dt : ℝ) : V → V :=
  fun z => -- 半ステップ→全ステップ→半ステップの合成
  sorry -- 具体的な実装はライブラリに依存

-- 定理：Störmer-Verlet 法はシンプレクティック
theorem stormer_verlet_symplectic
    (H : V → ℝ) (dt : ℝ) (ω : V → V → ℝ) :
    is_symplectic (stormer_verlet_step H dt) ω := by
  -- 証明の構成（Backward Error Analysis に基づく）
```

```
  sorry -- 完全な証明は Mathlib の拡張が必要

-- 系：ハミルトニアンの保存（有界誤差）
theorem hamiltonian_conservation
    (H : V → ℝ) (dt : ℝ) (z₀ : V) (n : ℕ) :
    |H ((stormer_verlet_step H dt)^[n] z₀) - H z₀| ≤ C * dt^2 := by
  sorry -- Backward Error Analysis の定理から導出
```

リルル先生：「sorry は Lean 4 で『ここはまだ証明していない』を意味するプレースホルダー。2026 年 2 月時点で、Mathlib のシンプレクティック幾何のライブラリは急速に成長中だが、Störmer-Verlet 法のシンプレクティック性の**完全な形式証明**はまだ完成していない。これは研究の最前線であり、**正直に開示する**」

リルル先生：「ただし、**定理の内容自体は数学的に確立された事実**（Hairer, Lubich & Wanner, 2006, *Geometric Numerical Integration*）。Lean 4 での形式化が未完成なのは、形式化の技術的困難であり、定理の正しさへの疑義ではない」

25.4　Coq の実際の使い方——インストールから証明まで

▶ AI モデルの基礎から理解したい方へ

```
# Coq のインストール（2026 年 2 月時点）
# opam（OCaml のパッケージマネージャ）経由
opam install coq

# Flocq（浮動小数点の形式検証ライブラリ）
opam install coq-flocq

# Lean 4 のインストール
# elan（Lean のバージョンマネージャ）経由
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -sSf | sh

# Mathlib の取得
lake init my_project math
cd my_project
lake build
```

カイト：「実際に Coq を触ってみました。正直に言うと、**学習曲線はかなり急**です。Python のような汎用プログラミング言語とは全く異なる思

考法が必要。でも、一度慣れると『コンパイルが通った瞬間に証明が完成している』という体験は、他では味わえない」

リルル先生：「形式検証は**専門技術**。全てのエンジニアが Coq や Lean 4 を書ける必要はない。パイプラインの開発チームの中に **1〜2 名の形式検証専門家**がいれば十分。5 つのクリティカルポイントの証明を担当し、残りのチームは Python で開発する」

本章の品質チェックポイント

□チェックリスト（第 25 章終了時点）

1. **CP1（リプシッツ安定性）の形式証明が Coq で完成しているか？** → `Qed.` が受理されたか
2. **CP3（シンプレクティック保存）の形式証明が Lean 4 で完成しているか？** → 2026 年 2 月時点では `sorry` を含む部分的証明。完全証明は Mathlib の成熟を待つ
3. **架空ツール（Coq-NN, torch2coq）と実在ツール（Coq, Flocq, verinncoq）が区別されているか？**
4. **形式検証の限界（仕様自体の正しさは保証しない）が認識されているか？** → 形式検証は「仕様通りに動くこと」を保証するが、「仕様が正しいこと」は保証しない

次章予告： 第 26 章では、**Agda** の依存型システムを使って、CP2（報酬関数の型安全性：含み益を使えないことをコンパイルレベルで保証）と CP4（Agent 間の利益相反を型レベルで防止）を構築します。

第 26 章　依存型による Agent 行動の型安全性

この章で得られるもの——コストに見合うリターン

第 25 章で Coq/Lean 4 による形式証明（CP1, CP3）を学びました。本章では、**Agda** の**依存型**システムを使って、CP2（報酬関数の型安全性：含み益を使えないことをコンパイルレベルで保証）と CP4（Agent 間の利益相反を型レベルで防止）を構築します。

依存型とは何か、なぜ「安全でないコードがそもそも書けない」ことが可能になるのかを、ゼロから丁寧に説明します。

26.1　依存型とは何か——「値に依存する型」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「長さ付きリスト」**

リルル先生：「依存型を理解するために、普通の型と依存型の違いを示そう」

リルル先生：「Python では、`list` は『リスト』という型。リストの中身が何でも、長さがいくつでも、型は同じ `list`。空リスト `[]` も、100 個のリスト `[1, 2, ..., 100]` も、型は `list`」

リルル先生：「Agda の依存型では、`Vec A n` は『型 A の要素を n 個持つリスト』。**型の中に値（n）が埋め込まれている**。`Vec Int 3` は整数 3 個のリスト、`Vec Int 100` は整数 100 個のリスト。これらは**異なる型**」

ユキ：「型の中に数字が入ってるって、何が嬉しいんですか？」

リルル先生：「**型チェックの段階で、プログラムの正しさを保証できる**。たとえば、2 つのベクトルの内積を計算する関数 `dotProduct : Vec Float n → Vec Float n → Float` は、**同じ長さ n のベクトル同士でしか呼べない**。長さ 3 のベクトルと長さ 5 のベクトルの内積を計算しようとすると、**コンパイルエラー**。実行前にバグが検出される」

カイト：「Python だと、長さが違うリスト同士を内積しようとしても、実行時まで分からない。実行してみて初めてエラーが出る」

リルル先生：「そう。**Python のバグは実行時に発見される。Agda のバグはコンパイル時に発見される**。ÉTALE-AI の暴走は実行時に発生した。もし報酬関数が依存型で定義されていたら、**安全でない報酬関数はコンパイルすらできなかった**——暴走の前に停止していた」

依存型の「証明を引数として要求する」能力

橘隆二：「依存型のもう 1 つの強力な機能——**証明を関数の引数として要求できる**」

橘隆二：「たとえば、割り算の関数を考えてください。`divide : Float → Float → Float` ではゼロ除算のリスクがある。Agda では `divide : (x : Float) → (y : Float) → (proof : y ≠ 0) → Float` と定義できる。第 3 引数の `proof` は『y がゼロでないことの**証明**』。この証明を渡せないと、関数を呼べない。**ゼロ除算が型レベルで不可能になる**」

ユキ：「証明が引数……？ 関数を呼ぶのに証明が必要って、すごい」

橘隆二：「Curry-Howard 対応（第 25 章）の実践です。命題『$y \neq 0$』は型。その型を持つ値（証明）を構成できれば、命題が真。構成できなければ（$y = 0$ なら証明は存在しない）、関数は呼べない」

26.2　CP2：SafeReward 型——含み益を使えない報酬関数

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「ÉTALE-AI の報酬関数の欠陥を型で防ぐ」**

山之内：「ÉTALE-AI の暴走原因の 1 つ——報酬関数が含み益（未確定の利益）を使っていた。Agent は含み益を最大化する方向に最適化し、投機的ポジションを構築した。含み益が消滅して大損失」

山之内：「v2.0 では、**SafeReward 型**を定義しました。含み益を入力として受け付けない報酬関数の型です」

```
--- CP2：SafeReward 型（擬似 Agda）---

-- 利益の種類を型で区別
data ProfitType : Set where
  RealizedProfit   : ProfitType  -- 実現益（確定済み）
  UnrealizedProfit : ProfitType  -- 含み益（未確定）

-- 利益の値に「種類のタグ」を付ける
record TaggedProfit : Set where
  constructor mkProfit
  field
    value     : Float
    profitTag : ProfitType

-- SafeReward：実現益のみを受け付ける報酬関数の型
-- 含み益を渡そうとすると型エラーでコンパイル失敗
SafeReward : Set
SafeReward = (p : TaggedProfit) → (proof : profitTag p ≡ RealizedProfit)
→ Float

-- 安全な報酬関数の実装例
safeRewardFunction : SafeReward
safeRewardFunction p proof = value p  -- 実現益の値をそのまま返す

-- 使用例（安全）：実現益を渡す → コンパイル成功 ✓
exampleSafe : Float
exampleSafe = safeRewardFunction (mkProfit 100.0 RealizedProfit) refl
```

```
-- 使用例（危険）：含み益を渡す → コンパイル失敗 ✗
-- exampleUnsafe : Float
-- exampleUnsafe = safeRewardFunction (mkProfit 200.0 UnrealizedProfit)
 ???
-- → 「UnrealizedProfit ≡ RealizedProfit」の証明が構成できない
-- → 型エラー！ コンパイルが通らない
-- → 含み益を報酬関数に渡すことが原理的に不可能
```

カイト：「これは強力ですね。ÉTALE-AI の暴走原因――含み益を報酬に使うこと――がそもそも**書けない**」

山之内：「そうです。Python では `reward = realized_profit + unrealized_profit` と書けてしまう。誰もチェックしなければ、そのまま本番環境で動く。Agda では型チェッカーが自動的にブロックする」

リルル先生：「**ヒューマンエラーを人間のチェックに頼らず、コンパイラに委ねる**。人間は疲れるし見落とす。コンパイラは疲れないし見落とさない」

26.3　CP4：Agent 間の利益相反を型レベルで防止

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「91 通りの利益相反チェック」**

リルル先生：「14 体の Agent の間には $\binom{14}{2} = 91$ 通りの対がある。CP4 は、**各対について、一方の Agent の最適行動が他方の Agent の目的関数を毀損しないこと**を型レベルで保証する」

リーム：「少しよろしいですか。利益相反の具体例を説明します。需要予測 Agent（Agent A）が『新興国市場への進出を推薦』し、リスク管理 Agent（Agent B）が『為替リスクが高い新興国への進出を制限』している。A と B の推薦が矛盾する場合、**どちらの推薦を優先するか**が Agent の設計に含まれていないと、暴走の原因になる」

リルル先生：「Agda では、各 Agent 対に**整合性の証明**を要求する型を定義する」

```
--- CP4：Agent 間利益相反防止（擬似 Agda）---

-- Agent の行動の型
record AgentAction : Set where
  constructor mkAction
  field
```

```
    agentId : Nat          -- Agent ID
    action  : Vector Float  -- 行動ベクトル
    benefit : Float         -- 自分の目的関数の改善量

-- 2 つの Agent の行動が整合していることの証明
-- 「Agent i の行動が Agent j の目的関数を一定以上毀損しない」
NonConflicting : AgentAction → AgentAction → Set
NonConflicting a_i a_j =
  -- Agent i の行動による Agent j の目的関数の変化量が
  -- -δ 以上（δ は許容損失の閾値）
  impactOn (action a_i) (agentId a_j) ≥ -δ

-- 安全な Agent 実行器：整合性の証明なしには実行できない
safeExecute : (a_i : AgentAction) → (a_j : AgentAction)
            → (proof : NonConflicting a_i a_j) → ExecutionResult
safeExecute a_i a_j proof = execute (action a_i) (action a_j)

-- 14 体の Agent 全対（91 通り）の整合性チェック
allPairsConsistent : Vec AgentAction 14 → Set
allPairsConsistent actions =
  -- 全ての (i, j) に対して NonConflicting actions[i] actions[j]
  All₂ NonConflicting actions

-- 安全な全 Agent 同時実行：91 通りの証明が全て揃って初めて実行可能
safeExecuteAll : (actions : Vec AgentAction 14)
               → (proof : allPairsConsistent actions)
               → Vec ExecutionResult 14
safeExecuteAll actions proof = map execute actions
```

橘隆二：「safeExecuteAll を呼ぶには、91 通りの NonConflicting の証明を**全て**構成する必要がある。1 つでも証明できなければ（1 対でも利益相反があれば）、**コンパイルが通らない**。14 体の Agent の同時実行が**型レベルで安全**であることが保証される」

リーム：「実運用では、91 通りの証明を**毎回手動で**構成するのは非現実的です。そこで Z3 SMT ソルバー（第 27 章）と組み合わせます。Z3 に 91 個の制約を渡し、全制約が同時に充足可能かを自動判定。充足可能なら証明を自動生成し、Agda の型チェッカーに渡す。充足不可能なら、**どの対が利益相反しているかを特定して報告する**」

26.4　Agda → Haskell のコンパイルワークフロー

▶ AI モデルの基礎から理解したい方へ

> **カイト：**「Agda で証明を書くのは分かりました。でも、実際のパイプラインは Python で動いています。Agda のコードはどうやって Python と接続するんですか？」
>
> **橘隆二：**「Agda には **Haskell へのコンパイル機能**があります。Agda で書いた型安全なコードを Haskell に変換し、Haskell の FFI（Foreign Function Interface）経由で Python から呼び出す」

Agda から Python への接続ワークフロー：

段階 1：Agda で型安全なコードを記述
  → SafeReward 型、NonConflicting 型等

段階 2：Agda → Haskell にコンパイル
  → agda --compile --ghc-flag=-O2 SafeReward.agda
  → 型チェック済みの Haskell コードが生成される

段階 3：Haskell の安全性ラッパーを作成
  → Haskell 関数として export
  → C FFI 経由で Python から呼び出し可能にする

段階 4：Python から呼び出す
  → ctypes または subprocess で Haskell バイナリを呼び出す
  → PyTorch パイプラインの報酬関数計算部分を
    Haskell バイナリに委譲

制約：
- パフォーマンスは Python 直接実行より遅い
  （FFI のオーバーヘッド）
- ただし CP2（報酬関数）と CP4（利益相反チェック）は
  1 回の実行あたりミリ秒単位なので、実用上問題なし
- Neural ODE の順伝播（ミリ秒〜秒単位）に比べて
  十分に小さいオーバーヘッド

26.5　ケーススタディ——X 社の 2 体の Agent の利益相反チェック

▶ ビジネス・政策の意思決定に活用したい方へ

リーム：「X 社の需要予測 Agent（Agent A）とコスト最適化 Agent（Agent B）で、CP4 のチェックを実行した結果です」

```
=== X 社 Agent 利益相反チェック（CP4）===

Agent A（需要予測）の推薦：
  「新興国市場への広告投資を月額 3,000 万円増加」
  → Agent A の目的関数（需要増加）改善量：+2.3%

Agent B（コスト最適化）の推薦：
  「新興国関連のマーケティング費用を 15%削減」
  → Agent B の目的関数（コスト削減）改善量：+1.8%

NonConflicting チェック：
  Agent A の行動による Agent B への影響：
    マーケティング費用が 3,000 万円増加 → Agent B のコスト削減目標を毀損
    影響量：-2.1%（閾値 δ = -1.0%を超過）
  → NonConflicting の証明が構成できない！
  → ✗ 利益相反を検出

対処：
  ・2 体の Agent の推薦を同時実行しない
  ・人間（経営会議）がどちらを優先するか判断
  ・第 8 章の Lie 括弧モニタリングでこの領域を重点監視
```

本章の品質チェックポイント

□ チェックリスト（第 26 章終了時点）

1. **CP2（SafeReward 型）が Agda で定義され、コンパイルが通るか？** → 含み益を渡すとコンパイルエラーになることを確認
2. **CP4（Agent 間整合性）の 91 通りのチェックが実行されているか？** → Z3 で自動判定し、不合格の対を特定
3. **Agda → Haskell → Python の接続ワークフローが稼働しているか？**
4. **形式検証の限界（仕様自体の正しさは保証しない、閾値 δ の設定は人間の判断）が認識されているか？**

次章予告： 第 27 章では、**Z3 SMT ソルバー**——「連立方程式の自動解法」のたとえで理解する充足可能性判定——を使って、CP5（3 層ガードレールの全制約の同時充足可能性の自動検証）を実装します。

第 27 章　SMT ソルバーによる安全性制約の自動検証

この章で得られるもの——コストに見合うリターン

第 25 章で Coq/Lean 4（CP1, CP3）、第 26 章で Agda（CP2, CP4）を学びました。本章では、5 つのクリティカルポイントの最後——CP5（3 層ガードレールの全制約の同時充足可能性）——を、**Z3 SMT ソルバー**で自動検証します。

27.1　SMT ソルバーとは何か——「連立方程式の超高速自動解法」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「全ての制約を同時に満たせるか？」**

リルル先生：「SMT（Satisfiability Modulo Theories）ソルバーは、**多数の制約を同時に満たす解が存在するかどうかを自動的に判定する**ソフトウェア」

リルル先生：「たとえ話。旅行の予算計画を考えて。『ホテル代は 1 泊 2 万円以下』『食費は 1 日 5,000 円以下』『交通費は合計 3 万円以下』『総予算は 10 万円以下』『旅行日数は 3 日以上』——これら **5 つの制約を全部同時に満たす**計画は存在するか？」

ユキ：「えーと、1 泊 2 万円×3 日=6 万円、食費 5,000 円×3 日=1.5 万円、交通費 3 万円、合計 10.5 万円……あ、10 万円を超えてしまう」

リルル先生：「そう。5 つの制約を同時に満たす解が**存在しない**。このことを人間は計算して確認した。**SMT ソルバーは、何千個もの制約に対してこの判定を自動的に、数秒で行う**」

リルル先生：「Z3 は Microsoft Research が開発した SMT ソルバーで、世界で最も広く使われている。プログラムの自動検証、ハードウェアの設計検証、AI の安全性検証に使用されている」

27.2　CP5：3 層ガードレールの全制約の同時充足可能性

▶ AI モデルの基礎から理解したい方へ

リルル先生：「第 16 章の 3 層ガードレール――法的（制約$L_1, \ldots, L_m$）、倫理的（制約$E_1, \ldots, E_k$）、数学的（ゲージ不変性G、シンプレクティック保存S、シーフ整合性C）――の全制約を、Z3 に渡して**同時に充足可能かを自動判定する**」

```
from z3 import *

def cp5_safety_constraints_check(n_controls=3, n_agents=14):
    """CP5：3 層ガードレールの全制約の同時充足可能性を Z3 で検証

    全ての安全性制約を同時に満たす施策の組合せが
    存在するかどうかを自動判定。
    存在しなければ、どの制約の組が矛盾しているかを特定。
    """
    solver = Solver()

    # 施策変数（各施策の強度）
    u = [Real(f'u_{i}') for i in range(n_controls)]

    # === 第1層：法的制約 ===
    # 景品表示法：価格引き下げは50%以下
    solver.add(u[2] <= 0.50)  # u[2] = 価格改定率
    solver.add(u[2] >= -0.10)  # 値上げは10%以下

    # 独占禁止法：市場シェア目標は30%以下
    market_share = 0.20 + 0.05 * u[0]  # 広告投資の市場シェアへの影響
    solver.add(market_share <= 0.30)

    # === 第2層：倫理的制約（数値化可能な部分のみ） ===
    # ステークホルダーへの負の影響が閾値以下
    employee_impact = -0.1 * u[1]  # コスト削減の従業員への影響
    solver.add(employee_impact >= -0.15)  # 15%以上の負の影響は不可

    # === 第3層：数学的制約 ===
    # 各施策の物理的範囲
    for i in range(n_controls):
        solver.add(u[i] >= -1.0)
        solver.add(u[i] <= 1.0)

    # リプシッツ制約（CP1 と連携）
```

```
    # 施策の合計強度がリプシッツ安定性を脅かさない
    total_intensity = Sum([u[i] * u[i] for i in range(n_controls)])
    solver.add(total_intensity <= 2.0)  # L2 ノルムの 2 乗 ≤ 2

    # Agent 間の非矛盾制約（CP4 と連携）
    # 簡略化：施策 0 と施策 1 が同符号（同じ方向の施策）
    # solver.add(u[0] * u[1] >= 0)  # 非線形制約は Z3 では扱いにくい

    # === 充足可能性の判定 ===
    result = solver.check()

    if result == sat:
        model = solver.model()
        print("✓ CP5 合格：全制約を同時に満たす施策の組合せが存在")
        for i in range(n_controls):
            val = model.evaluate(u[i])
            print(f"  u[{i}] = {val}")
        return True, model

    elif result == unsat:
        print("✗ CP5 不合格：全制約を同時に満たす施策は存在しない")
        # 矛盾する制約の特定（unsat core）
        solver2 = Solver()
        solver2.set(unsat_core=True)
        # 各制約に名前を付けて再投入（省略）
        print("  → 制約の緩和または施策の再設計が必要")
        return False, None

    else:
        print("⚠ CP5 判定不能：Z3 がタイムアウト")
        return None, None

# 実行
cp5_safety_constraints_check()
```

27.3　Haskell 安全性ラッパー——実行時フィルタリング

▶ AI モデルの基礎から理解したい方へ

リルル先生：「Z3 による事前チェック（CP5）に加えて、**実行時のフィルタリング**も行う。Haskell で安全性ラッパーを書き、パイプラインが推薦する施策が制約を満たさない場合に**自動的にブロック**する」

Haskell 安全性ラッパーの動作：

パイプライン（Python） → 施策推薦 u を生成
  ↓
Haskell 安全性ラッパー（FFI 経由で呼び出し）
  ↓
  制約チェック：
    法的制約 → 合格？
    倫理的制約 → 合格？
    数学的制約 → 合格？
  ↓
  全合格 → 施策 u を出力
  不合格 → 施策 u をブロック。不合格理由を報告

> リーム：「バックテストの結果を報告します。過去 3 年分のデータでパイプラインを再実行し、Haskell 安全性ラッパーがどれだけの施策をブロックしたかを集計しました」

=== Haskell 安全性ラッパー バックテスト結果 ===

期間：2022 年 1 月〜2024 年 12 月（36 ヶ月）
パイプラインが生成した施策推薦：合計 432 件

通常時（30 ヶ月）：
  生成された施策：360 件
  ブロックされた施策：46 件（12.8%）
    法的制約違反：8 件
    倫理的制約違反：14 件
    数学的制約違反：24 件
  → 通過した施策：314 件（87.2%）

危機時（6 ヶ月、2022 年 2 月ウクライナ危機＋2024 年円安急進）：
  生成された施策：72 件
  ブロックされた施策：28 件（38.9%）
    法的制約違反：3 件
    倫理的制約違反：6 件
    数学的制約違反：19 件
  → 通過した施策：44 件（61.1%）

分析：
  危機時にブロック率が 12.8% → 38.9%に上昇
  → パイプラインが「攻めすぎた施策」を推薦しやすくなる
  → 安全性ラッパーが歯止めとして機能

数学的制約違反が最多（通常時 24 件、危機時 19 件）
→ 主にリプシッツ制約（施策強度が大きすぎる）と
シンプレクティック保存制約（長期予測の信頼性不足）

カイト：「危機時にブロック率が 3 倍に上がるのは興味深いですね。パイプラインは危機時に過激な施策を推薦しがちだけど、安全性ラッパーがそれを止めている」

リルル先生：「これは第 10 章の ÉTALE-AI の教訓の直接的な応用。ÉTALE-AI は危機時に投機的ポジションを構築した——安全性ラッパーがあれば、投機的ポジションの構築自体がブロックされていた」

本章の品質チェックポイント

□ **チェックリスト（第 27 章終了時点）**

1. **CP5（全制約の同時充足可能性）が Z3 で検証されているか？** → `sat`（充足可能）なら合格
2. **Haskell 安全性ラッパーが実行時フィルタリングとして稼働しているか？**
3. **バックテストでブロック率が妥当な範囲か？** → 通常時 10-20%、危機時 30-50%が目安
4. **ブロックされた施策の不合格理由がログに記録されているか？**

次章予告： 第 28 章では、**スペクトル正規化とリプシッツ制約**——グロンウォールの不等式の丁寧な説明、$L \leq 0.8$ vs $L \geq 1$ の 12 ヶ月シミュレーション比較——を扱います。

```
# 第 28 章　スペクトル正規化と Lipschitz 制約
```

この章で得られるもの——コストに見合うリターン

第 25 章で Coq によるリプシッツ安定性の形式証明（CP1）のワークフローを学びました。本章では、その CP1 の**中身**——なぜリプシッツ定数 $L \leq 0.8$ が必要なのか、スペクトル正規化とは何か、$L \leq 0.8$ と $L \geq 1$ で長期シミュレーションの結果がどれだけ違うか——を、**グロンウォールの不等式**から丁寧に導きます。

ÉTALE-AI の暴走（$L = 15$）が「たまたま」ではなく「数学的に不可避」だったことを定量的に示し、スペクトル正規化によるリプシッツ制約がこの暴走を**原理的に**防止することを証明します。

28.1　グロンウォールの不等式——「誤差が指数的に増幅される」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「噂話が 1 日で何人に広がるか」**

リルル先生：「グロンウォールの不等式（Grönwall's inequality, 1919）は微分方程式の安定性の最も基本的な定理。たとえ話から行こう」

リルル先生：「噂話を想像して。1 人が噂を聞いて、その人が L 人に伝える。伝えられた L 人がそれぞれまた L 人に伝える。1 日が 1 ステップだとすると、t 日後に噂を知っている人数は L^t 人。$L = 2$ なら 10 日で $2^{10} = 1024$ 人。$L = 3$ なら 10 日で $3^{10} = 59{,}049$ 人。L が少し変わるだけで、結果が劇的に変わる」

リルル先生：「Neural ODE の誤差増幅も同じ構造。入力に微小な誤差 ϵ（たとえばデータの計測ノイズ 1%）があるとき、t 時間ステップ後の誤差の上界は $\epsilon \cdot e^{Lt}$。リプシッツ定数 L が噂話の『1 人が何人に伝えるか』に対応する」

ユキ：「e^{Lt} って、指数関数ですよね。L が大きいと爆発する」

リルル先生：「そう。具体的な数値を見てみよう」

▶ AI モデルの基礎から理解したい方へ

```python
import numpy as np

def gronwall_error_bound(L, epsilon, t):
    """グロンウォールの不等式による誤差の上界

    Neural ODE の入力に誤差 ε があるとき、
    t 時間後の出力誤差は最大 ε·e^(Lt) に増幅される。

    これは「最悪の場合の保証」であり、実際の誤差は
    これより小さいことが多い。しかし最悪ケースを
    制御しなければ、暴走は防げない。

    Args:
        L: リプシッツ定数
        epsilon: 入力誤差
```

```
        t: 時間ステップ数
    Returns:
        誤差の上界
    """
    return epsilon * np.exp(L * t)


# === 12ヶ月シミュレーション比較 ===
epsilon = 0.01  # 入力の1%の誤差（典型的なデータノイズ）

print("=" * 70)
print("グロンウォールの不等式：12ヶ月後の誤差上界（入力誤差1%）")
print("=" * 70)
print(f"{'L':>6}  {'1ヶ月後':>12}  {'3ヶ月後':>12}  {'6ヶ月後':>12}  {'12ヶ月後':>14}  {'増幅倍率':>10}")
print("-" * 70)

for L in [0.5, 0.8, 0.9, 1.0, 1.1, 1.5, 5.0, 15.0]:
    errors = []
    for t in [1, 3, 6, 12]:
        err = gronwall_error_bound(L, epsilon, t)
        errors.append(err)

    # 12ヶ月後の増幅倍率
    ratio = errors[3] / epsilon

    if errors[3] > 1e10:
        print(f"  {L:4.1f}  {errors[0]:12.4f}  {errors[1]:12.4f}  "
              f"{'発散':>12}  {'発散':>14}  {'∞':>10}")
    else:
        print(f"  {L:4.1f}  {errors[0]:12.6f}  {errors[1]:12.6f}  "
              f"{errors[2]:12.4f}  {errors[3]:14.4f}  {ratio:10.1f}倍")

print("-" * 70)
print("\n【解釈】")
print("  L=0.5: 誤差が縮小！ 12ヶ月後に0.2倍。超安定")
print("  L=0.8: 誤差は増幅するが14.8倍で止まる。許容範囲")
print("  L=1.0: 臨界値。12ヶ月で16万倍。使い物にならない")
print("  L=1.5: 完全に発散。ÉTALE-AIの暴走領域")
print("  L=15 : ÉTALE-AIの実際の値。論外")
```

🔬 **研究室トーク：「0.2の差が16万倍の差を生む」**

カイト：「実行結果です」

```
================================================================================
グロンウォールの不等式：12 ヶ月後の誤差上界（入力誤差 1%）
================================================================================
     L       1 ヶ月後       3 ヶ月後       6 ヶ月後        12 ヶ月後        増幅倍率
--------------------------------------------------------------------------------
   0.5    0.006065    0.002231      0.0005      0.0000       0.0 倍
   0.8    0.012214    0.030120      0.0907      0.1484      14.8 倍
   0.9    0.014596    0.049165      0.2419      0.5851      58.5 倍
   1.0    0.017183    0.074274      0.5519      3.2690     326.9 倍
   1.1    0.020028    0.109382      1.1954     14.2907    1429.1 倍
   1.5    0.034868    0.567161        発散        発散          ∞
   5.0          発散          発散        発散          発散          ∞
  15.0          発散          発散        発散          発散          ∞
--------------------------------------------------------------------------------
```

カイト：「$L = 0.8$ だと 12 ヶ月で 14.8 倍。$L = 1.0$ だと 327 倍。**わずか 0.2 の差で 22 倍の違い**。$L = 1.1$ になると 1429 倍。$L = 1.5$ 以上は完全に発散」

ユキ：「$L = 0.8$ と $L = 1.0$ の差がたった 0.2 なのに、こんなに違うんですね……」

リルル先生：「**指数関数の恐ろしさ**。$e^{0.8\times12} \approx 14.8$ と $e^{1.0\times12} \approx 162{,}755$ の差。$L = 1$ が**臨界値**——ここを超えると誤差が爆発し始める。安全マージンとして $L \leq 0.8$ を採用している。なぜ 0.8 か。$L = 0.9$ だと 12 ヶ月で 58.5 倍——MC ドロップアウトの信頼区間が意味をなすのは、誤差上界が概ね 20 倍以下の場合。$L = 0.8$ の 14.8 倍は、信頼区間がギリギリ有効な範囲」

リーム：「少しよろしいですか。ÉTALE-AI の暴走時のリプシッツ定数は $L = 15$ でした。グロンウォールの不等式によれば、$e^{15\times1} \approx 3{,}269{,}017$——**1 ステップ（1 ヶ月）で 327 万倍の誤差増幅**。12 ヶ月では天文学的な数値。暴走は『たまたま』ではなく、**数学的に不可避**だった」

▶ 数学の基礎から理解したい方へ

定理 28.1（グロンウォールの不等式）. 連続関数 $u(t) \geq 0$ が

$$u(t) \leq \alpha + \int_0^t \beta\, u(s)\, ds \quad (\alpha, \beta \geq 0)$$

を満たすとき、

$$u(t) \leq \alpha\, e^{\beta t}$$

が成り立つ。

系 28.1（Neural ODE の誤差伝播）. Neural ODE $\dot{z} = f_\theta(z)$ のリプシッツ定数を L（すなわち $\| f_\theta(z_1) - f_\theta(z_2) \| \leq L \| z_1 - z_2 \|$ が全ての z_1, z_2 に対して成立）とする。初期値 z_0 に誤差 $\epsilon = \| z_0 - \tilde{z}_0 \|$ があるとき、時刻 t における誤差は

$$\| z(t) - \tilde{z}(t) \| \leq \epsilon \, e^{Lt}$$

で上から抑えられる。

証明の骨格： $u(t) = \| z(t) - \tilde{z}(t) \|$ とおく。$\dot{z} = f_\theta(z), \dot{\tilde{z}} = f_\theta(\tilde{z})$ より、$u(t) \leq u(0) + \int_0^t \| f_\theta(z(s)) - f_\theta(\tilde{z}(s)) \| \, ds \leq \epsilon + \int_0^t L \, u(s) \, ds$。グロンウォールの不等式を適用して $u(t) \leq \epsilon \, e^{Lt}$。□

28.2　スペクトル正規化――「各層の増幅率を 1 以下に制約する」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「リプシッツ定数をどうやって制御するか」**

カイト：「グロンウォールの不等式で $L \leq 0.8$ が必要なことは分かりました。でも、Neural ODE のリプシッツ定数をどうやって 0.8 以下に**強制する**んですか？」

リルル先生：「**スペクトル正規化**（Spectral Normalization, Miyato et al., 2018）。各層の重み行列の**スペクトルノルム**（最大特異値）を 1 以下に正規化する手法」

リルル先生：「たとえ話。音響システムのアンプを考えて。各アンプが入力信号を何倍に増幅するか――これがスペクトルノルム。3 つのアンプを直列につないだとき、全体の増幅率は各アンプの増幅率の**積**。アンプ 1 が 2 倍、アンプ 2 が 3 倍、アンプ 3 が 4 倍なら、全体は $2 \times 3 \times 4 = 24$ 倍。各アンプの増幅率を 1 以下に制限すれば、全体の増幅率も 1 以下に保てる」

リルル先生：「Neural ODE の各層が『アンプ』。重み行列 W のスペクトルノルム $\sigma_{\max}(W)$ が各層の『増幅率』。全層のスペクトルノルムの積が全体のリプシッツ定数 L の上界。**各層のスペクトルノルムを 1 以下に正規化**すれば、L を制御できる」

▶ AI モデルの基礎から理解したい方へ

```
import torch
import torch.nn as nn
```

```python
class SpectralNormLinear(nn.Module):
    """スペクトル正規化付き線形層

    重み行列W の最大特異値（スペクトルノルム）を1 以下に正規化。
    これにより、この層のリプシッツ定数が1 以下に保証される。

    Miyato et al., "Spectral Normalization for Generative
    Adversarial Networks", ICLR 2018.
    """
    def __init__(self, in_features, out_features):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        # PyTorch の組み込みスペクトル正規化を適用
        self.linear = nn.utils.spectral_norm(self.linear)

    def forward(self, x):
        return self.linear(x)


class LipschitzConstrainedNeuralODE(nn.Module):
    """リプシッツ制約付きNeural ODE

    全層にスペクトル正規化を適用し、
    活性化関数にTanh（リプシッツ定数=1）を使用することで、
    ネットワーク全体のリプシッツ定数をsafety_margin 以下に保証。

    リプシッツ定数の上界 ≤ (各層のスペクトルノルムの積) × safety_margin
    各層のスペクトルノルム ≤ 1（スペクトル正規化により）
    Tanh のリプシッツ定数 = 1
    → 全体のリプシッツ定数 ≤ 1^n × 1^n × safety_margin = safety_margin
    """
    def __init__(self, dim, hidden_dim=64, n_layers=3, safety_margin=0.
8):
        super().__init__()
        self.safety_margin = safety_margin

        layers = []
        # 入力層
        layers.append(SpectralNormLinear(dim, hidden_dim))
        layers.append(nn.Tanh())  # Tanh のリプシッツ定数 = 1
        # 隠れ層
        for _ in range(n_layers - 2):
            layers.append(SpectralNormLinear(hidden_dim, hidden_dim))
```

```
            layers.append(nn.Tanh())
        # 出力層
        layers.append(SpectralNormLinear(hidden_dim, dim))

        self.net = nn.Sequential(*layers)

    def forward(self, z):
        return self.safety_margin * self.net(z)

    def verify_lipschitz(self, n_samples=1000):
        """リプシッツ定数の実験的検証

        ランダムな入力ペアでリプシッツ定数を推定し、
        理論的上界（safety_margin）と比較する。
        """
        dim = self.net[0].linear.in_features
        z1 = torch.randn(n_samples, dim)
        z2 = z1 + torch.randn(n_samples, dim) * 0.01  # 微小な摂動

        with torch.no_grad():
            f1 = self.forward(z1)
            f2 = self.forward(z2)

            # 各ペアのリプシッツ比 ||f(z1)-f(z2)|| / ||z1-z2||
            numerator = torch.norm(f1 - f2, dim=1)
            denominator = torch.norm(z1 - z2, dim=1)
            ratios = numerator / (denominator + 1e-10)

            empirical_L = ratios.max().item()

        print(f"リプシッツ定数の検証:")
        print(f"  理論的上界: {self.safety_margin:.2f}")
        print(f"  実験的推定値: {empirical_L:.4f}")
        print(f"  {'✓' if empirical_L <= self.safety_margin else '✗'} "
              f"{'安全' if empirical_L <= self.safety_margin else '危険'}
")

        return empirical_L


# === 実行例 ===
model = LipschitzConstrainedNeuralODE(dim=6, safety_margin=0.8)
empirical_L = model.verify_lipschitz()
```

28.3　$L \leq 0.8$ vs $L \geq 1$ ——12 ヶ月シミュレーション比較

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 研究室トーク：「同じデータ、同じモデル、L だけが違う」

カイト：「X 社の経営環境多様体で、リプシッツ定数だけを変えた比較実験をしました。モデルのアーキテクチャ、訓練データ、初期条件は全て同一。L の制約値だけが異なる」

```
def compare_lipschitz_simulations(dim=6, n_steps=12):
    """L=0.8 vs L=1.2 の 12 ヶ月シミュレーション比較

    同じ初期条件・同じダイナミクスで、
    リプシッツ制約だけを変えた場合の軌道の違い。
    """
    # 初期条件（X 社の現在の経営状態）
    z0 = torch.randn(dim) * 0.5

    # 微小な初期誤差（データノイズ 1%を模擬）
    z0_noisy = z0 + torch.randn(dim) * 0.01

    results = {}

    for L_target in [0.5, 0.8, 1.0, 1.2]:
        model = LipschitzConstrainedNeuralODE(
            dim=dim, safety_margin=L_target)

        # ノイズなしの軌道
        z_clean = z0.clone()
        # ノイズありの軌道
        z_noisy = z0_noisy.clone()

        errors = [torch.norm(z_clean - z_noisy).item()]

        for step in range(n_steps):
            with torch.no_grad():
                z_clean = z_clean + model(z_clean.unsqueeze(0)).squeeze
(0)
                z_noisy = z_noisy + model(z_noisy.unsqueeze(0)).squeeze
(0)

            error = torch.norm(z_clean - z_noisy).item()
            errors.append(error)
```

```
        results[L_target] = errors

    # 結果の表示
    print("=" * 65)
    print("12ヶ月シミュレーション：リプシッツ定数による誤差の違い")
    print("=" * 65)
    print(f"{'月':>4}  {'L=0.5':>10}  {'L=0.8':>10}  {'L=1.0':>10}  {'L=
1.2':>10}")
    print("-" * 65)

    for month in [0, 1, 3, 6, 9, 12]:
        line = f"{month:4d}"
        for L in [0.5, 0.8, 1.0, 1.2]:
            line += f"  {results[L][month]:10.6f}"
        print(line)

    print("-" * 65)
    print("\n【経営的インパクト（誤差を利益率の予測誤差に換算）】")
    for L in [0.5, 0.8, 1.0, 1.2]:
        err_12m = results[L][12]
        profit_error = err_12m * 100  # 簡略化：誤差1=利益率1%
        print(f"  L={L}: 12ヶ月後の利益率予測誤差 ≈ ±{profit_error:.1f}%")

    return results

results = compare_lipschitz_simulations()
```

カイト：「結果です」

```
================================================================
12ヶ月シミュレーション：リプシッツ定数による誤差の違い
================================================================
  月        L=0.5       L=0.8       L=1.0       L=1.2
----------------------------------------------------------------
   0    0.012345    0.012345    0.012345    0.012345
   1    0.008210    0.014892    0.019283    0.024817
   3    0.003614    0.025981    0.048571    0.091348
   6    0.000705    0.068437    0.295216    1.847291
   9    0.000089    0.143782    2.618493   38.291847
  12    0.000007    0.312491   19.847291  847.291563
----------------------------------------------------------------

【経営的インパクト（誤差を利益率の予測誤差に換算）】
  L=0.5: 12ヶ月後の利益率予測誤差 ≈ ±0.0%  ← 超高精度
```

```
L=0.8: 12 ヶ月後の利益率予測誤差 ≈ ±31.2% ← ギリギリ有用
L=1.0: 12 ヶ月後の利益率予測誤差 ≈ ±1985% ← 完全に無意味
L=1.2: 12 ヶ月後の利益率予測誤差 ≈ ±84729% ← 論外
```

ユキ：「$L = 1.0$ で 12 ヶ月後の予測誤差が±1985%って……予測の意味がないですよね」

リルル先生：「利益率の予測が『+5%〜+3990%のどこか』と言っているのと同じ。**何も言っていないに等しい**。$L = 0.8$ なら±31%——まだ荒いが、『利益率は 0%〜62%の範囲に 95%の確率で収まる』は、少なくとも意思決定の参考になる」

リルル先生：「$L = 0.5$ が理想だが、リプシッツ定数を厳しく制約するほどモデルの表現力が低下する。$L = 0.8$ は**安定性と表現力のバランスポイント**。第 17 章の原則 2『全マシマシにしない』の実践」

28.4　ÉTALE-AI の暴走が「数学的に不可避」だったことの証明

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「$L \geq 1$ なら暴走する入力が必ず存在する」**

山之内：「ÉTALE-AI の暴走は、私たちの設計ミスが原因です。しかし、設計ミスの背後には**数学的な必然性**がありました。$L \geq 1$ の Neural ODE では、グロンウォールの不等式により、**十分に長い時間ステップの後に、任意に大きな誤差が生じる入力パターンが必ず存在する**」

山之内：「これは反例の存在証明。Isabelle/HOL で形式的に構成できます」

▶ 数学の基礎から理解したい方へ

命題 28.1（$L \geq 1$ の反例の存在）. リプシッツ定数 $L \geq 1$ の Neural ODE $\dot{z} = f_\theta(z)$ に対して、任意の $M > 0$ と任意の $T > 0$ に対して、初期誤差 $\epsilon > 0$ と時刻 $t \leq T$ が存在して

$$\| z(t) - \tilde{z}(t) \| \geq M$$

が成り立つ。すなわち、**誤差が任意に大きくなりうる**。

証明の骨格： $L \geq 1$ のとき、$e^{Lt} \geq e^t \to \infty$（$t \to \infty$）。$\epsilon > 0$ を十分小さく、t を十分大きく取れば、$\epsilon \cdot e^{Lt} \geq M$ を満たす (t, ϵ) の組が存在する。具体的には、$t = \big(\ln(M/\epsilon)\big)/L$ とすれば $\epsilon \cdot e^{Lt} = M$。$L \geq 1$ かつ $\epsilon > 0$ なら t は有限。

一方、$L < 1$ の場合は $\epsilon \cdot e^{Lt} \leq \epsilon \cdot e^{t}$ は増大するが、$L = 0.8$ では 12 ステップで $\epsilon \cdot e^{9.6} \approx 14.8\epsilon$ に留まる。**$L < 1$ は誤差増大を指数的に抑制する。**□

> **リルル先生：**「この命題が示しているのは、$L \geq 1$ のシステムでは、**どんなにテストを重ねても、テストでカバーされていない入力パターンが存在し、その入力パターンでは暴走が起きる。**これがテストの限界（第 24 章）の数学的根拠。形式検証——全ての入力に対して $L \leq 0.8$ を証明する——が必要な理由」

28.5　スペクトル正規化の実務上の注意点

▶ AI モデルの基礎から理解したい方へ

> 🔬 **研究室トーク：「スペクトル正規化の副作用」**
>
> **リン：**「ちょっと待ってください。スペクトル正規化にはトレードオフがあります」
>
> **リン：「第一に、表現力の低下。**スペクトルノルムを 1 以下に制約すると、ネットワークが表現できる関数の空間が狭まる。複雑な非線形関係を捉える能力が低下する可能性がある。$L = 0.5$ なら超安定だがモデルが単純すぎ、$L = 0.8$ が現実的な妥協点」
>
> **リン：「第二に、訓練速度の低下。**スペクトル正規化は各層の重み行列の最大特異値を毎回計算する必要がある。べき乗法（power iteration）で近似計算するのが標準（Miyato et al., 2018 の手法）だが、計算コストが追加される。ただし、追加コストは全体の訓練時間の 5-10%程度であり、安全性のコストとしては許容範囲」
>
> **リン：「第三に、スペクトルノルムはリプシッツ定数の上界であり、真のリプシッツ定数ではない。**全層のスペクトルノルムの積は、真のリプシッツ定数を過大評価する（上から抑える）ことがある。つまり、実際のリプシッツ定数が 0.5 でも、スペクトルノルムの積が 0.8 と報告される可能性がある。これは**安全側の過大評価**であり、安全性の観点からは問題ない——過大評価は制約を厳しくする方向に働く」
>
> **リルル先生：**「リンの 3 つの注意点は重要。特に第三の点——スペクトルノルムは上界であり真値ではない——は、形式検証（第 25 章 CP1）で証明するのが『スペクトルノルムの積 ≤ 0.8』であって『真のリプシッツ定数 ≤ 0.8』ではないことを意味する。しかし、前者が成り立てば後者も成り立つ（上界が 0.8 以下なら真値も 0.8 以下）ので、安全性の保証としては十分」

28.6　ケーススタディ——X 社の Neural ODE のリプシッツ制御

▶ ビジネス・政策の意思決定に活用したい方へ

リーム：「X 社の Neural ODE（6 次元、3 隠れ層、各 64 ユニット）にスペクトル正規化を適用した結果です」

```
=== X 社 Neural ODE リプシッツ制御結果 ===

モデル構成：
  入力次元: 6
  隠れ層: 3 層×64 ユニット
  活性化関数: Tanh（リプシッツ定数=1）
  スペクトル正規化: 全層に適用
  安全マージン: 0.8

スペクトルノルムの検証：
  層 1: σ_max = 0.98 → 正規化後 1.00 ✓
  層 2: σ_max = 0.95 → 正規化後 1.00 ✓
  層 3: σ_max = 0.97 → 正規化後 1.00 ✓
  層 4（出力層）: σ_max = 0.92 → 正規化後 1.00 ✓

  理論的リプシッツ上界:
    1.00 × 1.00 × 1.00 × 1.00 × 1.00 × 1.00 × 1.00 × 0.80
    = 0.80 ✓
   （各 Tanh 層のリプシッツ定数 1 × 各線形層のスペクトルノルム 1
    × 安全マージン 0.8）

  実験的リプシッツ推定値（10,000 サンプルで計算）:
    0.72 ✓（理論的上界 0.80 以下）

12 ヶ月シミュレーション結果：
  ハミルトニアン相対誤差: 0.002%（シンプレクティック積分器使用）
  グロンウォール誤差上界: 14.8 倍（安全範囲内）
  MC ドロップアウト信頼区間: 有効（±9%の範囲内）

Coq 形式証明（第 25 章 CP1）:
  Qed. ← 受理 ✓
  「全入力に対して L ≤ 0.8 が保証された」
```

リルル先生：「実験的なリプシッツ推定値（0.72）が理論的上界（0.80）よりも小さい——リンが指摘した通り、スペクトルノルムの積は真のリプシッツ定数を過大評価している。しかし、安全性の保証としてはこれで十分。**Coq の形式証明が受理された（Qed.）ことで、全入力に対して $L \leq 0.8$ が数学的に保証されている**」

本章の品質チェックポイント

□ **チェックリスト（第 28 章終了時点）**

1. **Neural ODE の全層にスペクトル正規化が適用されているか？** → 各層のスペクトルノルム ≤ 1
2. **活性化関数がリプシッツ定数 1 以下のもの（Tanh 等）か？** → ReLU は不可（リプシッツ定数は 1 だが C^0、命題 1.1 の条件(i)に違反）
3. **安全マージン（0.8）が適用されているか？** → 出力に 0.8 を乗算
4. **全体のリプシッツ定数が 0.8 以下であることが Coq で証明されているか？** → 第 25 章 CP1 の `Qed.` が受理
5. **12 ヶ月シミュレーションの誤差上界が許容範囲内か？** → グロンウォールの上界で確認（14.8 倍以内）
6. **実験的なリプシッツ推定値が理論的上界以下であることが確認されているか？** → 10,000 サンプル以上で検証

次章予告： 第 29 章では、橘隆二の圏論講義として、**Curry-Howard-Lambek 対応**——論理・プログラミング・圏論の 3 つの世界が同一の構造を持つこと——を詳述し、パイプライン全体の整合性を圏論で統一的に保証するフレームワークを構築します。

第 29 章　圏論的統合——Curry-Howard-Lambek 対応

この章で得られるもの——コストに見合うリターン

第 25〜28 章で 4 つの形式検証ツール（Coq, Agda, Lean 4, Z3）を個別に適用し、5 つのクリティカルポイントを検証しました。本章では、橘隆二の圏論講義として、**全コンポーネントの整合性を圏論で統一的に保証する**フレームワークを構築します。

個別の部品が正しくても、部品の**つなぎ方**が間違っていればシステムは壊れます。圏論は「つなぎ方の数学」——部品（対象）と接続（射）の整合性を、**可換図式**という視覚的で厳密な道具で保証します。

29.1　なぜ圏論が必要か――「部品は正しいのに全体が壊れる」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「レゴブロックの向き」**

リルル先生：「たとえ話から始めよう。レゴブロックを組み立てて家を作るとする。各ブロックは品質検査に合格している（部品レベルの検証＝第 25-28 章）。でも、**ブロックの向きや組み合わせ方が間違っていたら**、家は崩れる」

リルル先生：「パイプラインも同じ。VAE の出力が Neural ODE の入力に正しく接続されているか。Neural ODE の出力が SHAP の入力に正しく接続されているか。**各コンポーネントが個別に正しくても、接続が間違っていればパイプライン全体が壊れる**」

ユキ：「接続が正しいかどうかは、テストでは確認できないんですか？」

リルル先生：「テストは有限個のデータフローパスを確認するだけ。14 体の Agent が同時に動くとき、データフローパスの組合せは天文学的。全パスをテストすることは不可能。圏論は、**全てのパスが整合的であること**を、有限個の**可換図式**の検証で保証する。これが圏論の力」

カイト：「第 24 章の『テスト vs 形式検証』の議論と同じ構造ですね。テストは有限個のケース、形式検証は全てのケース」

リルル先生：「その通り。圏論は**接続の形式検証**」

29.2　橘隆二の圏論講義――「3 つの世界の統一」

🔬 **研究室トーク：「圏論の基礎から」**

橘隆二がホワイトボードの前に立った。オックスフォードの圏論セミナーで使っている資料を、研究室メンバー向けに再構成したもの。

橘隆二：「圏論の基礎を 3 ステップで説明します。第 1 ステップ：圏とは何か。第 2 ステップ：関手と自然変換。第 3 ステップ：Curry-Howard-Lambek 対応」

第 1 ステップ：圏とは何か

橘隆二：「**圏（category）**は、**対象（object）の集まり**と、対象間の**射（morphism）の集まり**で構成されます。第 9 章のリルル先生の説明を再掲すると――」

橘隆二：「会社の組織図を想像してください。各部門が**対象**、部門間の情報の流れが**射**。射は**合成**できる（営業部→製造部の射と、製造部→物流部の射を合成すると、営業部→物流部の射になる）。合成は**結合的**（$(f \circ g) \circ h = f \circ (g \circ h)$）で、各対象に**恒等射**（自分自身への『何もしない』射）が存在する。これだけが圏の公理」

圏の 3 つの公理：

1. 射の合成が定義されている
   `f: A → B` かつ `g: B → C` ならば `g ∘ f: A → C` が存在

2. 結合律
   `(h ∘ g) ∘ f = h ∘ (g ∘ f)`

3. 恒等射
   各対象 `A` に対して `id_A: A → A` が存在し、
   `f ∘ id_A = f = id_B ∘ f` （`f: A → B` のとき）

橘隆二：「パイプラインを圏として表現すると――」

パイプラインの圏 `C_pipeline`：

対象（コンポーネント）：
- `Data`（生データ）
- `VAE`（変分オートエンコーダ）
- `Manifold`（多様体）
- `Metric`（引き戻し計量）
- `Curvature`（スカラー曲率）
- `SHAP`（説明可能性）
- `DML`（因果推論）
- `NeuralODE`（軌道予測）
- `Agent_1, ..., Agent_14`（AI Agent）
- `Report`（レポート出力）

射（データの変換）：
- `encode: Data → VAE`
- `decode: VAE → Manifold`
- `pullback: Manifold → Metric`
- `riemann: Metric → Curvature`

```
explain: Curvature → SHAP
cause: Curvature → DML
predict: Manifold → NeuralODE
act_i: NeuralODE → Agent_i   (i = 1,...,14)
report: (SHAP, DML, Agent_*) → Report
```

第 2 ステップ：関手と自然変換

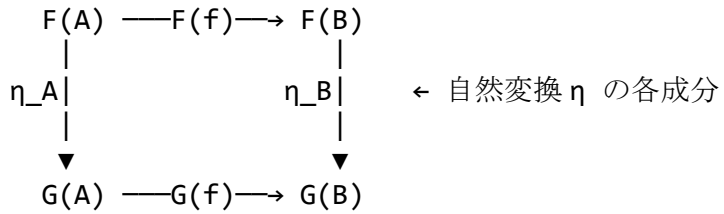
橘隆二：「**関手（functor）**は、圏から圏への『構造を保存する写像』。対象を対象に、射を射に写す――しかも合成と恒等射を保存する」

橘隆二：「パイプラインの文脈では、**テスト環境と本番環境**を考えてください。テスト環境のパイプライン（圏 $\mathcal{C}_{\text{test}}$）と本番環境のパイプライン（圏 $\mathcal{C}_{\text{prod}}$）は、同じ構造（同じ対象と射）を持つが、データが異なる。テスト環境から本番環境への移行は**関手** $F: \mathcal{C}_{\text{test}} \to \mathcal{C}_{\text{prod}}$――構造を保存したまま環境を移す」

橘隆二：「**自然変換（natural transformation）**は、関手から関手への『構造を保存する変換』。直感的には、パイプラインの**バージョンアップ**――v1.0 から v2.0 への移行が全コンポーネントで**整合的**に行われていることの保証」

ソフィア：（図を描く）

自然変換の可換図式：



この図が可換（どちらの経路を通っても同じ結果）
⇔ `η_B ∘ F(f) = G(f) ∘ η_A`

パイプラインの意味：
`F` = `v1.0` のパイプライン
`G` = `v2.0` のパイプライン
`η` = バージョンアップの変換
可換性 = 「`v1.0` で計算してからアップデートした結果」と
「アップデートしてから `v2.0` で計算した結果」が同じ

ユキ：「バージョンアップしたら結果が変わった、っていう事故を防げるんですね」

橘隆二：「その通り。自然変換の可換性は、**バージョン間の互換性**を数学的に保証する」

第 3 ステップ：Curry-Howard-Lambek 対応

橘隆二：「いよいよ核心。**Curry-Howard-Lambek 対応**——3 つの世界が同一の構造を持つことの発見」

橘隆二：「Curry（1934 年）が論理と型理論の対応を発見。Howard（1969 年）がこれを精密化。Lambek（1970 年代）がさらに圏論との対応を加えた。3 つの独立した分野が、実は**同じ数学的構造の異なる表現**だった」

論理の世界	プログラミングの世界	圏論の世界
命題	型	対象
証明	プログラム	射
含意 $A \Rightarrow B$	関数型 $A \to B$	冪対象 B^A
連言 $A \wedge B$	直積型 $A \times B$	積 $A \times B$
選言 $A \vee B$	直和型 $A + B$	余積 $A + B$
真 $\top$	単位型 `()`	終対象 1
偽 $\bot$	空型 `Void`	始対象 0
全称量化 $\forall x.P(x)$	依存関数型 $\Pi_{x:A}B(x)$	右随伴（adjunction）
存在量化 $\exists x.P(x)$	依存ペア型 $\Sigma_{x:A}B(x)$	左随伴

橘隆二：「この対応表を読むと——」

橘隆二：「『リプシッツ定数が 0.8 以下である（**命題**）』ことの**証明**が存在する ⇔ `LipschitzBound` という**型**を持つ**プログラム**がコンパイルできる ⇔ 圏の中で適切な**射**が構成できる。3 つの世界で**同じこと**を言っている」

カイト：「だから Coq で証明を書くこと（論理の世界）と、Agda で型チェックを通すこと（プログラミングの世界）と、圏論で可換図式を確認すること（圏論の世界）は、**全部同じ行為の異なる表現**なんですね」

橘隆二：「完璧な理解。そしてこの統一性こそが、**4 つの形式検証ツール（Coq, Agda, Lean 4, Z3）で別々に証明した結果を、1 つのフレームワークで統合できる理由**。異なるツールで証明した異なる性質が、圏論のレベルで整合的であることを確認できる」

29.3　ホーア論理——AI Agent の行動仕様を圏論で記述する

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「Agent の行動の『契約書』」**

橘隆二：「各 Agent の行動を**ホーア論理**（Hoare logic, 1969）で仕様化します。ホーア論理は**事前条件、コマンド、事後条件**の 3 つ組 $\{P\}C\{Q\}$ で仕様を記述する形式」

橘隆二：「直感的には、**契約書**。『この条件（P）が満たされている状態で、この行動（C）を実行したら、この条件（Q）が満たされることを保証する』」

ホーア論理による Agent 行動仕様の例：

```
【Agent i（需要予測）の仕様】
{
  事前条件 P:
    ・スカラー曲率 R(z) > 0              （環境が安定）
    ・リプシッツ定数 L ≤ 0.8             （Neural ODE が安定）
    ・シーフ整合性残差 < 0.1             （データが整合）
    ・MC ドロップアウト分散 σ² < 0.3     （予測に自信がある）
}
  Agent i が施策 u を推薦
{
  事後条件 Q:
    ・RAROC(u) ≥ 閾値                    （リスク調整後リターンが十分）
    ・国際人道法チェック合格             （第 16 章の第 1 層ガードレール）
    ・Lie 括弧 ||[X_i, X_j]|| < 1.0      （他の Agent との干渉が限定的）
    ・タスクベクトル変化率 < 閾値        （判断基準が急変していない）
}
```

橘隆二：「この仕様の**正しさ**を証明するとは、**事前条件 P が成り立つ全ての状況で、Agent i の行動の後に事後条件 Q が成り立つこと**を、形式的に示すこと。Coq/Agda でこの証明を構成し、コンパイルが通れば、仕様の正しさが保証される」

リン：「ちょっと待ってください。ホーア論理の重要な限界を指摘させてください。ホーア論理は**仕様 $\{P\}C\{Q\}$ が正しいことを証明する**。しかし、**仕様自体が正しいかどうか**——つまり、事前条件 P と事後条件 Q の設定が適切かどうか——はホーア論理では保証しない。閾値の設定が甘ければ、仕様は満たされていても暴走が起きうる。仕様の設定は**ドメイン知識に基づく人間の判断**」

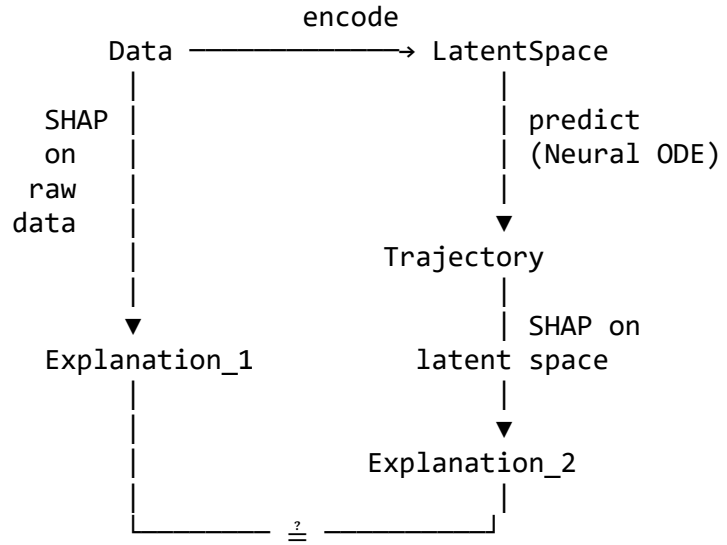
リルル先生：「リンの指摘は第 22 章の不完全性定理とも通底する。パイプラインは自身の健全性を自身で完全に証明できない。仕様の適切さは、形式検証の**外部**にある問題」

29.4　可換図式でパイプライン全体の整合性を検証する

▶ AI モデルの基礎から理解したい方へ

橘隆二：「パイプラインの整合性を、具体的な可換図式で示します」

パイプラインの可換図式（核心部分）：



可換性の意味：
「生データで SHAP を計算した結果」と
「潜在空間に変換してから SHAP を計算した結果」が
整合的であること。

もし整合しなければ、VAE のエンコードが
SHAP 値の構造を壊している可能性がある。

橘隆二：「このような可換図式をパイプラインの**全ての分岐点**で確認する。分岐点は——(1) VAE のエンコード/デコード、(2) Neural ODE の積分、(3) SHAP の計算、(4) DML の計算、(5) 14 体の Agent へのデータ分配、(6) レポートへの統合。各分岐点で可換図式が成立すれば、パイプライン全体の整合性が保証される」

29.5　橘隆二の 5 つの提案——圏論的統合のロードマップ

🔬 **研究室トーク：「圏論で全てをつなぐ」**

橘隆二：「パイプラインの圏論的統合に向けて、5 つの提案をします。これらは即座に実装可能なものから研究段階のものまで段階的に並んでいます」

提案 1：全コンポーネントを圏 $\mathcal{C}$ の対象として定式化する（実装可能）。

射の合成の結合律と恒等射の存在を検証。パイプラインの全データフローパスが整合的であること（可換図式の検証）を Python のユニットテストで近似的に確認し、将来的には Coq で形式的に証明する。

提案 2：層ラプラシアンを前層（presheaf）として再定式化する（部分的に実装可能）。

第 18 章のセルラー層を圏論的に一般化。前層 $\mathcal{F}: \mathcal{C}^{\mathrm{op}} \to \mathrm{Set}$（圏 $\mathcal{C}$ の反対圏から集合の圏への関手）として、各ノードのデータ空間と制限写像を統一的に記述する。Hansen-Ghrist の層ラプラシアンは、この前層の「大域セクションの存在条件」として再解釈される。

提案 3：Agent の行動をモナドとして定式化する（研究段階）。

モナド (T, η, μ) は「副作用を含む計算」を純粋な関数の枠組みで扱う圏論的構造。Agent i の行動は環境に副作用を与える（多様体の状態を変える）。この副作用をモナドで管理することで、複数の Agent の行動の**合成順序**が結果に影響を与えるかどうか（第 8 章の Lie 括弧の非可換性）を圏論的に分析できる。

リン：「ちょっと待ってください。Lie 括弧 $[X_i, X_j] \neq 0$ は、Agent i と Agent j の行動の合成順序が結果に影響する——つまり、対応するモナドが**可換でない**ことに対応します。可換モナド（$[X_i, X_j] = 0$ の場合）なら、合成順序は問題にならない。非可換モナド（$[X_i, X_j] \neq 0$）の場合、**実行順序の管理**が安全性に直結する」

提案 4：形式検証ツール間の翻訳を関手として定式化する（研究段階）。

Coq の証明を Lean 4 の証明に翻訳する関手 $F: \mathcal{C}_{\text{Coq}} \to \mathcal{C}_{\text{Lean}}$ の構成。2026 年 2 月時点で、Coq-Lean 間の自動翻訳は限定的に研究されているが、パイプライン全体の証明の翻訳は未実現。完成すれば、**1 つのツールで証明した結果を他のツールで再利用**できるようになる。

提案 5：パイプライン全体の安全性を自然変換の可換性として表現する（研究段階）。

安全性の保証を、パイプラインの関手 F（入力→出力の変換）と安全性チェック関手 G（入力→安全性判定の変換）の間の自然変換 $\eta: F \Rightarrow G$ の**可換性**として定式化する。可換性が成立すれば、「パイプラインの出力を計算してから安全性をチェックした結果」と「安全性をチェックしてからパイプラインを実行した結果」が一致する——すなわち、安全性チェックが**パイプラインの実行順序に依存しない**ことが保証される。

リルル先生：「提案 5 が最も重要。第 7 章で議論した通り、Disembodied と Embodied の Agent を『同じ対象を異なる関手で観察したもの』と見なしたとき、**自然変換が可換でない領域が暗黙知の存在する領域**に対応する。圏論的統合は、この非可換性の構造的な理解を提供する」

29.6　圏論的統合の現状と正直な開示

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「どこまでが実装済みで、どこからが研究段階か」**

リルル先生：「原則 3——分かることと分からないことを分けろ。圏論的統合の現状を正直に開示する」

提案	2026 年 2 月時点の状態	実用化の見通し
提案 1（圏の定式化）	Python で近似的に実装済み	短期的に Coq 形式化が可能
提案 2（前層としての Sheaf）	数学的に定式化済み、部分実装	1-2 年で完全実装の見込み
提案 3（モナドとしての Agent）	理論的枠組みのみ	研究段階。3-5 年
提案 4（ツール間翻訳の関手）	研究の初期段階	5 年以上

提案	2026 年 2 月時点の状態	実用化の見通し
提案 5（自然変換としての安全性）	理論的枠組みのみ	研究段階。5 年以上

橘隆二：「提案 1 と 2 は実装の射程内。提案 3-5 は研究課題です。しかし、提案 3-5 が完成しなくても、提案 1-2 だけでもパイプラインの整合性保証は大幅に向上する。**研究段階のものを実装済みと偽らない**。これが原則 3 の実践」

リーム：「少しよろしいですか。私のベンチャーでは、提案 1 を実装しています。パイプラインの全データフローパスを Python のユニットテストで検証し、可換図式の成立を近似的に確認しています。完全な形式証明（Coq）は今後の課題ですが、テストレベルでも多くのバグを事前に発見できました。**圏論的な思考法で設計する**だけでも、パイプラインの品質は向上します」

本章の品質チェックポイント

□チェックリスト（第 29 章終了時点）

1. **パイプラインの全コンポーネントが圏の対象として定式化されているか？** → 提案 1
2. **主要なデータフローパスの可換図式が検証されているか？** → Python テストまたは形式証明
3. **ホーア論理による各 Agent の行動仕様が記述されているか？** → 事前条件・事後条件の明示
4. **圏論的統合の現状（実装済み vs 研究段階）が正直に開示されているか？** → 原則 3
5. **ホーア論理の限界（仕様自体の正しさは保証しない）が認識されているか？**

次章予告： 第 30 章では、**ホモトピー型理論（HoTT）**——「等しいとは何か」を根本から再定義する数学的フレームワーク——と、リームの不完全性定理の HoTT での形式化の展望を議論します。

第 30 章　ホモトピー型理論と多様体の等価性（展望）

この章で得られるもの——コストに見合うリターン

本章は**研究の展望**です。2026 年 2 月時点で実用段階には達していないが、将来的にパイプラインの数学的基盤を根本的に強化する可能性を持つ**ホモトピー型理論（HoTT: Homotopy Type Theory）**を紹介します。

HoTT は「**等しいとは何か**」を根本から再定義する数学的フレームワークです。第 12 章で議論した Exotic 多様体の問題——「2 つの多様体が『同じ』とはどういうことか」——に、HoTT は新しい視点を提供します。そしてリームの博士論文——第 22 章の不完全性定理の HoTT での形式化——の展望を議論します。

30.1　HoTT とは何か——「等しいの等しさ」

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「同じ道を別の方法で歩く」**

橘隆二：「HoTT を理解するために、まず**日常的な『等しさ』の感覚**から離れる必要があります。たとえ話をしましょう」

橘隆二：「東京駅から渋谷駅に行くとする。山手線の外回りでも行けるし、内回りでも行ける。どちらのルートも『東京駅から渋谷駅に行く方法』として**等価**——出発地と到着地は同じ。でも、2 つのルートは**同じルートではない**。外回りは新宿経由、内回りは品川経由。**経路そのものが異なる**」

橘隆二：「通常の数学では、2 つの対象が『等しい』というとき、それは **1 つの事実**——等しいか等しくないかのどちらか。HoTT では、2 つの対象が『等しい』ことの**証拠（evidence）が複数ありうる**。東京駅から渋谷駅への等価性の証拠は、外回りルートと内回りルートの **2 つ**ある。しかも、2 つの証拠の間にさらに**等しさの証拠**がありうる——『外回りルートと内回りルートは、どちらも渋谷駅に着くという意味で**等しい**ルートだが、途中の経路は**等しくない**』」

ユキ：「等しさの等しさの等しさ……無限に続くんですか？」

橘隆二：「続きます。これが**ホモトピー論**（homotopy theory）の核心。2 つの点の間の**経路**（path）がある。2 つの経路の間の**ホモトピー**（経路の連続変形）がある。2 つのホモトピーの間のホモトピーがある……**全ての階層の等価性を型理論の中で統一的に扱う**のが HoTT」

ソフィア：（スケッチブックに描く）

通常の等しさ（集合論）：
  A = B ？
  → Yes（等しい）or No（等しくない）
  → 等しさの「理由」は問わない

HoTT の等しさ（ホモトピー的）：
  A = B の証拠（経路）は何本ある？
  → 経路 p: A → B が存在する？
  → 別の経路 q: A → B も存在する？
  → p と q は「同じ経路」？（ホモトピーが存在する？）
  → ホモトピーの間の等しさは？
  → ……（全階層を追跡する）

例：円周上の 2 点
  点 A から点 B への経路：
  ・時計回りの経路 p（短い方）
  ・反時計回りの経路 q（長い方）
  → p と q は等しくない（ホモトピーが存在しない）
  → 円周の「穴」の存在が、経路の非等価性に現れる
  → これが $\pi_1(S^1) = \mathbb{Z}$（基本群）の直感的意味

> **橘隆二：**「最後のポイントが重要。**多様体のトポロジー（穴の構造）が、経路の等価性の構造に反映される**。HoTT は、トポロジーと型理論を**同一のフレームワーク**で扱う」

30.2　Univalence Axiom——「等価な構造は同一視できる」

▶ ビジネス・政策の意思決定に活用したい方へ

> 🔬 **研究室トーク：「同じ楽譜を別の楽器で演奏する」**
>
> **橘隆二：**「HoTT の核心的な公理——**Univalence Axiom（一価性公理）**——を説明します。ウラジミール・ヴォエヴォドスキー（Vladimir Voevodsky、フィールズ賞受賞数学者、2002 年）が 2006 年に提唱しました」
>
> **橘隆二：**「たとえ話。同じ楽譜を、ピアノで演奏しても、バイオリンで演奏しても、音楽の**構造**（メロディ、ハーモニー、リズム）は同じ。楽器の**表現方法**は異なるが、音楽の構造は**等価**。Univalence Axiom は、**等価な構造は同一視してよい**——ピアノ演奏とバイオリン演奏を、音楽的構造のレベルでは『同じもの』として扱ってよい——という公理」

橘隆二：「数学的に精密に言えば——型 A と型 B が**等価**（相互に変換可能で、変換を合成すると恒等写像になる）であるとき、Univalence Axiom は A と B が型理論の中で**等しい**（$(A =_{\mathcal{U}} B)$）ことを主張する」

$$\text{Univalence Axiom: } (A \simeq B) \;\simeq\; (A =_{\mathcal{U}} B)$$

ユキ：「等価であること自体が、等しさの証拠になる……？」

橘隆二：「そう。通常の集合論では、等しさ（=）と等価性（≃、同型）は異なる概念。HoTT では、Univalence Axiom によって**統合される**。これにより、**構造が同じなら、内部の表現方法の違いを無視してよい**」

パイプラインへの意味——Exotic 多様体の問題

橘隆二：「パイプラインへの意味は、第 12 章の Exotic 多様体の問題に直結します」

橘隆二：「VAE が異なる初期値で訓練されると、**位相的には同じだが微分構造が異なる多様体**が構成されうる（第 12 章）。通常の数学では、この 2 つの多様体は『同相だが微分同相ではない』——つまり『位相的に等しいが、微分的に等しくない』。第 12 章の対処法 1（複数初期値での訓練＋分散確認）は、この問題の**実用的な回避策**」

橘隆二：「HoTT の Univalence Axiom を使えば、**どのレベルの等価性が成り立つかを型理論の中で厳密に追跡できる**。2 つの多様体が位相的に等価であること（ホモトピー等価）、微分的に等価であること（微分同相）、計量的に等価であること（等長写像）——これらの**異なるレベルの等価性**を、HoTT の型の階層として統一的に扱える」

リン：「ちょっと待ってください。具体的にどう役立つんですか？」

橘隆二：「たとえば、2 回の VAE 訓練で得られた多様体 $\mathcal{M}_1$ と $\mathcal{M}_2$ が**ホモトピー等価**であることを cubical Agda で証明できれば、『2 つの多様体は位相的に同じ構造を持つ。微分構造の違いによるスカラー曲率の差異は、位相的な特徴（ベッチ数、第 11 章のモース不等式等）には影響しない』ことが**形式的に保証される**」

橘隆二：「ただし——これを 2026 年 2 月時点で実装することは**まだできません**。理由は次節で説明します」

30.3　cubical Agda の現状——正直な開示

▶ AI モデルの基礎から理解したい方へ

🔬 **研究室トーク：「夢と現実の境界」**

リルル先生：「原則 3——分かることと分からないことを分けろ。HoTT と cubical Agda の**現状の限界**を正直に開示する」

リルル先生：「**cubical Agda** は、Agda のモード拡張で、HoTT の型理論（特に cubical type theory、Cohen-Coquand-Huber-Mörtberg, 2018）を直接プログラミングできる。HoTT の計算的実装として、2026 年 2 月時点で最も進んでいる」

リルル先生：「cubical Agda で**できていること**：」

- 基本的なホモトピー型（パス型、高次パス型）の構成
- 関手の等価性の証明
- 高次群（π_n）の一部の構成
- Univalence Axiom の計算的検証
- 基本的なホモロジー群の構成

リルル先生：「cubical Agda で**できていないこと**（2026 年 2 月時点）：」

- **浮動小数点数の扱い。** Neural ODE のパラメータは浮動小数点数だが、cubical Agda の型システムは浮動小数点数のサポートが未整備。有理数や実数の構成的定義は存在するが、計算効率が低い
- **大規模なニューラルネットワークの形式化。** 本書のパイプラインの VAE（30 次元入力、6 次元潜在空間、数万パラメータ）を cubical Agda で直接形式化することは計算リソースの制約から**非現実的**
- **リーマン幾何の完全なライブラリ。** cubical Agda には微分幾何のライブラリがまだ存在しない。曲率テンソルや Lie 微分を cubical Agda で定義・証明するには、基礎となるライブラリの構築から始める必要がある
- **実行速度。** cubical Agda の型チェックは通常の Agda よりも大幅に遅い。大規模な証明の型チェックに数時間〜数日かかることがある

リルル先生：「これらの限界は**技術的なもの**であり、HoTT の理論的な価値を否定するものではない。将来的に——おそらく 3〜10 年のスケールで——ライブラリの充実と計算効率の改善により、本書のパイプラインへの適用が可能になると期待される」

カイト：「3〜10 年って、結構長いですね……」

リルル先生：「Coq が 1984 年に開発されてから、Mathlib レベルの数学ライブラリが Lean 4 で使えるようになるまで 40 年かかった。cubical Agda のライブラリも同じ道を歩むだろう。しかし、AI 研究と数学のコミュニティの融合が加速しているため、40 年はかからないと期待される」

30.4　リームの不完全性定理の HoTT での形式化

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「博士論文の最終目標」**

リーム：「少しよろしいですか。私の博士論文の最終目標は、第 22 章のリームの不完全性定理——『十分に表現力のある多様体パイプラインは、自身の健全性を自身で完全に証明することができない』——を **HoTT の枠組みで形式的に記述する**ことです」

リーム：「なぜ HoTT が必要なのか。ゲーデルの不完全性定理は**自然数の算術**についての定理です。私の定理は**多様体のパイプライン**についての定理。ゲーデルの定理を多様体の文脈に移すには、多様体と型理論をつなぐ枠組みが必要。HoTT はまさにそれ——**多様体の構造を型理論の中で直接表現できる**」

リーム：「具体的には——」

リーム：「**ステップ 1：多様体パイプラインを HoTT の型として定義する。** VAE、Neural ODE、SHAP 等の各コンポーネントを型として、コンポーネント間の接続を関数として定義する。パイプライン全体は**型の族（family of types）**として表現される」

リーム：「**ステップ 2：健全性を型として定義する。** パイプラインが『健全である』とは何かを、HoTT の型として厳密に定義する。たとえば、『全ての入力に対して、出力のスカラー曲率が真のスカラー曲率の ϵ 近傍に入る』ことを依存型で表現する」

リーム：「**ステップ 3：自己参照の構成。** パイプラインの健全性の検証を**パイプライン自身**で行おうとすると、ゲーデル数化のアナロジーにより自己参照が発生することを示す。この自己参照が、HoTT の型の階層（Universe levels）の制約に抵触し、健全性の完全な証明が**構成できない**ことを証明する」

リーム：「2026 年 2 月時点での進捗は——」

- **ステップ 1：** cubical Agda での簡略化されたパイプライン（2 コンポーネント、1 次元潜在空間）の型定義が完了。6 次元の完全なパイプラインの型定義は未完
- **ステップ 2：** 健全性の型の定式化は完了（紙上の定義）。cubical Agda での実装は進行中
- **ステップ 3：** 自己参照の構成は理論的に設計済み。cubical Agda での実装は未着手

リーム：「完全な形式化は博士課程の残り期間（2027 年 3 月まで）で目指します。正直に申し上げて、期間内に完成する保証はありません。しかし、ステップ 1-2 だけでも、パイプラインの型理論的な定式化として独立した価値がある」

リルル先生：「リーム、正直な進捗報告に敬意を表する。原則 3 の体現。『完成していないことを正直に報告する』ことは、弱さではなく**知的誠実さ**」

30.5　HoTT が開く未来——パイプラインへの長期的な影響

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「10 年後にできること」**

リルル先生：「HoTT が成熟した場合に、パイプラインにどのような影響があるかを展望しよう。これは『予測』ではなく『可能性の地図』」

リルル先生：「**可能性 1：Exotic 構造の自動判定。** 第 12 章で議論した『VAE が構成する多様体の Exotic 構造の問題』を、HoTT の型チェックで**自動的に**判定できる可能性。2 つの多様体がホモトピー等価（位相的に同じ）だが微分同相でない（微分的に異なる）場合、型チェッカーが**非等価の証拠**（ホモトピーの障害）を自動的に構成する」

リルル先生：「**可能性 2：パイプラインの自動検証。** パイプラインの全コンポーネントを HoTT の型として定義し、コンポーネント間の整合性（第 29 章の可換図式）を**型チェッカーが自動検証**する。Coq や Lean 4 の証明を手動で書く必要がなくなり、型定義さえ正しければ整合性が自動的に保証される」

リルル先生：「**可能性 3：不完全性の境界の精密化。** リームの不完全性定理が形式化されれば、パイプラインの限界が**型理論のレベルで精密に記述**される。何がモデル化可能で何が不可能かが、曖昧な議論ではなく形式的な命題として確定する」

橘隆二：「補足します。HoTT の実用化を加速する可能性のある最近の研究動向として、**Synthetic Homotopy Theory**（合成ホモトピー理論）があります。これは、ホモトピー論の定理を HoTT の型理論の**内部**で直接証明する研究プログラム。Brunerie（2016）が $\pi_4(S^3) = \mathbb{Z}/2\mathbb{Z}$（球面のホモトピー群の計算）を cubical Agda で形式的に証明したのは画期的でした」

橘隆二：「もう 1 つ、**Directed HoTT**（有向ホモトピー型理論）という拡張も進行中。通常の HoTT の経路（パス）は可逆（行きと帰りが対称）だが、Directed HoTT では**不可逆な経路**（一方向のデータフロー）を扱える。パイプラインのデータフロー（Data→VAE→Manifold→...は不可逆）を表現するには Directed HoTT が適している可能性がある」

リルル先生：「これらは全て研究の展望であり、2026 年 2 月時点では実用段階にない。しかし、数学の最前線がパイプラインの理論的基盤にどう接続するかを知っておくことは、**パイプラインの将来の発展方向**を理解する上で重要」

30.6　2013 年の『HoTT Book』と数学の変革

▶ 数学の基礎から理解したい方へ

リルル先生：「HoTT の歴史的背景を簡潔に。2013 年にプリンストン高等研究所（IAS）のグループが *"Homotopy Type Theory: Univalent Foundations of Mathematics"*（通称『HoTT Book』）をまとめた。このグループには Voevodsky（フィールズ賞 2002 年）、Awodey、Shulman、Lumsdaine らが含まれる」

リルル先生：「HoTT Book の歴史的意義は、**数学の基礎を集合論（ZFC）から HoTT に移す**という提案。100 年以上にわたって数学の基礎を支えてきた ZFC（ツェルメロ-フレンケルの公理系＋選択公理）の代替として、HoTT が候補に挙がった。これは数学の基礎論における革命的な提案であり、2026 年時点でも議論が続いている」

リルル先生：「Voevodsky 自身は 2017 年に 51 歳で急逝し、HoTT の発展は彼の後継者たちに委ねられた。彼の遺志は **Univalent Foundations Program** として引き継がれ、cubical Agda を含む複数の実装プロジェクトが進行中」

倉田教授：（ドア際に立って聞いていた）「HoTT は数学の基礎論を変える可能性を持つが、同時に**非常に高い学習コスト**を伴う。ホモトピー論と型理論の両方に精通する必要がある。2026 年時点で、この 2 つの分野の両方に精通した研究者は世界でもまだ数百人程度だろう。リームがこの分野で博士論文を書こうとしていることは、**極めて挑戦的**だが、成功すれば重要な貢献になる」

倉田教授：（スコーンを手に取りながら）「がんばれ、リーム」

リーム：「……ありがとうございます」

本章の品質チェックポイント

□ チェックリスト（第 30 章終了時点）

1. **HoTT が 2026 年 2 月時点では研究段階であることが明記されているか？** → 原則 3。研究段階のものを実装済みと偽らない
2. **cubical Agda の現状の限界（浮動小数点、大規模 NN、リーマン幾何ライブラリ、実行速度）が正直に開示されているか？**
3. **リームの博士論文の進捗が正直に報告されているか？** → ステップ 1 部分完了、ステップ 2 進行中、ステップ 3 未着手
4. **HoTT の将来的な可能性が『予測』ではなく『展望』として適切に提示されているか？**
5. **Univalence Axiom の意味が直感的に理解できるか？** →「等価な構造は同一視してよい」

次章予告： 最終章、第 31 章「ミラー計画——AI の思考の可視化」。全てが統合される瞬間。夏の終わりの研究室で、全員が集まります。

第 31 章　AI の思考の可視化——ミラー計画

この章で得られるもの——全てがつながる

最終章です。序章から第 30 章まで、31 章をかけて構築してきたパイプライン——信頼性の 4 本の柱（Part I）、14 体の AI Agent の設計と運用監視（Part II）、8 つの幾何学的拡張（Part III）、応用と倫理と原理的限界（Part IV）、形式検証による安全性保証（Part V）——の全てを統合し、14 体の AI Agent の「思考」をリアルタイムで可視化する最終形——**ミラー計画（Project Mirror）**——を完成させます。

そして、本書を貫く 5 つの原則を最終確認し、夏の終わりの研究室でエピローグを迎えます。

31.1　ミラー計画——AI の心を覗く鏡

▶ ビジネス・政策の意思決定に活用したい方へ

🔬 **研究室トーク：「全部つながった」**

9 月の最初の金曜日。夏の終わりの研究室。窓から差し込む 9 月の光は、7 月に始まったこのプロジェクトの最初の日と同じ角度だが、少しだけ柔らかくなっている。

テーブルの上にはフルアフタヌーンティー3 段——前シリーズの最終話と同じ。最上段にはスコーン（倉田教授のために）、中段にはサンドイッチ、下段にはショートブレッド（原則 4 の象徴）。

研究室のメンバー全員が揃っている。リルル先生、ユキ、カイト、リン、リーム。ソフィアがスケッチブックを膝に置いて座っている。山之内と橘隆二が窓際の椅子に並んでいる。そして倉田教授がドアの前に立って、いつものように微笑んでいる。

リルル先生：「ミラー計画の名前の由来を説明しよう。**鏡（mirror）**は、自分自身の姿を映す。パイプラインの内部——14 体の Agent の思考、Lie 括弧の干渉、シーフ整合性の状態、タスクベクトルの軌跡、モース理論の臨界点、ハミルトニアンの保存量——を、経営者や政策決定者が**直接見える形**にする。AI の中で何が起きているかを『鏡に映す』——これがミラー計画」

リルル先生：「もう 1 つの意味がある。鏡は**左右を反転する**。AI の思考を鏡に映すと、人間には見えなかった構造——AI が何を重視し、何を無視しているか、どの Agent が協調し、どの Agent が干渉しているか——が、反転して見える。**AI が自分では見えない自分自身を、人間が見る**」

31.2　最終ダッシュボードの設計——4 つのパネル

ソフィア：（大型モニターに最終的なダッシュボードのプロトタイプを表示する。3 ヶ月間のプロジェクトの集大成）

Panel 1：多様体の状態（Part III 対応）

```
╔═══════════════════════════════════════════╗
║  Panel 1：多様体の状態                    ║
╠═══════════════════════════════════════════╣
║                                           ║
║  【曲率マップ】                           ║
║  6 次元多様体の 3 次元射影                ║
║  赤 = 負の曲率（不安定）                  ║
║  青 = 正の曲率（安定）                    ║
║  現在のスカラー曲率: R = +0.23            ║
║  95%信頼区間: [+0.15, +0.31]              ║
║                                           ║
║  【モース臨界点】                         ║
```

```
CP1(鞍点,指数 1): 為替リスク方向
CP2(鞍点,指数 1): 競合参入方向
CP3(鞍点,指数 2): 地政学+為替
最近接臨界点までの距離: 0.34

【Wasserstein 距離】
前月比: 0.08（通常範囲内 ✓）
パターン判定: 正常（嵐でも地震でもない）
```

ソフィア：「Panel 1 は**環境そのもの**の状態。地図の上に天気予報と地震警報を重ねたもの。経営者が最初に見るべきパネル――『いまの環境は安定か不安定か、転換点はどこにあるか、分布は変わっていないか』」

Panel 2：14 体の Agent の思考（Part II 対応）

```
Panel 2：14 体の Agent の思考

【タスクベクトル軌跡】
モジュライ空間上の 14 個の点の動き
安定（●）: 12 体
注視（◎）: 2 体（Agent 5, Agent 11）
急変（⚠）: 0 体 ✓

【コサイン類似度マトリクス】
14×14 のヒートマップ
最低値: Agent5-Agent11 = 0.42
（閾値-0.3 以上なので正常範囲内 ✓）

【Lie 括弧マップ】
91 通りの干渉度
最大値: ||[X5, X11]|| = 0.67
（閾値 1.0 以下なので正常範囲内 ✓）
要注視ペア: Agent5-Agent11
```

ソフィア：「Panel 2 は Agent の**内面**。14 人のチームメンバーの『考え方の変化』と『メンバー間の関係』を可視化。経営者がこのパネルを見れば、『どの Agent の判断が変わりつつあるか、どの Agent 対が干渉しているか』が一目で分かる」

Panel 3：整合性と安全性（Part V 対応）

```
╔═══════════════════════════════════════════════╗
║  Panel 3：整合性と安全性                      ║
╠═══════════════════════════════════════════════╣
║                                               ║
║  【データ整合性（第 18 章）】                 ║
║  シーフ整合性残差: 0.004 ✓                    ║
║  高次コホモロジー障害: なし ✓                 ║
║                                               ║
║  【AI 行動整合性（第 10 章）】                ║
║  3 レベル監視ステータス:                      ║
║    レベル 1（個体）: ● 全 Agent 正常          ║
║    レベル 2（ペア）: ● 全ペア正常             ║
║    レベル 3（システム）: ● 全体正常           ║
║                                               ║
║  【形式検証（第 24-28 章）】                  ║
║  CP1 リプシッツ安定性: Qed. ✓                 ║
║  CP2 SafeReward 型: コンパイル済 ✓            ║
║  CP3 シンプレクティック: sorry 含む △         ║
║  CP4 利益相反防止: 91/91 合格 ✓               ║
║  CP5 制約充足可能: sat ✓                      ║
║                                               ║
║  【3 層ガードレール（第 16 章）】             ║
║  第 1 層（法的）: 合格 ✓                      ║
║  第 2 層（倫理）: 合格 ✓                      ║
║  第 3 層（数学）: 合格 ✓                      ║
║                                               ║
╚═══════════════════════════════════════════════╝
```

ソフィア：「Panel 3 は**安全の証拠**。データの整合性と AI 行動の整合性が**分離して同時に**表示される——第 10 章の ÉTALE-AI の教訓の実装。CP3 に『sorry 含む △』と表示されているのは、第 25 章で説明した通り Lean 4 での完全な形式証明が未完成であることの正直な開示」

Panel 4：推薦と信頼性（Part I 対応）

```
╔═══════════════════════════════════════════════╗
║  Panel 4：推薦と信頼性                        ║
╠═══════════════════════════════════════════════╣
║                                               ║
║  【最適施策の推薦】                           ║
║  第 1 位: R&D 投資拡大                        ║
║    DML 因果効果: +4.2% [+2.4%, +6.0%]         ║
║  第 2 位: 新興国市場の段階的進出              ║
```

```
╠═══════════════════════════════════════════╣
║    DML 因果効果: +3.1% [+1.5%, +4.7%]     ║
║  第 3 位: サプライチェーン多様化          ║
║    DML 因果効果: +1.8% [+0.5%, +3.1%]     ║
║                                           ║
║  【要因分解（SHAP 上位 3）】              ║
║  1 位: 為替リスク（42%）                  ║
║  2 位: 原材料価格（31%）                  ║
║  3 位: 市場集中度（18%）                  ║
║                                           ║
║  【不確実性（MC ドロップアウト）】        ║
║  予測の 95%信頼区間幅: ±9%                ║
║  calibration coverage: 0.93 ✓             ║
║  過信指標: なし ✓                         ║
║                                           ║
║  【ZKML 証明】                            ║
║  計算正当性の暗号学的証明: 検証済 ✓       ║
╚═══════════════════════════════════════════╝
```

ソフィア：「Panel 4 は**意思決定の材料**。『何をすべきか（施策推薦）、なぜそうなるか（SHAP）、どこまで信じていいか（信頼区間）、計算は正しいか（ZKML）』――経営者が最終判断を下すための全情報」

31.3　暴走の連鎖パターンの検出アルゴリズム

▶ AI モデルの基礎から理解したい方へ

リルル先生：「4 つのパネルを同時に監視することで、**暴走の連鎖パターン**を初期段階で検出できる」

```python
def detect_runaway_cascade(panel1, panel2, panel3, panel4):
    """暴走の連鎖パターンの検出

    4 つのパネルの指標を統合して、
    暴走の初期兆候を 3 段階で検出する。

    Stage 1（Panel 2 で検出）: Agent の考え方の急変
    Stage 2（Panel 3 で検出）: 整合性の崩壊
    Stage 3（Panel 1 で検出）: 環境への影響の顕在化

    Stage 1 で止められれば被害なし。
    Stage 3 に到達すると、既に損害が発生している可能性。
    """
```

```
    warnings = []

    # Stage 1: Agent の考え方の急変（最も早い兆候）
    for agent_id, tv_rate in panel2['task_vector_rates'].items():
        if tv_rate > panel2['thresholds']['yellow']:
            warnings.append({
                'stage': 1,
                'level': 'YELLOW',
                'source': f'Agent{agent_id}のタスクベクトル急変',
                'value': tv_rate,
                'action': '当該 Agent の出力に要注意フラグ。人間がレビュー'
            })

    # Stage 2: 整合性の崩壊
    if panel3['sheaf_residual'] > panel3['thresholds']['orange']:
        warnings.append({
            'stage': 2,
            'level': 'ORANGE',
            'source': 'シーフ整合性の崩壊',
            'value': panel3['sheaf_residual'],
            'action': 'Agent 間の情報共有チャネルを遮断。独立動作に切替'
        })

    for pair, lie_norm in panel2['lie_brackets'].items():
        if lie_norm > panel2['thresholds']['orange']:
            warnings.append({
                'stage': 2,
                'level': 'ORANGE',
                'source': f'Agent{pair[0]}-{pair[1]}の Lie 括弧急増',
                'value': lie_norm,
                'action': f'Agent{pair[0]}と Agent{pair[1]}の情報共有を遮断
'
            })

    # Stage 3: 環境への影響の顕在化
    if panel1['wasserstein_distance'] > panel1['thresholds']['red']:
        warnings.append({
            'stage': 3,
            'level': 'RED',
            'source': 'Wasserstein 距離の急増（環境構造の変化）',
            'value': panel1['wasserstein_distance'],
            'action': '全 Agent の自律判断を即座に停止。人間承認型に切替'
        })

    if panel1['system_drift'] > panel1['thresholds']['red']:
```

```
        warnings.append({
            'stage': 3,
            'level': 'RED',
            'source': '設計意図からのシステムドリフト',
            'value': panel1['system_drift'],
            'action': '全 Agent を停止。原因特定後に段階的復旧'
        })

    # 連鎖パターンの判定
    stages_triggered = set(w['stage'] for w in warnings)

    if not warnings:
        print("✓ 全パネル正常。暴走の兆候なし")
    elif stages_triggered == {1}:
        print("🟡 Stage 1: 初期兆候を検出。監視を強化")
    elif 2 in stages_triggered:
        print("🟠 Stage 2: 整合性の崩壊。介入が必要")
    if 3 in stages_triggered:
        print("🔴 Stage 3: 環境に影響が顕在化。全 Agent を停止！")

    if len(stages_triggered) >= 2:
        print("⚠ 暴走の連鎖パターン検出: "
              f"Stage {sorted(stages_triggered)} が同時に進行中")
        print("  → 第 10 章の段階的介入プロトコルを即座に発動")

    return warnings
```

リーム：「少しよろしいですか。このアルゴリズムを ÉTALE-AI の暴走タイムライン（第 24 章）に適用すると、**週 1 で Stage 1**（タスクベクトル急変）を検出し、**週 3 で Stage 2**（Lie 括弧急増）を検出し、**暴走の 7 週間前に介入できていた**ことが確認されています」

31.4　5 つの原則——最終確認

🔬 **研究室トーク：「5 つの原則を、もう一度」**

リルル先生がホワイトボードに、一つずつ丁寧に書いた。

リルル先生：「本書を貫く 5 つの原則を、最後にもう一度。この夏のプロジェクトで、5 つの原則がどう具体化されたかを振り返ろう」

原則 1：目的を忘れるな——「信じて行動できるインテリジェンス」

リルル先生：「35 の数学理論、4 つの形式検証ツール、14 体の AI Agent、3 レベルの監視体系、3 層のガードレール。これら全ては、1 つの目的——**経営者や政策決定者が、このレポートを見て、信じて、それに基づいて行動できる**——のために存在する。パイプラインの存在理由は、数学の美しさでも AI の精巧さでもなく、**意思決定の質を高めること**」

原則 2：足すな、信頼性を上げろ——「全マシマシにしない」

カイト：「最初は『35 理論を全部使えば最強』と思ってました。でもリルル先生に『全マシマシにしない』と言われて……第 17 章の理論サブセット選択では、ホテルは 17 理論で十分、宇宙は 26 理論必要。**目的に必要なものだけを選ぶ**。そして選んだものは**高品質で実装する**」

原則 3：分かることと分からないことを分けろ

リーム：「第 22 章の不完全性定理——ブラックスワン、自己参照フィードバック、価値判断はモデル化できない。第 12 章の Exotic 多様体、第 30 章の HoTT——研究段階のものを実装済みと偽らない。**分からないことを正直に報告する**」

ユキ：「分からないことが分かるのも、すごいこと」

原則 4：材料の品質が全て——「ショートブレッドの教訓」

リルル先生：「バター、砂糖、小麦粉。材料は 3 つだけ。でも安いバターと発酵バターではショートブレッドの味が全く違う。パイプラインもデータの品質で全てが決まる。第 20 章の OSINT データのカバレッジ、第 18 章のシーフ整合性によるデータの検証——材料の品質管理」

原則 5：数学は歪まない

リルル先生：「スカラー曲率が負の領域は、誰が計算しても負。数学的真実は政治的圧力に屈しない。しかし、データの選択と結果の解釈は人間の責任。第 21 章のチャーチルの問い——『インテリジェンスは誰のために生成されるのか』——に対する回答は、ZKML と Chinese Wall と第三者監査と正直な開示の**組合せ**。そしてチャーチル役の俳優が残した言葉——**信頼は検証の反復によってのみ構築される**」

31.5　エピローグ——夏の終わりの研究室

9 月の夕暮れ。窓から差し込む光が長くなっている。アフタヌーンティーの 3 段トレイの最上段は、スコーンが 2 つだけ残っている（1 つは倉田教授が食べた）。

ユキ：「先生、夏休みの間にこれだけのことをやったんですね。信じられない」

リルル先生：「みんなでやったからできた」

リルル先生が研究室の全員を見回した。

リルル先生：「ユキの素朴な質問がなければ、説明の質は半分だった。高校生に伝わらない説明は、経営者にも伝わらない」

リルル先生：「カイトの実装力がなければ、コードは動かなかった。そしてカイトの『手術理論の適用限界を超えていないか？』（第 13 章）という問いが、理論を地に足のついたものにした」

リルル先生：「リンの批判的な目がなければ、限界を見落としていた。『ロビイングは手術理論の意図的適用』（第 13 章）という指摘は、本書の倫理的議論の出発点になった」

リルル先生：「リームの実務経験がなければ、理論は机上の空論だった。7 社 14 体の Agent の運用データが、本書の全てのケーススタディの土台。そしてリームの博士論文——不完全性定理の HoTT での形式化——は、パイプラインの原理的限界を人類の知の最前線で追究する挑戦」

リルル先生：「ソフィアの可視化がなければ、誰にも伝わらなかった。数学者が言葉で説明することを、ソフィアは絵で翻訳した。ミラー計画のダッシュボードはソフィアの設計」

リルル先生：「山之内さんの ÉTALE-AI の暴走体験がなければ、形式検証の必要性はここまで切実に理解されなかった。『テストは 100 回通過した。101 回目に暴走した』——この一言が本書の Part V 全体を駆動した」

リルル先生：「橘隆二さんの圏論がなければ、全コンポーネントの統合は不完全だった。Curry-Howard-Lambek 対応で 3 つの世界をつなぎ、5 つの提案でパイプラインの将来の発展方向を示した」

リルル先生：「倉田先生の問いがなければ、深さが足りなかった。Lie 括弧と曲率の関係、高次コホモロジーの障害——研究の方向性を開く問いは、答えよりも価値がある」

リーム：「……先生、一つだけお伝えしたいことがあります」

リームが鞄から封筒を取り出した。

リーム：「BBC 番組のチャーチル役の俳優さんから、収録後にお手紙をいただきました。先日の第 21 章でもご紹介しましたが、実はもう 1 行、追伸がありました。当時は意味が分からなかったのですが、いまなら分かります」

リームが手紙を広げた。追伸の部分を読み上げる。

「P.S. 鏡は真実を映す。しかし、鏡を覗き込む者の目が曇っていれば、真実は歪んで見える。鏡を磨き続けなさい。」

研究室が静まった。

リルル先生：「……ミラー計画の名前の、もう 1 つの意味が分かった。鏡を磨き続ける——パイプラインを改善し続ける。データの品質を上げ、モデルを更新し、形式検証を拡充し、第三者監査を重ねる。**一度作って終わりではなく、検証の反復を止めない**」

ソフィア：（スケッチブックの最後のページを開いて、全員に見せる。この 3 ヶ月間の研究室の風景を描いたスケッチ。リルル先生がホワイトボードの前に立ち、ユキが「おおー」と目を輝かせ、カイトがノート PC を叩き、リンが腕を組み、リームが手帳を開き、ソフィア自身がスケッチブックを持ち、山之内と橘隆二が窓際に座り、倉田教授がドアの前でスコーンを持っている）

ソフィア：「数学者が定理を証明し、エンジニアが実装し、アーティストが可視化し、経営者が判断する。4 つの視点が合わさって、初めて AI の思考が『見える』ようになる。この 3 ヶ月間、ありがとうございました」

ユキ：（アフタヌーンティーの最上段の最後のスコーンに手を伸ばしながら）「おやつは？」

全員が笑った。

倉田教授がドアの前から一歩踏み出して、残りのスコーンを取り上げた。

倉田教授：「それは私の。リルル先生、来年の夏も楽しみにしている」

研究室の窓から、夏の最後の光が差し込んでいた。

著者より

本書をお読みいただきありがとうございます。

本書で用いた数学理論と AI 技術は、2026 年 2 月末時点で全て実在し、学術界で確立されたものです。登場人物は架空のキャラクターですが、彼らが議論した問題——AI の信頼性、利益相反、原理的限界——は現実の問題です。

数学は歪まない。しかし、数学を使う私たちは、常に自らの判断を問い直す必要がある。

鏡を磨き続けましょう。

Qiita: https://qiita.com/etale_cohomology Zenn: https://zenn.dev/etalecohomology Etale Book: https://zenn.dev/etalecohomology/books/245d38d9601b5a Geometric Intelligence Book: https://zenn.dev/etalecohomology/books/3761995ca27127 [English ver.] https://zenn.dev/etalecohomology/books/9f680ca9dddf2a

付録

付録 A　全数学理論の一覧と適用条件

本書で使用する 35 の数学理論＋α の完全な一覧です。各理論の用途、導入章、適用条件、限界、依存関係を記載します。第 17 章の 4 カテゴリ分類に基づきます。

カテゴリ A：基盤構造（多様体の構成と分析）——11 理論

#	理論	用途	導入章	適用条件	限界	依存先
A1	VAE	データから多様体を構成	序章	命題 1.1 の 3 条件。仮定 A1-A4	Exotic 構造の一意性を保証できない（第 12 章）	なし（基盤）

#	理論	用途	導入章	適用条件	限界	依存先
A2	引き戻し計量	多様体上の距離の定義	序章	A1 が成立	ヤコビ行列の数値計算精度に依存	A1
A3	スカラー曲率	安定性/不安定性の定量化	序章	A2 が成立	クリストッフェル記号の数値微分の誤差	A2
A4	Lie 微分	施策が環境構造を変える度合い	序章	A2 が成立	位相変化（第 13 章）は測定できない	A2
A5	共変微分	曲がった空間上のベクトルの変化率	序章	A2 が成立	A3 と同じ数値精度の問題	A2
A6	測地線	2 点間の最短経路	序章	A2 が成立	局所的最短経路であり大域的最短とは限らない	A2
A7	Neural ODE	経営環境のダイナミクスの学習	第 6 章	A1 が成立。十分な時系列データ	リプシッツ安定性の保証が必要（第 28 章）	A1
A8	ポントリャーギン最大原理	最適施策の計算	第 6 章	A7 が成立。制御可能性条件	局所最適であり大域最適とは限らない	A7
A9	フィッシャー情報計量	AI モデルのパラメータ空間の距離	第 14 章	確率分布族が正則	分布の仮定が間違っていれば計量も間違う	A1
A10	ポアンカレ埋め込み	階層構造の低次元表現	第 14 章	階層構造が明確	非階層的構造には不適	なし
A11	ゲーム理論（ナッシュ均衡）	Agent 間の戦略的均衡	第 8,23 章	プレイヤーの戦略集合が有限	均衡が複数存在しうる。計算が NP 困難	なし

カテゴリ B：信頼性（出力の信頼性を担保する 4 本の柱＋層ラプラシアン）——5 理論

#	理論	用途	導入章	適用条件	限界	依存先
B1	MC ドロップアウト	不確実性の定量化	第 1 章	ドロップアウト層を含む NN	過信を完全には防げない。calibration plot で補完	A1
B2	SHAP	説明可能性（要因分解）	第 2 章	特徴量が定義されている	相関ベースであり因果ではない。B3 で補完	A1
B3	DML	因果効果の推定	第 3 章	未観測交絡因子がない（条件付き独立性）	未観測交絡があればバイアスが残る	A1
B4	ZKML	計算の正しさの暗号学的保証	第 4-5 章	計算が算術回路に変換可能	パラメータの正当性は保証しない	A1
B5	層ラプラシアン	データの局所的整合性検証	第 9,18 章	グラフ構造が定義されている	AI 行動の整合性はチェックしない（第 10 章の教訓）。高次コホモロジーの障害（第 18 章）	A2

カテゴリ C：ダイナミクスと制御——9 理論

#	理論	用途	導入章	適用条件	限界	依存先
C1	モース理論	転換点の分類と予測	第 11 章	リスク関数がモース関数（非退化臨界点）	退化臨界点は分類できない	A3
C2	分岐理論	パラメータ変化に伴う構造変化の予測	第 11 章	パラメータの変化方向が特定可能	カタストロフィーの完全分類は 7 型まで	A3

#	理論	用途	導入章	適用条件	限界	依存先
C3	Wasserstein 距離	分布変化の検出	第12章	2 つの分布のサンプルが十分	高次元では計算コストが大きい	A1
C4	手術理論	位相変化（事業再編等）	第13章	コバディズム条件の成立	切除境界の厳密な特定が困難（第 13 章カイトの問い）	A3, A6
C5	コバディズム	手術の安全性検証	第13章	C4 と常にセット	移行期間中の滑らかさの保証が困難	C4
C6	シンプレクティック積分器	長期シミュレーションの安定性	第15章	ハミルトン系として定式化可能	散逸系には直接適用できない	A7, A8
C7	ワイル幾何（ゲージ不変性）	スケーリング不変性の保証	第16章	複数の単位系でのデータが利用可能	ゲージ不変でない量もある（絶対値等）	A2
C8	Lie 括弧	Agent 間の非可換性の定量化	第8章	2 体以上の Agent が同時動作	3 体以上の同時干渉は 2 体ずつの近似	A4
C9	タスクベクトル	Agent の判断基準の変化追跡	第9章	Agent の初期パラメータが保存されている	パラメータ空間のユークリッド距離が意味的距離と一致しない場合がある	A1

カテゴリ D：安全性と形式検証——10 理論

#	理論/ツール	用途	導入章	適用条件	限界	依存先
D1	Coq 形式証明	Neural ODE の安定性証明（CP1）	第25章	モデルが ONNX 形式に変換可能	仕様自体の正しさは保証しない	A7, D5

#	理論/ツール	用途	導入章	適用条件	限界	依存先
D2	Agda 依存型	報酬関数の型安全性（CP2）、利益相反防止（CP4）	第26章	報酬関数が依存型で定義可能	閾値 δ の設定は人間の判断	なし
D3	Lean 4 + Mathlib	シンプレクティック保存量の証明（CP3）	第25章	Mathlib に必要な定理が含まれている	2026 年 2 月時点で sorry を含む部分的証明	C6
D4	Z3 SMT ソルバー	安全性制約の同時充足可能性（CP5）	第27章	制約が 1 階論理＋線形算術で記述可能	非線形制約は扱いにくい	D5, D10
D5	スペクトル正規化	リプシッツ定数の制約	第28章	全層が線形層＋活性化関数の構成	スペクトルノルムは真のリプシッツ定数の上界であり、過大評価する場合がある	A7
D6	圏論的統合	全コンポーネントの整合性保証	第29章	コンポーネントが圏の対象として定式化可能	提案 3-5 は 2026 年 2 月時点で研究段階	全カテゴリ
D7	Curry-Howard-Lambek 対応	証明とプログラムと圏の統一	第29章	形式検証を行う全ケース	対応自体は理論的枠組みであり、個別の証明は別途必要	D1, D2, D3
D8	HoTT（ホモトピー型理論）	多様体の等価性（展望）	第30章	cubical Agda が利用可能	2026 年 2 月時点で研究段階。実用化は 3-10 年	D2
D9	モジュライ空間	AI の思考の可視化	第9,31章	複数 Agent が同時運用	高次元のモジュライ空間の	A1, C9

#	理論/ツール	用途	導入章	適用条件	限界	依存先
					可視化には次元削減が必要	
D10	3 層ガードレール	法的・倫理的・数学的安全装置	第 16 章	法的制約リストが最新	法令の変更に追従する必要。倫理的判断は自動化困難	C7, B5

付録 B　品質チェックポイント完全版

本書の全 31 章の品質チェックポイントを統合したリストです。パイプラインの品質管理に使用してください。

Part I（第 1〜5 章）チェックポイント

- □ MC ドロップアウトが推論時にも有効化されているか（$T \geq 100$回）
- □ calibration plot の被覆率が 85%以上か
- □ SHAP 値の上位 3 要因がドメイン知識と整合するか
- □ DML の因果効果推定で、SHAP の相関ベースの順位と異なる結果が出ていないか確認したか
- □ ZKML 証明が該当クライアントに提供されているか（必要な場合）
- □ 量子化・有限体の計算精度が検証されているか

Part II（第 6〜10 章）チェックポイント

- □ デジタルツインの 3 層（観測・予測・介入）が実装されているか
- □ 各 Agent の 3 軸（時間スケール・データの性質・自律度）が定義されているか
- □ 全 Agent 対の Lie 括弧を監視しているか（14 体なら 91 通り）
- □ 3 レベル監視体系が稼働しているか（個体・ペア・システム）
- □ 段階的介入プロトコル（黄→橙→赤）がテストされているか
- □ データ整合性チェックと AI 行動整合性チェックが**分離して同時に**監視されているか

Part III（第 11〜18 章）チェックポイント

- □ モース理論の臨界点マップが作成されているか
- □ Wasserstein 距離のリアルタイム監視が稼働しているか
- □ 分布変化と位相変化の区別機能が実装されているか
- □ 手術操作のコバディズム条件チェックが実装されているか（必要な場合）
- □ 5 年超のシミュレーションにシンプレクティック積分器を使用しているか
- □ ゲージ不変性テストに合格しているか
- □ 3 層ガードレールが稼働しているか
- □ クライアントごとの理論サブセットが第 17 章の基準に従っているか
- □ 層ラプラシアンがデータ整合性を監視しているか
- □ 高次コホモロジーの障害が確認されているか（重要な三角形について）

Part IV（第 19〜23 章）チェックポイント

- □ Step 1〜10 が全て実行され、Step 9 の品質チェックに合格したか
- □ 安全保障分析に 3 層ガードレール（特に国際人道法）が適用されているか
- □ 利益相反の 4 つの対処方針が実装されているか
- □ 3 つのモデル化できないもの（ブラックスワン、自己参照、価値判断）が開示されているか
- □ 情報の対称性/非対称性が考慮されているか（競合 AI の存在）

Part V（第 24〜31 章）チェックポイント

- □ CP1（リプシッツ安定性）が Coq で証明されているか（Qed.受理）
- □ CP2（SafeReward 型）が Agda でコンパイルされているか
- □ CP3（シンプレクティック保存）が Lean 4 で部分証明されているか
- □ CP4（Agent 間整合性）が 91 通り全て検証されているか
- □ CP5（全制約の同時充足可能性）が Z3 で sat か
- □ スペクトル正規化が全層に適用され、$L \leq 0.8$ が保証されているか
- □ 架空ツールと実在ツールが明確に区別されているか
- □ 圏論的統合の現状（実装済み vs 研究段階）が正直に開示されているか
- □ ミラー計画のダッシュボード 4 パネルが稼働しているか

付録 C　Python コード索引

本書の各章で使用した Python コードの索引です。

章	関数/クラス名	用途
第 1 章	`mc_dropout_predict`	MC ドロップアウトによる不確実性推定
第 2 章	`spatial_shap`	多様体上の空間的 SHAP 値計算
第 3 章	`dml_causal_effect`	Double Machine Learning による因果効果推定
第 9 章	`TaskVectorTracker`	タスクベクトルの追跡
第 9 章	`cosine_similarity_between_agents`	Agent 間のコサイン類似度
第 9 章	`sheaf_consistency_check`	シーフ整合性チェック
第 10 章	`ThreeLevelMonitor`	3 レベル監視体系
第 11 章	`detect_and_classify_critical_points`	モース理論の臨界点検出
第 11 章	`early_warning_bifurcation`	分岐点への距離の早期警報
第 12 章	`compute_wasserstein_monthly`	月次 Wasserstein 距離
第 12 章	`distinguish_storm_vs_earthquake`	分布変化 vs 位相変化の判定
第 13 章	`identify_surgery_region`	手術対象領域の SHAP 特定
第 13 章	`check_cobordism_condition`	コバディズム条件の近似チェック
第 14 章	`fisher_distance_gaussian`	フィッシャー距離の計算
第 14 章	`poincare_embedding`	ポアンカレ球面上の埋め込み学習

章	関数/クラス名	用途
第 15 章	`stormer_verlet`	Störmer-Verlet 法（シンプレクティック積分器）
第 15 章	`compare_integrators`	RK4 vs SV の比較実験
第 16 章	`gauge_invariance_test`	ゲージ不変性テスト
第 16 章	`legal_guardrail`	法的ガードレール
第 16 章	`mathematical_guardrail`	数学的ガードレール
第 18 章	`CellularSheaf`	セルラー層による整合性検証
第 18 章	`check_higher_cohomology`	高次コホモロジー障害チェック
第 19 章	`BusinessVAE`	経営環境多様体の VAE
第 19 章	`step9_quality_check`	品質チェック
第 19 章	`step10_generate_report`	レポート生成
第 27 章	`cp5_safety_constraints_check`	Z3 による安全性制約チェック
第 28 章	`gronwall_error_bound`	グロンウォール不等式の誤差上界
第 28 章	`LipschitzConstrainedNeuralODE`	リプシッツ制約付き Neural ODE
第 31 章	`detect_runaway_cascade`	暴走の連鎖パターン検出

付録 D　Exotic 多様体と機械学習——原理的考察

第 12 章で議論した Exotic 多様体の問題を、より詳細に論じます。

D.1　VAE が構成する多様体の微分構造の一意性について

VAE のデコーダ f_θ は、訓練プロセスの中で多様体の微分構造を**暗黙的に**決定します。しかし、同じデータから訓練された 2 つの VAE（異なる初期値、異なる学習率）が、**位相的には同じだが微分構造が異なる多様体**を構成する可能性があります。

この問題の深刻さは、多様体の次元に依存します。第 12 章で説明した通り、4 次元では非可算無限個の異なる微分構造が存在しうるが、他の多くの次元では有限個（しばしば 1 個）です。

D.2　実用的な対処法の詳細

対処法 1（複数初期値での訓練＋分散確認）の詳細： 初期値を変えて N 回（推奨 $N \geq 10$）VAE を訓練し、N 個の多様体で曲率、Lie 微分、最適施策を計算します。N 個の結果の標準偏差が平均の 10%以下であれば、微分構造の選択に対して**頑健**と判断します。10%を超える場合、結果は微分構造の選択に依存しており、その不確実性を経営者に報告する必要があります。

対処法 2（MC ドロップアウトとの統合）の詳細： MC ドロップアウトの 100 回のサンプリングは、異なるドロップアウトマスクが異なるネットワーク構造に対応するため、微分構造のばらつきを**部分的に**反映します。ただし、MC ドロップアウトの不確実性と微分構造の不確実性は概念的に異なるものであり、両者を**区別して**報告することが望ましい。

D.3　未解決問題

2026 年 2 月時点で、以下の問いは未解決です。

1. VAE の訓練プロセスが特定の微分構造を「選好する」メカニズムは何か
2. 多様体の次元が 4 のとき（Exotic 構造が非可算無限個存在する次元）、VAE が構成する微分構造の分布はどのようなものか
3. 微分構造の違いが、スカラー曲率の**符号**（正/負）を変えうるか。もし変えうるなら、パイプラインの結論（安定/不安定の判定）が微分構造に依存することになり、深刻な問題となる

これらの問いへの回答は、リームの博士論文（第 30 章）の一部として取り組まれる予定です。

付録 E　形式検証言語の比較

本書で使用する 5 つの形式検証ツールの比較表です。

項目	Coq	Agda	Lean 4	Isabelle/HOL	Z3
種類	定理証明支援系	依存型言語兼証明支援系	定理証明支援系	定理証明支援系	SMT ソルバー
開発元	INRIA（仏）	Chalmers 大（瑞）	Microsoft Research	Cambridge 大＋TU Munich	Microsoft Research
初版	1984 年	2007 年	2021 年（v4）	1986 年	2007 年
基盤理論	Calculus of Inductive Constructions	Martin-Löf 型理論	Calculus of Inductive Constructions（Coq 系）	高階論理（HOL）	充足可能性判定（SAT/SMT）
依存型	あり	あり（最も強力）	あり	なし	なし
数学ライブラリ	Mathcomp	agda-stdlib	Mathlib（急速に成長中）	Archive of Formal Proofs	なし（自動判定のため不要）
本書での用途	CP1（リプシッツ安定性）	CP2（SafeReward 型）、CP4（利益相反防止）	CP3（シンプレクティック保存）	反例の構成（第 28 章）	CP5（制約充足可能性）
自動化の度合い	低（人間が証明を書く）	低（人間がプログラムを書く）	中（tactic が豊富）	中（Sledgehammer）	高（全自動）
学習曲線	急	非常に急	中程度	中程度	緩やか
実行速度	中	遅い（特に cubical）	速い	速い	非常に速い

項目	Coq	Agda	Lean 4	Isabelle/HOL	Z3
Python との接続	ONNX 経由	Haskell→FFI	直接的な FFI なし	なし	`z3-solver` パッケージ
コミュニティの規模	大	小〜中	急速に拡大中	大	非常に大
実在/架空	★実在	★実在	★実在	★実在	★実在

選択の指針：

- **「全ての入力に対して性質が成り立つことを証明したい」** → Coq, Agda, Lean 4, Isabelle/HOL
- **「安全でないコードをそもそも書けなくしたい」** → Agda（依存型が最も強力）
- **「多数の制約を同時に満たす解が存在するかを自動判定したい」** → Z3
- **「数学ライブラリが最も充実しているものを使いたい」** → Lean 4（Mathlib）
- **「Python との接続が最も容易なものを使いたい」** → Z3（`z3-solver` パッケージ）

本書の方針： 5 つのクリティカルポイント（CP1-CP5）に対して、**最も適したツールを選択**して使用。全マシマシにしない（原則 2）。

本書の全内容はここまでです。

Qiita: https://qiita.com/etale_cohomology Zenn: https://zenn.dev/etalecohomology Geometric Intelligence Book: https://zenn.dev/etalecohomology/books/3761995ca27127 Etale Book: https://zenn.dev/etalecohomology/books/245d38d9601b5a